

AWS – Initiation et approfondissement

Support consolide pour lecture et impression. Les quiz interactifs restent disponibles dans la version web.

Informations administratives

Cette page rassemble les informations pratiques à connaître avant d’aborder les chapitres techniques.

- L’émargement est réalisé deux fois par journée de formation.
- Les besoins et attentes sont recueillis au démarrage de la session.
- Les accès temporaires et les consignes propres aux labs sont communiqués pendant la session et ne sont pas publiés dans ce support.

Horaires

Période	Matin	Après-midi
Lundi	9 h 30 – 12 h 30	13 h 30 – 17 h 30
Mardi à vendredi	9 h 00 – 12 h 30	13 h 45 – 17 h 00

Une pause de 15 minutes est prévue le matin et l’après-midi.

Tour de table

Avant d’aborder les concepts techniques, ce tour de table permet d’adapter les exemples et le niveau d’accompagnement au contexte professionnel du groupe :

- Quel est votre métier et votre environnement technique actuel ?
- Avez-vous déjà utilisé un cloud public ou administré une infrastructure virtualisée ?
- Quels services ou sujets AWS rencontrez-vous dans vos projets ?
- Quel objectif professionnel souhaitez-vous atteindre avec cette formation ?

- Existe-t-il une contrainte particulière à prendre en compte : sécurité, réseau, coûts, exploitation ou architecture ?

Les réponses servent à choisir les exemples et le niveau d'approfondissement sans modifier la couverture du programme.

Important

Règle d'or de cette formation : interrompre le formateur dès qu'une notion n'est pas comprise ou qu'une question se présente ; partager son écran en cas de blocage technique afin de permettre un diagnostic collectif ; participer activement aux échanges et aux validations.

Finalité de la formation

La formation apporte les bases nécessaires pour analyser une architecture AWS, comprendre les responsabilités de sécurité, sélectionner les principaux services de calcul, de stockage, de réseau et de données, puis introduire l'automatisation et la supervision.

Le support sépare volontairement trois activités :

- le **cours**, consacré aux concepts, au fonctionnement des services et aux décisions d'architecture ;
- les **travaux pratiques**, réalisés dans l'environnement temporaire remis au début de la formation ;
- les **quiz**, utilisés pour vérifier immédiatement la compréhension de chaque chapitre.

Objectifs de la formation

À l'issue de la formation, vous saurez :

- **expliquer** les fondamentaux du Cloud Computing et le modèle de responsabilité partagée pour choisir une région ou une architecture multi-AZ adaptée ;
- **concevoir** une gestion des identités et des accès sécurisée avec IAM (rôles temporaires, moindre privilège, MFA, traçabilité) ;
- **concevoir** un VPC segmenté et choisir un service de données (relationnel, NoSQL ou cache) adapté à un besoin donné ;
- **sélectionner et configurer** les services de calcul et de stockage adaptés (EC2, Lambda, S3, EBS, EFS) en tenant compte de l'élasticité, de la disponibilité et de la reprise d'activité ;

- **automatiser** une infrastructure avec CloudFormation, l'administrer avec Systems Manager et l'évaluer avec CloudWatch et les six piliers du AWS Well-Architected Framework.

Public et prérequis

Le parcours s'adresse aux professionnels de l'informatique, de l'exploitation, du développement, de la sécurité ou de l'architecture qui souhaitent comprendre ou consolider leur pratique d'AWS.

Les manipulations nécessitent :

- un ordinateur disposant d'un navigateur web récent ;
- une connexion Internet stable ;
- l'accès aux environnements temporaires remis en début de formation.

Aucun compte AWS personnel, aucune carte bancaire et aucune installation locale de la CLI AWS ne sont exigés pour suivre les activités prévues.

Méthode de travail

1. Lire les objectifs du chapitre.
2. Parcourir les concepts dans l'ordre proposé ou accéder directement à une notion précise.
3. Consulter le glossaire et la cheat sheet depuis les liens du chapitre.
4. Réaliser les modules et labs indiqués dans l'environnement de formation.
5. Terminer par le quiz interactif du chapitre.

Info

Cette formation débute par les fondations théoriques indispensables avant de manipuler la console AWS : sans comprendre ce qu'est réellement le Cloud Computing, ses modèles économiques et ses responsabilités, il est impossible de faire des choix d'architecture pertinents par la suite.

Objectifs du chapitre

À l'issue de ce chapitre, les stagiaires seront capables de :

- **Expliquer** ce qu'est AWS — historique, positionnement sur le marché, modèle de responsabilité partagée
- **Distinguer** les modèles cloud : IaaS/PaaS/SaaS, public/privé/hybride, CAPEX vs OPEX
- **Expliquer** les principes de virtualisation, conteneurisation et microservices
- **Situer** l'infrastructure mondiale AWS (régions, AZ, edge locations) et appliquer le Well-Architected Framework
- **Naviguer** dans l'AWS Management Console et maîtriser les outils de suivi budgétaire (Billing, Budgets, Cost Explorer)

1. Introduction

1.1 Qu'est-ce qu'AWS et pourquoi le découvrir ?

Amazon Web Services (AWS) est la division cloud du groupe Amazon. Elle propose aujourd'hui plus de 200 services : calcul, stockage, bases de données, sécurité, intelligence artificielle, DevOps.

Ce catalogue n'a pas surgi d'un coup. Il est le résultat de vingt ans de croissance quasi continue — la section suivante en retrace les jalons.

Ce qui rend AWS incontournable dans une formation cloud n'est pas seulement son ancienneté. C'est sa position de force sur le marché. AWS capte aujourd'hui un peu plus de **30 % du marché mondial des infrastructures cloud**. Cette part dépasse celle de ses deux principaux concurrents, Microsoft Azure et Google Cloud, réunis.

Cette domination a une conséquence directe pour vous. Les compétences AWS restent, pour l'instant, les plus recherchées sur le marché de l'emploi IT. Une grande partie des architectures cloud que vous croiserez en entreprise — quel que soit le secteur — auront été conçues avec ces services en tête. Apprendre AWS en premier, c'est donc apprendre un vocabulaire et des mécanismes qui se retrouvent, sous des noms différents, chez tous les fournisseurs cloud : régions, zones de disponibilité, modèle de responsabilité partagée.

Vous trouverez AWS dans quasiment tous les secteurs d'activité, pas seulement chez des startups qui testent une idée en quelques jours. Amazon documente publiquement des cas d'usage chez des entreprises que vous connaissez déjà :

- **Netflix** héberge sur AWS l'intégralité de son infrastructure de diffusion vidéo, jusqu'à un studio de production virtuel dans le cloud.
- **Airbnb** y fait tourner ses annonces, ses paiements sécurisés et son moteur de recommandation.
- La bourse **Nasdaq** y traite des données de marché critiques.
- La **NASA** y stocke et analyse les données de ses missions spatiales.
- Dans la santé, des groupes pharmaceutiques comme **Pfizer** s'appuient sur AWS pour leurs travaux de recherche.

Ces exemples ne sont pas anecdotiques. Ils montrent que le même socle de services — calcul, stockage, bases de données, sécurité — s'adapte à un site e-commerce comme à une mission spatiale ou un système bancaire. C'est cette polyvalence que ce chapitre pose comme fondation, avant d'entrer dans le détail des services.

1.2 Histoire d'AWS — d'un mémo interne au leader mondial du cloud

L'histoire d'AWS ne commence pas en 2006 avec le lancement grand public de S3 et EC2. Elle commence trois ans plus tôt, en interne chez Amazon.

En 2003, l'entreprise impose à toutes ses équipes une architecture orientée services (SOA). Chaque système doit communiquer avec les autres via des API bien définies, plutôt que par des accès directs à des bases de données partagées. Cette contrainte technique visait d'abord à fiabiliser le site e-commerce d'Amazon. Mais elle révèle un constat inattendu : les équipes passent un temps disproportionné à réinventer chacune leur propre infrastructure de serveurs et de stockage.

Deux ingénieurs, **Chris Pinkham** et **Benjamin Black**, proposent alors un mémo interne. Il décrit des services de calcul virtualisés standardisés. L'idée germe : Amazon pourrait vendre cette infrastructure interne à d'autres entreprises.

Qui sont ces personnes ?

- **Chris Pinkham** — ingénieur Amazon. Il dirigera ensuite le développement d'EC2 depuis une nouvelle filiale créée au Cap, en Afrique du Sud.
- **Benjamin Black** — ingénieur Amazon, co-auteur du mémo fondateur avec Pinkham. Il quittera l'entreprise avant le lancement public d'AWS en 2006.
- **Andy Jassy** — à l'époque cadre chez Amazon, pas encore dirigeant du groupe. Il porte le projet politiquement en interne et devient le premier responsable d'AWS, avant de diriger Amazon tout entier en 2021 (voir plus bas).

Andy Jassy porte le projet. Il rédige la note de six pages — un "six-pager", format classique chez Amazon pour formaliser une proposition stratégique. Le feu vert tombe : la première équipe est recrutée.

Le premier service à voir le jour, en 2004, est discret : **Amazon SQS** (Simple Queue Service), une file de messages entre applications. Pas spectaculaire, mais c'est la première pierre technique du futur AWS.

Le grand public découvre la plateforme en 2006, avec le lancement quasi simultané de deux services :

- **S3** (mars 2006) — stockage objet.
- **EC2** (août 2006) — calcul virtualisé à la demande.

Ces deux services incarnent encore aujourd’hui les deux piliers historiques du cloud : payer pour stocker des données, payer pour de la puissance de calcul — les deux à l’usage, jamais par achat de matériel.

La table ci-dessous reprend les jalons majeurs qui ont suivi ce lancement fondateur. Les dates comptent moins que ce qu’elles disent : AWS est passé d’un simple hébergeur de serveurs à une plateforme qui couvre aujourd’hui la donnée, la sécurité et l’intelligence artificielle.

Année	Événement	Explication pédagogique
2004	Lancement discret d'Amazon SQS	Premier service AWS jamais commercialisé, une simple file de messages — la brique technique fondatrice, avant même que le nom “AWS” ne soit public.
2006	Lancement public d'AWS avec S3 et EC2	AWS débute avec deux services fondamentaux : S3 pour le stockage objet (sauvegardes, fichiers, images) et EC2 pour le calcul (machines virtuelles). Ces deux services incarnent les piliers du cloud : stockage et puissance de calcul à la demande.
2009	Introduction de VPC et RDS	AWS ajoute VPC (Virtual Private Cloud) pour créer des réseaux privés isolés, et RDS (Relational Database Service) pour gérer des bases de données relationnelles sans administrer les serveurs. Cela marque l'arrivée des services réseau et des bases gérées.
2012	Lancement de DynamoDB	AWS introduit DynamoDB , une base NoSQL scalable et sans schéma, adaptée aux applications modernes (web, mobile, IoT). C'est le tournant vers les architectures serverless et les microservices.
2014	Lancement de AWS Lambda	AWS révolutionne le cloud avec Lambda , qui permet d'exécuter du code sans serveur. C'est le début du serverless computing , où l'infrastructure disparaît derrière la logique métier.
2021	Andy Jassy devient CEO d'Amazon	Le fondateur d'AWS quitte la direction d'AWS pour succéder à Jeff Bezos à la tête du groupe Amazon tout entier — signe du poids stratégique pris par la division cloud. Adam Selipsky (l'un des tout premiers VP d'AWS, embauché en 2005) reprend la direction d'AWS.
2024	Matt Garman devient CEO d'AWS	Vétéran d'AWS depuis 18 ans et premier product manager d'EC2 historiquement, Matt Garman succède à Adam Selipsky le 3 juin 2024 et dirige AWS aujourd'hui.
2025	Plus de 200 services , chiffre d'affaires record	AWS devient le fournisseur cloud le plus complet, couvrant tous les domaines : calcul, stockage, IA, sécurité, DevOps, IoT, bases de

Année	Événement	Explication pédagogique
		données, etc. La section suivante détaille les chiffres financiers de cette année.

Ces dates montrent une trajectoire claire : AWS est passé d'un simple fournisseur de serveurs et de stockage à une **plateforme cloud complète** — hébergement, sécurité, automatisation, intelligence artificielle.

Elles montrent aussi une continuité de gouvernance rare dans la tech. De 2003 à 2021, une seule personne — Andy Jassy — a porté le projet, du mémo initial jusqu'à la direction du groupe Amazon tout entier. C'est un signal fort : ce qui n'était au départ qu'un problème d'infrastructure interne a fini par devenir stratégique pour toute l'entreprise.

1.2 bis — Chiffres clés 2025 : le poids réel d’AWS dans le groupe Amazon

Amazon publie chaque année ses chiffres financiers dans son rapport annuel déposé auprès du régulateur américain (formulaire 10-K). Pour l'exercice 2025, déposé en février 2026 :

- Chiffre d'affaires total du groupe Amazon : **716,9 milliards de dollars**.
- Chiffre d'affaires AWS seule : **128,7 milliards de dollars**, soit environ **18 %** du total du groupe.
- Croissance d’AWS sur un an : **+20 %**, portée notamment par la demande liée à l’intelligence artificielle.
- Le seul quatrième trimestre 2025 a vu AWS croître de **24 %** sur un an — sa meilleure progression en treize trimestres.

Ce chiffre de 18 % du chiffre d'affaires masque le poids réel d’AWS : sa rentabilité. Sur ce même exercice 2025 :

- Résultat opérationnel total du groupe Amazon : **79,9 milliards de dollars**.
- Résultat opérationnel généré par AWS seule : **45,6 milliards de dollars**.

Autrement dit : AWS produit à elle seule plus de la **moitié du profit opérationnel de tout le groupe Amazon**, alors qu'elle ne représente qu'un cinquième de son chiffre d'affaires. Ce déséquilibre explique pourquoi la presse économique décrit souvent AWS comme le véritable “moteur de profit” d’Amazon. Les activités de vente en ligne (Amazon.com) génèrent l'essentiel du volume d'affaires. Mais elles opèrent avec des marges nettement plus faibles que celles du cloud — structurellement plus rentable une fois l'infrastructure de base amortie.



Info

À retenir : AWS $\approx$ 18 % du chiffre d'affaires d’Amazon, mais plus de 55 % de son résultat opérationnel. Un stagiaire qui comprend ce déséquilibre comprend aussi pourquoi Amazon continue d’investir massivement dans de nouvelles régions et de nouveaux services cloud : c’est la partie la plus rentable du groupe.

1.3 Certifications AWS

AWS structure son parcours de certification en quatre niveaux, pensés pour accompagner une montée en compétence progressive plutôt que pour classer les candidats par mérite. Chaque niveau valide un périmètre de responsabilités différent en entreprise, et il est tout à fait normal — recommandé, même — de les passer dans l'ordre.

Le niveau **Fondamental** (Cloud Practitioner) s'adresse à toute personne qui doit comprendre le vocabulaire et les grands principes du cloud AWS sans nécessairement configurer quoi que ce soit elle-même : commerciaux, chefs de projet, décideurs, ou stagiaires qui découvrent AWS pour la première fois. L'examen porte sur les concepts (modèles de tarification, responsabilité partagée, services principaux) plutôt que sur la mise en œuvre technique.

Le niveau **Associate** cible les professionnels qui déploient et opèrent concrètement des infrastructures AWS au quotidien. Il se décline selon le métier visé : **Solutions Architect – Associate** pour la conception d'architectures, **CloudOps Engineer – Associate (SOA-C03)** pour l'exploitation et la supervision — ce nom remplace SysOps Administrator depuis 2025 — et **Developer – Associate** pour l'intégration d'applications avec les API et services AWS. Le catalogue comprend également des certifications Associate orientées données et machine learning.

Le niveau **Professional** approfondit les deux certifications Architecte et DevOps du niveau Associate avec des scénarios de complexité réelle : migrations à grande échelle, architectures multi-comptes, optimisation fine des coûts et de la résilience. Ces examens supposent une expérience pratique significative — AWS les recommande après plusieurs années d'usage professionnel du cloud.

Le niveau **Spécialité** (Specialty) valide une expertise pointue plutôt qu'une vision généraliste. Le catalogue évolue régulièrement : Database Specialty et plusieurs autres examens ont été retirés en 2024, puis Machine Learning Specialty le 31 mars 2026. Il faut donc consulter le catalogue officiel avant de présenter une liste d'examens comme actuelle. Security Specialty et Advanced Networking Specialty restent des repères pertinents pour les domaines abordés ici.

Pour cette formation, le niveau visé est le **Solutions Architect Associate (SAA-C03)** : c'est la certification la plus demandée sur le marché de l'emploi et celle qui couvre le plus largement les services abordés dans les chapitres suivants (EC2, S3, VPC, RDS, IAM, CloudFormation). Le Chapitre 5 détaillera la pondération exacte des domaines de l'examen une fois tous les services vus.



 [Certifications AWS](#)

2. Fondamentaux du Cloud Computing

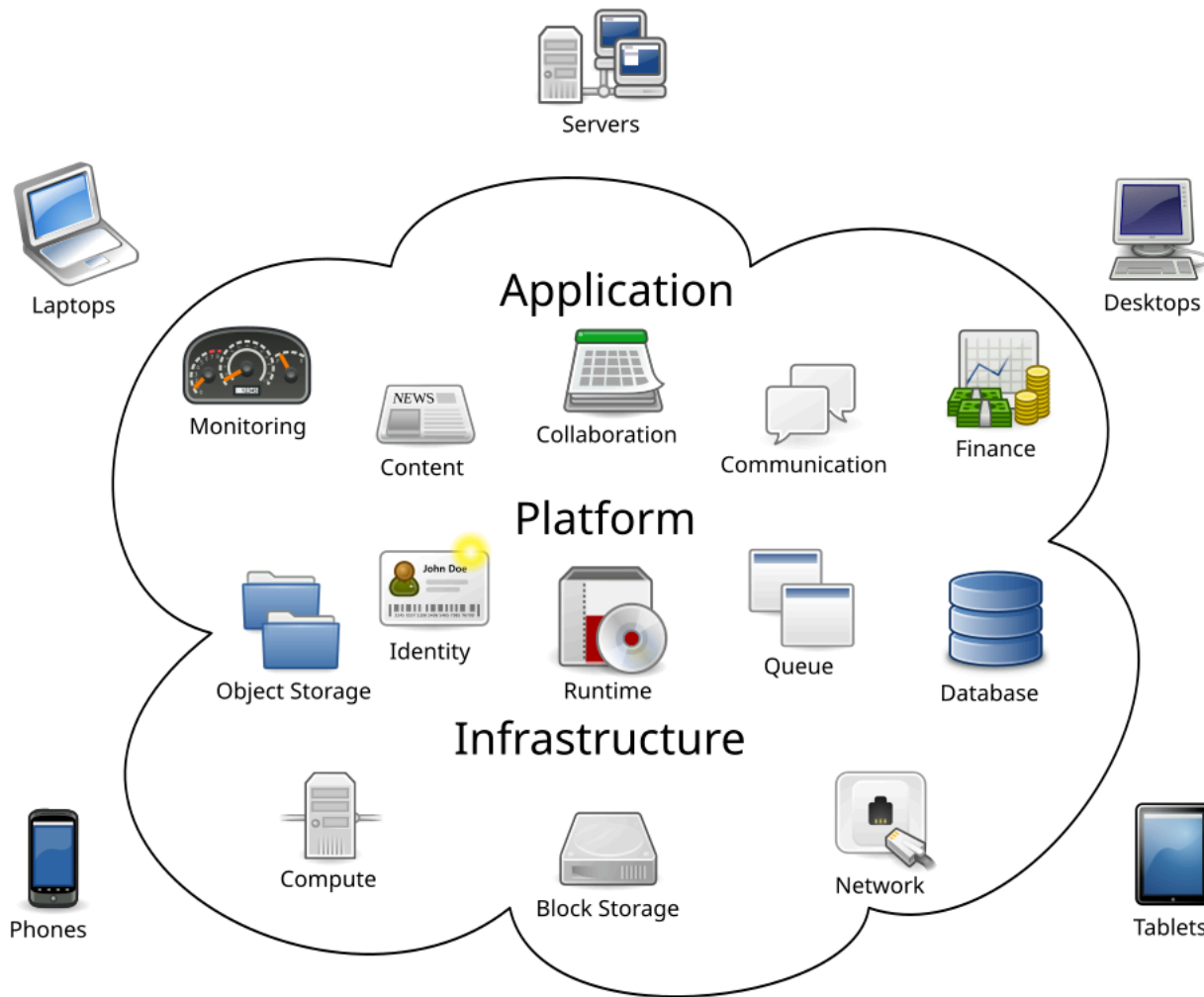
2.1 Qu'est-ce que le Cloud Computing ?

Le **Cloud Computing** est une évolution majeure dans la manière dont les systèmes d'information sont conçus, déployés et exploités.

Pendant des décennies, les entreprises ont investi massivement dans leurs propres infrastructures : datacenters internes, salles machines, serveurs physiques, systèmes de climatisation, sécurité, licences logicielles, équipes d'exploitation dédiées.

Ce modèle repose sur des investissements lourds (**CAPEX**) et des cycles de décision longs.

Le Cloud vient bouleverser cette logique en offrant un modèle **flexible, scalable** et **orienté services** : les ressources sont **louées à la demande** et **payées à l'usage**.



 [Documentation officielle AWS — What is Cloud Computing](#)

2.2 Les cinq caractéristiques fondamentales du Cloud (NIST)

Le **NIST (National Institute of Standards and Technology)** a défini cinq caractéristiques fondamentales du Cloud Computing. Ces critères permettent de distinguer une véritable solution Cloud d'une simple externalisation :

1. Libre-service à la demande (On-demand self-service)

Un utilisateur peut provisionner des ressources informatiques (par exemple, une machine virtuelle) sans intervention humaine du fournisseur.

En pratique avec AWS : Vous cliquez sur « Lancer une instance EC2 », configurez les paramètres, et en quelques secondes, votre serveur est actif et prêt à l'emploi.

2. Mise en commun des ressources (Resource pooling)

Les ressources physiques (serveurs, stockage, réseau) sont partagées entre plusieurs clients mais restent isolées logiquement.

En pratique avec AWS : Vos instances EC2 exécutent sur du matériel physique partagé avec d’autres clients, mais AWS garantit une isolation complète — vos données restent vos données.

3. Élasticité et scalabilité (Rapid elasticity)

Les ressources peuvent être augmentées ou réduites dynamiquement en fonction de la demande.

En pratique avec AWS : Votre application reçoit soudain 10 fois plus de visiteurs. AWS augmente automatiquement la capacité de calcul, puis la réduit quand le trafic baisse.

4. Mesurabilité et facturation à l’usage (Measured service)

Chaque ressource est mesurée et facturée selon la consommation réelle : CPU, mémoire, bande passante, stockage, etc.

En pratique avec AWS : Vous payez par heure pour une instance EC2, par Go stocké sur S3, par requête sur Lambda. Pas de frais fixes, pas de ressources inutilisées payées « pour rien ».

5. Accès étendu au réseau (Broad network access)

Les utilisateurs gèrent leurs ressources via une console Web, des API ou des outils en ligne de commande (CLI).

En pratique avec AWS : Vous ne contactez pas AWS pour dire « besoin d’un serveur ». Vous vous connectez à la console, créez ce que vous voulez, et c’est instantané.

 [NIST — The NIST Definition of Cloud Computing](#)

2.3 CAPEX vs OPEX — Comprendre le changement de modèle économique

Historiquement, les entreprises ont fonctionné sur un modèle **CAPEX** (*Capital Expenditures*) : elles achetaient leurs équipements, les installaient dans leurs propres locaux et les exploitaient pendant plusieurs années.

Avec le Cloud, elles passent à un modèle **OPEX** (*Operational Expenditures*) : elles louent des ressources à la demande et paient uniquement pour leur consommation réelle.

Modèle	Définition	Exemple typique	Gestion
CAPEX	Investissement initial important amorti sur plusieurs années	Achat de serveurs physiques	Budget fixe
OPEX	Dépenses variables basées sur l’usage	Paiement horaire d’une instance EC2	Budget flexible

Exemple concret :

CAPEX (Avant le Cloud) : Une entreprise achète 10 serveurs physiques pour héberger une application interne. Coût initial : 50 000 €. Ils fonctionnent en permanence, même en période creuse, avec des coûts d'entretien et de maintenance élevés (climatisation, électricité, renouvellement matériel tous les 5-7 ans).

OPEX (Avec AWS) : La même entreprise utilise **Amazon EC2 (Elastic Compute Cloud)**, un service AWS permettant de lancer et gérer des **machines virtuelles**. Elle ne paie que les heures réellement consommées (ex. 100 € par mois en moyenne) et peut arrêter les instances lorsqu'elles ne sont pas nécessaires.

La différence ? Flexibilité, prévisibilité des coûts, et liberté d'adapter l'infrastructure en temps réel.

Warning

Attention aux coûts AWS — le modèle OPEX peut surprendre : Une instance EC2 laissée tournante 24h/7j sans utilisation reste facturée. Contrairement à un investissement CAPEX amorti sur plusieurs années, les coûts OPEX s'accumulent en temps réel. Activez toujours des **alertes de budget (AWS Budgets)** dès le premier jour.

 [AWS Economics Center](#)

2.4 La rupture culturelle du libre-service

Dans les environnements informatiques traditionnels, **les équipes métiers et techniques** devaient **passer par des processus centralisés** pour obtenir des ressources :

- Demande de serveurs à l'équipe IT
- Validation budgétaire et technique
- Commande physique, installation, configuration... 👉 Résultat : **des délais de plusieurs semaines** avant de pouvoir simplement démarrer un projet.

Avec **le Cloud AWS**, ce modèle est **complètement bouleversé**.

Cette rupture se traduit concrètement par quatre changements qui s'enchaînent : l'accès direct aux ressources, un nouveau cadre de gouvernance pour l'encadrer, une accélération du rythme d'innovation, et finalement un changement culturel plus profond que la seule technologie.

1. Libre-service immédiat

Le premier changement, et le plus visible, concerne la façon même dont une ressource est obtenue :

- Les équipes peuvent **provisionner elles-mêmes** des ressources (VM, bases de données, stockage...) **en quelques clics ou via des API**.

- Plus besoin d'attendre une autre équipe pour « débloquer » les moyens techniques.
- Cela change profondément la manière de travailler : les développeurs deviennent **autonomes**.

Cette autonomie technique ne va cependant pas sans contrepartie : elle doit s'accompagner d'un cadre qui évite les dérives, d'où le second changement.

2. Responsabilisation et gouvernance

Donner l'accès direct aux ressources ne signifie pas supprimer tout contrôle — cela déplace simplement le contrôle vers d'autres mécanismes :

- Le libre-service ne signifie pas « anarchie » : cela s'accompagne de **règles, quotas, politiques IAM, budgets**, etc.
- Les organisations doivent mettre en place une **gouvernance cloud** qui encadre cette autonomie.

Une fois ce cadre posé, l'autonomie retrouvée par les équipes se traduit directement par une capacité à innover plus vite.

3. Accélération de l'innovation

Ce gain de vitesse se manifeste concrètement dans la façon de tester et de valider des idées :

- Cette autonomie permet de tester une idée en quelques heures plutôt qu'en plusieurs semaines.
- Les équipes peuvent **échouer vite et pas cher**, puis pivoter ou étendre si le projet fonctionne.

Au-delà de la vitesse d'exécution, c'est surtout la façon de travailler des équipes qui doit évoluer en profondeur.

4. Une vraie rupture culturelle

Ce dernier point est le plus difficile à mettre en œuvre, car il ne se décrète pas au niveau d'un outil mais d'une organisation entière :

- Le passage au cloud AWS **n'est pas qu'une question de technologie**.
- Il faut **changer les mentalités** :
 - Déléguer le pouvoir de créer des ressources.
 - Accepter la rapidité et parfois le désordre initial.
 - Former et responsabiliser les équipes pour qu'elles deviennent **consommatrices actives** du cloud.

Ces quatre points résument pourquoi l'adoption du Cloud échoue souvent moins pour des raisons techniques que pour des raisons humaines et organisationnelles.

« Avant, il fallait une demande, une validation, une livraison. Aujourd'hui, on clique — ou on lance une commande API — et on a une infrastructure prête en quelques secondes. Cette autonomie est la clé du Cloud, mais elle exige une nouvelle culture : celle de la confiance, de la responsabilisation et de la gouvernance bien pensée. »

2.5 Les limites et contraintes du Cloud

Aucun modèle n'est parfait. Le Cloud introduit aussi plusieurs défis importants :

- **Dépendance au fournisseur (vendor lock-in)** : migrer entre AWS, Azure ou GCP peut nécessiter une réarchitecture importante.
- **Connexion Internet critique** : sans réseau, pas d'accès aux ressources hébergées dans le cloud.
- **Sécurité partagée** : certaines responsabilités ne sont **pas déléguées** au fournisseur.
- **Maîtrise des coûts** : une mauvaise configuration peut générer des dépenses importantes et rapides.

Nous allons détailler **la contrainte liée à la sécurité partagée**, car elle est souvent **mal comprise** par les entreprises débutant dans le cloud.

2.6 Le modèle de responsabilité partagée AWS

Avant d'étudier le tableau, utilisez le [simulateur interactif de responsabilité partagée](#). Comparez EC2, RDS et Lambda : plus le service est managé, plus AWS prend en charge de couches techniques, sans jamais devenir responsable de la classification des données ni des autorisations décidées par le client.

Le modèle de responsabilité partagée AWS répartit la sécurité entre deux parties, et confondre les deux moitiés est l'une des erreurs les plus fréquentes chez les entreprises qui découvrent le cloud.

AWS porte la responsabilité de la **sécurité du cloud**, c'est-à-dire de tout ce qui constitue l'infrastructure sous-jacente sur laquelle les clients construisent leurs services. Cela couvre la sécurité physique des data centers — accès contrôlé, vidéosurveillance, alimentation redondante — que le client n'a jamais à gérer lui-même. Cela couvre aussi le matériel et la couche de virtualisation qui isole chaque client des autres sur un même serveur physique, ainsi que la disponibilité globale de la plateforme, avec ses multiples régions et zones de disponibilité conçues pour survivre à la panne d'un data center entier.

Le **client**, de son côté, porte la responsabilité de la **sécurité dans le cloud** : tout ce qu'il configure et déploie au-dessus de cette infrastructure reste de son ressort. Il doit gérer les accès et les identités via IAM — qui a le droit de faire quoi sur son compte — et sécuriser ses données par le chiffrement et la rotation régulière des clés. Il doit configurer correctement son réseau (VPC, Security Groups, NACL) pour éviter d'exposer par erreur des ressources sensibles sur Internet. Et lorsqu'il utilise un service de type IaaS comme EC2, où AWS fournit la machine virtuelle mais pas ce qui tourne dessus, il reste responsable des sauvegardes, des correctifs de sécurité et des mises à jour du système d'exploitation.

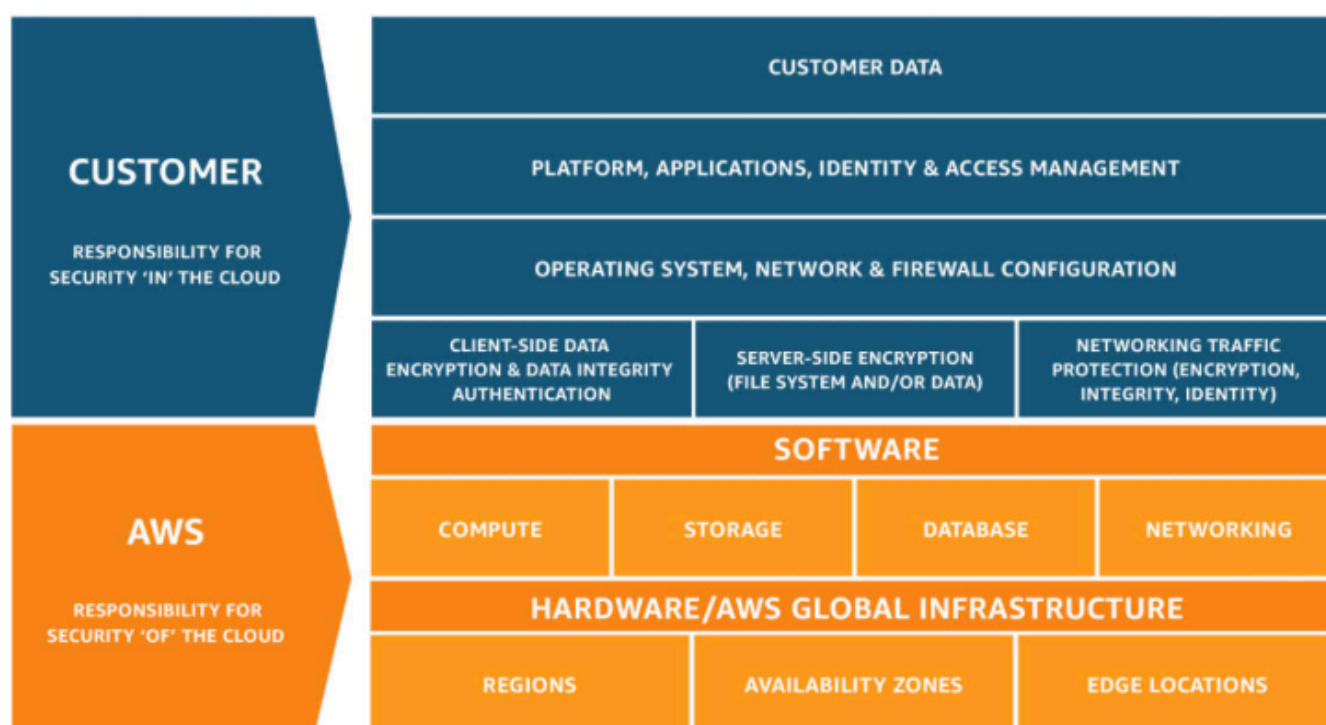
Cela signifie que même si AWS est hautement sécurisé, **une mauvaise configuration côté client peut compromettre la sécurité** (par ex. un bucket S3 public par erreur).

⚡ Danger

Responsabilité client — erreurs fréquentes en production :

- Un bucket S3 configuré en **accès public par erreur** expose toutes vos données sur Internet.
- L'utilisation du **compte root** pour les opérations quotidiennes est une faille de sécurité majeure.
- Un **Security Group ouvert sur 0.0.0.0/0 port 22** expose vos instances SSH au monde entier.
- L'absence de **MFA** sur le compte root est la première cause de compromission de compte AWS.

AWS ne peut pas vous protéger de vos propres erreurs de configuration — c'est votre responsabilité.



👤 Shared Responsibility Model – AWS [↗](#)

2.7 Sécurité de l'information dans le Cloud

La **sécurité de l'information** repose sur trois piliers fondamentaux (**CID**) :

Pilier	Définition	Exemple AWS
Confidentialité	Seules les personnes autorisées doivent accéder aux données.	IAM, KMS, Bucket Policies
Intégrité	Les données ne doivent pas être altérées ou corrompues.	Checksums, versioning S3
Disponibilité	Les systèmes doivent être accessibles à tout moment.	Multi-AZ, Load Balancer, snapshots

Ce tableau cite trois exemples AWS dont vous ne verrez le détail que plus tard dans la formation — c’est normal à ce stade : **IAM** (gestion des identités et des accès) est le sujet entier du Chapitre 2, **KMS** (Key Management Service, le service de gestion des clés de chiffrement) et les **Bucket Policies** (règles d’accès attachées à un bucket S3) seront présentés respectivement en section 6 de ce chapitre et au Chapitre 4. Retenez pour l’instant seulement le principe : chaque pilier CID s’appuie sur des outils AWS concrets que vous manipulerez au fil de la formation.

AWS fournit des outils et des certifications (ISO 27001, SOC 2, PCI-DSS...), mais **l’entreprise reste responsable de son usage**.

Exemples concrets de responsabilités du client :

- Un bucket S3 mal configuré (public par erreur) = **fuite de données** → responsabilité **du client**.
- Un mot de passe faible ou MFA non activée = **intrusion possible** → responsabilité **du client**.
- Absence de sauvegarde / réplication = **perte de disponibilité** → responsabilité **du client**.
- Firewall mal ouvert (0.0.0.0/0 sur SSH) = exposition à Internet → responsabilité **du client**.

C’est pourquoi **les bonnes pratiques de configuration** sont **aussi importantes que les services AWS eux-mêmes**.

En résumé :

Sécurité du cloud (AWS)	Sécurité dans le cloud (Client)
Data centers, matériel	Gestion des accès et des rôles (IAM)
Virtualisation et réseau global	Sécurisation des données et des flux
Résilience physique	Mise à jour, chiffrement, surveillance
Conformité et certifications	Respect des politiques internes et réglementations sectorielles

Cette répartition explique la plupart des incidents de sécurité observés dans le Cloud : ils viennent presque toujours de la colonne de droite (une mauvaise configuration côté client), rarement d'une faille dans l'infrastructure gérée par AWS.

2.8 Externalisation du Système d'Information (SI)

Pourquoi externaliser ?

Externaliser le SI vers le Cloud permet de :

- Réduire les coûts d'infrastructure et de maintenance.
- Accéder à une expertise et à des technologies de pointe.
- Améliorer la disponibilité et la continuité d'activité.

Mais cela implique aussi une **réévaluation des risques** :

- Dépendance contractuelle envers le fournisseur.
- Nécessité de revoir les politiques de sauvegarde et d'accès.
- Conformité légale (RGPD, hébergement des données, localisation géographique).

Conseil : Avant toute migration, réalisez une **analyse d'impact** (juridique, technique et organisationnelle). AWS met à disposition un outil appelé **AWS Migration Readiness Assessment (MRA)**.

 [AWS Migration Hub](#)

3. Modèles de services : IaaS, PaaS et SaaS

Les modèles de services sont essentiels pour comprendre **comment les entreprises consomment le Cloud**. Ils structurent les **couches techniques** (infrastructure, plateforme, application) et déterminent qui — du client ou du fournisseur — est responsable de chaque couche.

 [AWS Cloud Service Models](#)

Ces modèles sont une clé de lecture indispensable pour architecturer correctement une solution Cloud.

3.1 Infrastructure as a Service (IaaS)

Infrastructure as a Service (IaaS) désigne un modèle dans lequel le fournisseur met à disposition des ressources informatiques de base : **puissance de calcul, stockage, réseau et sécurité**. Le client gère et configure ces ressources

selon ses besoins.

Caractéristiques clés :

En IaaS, le fournisseur s'arrête à la couche matérielle et réseau : tout ce qui se trouve au-dessus reste sous la responsabilité du client.

- Accès à des **machines virtuelles** configurables.
- Gestion des disques, du réseau et de la sécurité.
- Contrôle total sur l'OS et les applications.
- Évolutivité à la demande.

Ce niveau de contrôle est aussi ce qui distingue l'IaaS des autres modèles : le client conserve la responsabilité du système d'exploitation, des correctifs de sécurité et de la configuration réseau, en échange d'une liberté totale sur l'architecture.

Exemples AWS :

Ces services illustrent concrètement chacune des caractéristiques ci-dessus :

- **Amazon EC2 (Elastic Compute Cloud)** : service qui permet de créer et gérer des machines virtuelles dans le Cloud.
- **Amazon EBS (Elastic Block Store)** : stockage en mode bloc attaché aux instances EC2.
- **Amazon VPC (Virtual Private Cloud)** : réseau virtuel isolé et configurable.
- **Elastic Load Balancing (ELB)** : répartition automatique du trafic entre plusieurs ressources.

Analogie : L'IaaS est comparable à la location d'un terrain pour construire une maison — le client choisit tout, mais doit aussi tout gérer.

3.2 Platform as a Service (PaaS)

Platform as a Service (PaaS) décharge le client de la gestion de l'infrastructure et du système d'exploitation. Le fournisseur gère les couches basses, le client se concentre sur le développement et la mise en production.

Caractéristiques clés :

Le PaaS déplace la frontière de responsabilité plus haut que l'IaaS : le fournisseur prend en charge tout ce qui touche à l'exploitation de l'environnement d'exécution.

- Pas de gestion des serveurs ni de patching système.
- Environnements prêts à l'emploi pour le déploiement applicatif.
- Scalabilité intégrée.

Le client garde ainsi la main sur le code et la logique métier, mais délègue tout ce qui concerne le maintien en condition opérationnelle de l'infrastructure sous-jacente.

Exemples AWS :

Ces trois services couvrent les usages PaaS les plus courants chez AWS, du déploiement web classique au calcul événementiel :

- **AWS Elastic Beanstalk** : service de déploiement et d'orchestration d'applications Web.
- **AWS Lambda** : service serverless qui exécute du code sans gérer de serveur.
- **Amazon RDS (Relational Database Service)** : base de données relationnelle gérée (patches, sauvegardes, monitoring automatisés).

 [Documentation AWS Elastic Beanstalk](#)

Le PaaS permet de **réduire la charge opérationnelle** et d'accélérer les cycles de développement.

3.3 Software as a Service (SaaS)

Software as a Service (SaaS) est un modèle dans lequel le fournisseur gère toute la pile technique — infrastructure, plateforme et application — et livre un service clé en main.

Caractéristiques clés :

Avec le SaaS, la frontière de responsabilité monte encore d'un cran : le client ne consomme plus qu'une fonctionnalité, sans jamais interagir avec la couche technique qui la fait fonctionner.

- Aucune gestion technique côté client.
- Accès via un navigateur ou une API.
- Maintenance et sécurité assurées par le fournisseur.

Le revers de cette simplicité est une perte de contrôle : le client ne peut ni personnaliser l'infrastructure ni choisir la version du logiciel qu'il utilise, il dépend entièrement de la feuille de route du fournisseur.

Exemples AWS :

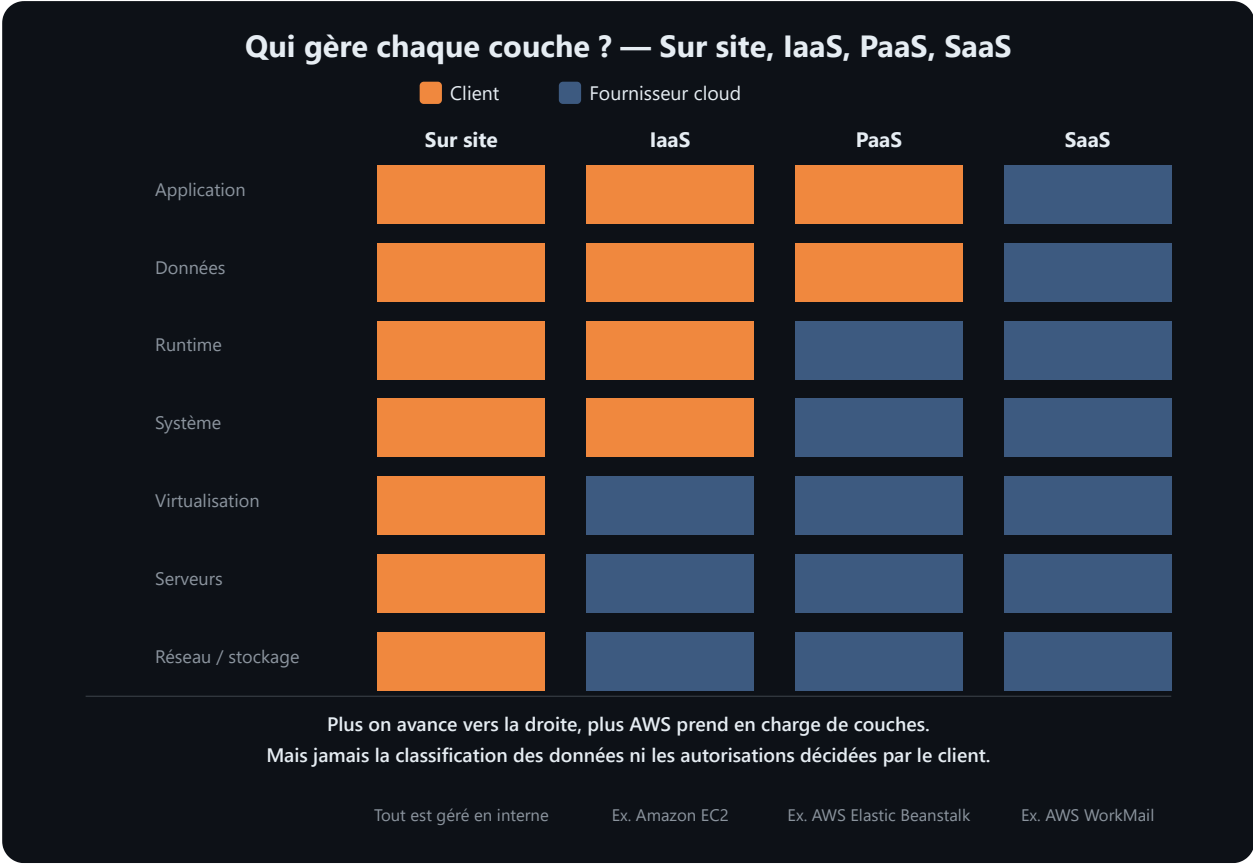
Voici quelques services SaaS courants, aussi bien chez AWS que chez d'autres éditeurs :

- **AWS WorkMail** : service de messagerie hébergée.
- **Salesforce** : CRM en ligne.
- **Microsoft 365, Google Workspace** : solutions collaboratives.

 [AWS SaaS Factory](#)

Retenons les cas d’usage typiques du SaaS — messagerie, CRM, outils de collaboration, ERP.

3.4 Comparatif synthétique des modèles



Ce schéma reprend, couche par couche, la bascule progressive de responsabilité déjà observée plus haut dans la définition de chaque modèle : sur site, tout reste à votre charge ; en IaaS, AWS ne prend en charge que la virtualisation, les serveurs et le réseau physique ; en PaaS, AWS gère aussi le runtime et le système d’exploitation ; en SaaS, seule la donnée que vous y déposez reste réellement de votre ressort.

Élément	IaaS	PaaS	SaaS
Flexibilité	Très élevée	Moyenne	Faible
Temps de déploiement	Plus long	Rapide	Instantané
Cas d’usage typique	Migration, environnements complexes	Déploiements rapides, automatisation	Solutions métiers clés en main

Ce tableau confirme la logique observée plus haut : plus le fournisseur prend en charge de couches, plus le déploiement est rapide, mais moins le client garde de marge de manœuvre — un compromis à évaluer projet par projet

plutôt qu'un modèle universellement supérieur aux autres.

4. Modèles de déploiement : Public, Privé et Hybride

Les **modèles de déploiement** définissent *où* sont hébergées les ressources Cloud et *qui* les gère. Ce choix a un impact direct sur la **gouvernance**, la **sécurité**, la **performance** et le **coût global** d'une solution Cloud.

Il existe trois grands modèles de déploiement reconnus par le NIST et adoptés par la majorité des entreprises : **Cloud Public**, **Cloud Privé** et **Cloud Hybride**.

Le modèle de déploiement choisi dépend souvent d'un compromis entre agilité, sécurité, conformité et maîtrise de l'environnement technique.

4.1 Le Cloud Public

Le **Cloud Public** est un modèle dans lequel les ressources sont hébergées dans les **datacenters du fournisseur Cloud** et partagées entre plusieurs clients. Les infrastructures sont **mutualisées**, mais chaque client bénéficie d'un environnement isolé logiquement.

Caractéristiques :

Ce modèle repose sur quelques propriétés techniques qui expliquent à la fois ses forces et ses limites.

- Hébergement sur l'infrastructure du fournisseur.
- Mutualisation des ressources physiques.
- Facturation à l'usage.
- Évolutivité quasi illimitée.
- Accès global via Internet sécurisé.

C'est précisément cette mutualisation des ressources physiques entre plusieurs clients qui permet au fournisseur de proposer une facturation à l'usage et une scalabilité que peu d'entreprises pourraient atteindre seules avec leur propre matériel.

Avantages :

Ces caractéristiques techniques se traduisent concrètement par les bénéfices suivants pour l'entreprise cliente :

- Pas de gestion matérielle.

- Déploiement rapide.
- Coût réduit grâce à la mutualisation.
- Accès à un large éventail de services.

Ces avantages ont toutefois une contrepartie directe, qui explique pourquoi certaines entreprises hésitent encore à tout migrer en Cloud public.

Inconvénients :

Cette contrepartie se résume à trois limites principales :

- Moins de contrôle sur l'infrastructure physique.
- Dépendance au fournisseur.
- Besoins accrus en gouvernance et sécurité.

Ces inconvénients ne remettent pas en cause le modèle en lui-même, mais imposent de mettre en place des garde-fous (contrats, politiques IAM, plans de sortie) dès l'adoption.

Exemple AWS :

AWS Public Cloud repose sur l'infrastructure mondiale d'**Amazon Web Services (AWS)**, composée de **régions** (Regions), **zones de disponibilité** (Availability Zones, AZ) et **points de présence** (Edge Locations).

- Une *région* est une zone géographique (par exemple *eu-west-3* pour Paris).
- Une *zone de disponibilité* (AZ) est un datacenter isolé dans cette région, assurant redondance et tolérance aux pannes.
- Les *points de présence* servent principalement aux services de diffusion de contenu comme Amazon CloudFront.

Services AWS typiques dans le Cloud public :

- **Amazon EC2** : instances de calcul à la demande
- **Amazon S3** : stockage objet scalable
- **AWS Lambda** : exécution de code sans serveur
- **Amazon RDS** : bases de données relationnelles gérées
- **Amazon CloudFront** : CDN mondial pour accélérer les contenus

Ces services couvrent l'essentiel des besoins d'une application moderne (calcul, stockage, base de données, distribution de contenu) sans qu'aucune infrastructure physique ne soit à gérer par l'entreprise.

 [AWS Global Infrastructure](#) 

4.2 Le Cloud Privé

Le **Cloud Privé** est un modèle dans lequel l'entreprise déploie ses propres ressources Cloud sur une **infrastructure dédiée**, souvent hébergée dans ses propres locaux ou dans un datacenter tiers, mais **non mutualisée** avec d'autres clients.

Caractéristiques :

Le fait de ne pas mutualiser l'infrastructure avec d'autres clients a des répercussions directes sur ce que l'entreprise gagne — et perd — par rapport au Cloud public.

- Infrastructure dédiée à une seule organisation.
- Contrôle complet sur l'environnement.
- Peut utiliser des technologies de virtualisation similaires à celles du Cloud public.
- Peut être automatisé et orchestré comme un Cloud public.

Ces caractéristiques rapprochent le Cloud privé d'une infrastructure on-premise classique, à ceci près que les outils et pratiques du Cloud public (automatisation, orchestration) peuvent aussi bien s'y appliquer.

Avantages :

Ne pas partager l'infrastructure avec d'autres clients se traduit directement par trois bénéfices :

- Meilleure maîtrise de la sécurité et de la conformité.
- Contrôle total sur les configurations.
- Plus de personnalisation possible.

Ce niveau de maîtrise a cependant un coût, qui se traduit par les inconvénients suivants.

Inconvénients :

Ce coût se manifeste principalement sur trois plans, financier, organisationnel et humain :

- Coût d'investissement plus élevé (proche d'un modèle CAPEX).
- Moins de flexibilité et de scalabilité.
- Nécessite des équipes internes pour la gestion.

C'est ce compromis entre contrôle et coût qui pousse encore aujourd'hui certaines organisations à conserver un Cloud privé plutôt que de tout basculer en Cloud public, notamment pour des raisons réglementaires.

Pourquoi le Cloud Privé existe-t-il encore ?

Avec la transformation digitale — certaines entreprises conservent des charges sensibles en interne pour des raisons réglementaires.

Exemple : le RGPD (Règlement Général sur la Protection des Données)

Le **RGPD** impose aux entreprises européennes une **maîtrise stricte** de la localisation, de la protection et de la gouvernance des données personnelles. De ce fait, certaines entreprises — notamment dans les secteurs **banque, santé** ou **secteur public** — choisissent de **conserver certaines données ou systèmes sensibles en interne** ou sur des infrastructures “souveraines” pour :

- Garantir que les données ne quittent pas l’Espace Économique Européen,
- Mieux contrôler les accès et traitements,
- Répondre aux obligations de **traçabilité** et de **confidentialité**.

Exemples concrets :

- **Système bancaire français** hébergeant les données de ses clients sur une infrastructure locale ou un cloud “de confiance” pour respecter :
 - L’article 44 du RGPD (transferts hors UE),
 - Les obligations de minimisation et de limitation de traitement (articles 5 et 6),
 - Les recommandations de la CNIL sur l’hébergement de données sensibles.
- **Code de la santé publique** → données de santé hébergées uniquement chez un **Hébergeur de Données de Santé (HDS)** agréé en France. Les hôpitaux et laboratoires doivent conserver certaines données sur site ou dans un cloud certifié HDS.
- **Secteur de la Défense** → obligations de **souveraineté et de sécurité** imposées par l’ANSSI (ex. SecNumCloud), qui limitent l’externalisation à des fournisseurs labellisés.

En résumé :

Législation / Réglementation	Secteur impacté	Conséquence sur l'hébergement
RGPD	Tous secteurs manipulant des données personnelles	Contrôle de localisation, transferts restreints
HDS	Santé	Hébergement uniquement chez des prestataires certifiés
ANSSI / SecNumCloud	Secteurs sensibles / OIV	Cloud souverain ou on-prem obligatoire pour certaines charges

Ce type de réglementation **explique pourquoi certaines entreprises gardent encore des charges sensibles “on-premise”** ou sur des **infrastructures dédiées** plutôt que dans le cloud public pur.

Warning

RGPD et localisation des données AWS : Par défaut, AWS peut stocker vos données dans n’importe quelle zone de disponibilité de la région choisie. Pour les données personnelles de citoyens européens, vous devez impérativement choisir une **région européenne** (ex. `eu-west-3` Paris, `eu-central-1` Francfort) et vérifier que les services utilisés ne transfèrent pas les données hors UE sans votre accord explicite.

AWS s’est adapté :

Face à ces contraintes réglementaires, AWS propose des services qui rapprochent l’expérience du Cloud public d’une infrastructure restant on-premise ou dédiée.

- Un groupe bancaire peut déployer un **Cloud privé** sur sa propre infrastructure virtualisée avec VMware ou OpenStack, tout en automatisant les déploiements comme dans AWS.

Cette approche permet de retrouver les mêmes outils d’automatisation et d’orchestration que sur AWS, tout en gardant la donnée dans un périmètre maîtrisé par l’entreprise.

 [VMware Cloud Foundation](#)  [OpenStack](#)

Services AWS typiques pour le Cloud privé :

- **AWS Outposts** : infrastructure AWS déployée dans le datacenter du client
- **VMware Cloud on AWS** : environnement VMware géré dans AWS
- **Amazon VPC** : réseau virtuel isolé
- **AWS Direct Connect** : liaison réseau privée entre le client et AWS



4.3 Le Cloud Hybride

Le **Cloud Hybride** combine le **Cloud Privé** et le **Cloud Public**, en permettant aux applications et données de circuler entre les deux environnements. C'est aujourd'hui le modèle **le plus répandu** dans les grandes organisations.

Caractéristiques :

Ce modèle hérite à la fois des avantages du Cloud public et de ceux du Cloud privé, mais aussi d'une complexité supplémentaire liée à la coexistence des deux environnements.

- Combinaison de ressources internes et publiques.
- Interopérabilité entre les environnements.

- Optimisation des coûts et de la sécurité.
- Support de scénarios de migration progressive.

Ces caractéristiques expliquent pourquoi le Cloud hybride est souvent choisi comme étape intermédiaire lors d'une migration progressive vers le Cloud, plutôt que comme destination finale.

Avantages :

Cette combinaison des deux mondes se traduit par quatre bénéfices concrets :

- Flexibilité maximale.
- Répartition intelligente des charges de travail.
- Sécurité renforcée pour les données sensibles.
- Possibilité d'exploiter le meilleur des deux mondes.

Cette flexibilité a néanmoins un prix opérationnel, détaillé dans les inconvénients ci-dessous.

Inconvénients :

Faire coexister deux environnements distincts a un coût, principalement organisationnel :

- Gestion plus complexe.
- Nécessite des compétences solides en architecture et interconnexion.
- Risque de dépendances multiples.

Cette complexité de gestion justifie l'existence de services AWS dédiés à l'interconnexion, qui simplifient la mise en œuvre d'une architecture hybride.

Exemple AWS :

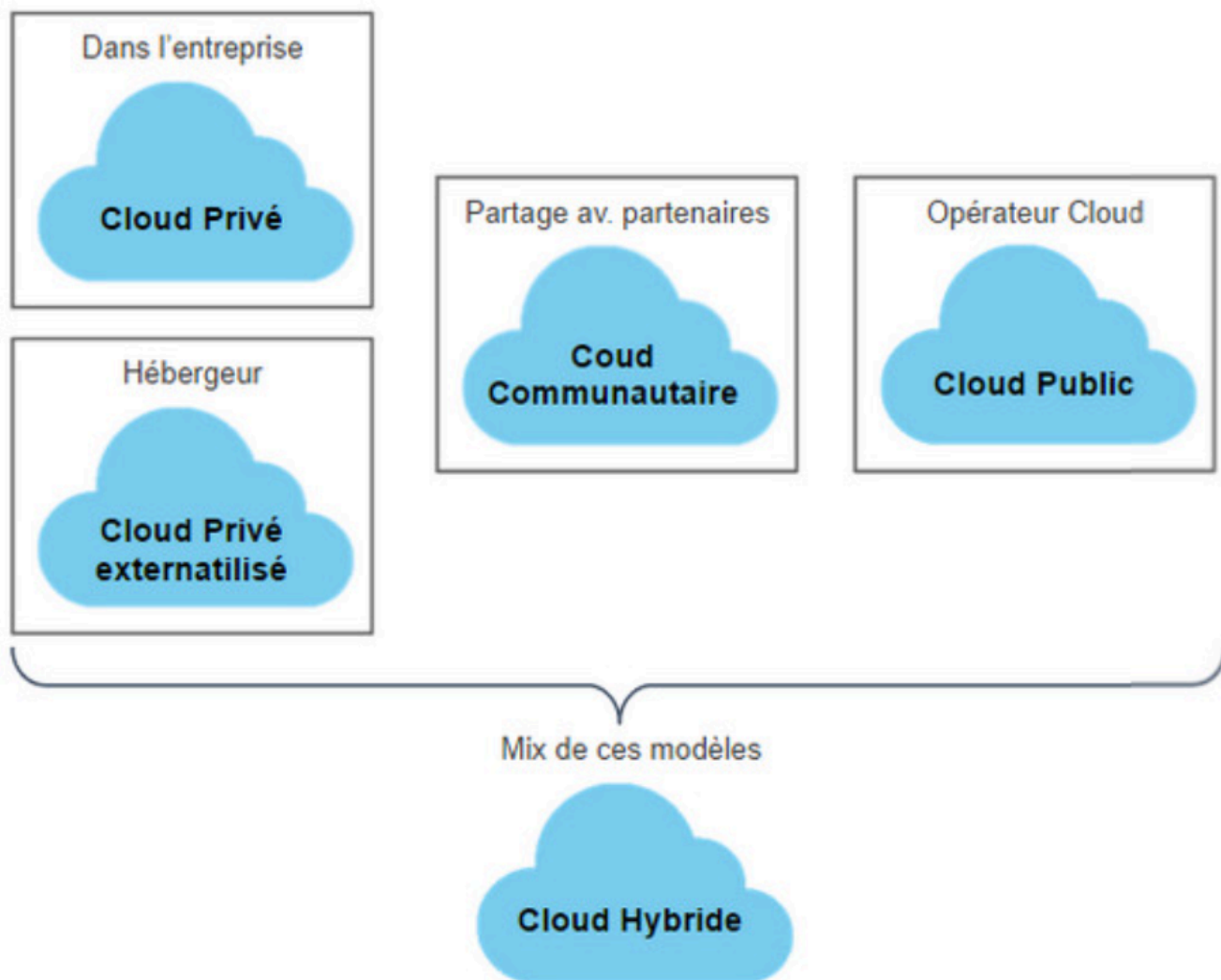
AWS Direct Connect est un service AWS qui permet de **créer une liaison réseau privée dédiée** entre l'infrastructure interne d'une entreprise et AWS. Cela permet de combiner la sécurité d'un Cloud privé avec la puissance d'un Cloud public.

Services AWS typiques pour le Cloud hybride :

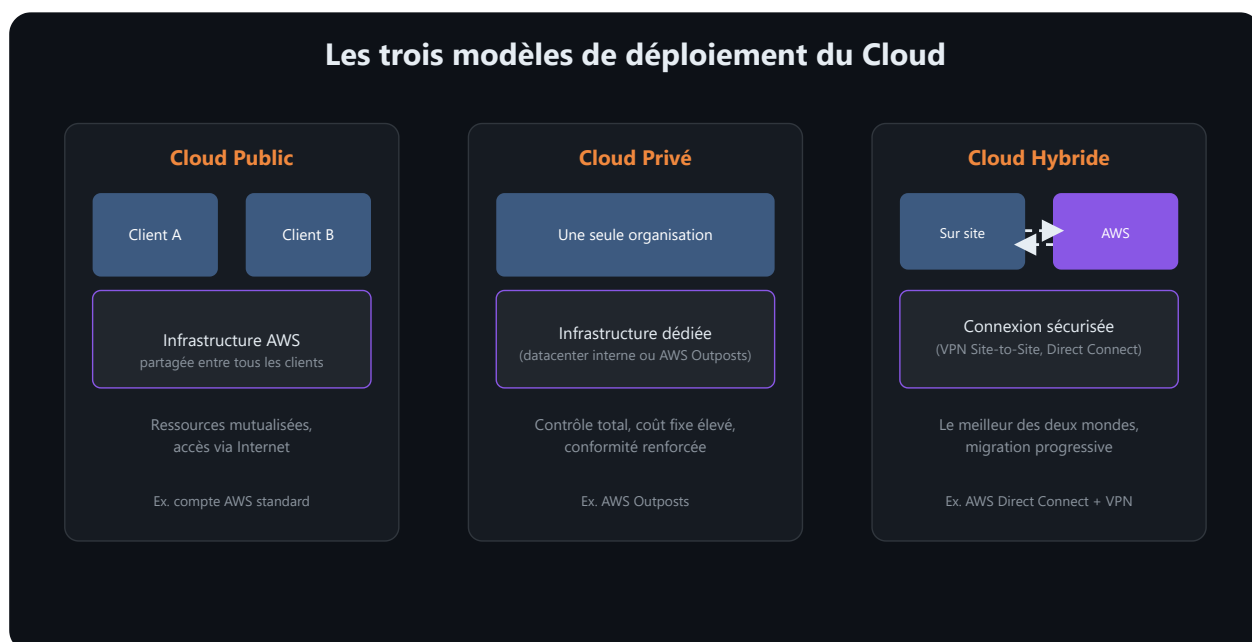
- **AWS Storage Gateway** : intégration du stockage local avec S3
- **AWS Transit Gateway** : interconnexion de VPC et réseaux sur site
- **AWS Systems Manager** : gestion centralisée des ressources hybrides
- **AWS Backup** : sauvegarde unifiée pour ressources cloud et sur site

 [AWS Direct Connect](#)

Le Cloud hybride est souvent une **étape transitoire** vers une adoption plus large du Cloud public.



4.4 Comparatif synthétique



Ce schéma résume visuellement ce qui distingue les trois modèles vus dans cette section : qui possède l’infrastructure, si elle est partagée ou dédiée, et comment les deux mondes se connectent dans le cas hybride. Le tableau ci-dessous détaille ces différences critère par critère.

Élément	Public Cloud	Private Cloud	Hybrid Cloud
Propriété de l’infrastructure	Fournisseur (ex. AWS)	Entreprise	Mixte
Isolation	Mutualisée, logique	Dédiée, physique	Mixte
Coûts	OPEX, à la demande	CAPEX, investissement initial	Mixte
Sécurité	Standardisée, contrôlée par fournisseur	Complète, contrôlée par l’entreprise	Partagée
Scalabilité	Très élevée	Limitée	Élevée
Gouvernance	Fournisseur	Entreprise	Mixte

Ce tableau montre que le Cloud hybride n’est pas un compromis médiocre entre les deux autres modèles, mais une combinaison qui hérite spécifiquement de la scalabilité du public sur les charges qui le permettent, tout en conservant le contrôle du privé sur celles qui l’exigent.

4.5 Recommandations par contexte

Le tableau ci-dessous traduit ces critères techniques en recommandation concrète selon le profil et les contraintes de l’organisation.

Modèle	Recommandé pour...	Explication simplifiée
Cloud public	Startups, PME, environnements Dev/Test, charges variables	AWS gère toute l'infrastructure. Vous utilisez les services à la demande, sans rien installer ni maintenir. Idéal pour démarrer vite, tester, ou adapter les ressources selon le trafic.
Cloud privé	Organisations réglementées, données sensibles, applications critiques	L'environnement est dédié à une seule entreprise. Plus de contrôle, plus de sécurité, mais aussi plus de configuration. Utilisé quand les données ne doivent pas sortir d'un périmètre défini (ex : santé, finance).
Cloud hybride	Entreprises en transition, contraintes réglementaires, intégration avec systèmes existants	Combine les deux : une partie des ressources reste sur site ou en cloud privé, l'autre est dans le cloud public. Permet de migrer progressivement, tout en respectant les contraintes métier ou techniques.

AWS permet aux entreprises de choisir le **modèle le plus adapté à leur contexte** :

- Le **cloud public** est parfait pour démarrer rapidement, tester des idées, ou gérer des pics de trafic.
- Le **cloud privé** offre un **contrôle maximal** pour les secteurs sensibles ou réglementés.
- Le **cloud hybride** est une **solution intermédiaire** qui permet de **connecter l'ancien et le nouveau**, en douceur.

Cette flexibilité est l'un des grands atouts d'AWS : vous pouvez commencer petit, évoluer progressivement, et adapter votre architecture à vos besoins métier, techniques et réglementaires.

5. Fondamentaux techniques : Virtualisation et Conteneurs

Derrière le Cloud se cachent des **technologies fondamentales** qui le rendent possible. Deux piliers en particulier ont permis l'essor massif des infrastructures à la demande :

- La **virtualisation**, qui permet de découper une machine physique en plusieurs machines virtuelles indépendantes.

- La **conteneurisation**, qui permet d'isoler et d'exécuter des applications de manière légère et flexible.

Comprendre ces concepts est essentiel pour appréhender le fonctionnement d'**Amazon Web Services (AWS)** et de nombreux autres fournisseurs Cloud.

 [AWS Compute Documentation](#)  [Introduction to Virtualization — VMware](#)  [Docker Documentation](#) 

Insistons ici sur la **différence entre virtualisation et conteneurisation**, car cette distinction conditionne la compréhension des services comme Amazon EC2, ECS ou EKS.

5.1 Virtualisation — Principe et rôle dans le Cloud

Définition

La **virtualisation** est une technologie qui permet d'exécuter plusieurs **machines virtuelles (VM)** sur un seul serveur physique. Chaque VM fonctionne comme une machine indépendante avec son propre système d'exploitation et ses ressources allouées (CPU, mémoire, stockage, réseau).

La virtualisation est gérée par une couche logicielle appelée **hyperviseur**.

Types d'hyperviseurs

On distingue deux familles d'hyperviseurs selon qu'ils s'installent directement sur le matériel ou par-dessus un OS existant :

- **Type 1 (bare-metal)** : installé directement sur le matériel physique. Exemples : VMware ESXi, Microsoft Hyper-V, KVM.
- **Type 2 (hosted)** : installé au-dessus d'un système d'exploitation. Exemples : VirtualBox, VMware Workstation.

Les hyperviseurs de type 1 sont ceux utilisés en production dans les datacenters (dont ceux d'AWS), car ils offrent de meilleures performances en s'affranchissant d'un OS hôte intermédiaire ; les hyperviseurs de type 2 restent réservés à des usages de poste de travail ou de test.

Avantages de la virtualisation :

Ces bénéfices expliquent l'adoption quasi universelle de la virtualisation dans les datacenters modernes :

- Meilleure utilisation des ressources matérielles.
- Isolation entre les environnements.
- Déploiement rapide.
- Maintenance simplifiée.

Ces avantages expliquent pourquoi la virtualisation est restée, plus de vingt ans après son adoption massive, le socle technique sur lequel repose la quasi-totalité des offres Cloud actuelles, AWS compris.

Virtualisation et AWS

Sur AWS, la virtualisation est au cœur de nombreux services, en particulier :

- **Amazon EC2 (Elastic Compute Cloud)** : service permettant de créer et gérer des machines virtuelles dans le Cloud AWS.
- **Amazon EBS (Elastic Block Store)** : service de stockage en mode bloc utilisé par les instances EC2.
- **Amazon VPC (Virtual Private Cloud)** : service qui fournit un réseau virtuel isolé dans AWS pour héberger les ressources.

Ces trois services montrent que la virtualisation n'est pas un détail d'implémentation invisible chez AWS : elle est directement exposée à l'utilisateur, qui choisit le type et la taille de sa VM (EC2), son stockage (EBS) et son réseau (VPC).

5.2 Paravirtualisation et Émulation

Deux modes techniques sont utilisés dans la virtualisation Cloud :

- **Émulation** : le matériel est simulé intégralement par logiciel. Plus flexible mais moins performant.
- **Paravirtualisation** : le système invité est conscient qu'il est virtualisé et interagit directement avec l'hyperviseur, ce qui offre de meilleures performances.

AWS utilise principalement des technologies de **paravirtualisation** et de **virtualisation matérielle assistée** pour offrir des performances quasi natives.

 **AWS Nitro System** — La plateforme AWS Nitro est une combinaison de matériel dédié et de logiciels légers qui améliore la sécurité et les performances de la virtualisation EC2.

5.3 Conteneurisation — Une approche plus légère

Définition

La **conteneurisation** est une méthode d'isolation applicative qui permet d'exécuter plusieurs applications sur le même système d'exploitation, dans des environnements isolés appelés **conteneurs**.

Contrairement à une machine virtuelle, un conteneur ne contient pas son propre OS complet — il partage le noyau de l'hôte, ce qui le rend **beaucoup plus léger**.

Avantages des conteneurs :

Le fait de partager le noyau de l'hôte plutôt que d'embarquer un OS complet se traduit par plusieurs bénéfices concrets :

- Démarrage quasi instantané.
- Consommation réduite de ressources.
- Facilité de portabilité et de déploiement.
- Standardisation des environnements.

Cette légèreté permet de faire tourner beaucoup plus de conteneurs que de VM sur un même serveur physique, ce qui explique pourquoi les architectures microservices modernes s'appuient massivement sur la conteneurisation plutôt que sur des VM dédiées par service.

Outils et services associés

Plusieurs outils et services permettent de créer, exécuter et orchestrer ces conteneurs, du poste de développement jusqu'à la production à grande échelle sur AWS :

- **Docker** : technologie de conteneurisation la plus répandue, permettant de créer, packager et exécuter des conteneurs.
- **Amazon ECS (Elastic Container Service)** : service AWS permettant de déployer et gérer des conteneurs Docker à grande échelle.
- **Amazon EKS (Elastic Kubernetes Service)** : service AWS qui fournit une plateforme Kubernetes managée pour orchestrer des conteneurs.

 [Amazon ECS Documentation](#)  [Amazon EKS Documentation](#) 

Il est important de comprendre la différence clé — VM = OS complet, Conteneur = application isolée. C'est une **rupture technologique** qui a permis l'essor de l'architecture microservices.

5.4 Microservices — Architecture distribuée et scalable

Les **microservices** sont une approche architecturale qui consiste à **découper une application monolithique en petits services indépendants**, chacun responsable d'une fonction métier spécifique.

Exemple : Une application de e-commerce ne sera plus UNE grosse application, mais plusieurs petits services :

- Service d'authentification
- Service de gestion de panier
- Service de paiement
- Service de recommandations
- Service de livraison

Ce découpage par fonction métier n'est pas qu'un détail d'organisation du code : il conditionne directement les bénéfices obtenus par rapport à une application monolithique.

Avantages des microservices

Le monolithe (une application unique de 500k lignes, une seule technologie, un déploiement lent et une scalabilité limitée) s'oppose aux microservices (Auth, Cart, Payment, Reco découpés en services indépendants), qui autorisent des technologies différentes par service, un déploiement rapide et une scalabilité flexible par équipe.

- **Agilité** : chaque équipe développe son service indépendamment.
- **Scalabilité** : seul le service de paiement reçoit beaucoup de trafic ? Scalez juste ce service.
- **Résilience** : si le service de recommandations est down, le reste de l'app fonctionne.
- **Technologie** : chaque service peut utiliser Node.js, Python, Java... selon le besoin.

Ces quatre bénéfices ont toutefois un prérequis technique implicite : découper une application en dizaines de petits services n'a de sens que si l'on dispose d'un moyen léger de les déployer et de les faire communiquer indépendamment — c'est le rôle des conteneurs.

Microservices et conteneurs

Les microservices **sans conteneurs** seraient très compliqués à gérer. C'est pourquoi **Docker + Kubernetes** (ou **AWS ECS**) sont devenus essentiels.

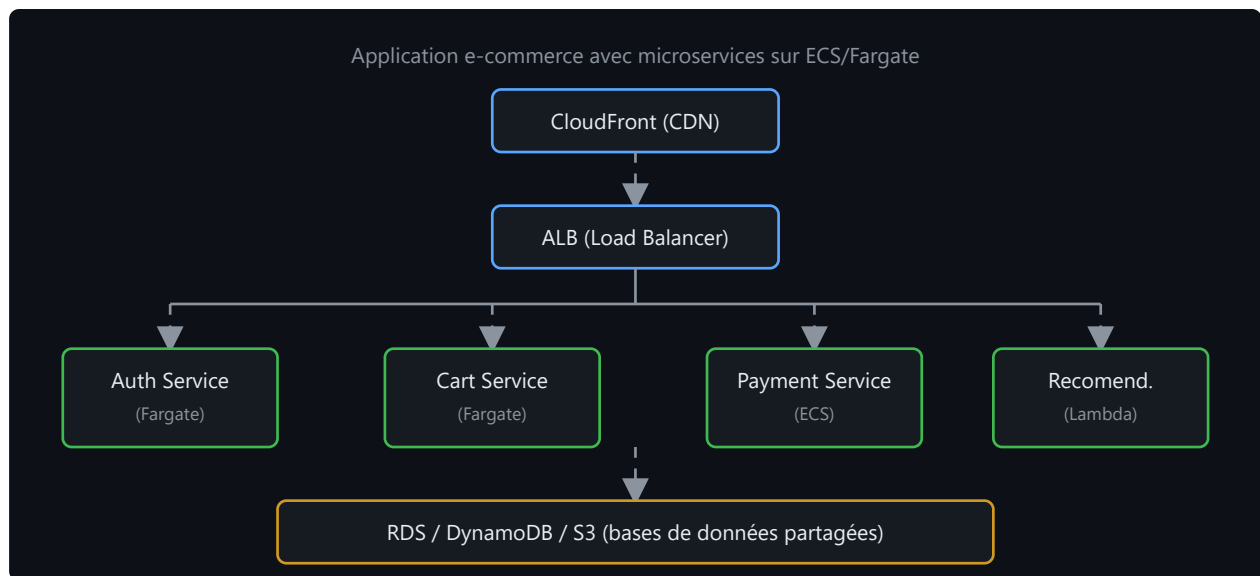
Chaque microservice est empaqueté dans son propre **conteneur Docker**, puis orchestré par un outil qui gère les déploiements, la scalabilité et la résilience.

Services AWS pour microservices :

- **Amazon ECS (Elastic Container Service)** : orchestration native AWS, plus simple que Kubernetes.
- **Amazon EKS (Elastic Kubernetes Service)** : Kubernetes managé, si vous avez une flotte massive.
- **AWS Fargate** : exécution serverless de conteneurs (sans gérer les instances EC2).
- **AWS Lambda** : exécution de fonctions sans serveur (cas de microservices très légers).

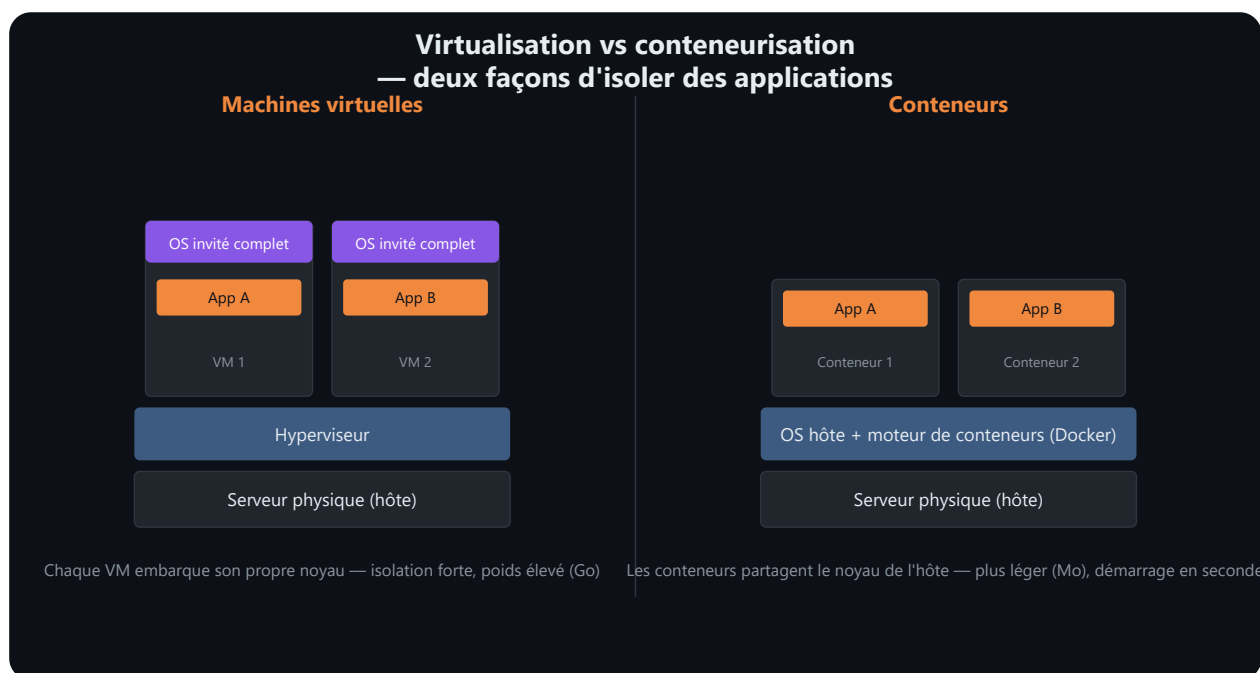
Le choix entre ces quatre services dépend surtout du niveau de contrôle recherché : ECS et Fargate privilégient la simplicité opérationnelle, EKS convient aux équipes déjà investies dans l'écosystème Kubernetes, et Lambda s'adresse aux microservices les plus légers et événementiels.

Un exemple en image



[Microservices Documentation AWS](#)

5.5 Virtualisation vs conteneurisation — tableau comparatif



La différence fondamentale entre les deux approches se voit dans l'empilement des couches : une VM embarque un système d'exploitation invité complet pour chaque application, tandis que plusieurs conteneurs partagent le même noyau hôte via le moteur de conteneurs. C'est ce partage du noyau qui explique l'écart de poids et de vitesse de démarrage entre les deux modèles.

Élément	Virtualisation (VM)	Conteneurisation
Système	OS complet par machine virtuelle	Noyau partagé, application isolée
Démarrage	Lent (minutes)	Rapide (secondes)
Consommation de ressources	Élevée	Faible
Portabilité	Moins flexible	Très flexible
Cas d'usage typiques	Migration d'applications legacy, serveurs complets	Microservices, CI/CD, déploiements rapides

Ce tableau permet de bien fixer les différences entre VM et conteneurs.

6. Présentation d’AWS et de son écosystème

6.1 Vue d’ensemble

Amazon Web Services (AWS) est aujourd’hui le leader mondial du Cloud public. Lancé en 2006, AWS est une filiale d’Amazon qui propose une **plateforme Cloud complète** couvrant :

- L’infrastructure (serveurs, stockage, réseau),
- Les plateformes de développement et déploiement applicatif,
- Les services managés et serverless,
- L’intelligence artificielle, la data et l’IoT.

AWS est présent dans la quasi-totalité des secteurs d’activité : industrie, banque, administration publique, éducation, santé, etc.

 [Site officiel AWS](#)  [AWS Overview Documentation](#) 

6.2 Les applications dans les offres Cloud

AWS n'est pas seulement une plateforme d'infrastructure : c'est aussi un **écosystème d'applications et de solutions métiers**.

Quelques exemples typiques :

- **Hébergement web et API** : via Amazon EC2, Elastic Beanstalk ou Lightsail.
- **Stockage et sauvegarde** : avec S3, EBS et Glacier.
- **Analyse de données** : grâce à Redshift et Athena.
- **Sécurité et conformité** : via IAM, CloudTrail et Security Hub.

Ces briques sont interconnectées. Une **application complète AWS** s'appuie généralement sur plusieurs de ces services, orchestrés ensemble pour offrir disponibilité, sécurité et performance.

6.3 Les principaux acteurs du marché du Cloud

Le Cloud Computing est un marché concurrentiel dominé par **trois géants**, avec quelques acteurs spécialisés ou souverains :

Panorama comparatif simplifié

Avant de décortiquer AWS, il est utile de comprendre son positionnement face à ses concurrents principaux.

Fournisseur	Origine	Points forts	Contexte d'utilisation	Position sur le marché
AWS	Amazon (2006)	Large catalogue (200+ services), maturité, écosystème, documentation, communauté	Startups, PME, grandes entreprises, multinationales	Leader (~32 % du marché)
Microsoft Azure	Microsoft (2010)	Intégration native Windows/Office 365/AD, modèle hybride fort, support entreprise	Organisations Microsoft existantes, conformité européenne	2 ^e position (~23 %)
Google Cloud Platform (GCP)	Google (2008)	Outils data/ML excellents, BigQuery, Vertex AI, simplicité d'usage	Projets data-driven, IA/ML, startups tech	3 ^e position (~10 %)
OpenStack (Open Source)	Communauté	Infrastructure privée, totale maîtrise, coût réduit	Cloud privé, institutions académiques, hébergeurs	Marché de niche
OVHcloud (Européen)	OVH (France)	Souveraineté données EU, prix compétitifs	PME EU, données sensibles, RGPD stricte	Marché de niche EU
IBM Cloud, Oracle Cloud	IBM / Oracle	Spécialisés (IA, bases données massives)	Grandes entreprises, solutions legacy	Marché de niche

Ce panorama confirme un marché dominé par trois hyperscalers généralistes (AWS, Azure, GCP), à côté duquel subsistent des acteurs de niche positionnés sur la souveraineté ou l'open source plutôt que sur le volume.

Comparatif détaillé : AWS vs Azure vs GCP

Le panorama ci-dessus donne une vue d'ensemble ; le tableau suivant entre dans le détail service par service pour les trois principaux fournisseurs, avec quelques lignes qui méritent une explication complémentaire.

Aspect	AWS	Azure	GCP
Nombre de services	200+	200+	100+
Services de calcul	EC2, Lambda, Fargate	VMs, App Services, AKS	Compute Engine, Cloud Functions
Bases de données	RDS, DynamoDB, Aurora	SQL Database, Cosmos DB	Cloud SQL, Firestore
Données et IA	SageMaker, Glue	Azure ML, Synapse	BigQuery, Vertex AI ★
Points de présence	30+ régions	60+ régions	40+ régions
Matériel propriétaire	AWS Nitro	Optimisé	Processeurs TPU (IA)
Force communautaire	Très forte	Forte (corporations)	Croissante (startups)
Apprentissage	Longue courbe	Intégré MS, plus facile	Pythoniste-friendly
Certifications	Cloud Practitioner, CloudOps Engineer, Solutions Architect	AZ-900, AZ-104	Cloud Digital Leader, Associate Cloud Engineer, Professional

Ce que signifient les lignes moins évidentes de ce tableau :

- **Matériel propriétaire** : chaque fournisseur a développé du matériel sur mesure pour ses data centers plutôt que d'utiliser des composants standards du marché. **AWS Nitro** (déjà présenté en section 5.2 de ce chapitre) est un système matériel et logiciel conçu par AWS qui décharge la virtualisation, le réseau et la sécurité du CPU principal du serveur physique — cela signifie que quasiment 100 % de la puissance de la machine physique est disponible pour vos instances EC2, au lieu qu'une partie soit consommée par l'hyperviseur lui-même. Chez GCP, les **TPU (Tensor Processing Units)** sont des puces conçues spécifiquement pour accélérer les calculs de machine

learning (entraînement et inférence de modèles d'IA) — un TPU va beaucoup plus vite qu'un CPU classique sur ce type de calcul précis, mais ne sert à rien pour un usage généraliste.

- **Force communautaire** : désigne le volume et l'activité de la communauté d'utilisateurs autour du fournisseur — tutoriels, forums (Stack Overflow, Reddit), projets open source, meetups. Plus cette communauté est active, plus il est facile de trouver de l'aide en cas de blocage. AWS bénéficie ici de son avance historique (premier arrivé en 2006) : davantage de contenus accumulés sur près de 20 ans.
- **Apprentissage** : la "courbe d'apprentissage" désigne la difficulté et le temps nécessaires pour devenir opérationnel sur la plateforme. AWS a la réputation d'avoir une courbe plus longue du fait de son catalogue de 200+ services parfois redondants (plusieurs façons de faire la même chose). Azure est souvent perçue comme plus accessible pour des équipes déjà habituées à l'écosystème Microsoft (Windows Server, Active Directory, Office 365) — les concepts et l'interface leur sont familiers. **"Pythoniste-friendly"** signifie que GCP a orienté beaucoup de ses outils et de sa documentation autour de Python, le langage dominant en data science — un développeur Python s'y retrouve plus vite, notamment pour les services data/IA (BigQuery, Vertex AI).
- **Certifications** : chaque fournisseur a son propre parcours de certification, avec des noms et des codes différents mais un principe similaire (un premier niveau généraliste, puis des spécialisations). Côté Azure, **AZ-900** est la certification fondamentale (équivalent du AWS Cloud Practitioner) et **AZ-104** la certification Administrator Associate (gestion quotidienne des ressources Azure, équivalent du AWS SysOps). Les certifications AWS sont détaillées à la section 1.3 de ce chapitre.

Ce détail ligne par ligne montre que le choix entre AWS, Azure et GCP dépend rarement d'un seul critère : il se joue souvent sur l'écosystème déjà en place dans l'entreprise (Microsoft, data science) ou sur des besoins matériels très spécifiques (TPU pour l'IA).

AWS, pourquoi c'est le leader ?

Les trois raisons principales :

1. **First-mover advantage** : AWS a lancé la révolution du cloud en 2006. Les équipes AWS ont mûri progressivement, corrigé les bugs, collecté les retours.
2. **Catalogue massif** : Plus de 200 services interconnectés permettent de construire à peu près n'importe quelle architecture.
3. **Écosystème** : Partenaires, formations, documentation, communauté, marketplace. Plus vous apprenez AWS, moins vous avez intérêt à partir.

C'est pourquoi AWS représente environ **32 % du marché cloud**, loin devant Azure (23 %) et GCP (10 %).

Solutions open source

Enfin, une mention importante : **OpenStack**.

OpenStack est une plateforme **open source complète** qui permet à toute organisation de **construire son propre cloud privé**, sans dépendre d'un fournisseur unique. Elle est utilisée par :

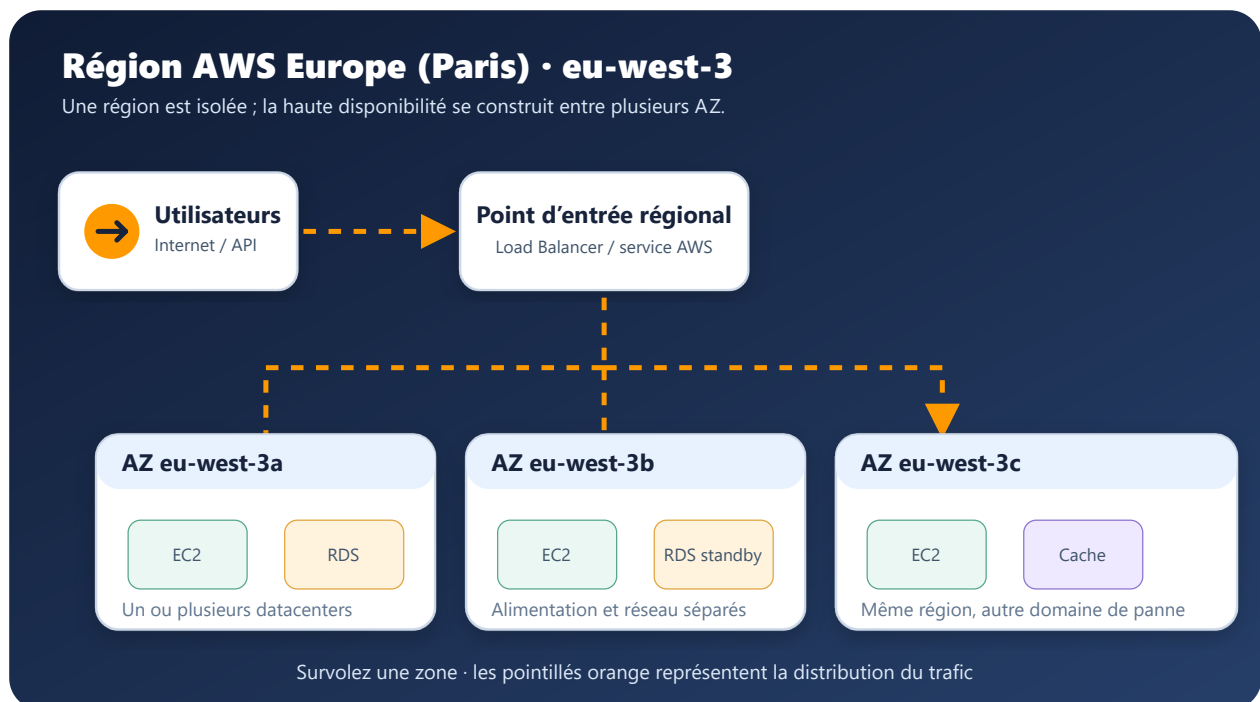
- Universités et instituts de recherche
- Gouvernements (souveraineté)
- Hébergeurs et fournisseurs de services cloud alternatifs
- Organisations ayant besoin de total contrôle

OpenStack ne joue pas sur le même terrain qu’AWS/Azure/GCP (qui sont des services managés) — c’est une brique technologique que vous installez et maintenez vous-même.

 [Gartner Magic Quadrant 2024 – Cloud Infrastructure and Platform Services](#)

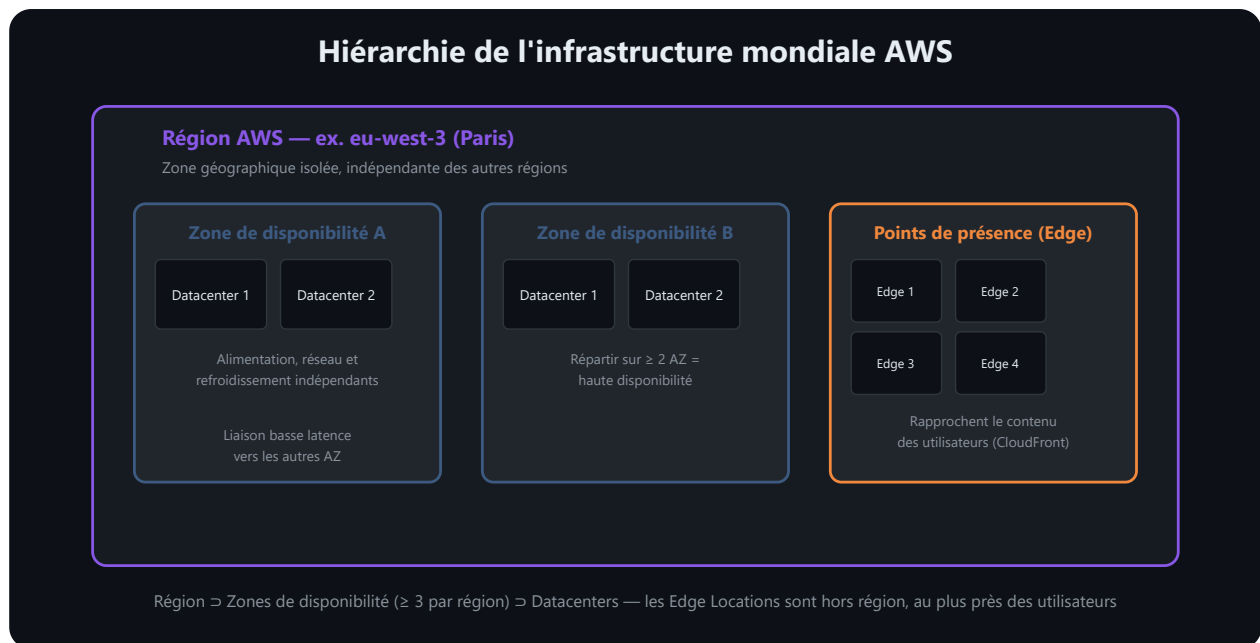
6.4 L’infrastructure mondiale d’AWS

Le schéma suivant part d’une requête utilisateur et montre pourquoi une région ne suffit pas, à elle seule, à garantir la disponibilité. Le trafic doit être distribué vers plusieurs zones indépendantes et les ressources zonales doivent réellement être dupliquées.



Les trois AZ appartiennent à la même région et communiquent par le réseau régional AWS, mais constituent des domaines de défaillance distincts. Déployer une instance dans `eu-west-3a` et une base uniquement dans cette même AZ reste une architecture mono-AZ, même si la région en propose trois.

L’un des piliers majeurs d’AWS est son **infrastructure mondiale** extrêmement étendue et redondée. Elle est structurée en trois grandes composantes, organisées selon une hiérarchie précise :



Ce schéma montre l’emboîtement des trois niveaux : une région regroupe plusieurs zones de disponibilité, chaque zone regroupant elle-même plusieurs datacenters physiques. Les points de présence, à l’inverse, ne font pas partie de cette hiérarchie régionale — ils sont disséminés indépendamment, au plus près des utilisateurs finaux.

1. Régions AWS (Regions)

Le niveau le plus large de cette hiérarchie est la région, qui définit avant tout où sont physiquement situées les données et les ressources.

- Une *région* est une zone géographique physique (exemple : `eu-west-3` pour Paris).
- Chaque région contient plusieurs datacenters appelés Zones de disponibilité (AZ).
- Les services AWS sont déployés par région et choisis par l'utilisateur.

Le choix de région n'est donc pas qu'une question de proximité géographique : il détermine aussi la juridiction sous laquelle sont hébergées les données et la liste des services AWS effectivement disponibles.

2. Zones de disponibilité (Availability Zones - AZ)

À l'intérieur de chaque région se trouve le niveau qui garantit concrètement la résilience face à une panne locale : la zone de disponibilité.

- Une AZ correspond à un ou plusieurs datacenters indépendants reliés entre eux par des liens à très faible latence.
- Les AZ sont conçues pour **garantir la haute disponibilité et la tolérance aux pannes**.

Contrairement aux régions et aux AZ, les points de présence ne servent pas à héberger des charges de travail mais à rapprocher le contenu de l'utilisateur final.

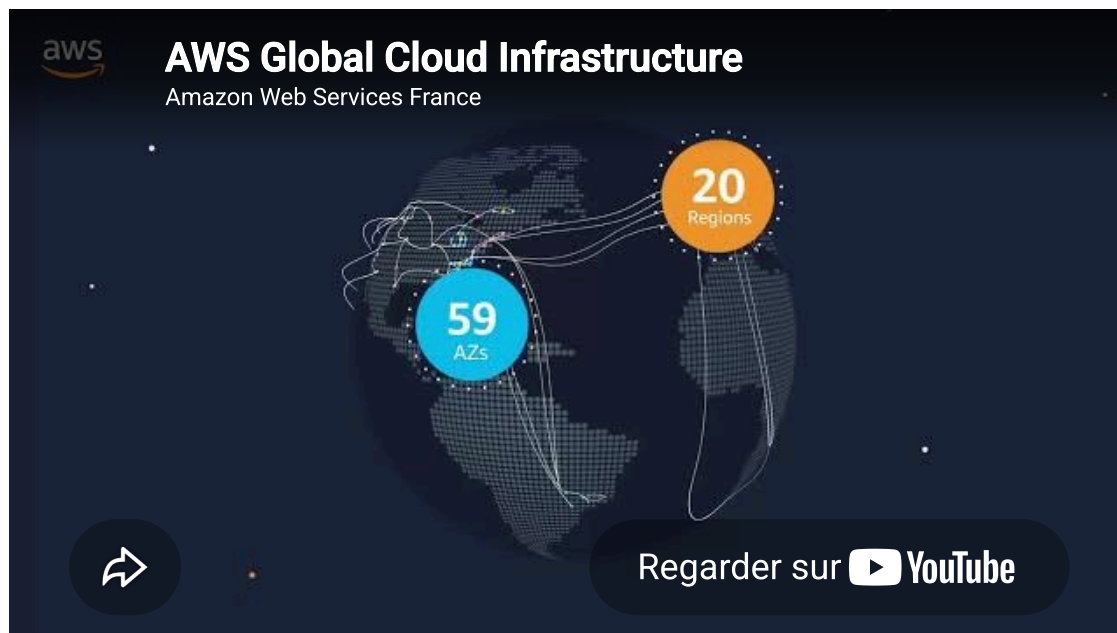
3. Points de présence (Edge Locations)

Ce troisième niveau répond à un besoin différent : la performance perçue plutôt que la résilience.

- Répartis dans le monde entier, ils permettent d'accélérer la diffusion de contenu et d'optimiser les performances réseau via des services comme **Amazon CloudFront** (réseau de diffusion de contenu — CDN).

Ces trois niveaux ne se substituent pas l'un à l'autre : une architecture résiliente combine en général le bon choix de région (contrainte légale) avec une répartition sur plusieurs AZ (contrainte de disponibilité), les points de présence intervenant en complément pour la performance perçue par l'utilisateur.

Vidéo explicative : infrastructure AWS



Retenons que le concept de *région* et de *localisation géographique des données* est souvent déterminant pour des raisons légales et de performance.

Décision d'architecture — maintenant que vous connaissez régions et AZ

Une application française contient des données sensibles et doit rester disponible lors de la panne d'un datacenter. Quel premier choix faut-il formuler ?

► Afficher la réponse raisonnée

6.5 Principes de tarification AWS

L'un des aspects stratégiques de l'adoption du Cloud est la **tarification à l'usage**. AWS applique un modèle de facturation souple et transparent basé sur la consommation réelle.

Principes clés :

Ce modèle de facturation repose sur quatre principes qui se combinent entre eux :

- **Pay-as-you-go** : paiement en fonction de l'utilisation réelle des ressources.
- **Sans engagement initial** : pas de coût d'entrée, contrairement au modèle CAPEX.
- **Économies d'échelle** : les tarifs peuvent diminuer avec l'usage.
- **Facturation par service** : chaque service (EC2, S3, RDS...) est facturé séparément selon ses métriques propres.

Ces quatre principes se combinent : un service EC2 facturé à l'heure de fonctionnement peut par exemple bénéficier de remises de volume (Savings Plans) sans jamais nécessiter d'engagement contractuel initial, ce qui distingue fondamentalement la logique tarifaire du Cloud de celle des infrastructures traditionnelles.

Selon le service et l'engagement pris, plusieurs modes de tarification existent — à la demande, instances réservées, Savings Plans, Spot Instances. Le Chapitre 4 détaille chacun de ces modes avec des cas métier chiffrés, au moment où vous configurerez vos premières instances EC2.

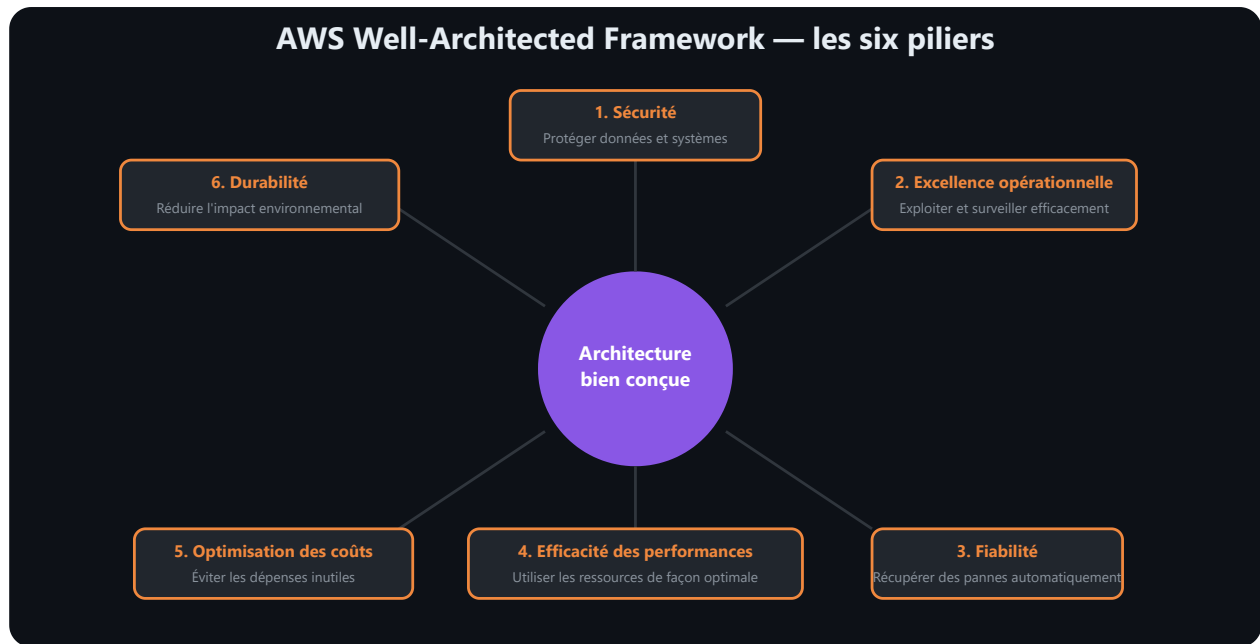
 [AWS Pricing Overview](#)   [AWS Pricing Calculator](#) 

7. AWS Well-Architected Framework

Le **Well-Architected Framework** est une méthodologie officielle AWS qui permet de **concevoir des architectures Cloud robustes**, sécurisées, performantes, optimisées en coûts et **durables**.

C'est une **philosophie de design** qui s'applique à chaque projet AWS, du plus petit au plus grand.

Ce framework repose sur **six piliers**, souvent représentés par le sigle **SOPREC** :



Les six piliers ne s'appliquent pas en séquence : ils se travaillent en continu et souvent en tension les uns avec les autres (par exemple, plus de résilience implique généralement plus de coûts) — le rôle de l'architecte est de trouver le bon équilibre pour chaque projet, pas de maximiser chaque pilier isolément.

- 🏛️ **Security** → Sécurité, IAM, chiffrement
- ⚙️ **Operational Excellence** → Monitoring, automatisation, processus
- 💪 **Reliability** → Résilience, haute disponibilité
- ⚡ **Performance** → Efficacité des ressources
- 💰 **Cost Optimization** → Optimisation des dépenses
- 🌱 **Sustainability** → Impact environnemental, efficacité



Tip

Vérification AWS CLI — liste des piliers WAF disponibles pour une review :

```
1 aws wellarchitected list-lenses --lens-type AWS_OFFICIAL
```

```
1 {
2     "LensSummaries": [
3         { "LensAlias": "wellarchitected", "LensName": "AWS Well-Architected
Framework",
4           "LensVersion": "2023-04-10", "Description": "Six pillars review"
5         },
6         { "LensAlias": "serverless", "LensName": "Serverless Lens",
7           "LensVersion": "3.0" },
8         { "LensAlias": "saas", "LensName": "SaaS Lens", "LensVersion":
9           "1.0" }
10    ]
11 }
```

Le Framework Well-Architected est disponible depuis la console et via API. Vous pouvez lancer une revue avec **AWS Well-Architected Tool** : les questions sont organisées selon les six piliers. Ce service ne doit pas être confondu avec **AWS WAF**, le pare-feu applicatif web.

1. Security (Sécurité)

Mettre en œuvre des mesures de protection fortes à tous les niveaux — **IAM, chiffrement, sécurité réseau, MFA, audit**.

En pratique :

- Créer des rôles IAM spécifiques plutôt que d'utiliser le compte root.
- Chiffrer les données au repos et en transit.
- Utiliser des security groups et NACL pour isoler les ressources.
- Activer CloudTrail pour auditer les accès.

La sécurité posée, le pilier suivant s'intéresse à la façon de faire fonctionner l'infrastructure au quotidien sans intervention manuelle permanente.

2. Operational Excellence (Excellence opérationnelle)

Surveiller et améliorer continuellement les opérations — **mise en place de processus, d'automatisation et de monitoring.**

En pratique :

- Utiliser CloudWatch pour surveiller les ressources.
- Automatiser les déploiements avec CloudFormation ou Terraform.
- Documenter les procédures d'incident.
- Faire des reviews régulières de l'architecture.

Une bonne automatisation opérationnelle facilite directement le pilier suivant, qui vise à garantir que le système continue de fonctionner même en cas de panne.

3. Reliability (Fiabilité / Résilience)

Assurer la **résilience** et la **tolérance aux pannes** — **redondance, sauvegarde, failover automatique.**

En pratique :

- Déployer les applications dans plusieurs **Availability Zones** pour la haute disponibilité.
- Mettre en place des sauvegardes automatiques (EBS snapshots, RDS backups).
- Utiliser des auto-scaling groups pour ajuster le nombre d'instances en cas de charge.
- Tester régulièrement les scénarios de récupération d'urgence (RTO/RPO).

Une fois la résilience assurée, encore faut-il que les ressources déployées pour l'atteindre soient correctement dimensionnées, sans sur- ni sous-provisionnement.

4. Performance Efficiency (Efficacité des performances)

Utiliser les ressources de manière **efficace et évolutive** — **choix des bonnes instances, optimisation des requêtes.**

En pratique :

- Choisir le bon type d'instance EC2 selon le workload (t3 pour web, m5 pour applications, c5 pour calcul).
- Utiliser des caches (ElastiCache, CloudFront) pour réduire la latence.
- Optimiser les requêtes de bases de données.
- Utiliser AWS Compute Optimizer pour recommander les bonnes tailles.

Ce dimensionnement au plus juste a une conséquence directe sur le pilier suivant : moins de ressources surdimensionnées signifie mécaniquement moins de coûts gaspillés.

5. Cost Optimization (Optimisation des coûts)

Éviter les dépenses inutiles et **ajuster la capacité à la demande** — **monitoring des coûts, réservation d’instances, suppression des ressources non utilisées.**

En pratique :

- Utiliser des **Reserved Instances** ou **Savings Plans** pour réduire les coûts.
- Mettre en place des **budgets d’alerte** dans AWS Billing.
- Arrêter les ressources non utilisées (instances EC2, RDS développement).
- Analyser les logs de coûts avec Cost Explorer ou AWS Cost Anomaly Detection.

Le sixième et dernier pilier prolonge cette logique d’optimisation des ressources au-delà du seul aspect financier, en y ajoutant une dimension environnementale.

6. Sustainability (Durabilité) — Le pilier le plus récent ✨

Minimiser l’**impact environnemental** de votre infrastructure Cloud — **efficacité énergétique, utilisation optimale des ressources, choix des régions.**

En pratique :

- Utiliser les régions AWS qui utilisent des énergies renouvelables (ex : `eu-west-1` en Irlande, alimentée à >80% par des renouvelables).
- Éteindre les ressources inutilisées plutôt que de les laisser tourner.
- Utiliser le **Compute Optimizer** pour ajuster les tailles d’instances et réduire la consommation.
- Préférer les architectures **serverless** (Lambda, Fargate) qui consomment moins que les EC2 permanentes.
- Monitorer l’empreinte carbone de vos déploiements via le **AWS Customer Carbon Footprint Tool**.

C’est un enjeu de plus en plus important pour les entreprises souhaitant atteindre leurs **objectifs de développement durable (ODD)** et de **net-zéro** (carbon-neutral).

 [AWS Well-Architected Framework](#)  [AWS Sustainability — Carbon Footprint Tool](#)

Appliquer le Framework au quotidien

Lors de **chaque conception d’architecture AWS**, on doit se poser ces questions :

-  **Sécurité** : Les données sont-elles protégées ? IAM bien configuré ?
-  **Opérations** : Comment monitorer et alerter ? Comment déployer sans risque ?
-  **Résilience** : Que se passe-t-il si un service tombe ? Avons-nous des sauvegardes ?
-  **Performance** : Les ressources sont-elles bien dimensionnées ? Y a-t-il des goulots ?
-  **Coûts** : Payons-nous ce que nous utilisons réellement ? Pouvons-nous optimiser ?
-  **Durabilité** : Quel est notre impact environnemental ? Pouvons-nous le réduire ?

Cette approche sera **reprise au fil des jours**, notamment lors des ateliers EC2, S3, RDS et VPC.

8. AWS Management Console

La **console AWS** est une interface Web centralisée qui permet de gérer tous les services AWS depuis un seul endroit.

8.1 Principales fonctionnalités

La console web regroupe en un seul endroit tout ce dont un administrateur a besoin pour piloter son compte AWS au quotidien, et c'est justement ce qui la rend accessible aux débutants malgré la richesse du catalogue de services.

Elle donne d'abord accès à l'ensemble des services AWS disponibles dans la région sélectionnée : EC2, S3, RDS, Lambda et les centaines d'autres services se retrouvent derrière une barre de recherche unique, sans avoir à mémoriser des noms de commandes ou des endpoints d'API. Elle permet ensuite de visualiser en temps réel l'état des ressources déjà créées — combien d'instances EC2 tournent, quels buckets S3 existent, comment est configuré le VPC — ce qui en fait le point d'entrée naturel pour diagnostiquer un problème avant de creuser en ligne de commande. Elle sert aussi de porte d'entrée vers **AWS IAM (Identity and Access Management)**, où se gèrent les comptes, les utilisateurs et leurs permissions : c'est depuis la console que l'on crée les premiers utilisateurs IAM et qu'on leur attribue des droits, avant même de savoir écrire une policy JSON. Enfin, elle donne un accès direct aux métriques d'usage et à la facturation, ce qui permet de surveiller les coûts sans quitter l'interface — un réflexe indispensable pour éviter les mauvaises surprises en fin de mois.

8.2 Structure de la console

L'interface de la console se découpe en quelques zones fixes, présentes quelle que soit la page ou le service consulté, ce qui facilite la navigation une fois ces repères identifiés.

Zone	Rôle
Barre de recherche	Trouver rapidement un service (ex : EC2, S3, IAM)
Panneau de navigation	Accéder aux sections : Services, Ressources, Facturation
Région	Indique la localisation actuelle (ex : Paris eu-west-3)
Compte utilisateur	Informations, factures, connexions IAM
Notifications	Alertes et mises à jour
Support	Documentation, centre d'aide, tickets AWS

 [AWS Console](#)  [Guide de l'interface AWS Console](#)



Tip

Vérification AWS CLI — lister les régions disponibles :

```
1 aws ec2 describe-regions --output table
```

```
-----
|                               DescribeRegions                               |
+-----+-----+-----+-----+
| RegionName | Endpoint |                               |
+-----+-----+-----+-----+
| eu-west-3  | ec2.eu-west-3.amazonaws.com | ← Paris
| eu-west-1  | ec2.eu-west-1.amazonaws.com | ← Irlande
| eu-central-1 | ec2.eu-central-1.amazonaws.com | ← Francfort
| us-east-1  | ec2.us-east-1.amazonaws.com | ← Virginie du Nord
| ap-southeast-1 | ec2.ap-southeast-1.amazonaws.com | ← Singapour
+-----+-----+-----+-----+
```

La région **eu-west-3** (Paris) est votre région de travail par défaut pour cette formation. Vérifiez toujours que vous êtes bien dans la bonne région avant de créer une ressource — une instance EC2 créée dans **us-east-1** par erreur sera difficile à retrouver.

Rappelons que la console est un moyen parmi d'autres d'interagir avec AWS — il existe également des API, des SDK et la **CLI AWS** pour les automatisations.

8.3 Bonnes pratiques de base

Avant d'aller plus loin dans les services eux-mêmes, quelques réflexes simples à prendre dès l'utilisation de la console évitent la majorité des incidents de sécurité et de facturation rencontrés par les débutants.

- Toujours vérifier la **région active** avant de créer des ressources.
- Fermer les ressources non utilisées pour éviter des coûts inutiles.
- Utiliser **IAM** pour créer des comptes utilisateurs distincts plutôt que d'utiliser le compte root — rappel : voir l'encart sur le compte root en section 2.6 de ce chapitre.
- Activer la **MFA (Multi-Factor Authentication)** pour sécuriser les accès.
- Surveiller les coûts dans **Billing & Cost Management**.

 [Best Practices AWS IAM](#)

8.4 Gestion du budget et des coûts AWS

L'un des **risques majeurs** du cloud est la **perte de contrôle des coûts**. Un bucket S3 public par erreur, une instance EC2 oubliée, un transfert de données non optimisé — et la facture peut exploser en quelques jours.

AWS fournit des outils puissants pour **monitorer, budgéter et optimiser** les coûts.

Principes clés de la tarification AWS

Ces principes reprennent et détaillent, dans le contexte spécifique du suivi budgétaire, la logique de facturation déjà introduite en section 6.5 de ce chapitre.

1. **Pay-as-you-go** : payer ce qu'on utilise réellement.
2. **Granularité** : chaque service a ses propres métriques — EC2 facturé par heure (ou seconde), S3 par Go stocké + requêtes, RDS par heure + stockage + transfert.
3. **Réductions possibles** : Reserved Instances (engagement 1 ou 3 ans), Savings Plans (engagement flexible), Spot Instances (capacité excédentaire à -70%).
4. **AWS Free Tier** : pour les comptes créés depuis le 15 juillet 2025, AWS propose un système de crédits — 100 USD à l'inscription et jusqu'à 100 USD supplémentaires via des activités — ainsi qu'un plan gratuit limité à six mois. Les comptes plus anciens restent soumis au régime historique, qui comportait notamment certaines offres gratuites pendant douze mois. Dans tous les cas, un service hors périmètre, un dépassement de crédit ou un dépassement de quota est facturé au tarif normal.

Le point le plus piégeux reste le quatrième : le Free Tier n'est pas un mode gratuit permanent, seulement une couverture partielle et temporaire — d'où l'intérêt de la commande de vérification ci-dessous pour confirmer ce qui est réellement facturé.



Tip

Exemple de commande AWS CLI pour vérifier votre consommation du Free Tier :

```
1 aws ce get-cost-and-usage \  
2   --time-period Start=2024-01-01,End=2024-01-31 \  
3   --granularity MONTHLY \  
4   --metrics BlendedCost \  
5   --group-by Type=DIMENSION,Key=SERVICE
```

```
1 {  
2   "ResultsByTime": [{  
3     "TimePeriod": { "Start": "2024-01-01", "End": "2024-01-31" },  
4     "Groups": [  
5       { "Keys": ["Amazon EC2"], "Metrics": { "BlendedCost": {  
"Amount": "0.00", "Unit": "USD" } } },  
6       { "Keys": ["Amazon S3"], "Metrics": { "BlendedCost": {  
"Amount": "0.23", "Unit": "USD" } } },  
7       { "Keys": ["Amazon RDS"], "Metrics": { "BlendedCost": {  
"Amount": "0.00", "Unit": "USD" } } }  
8     ]  
9   }]  
10 }
```

Dans cet exemple fictif, les crédits promotionnels absorbent encore la consommation EC2 et RDS. S3 affiche un coût car l'usage concerné n'est plus entièrement couvert. Le résultat réel dépend de la date de création du compte, du plan choisi, du solde de crédits, de la région et des services utilisés : il faut toujours vérifier la page **Free Tier** et la facturation du compte actif.

AWS Billing Dashboard

Accessible depuis la console AWS, le **Billing Dashboard** affiche :

- Facture du mois en cours
- Prévisions de coûts (estimé) jusqu'à fin du mois
- Services coûtant le plus

C'est un premier coup d'œil rapide, mais peu détaillé.

AWS Cost Explorer

Cost Explorer permet une **analyse fine des dépenses** :

- Filtrer par service (EC2, S3, RDS...)
- Filtrer par période (jour, mois, année)
- Filtrer par région
- Voir l'historique et les tendances

Cas d'usage : « Pourquoi ma facture a-t-elle explosé en janvier ? Quel service en est responsable ? »

AWS Budgets

AWS Budgets permet de **définir des seuils d'alerte** :

- Budget mensuel maximum (ex : 500 €/mois)
- Alertes si dépassement imminent
- Alertes si usage atypique détecté

Cas d'usage : Recevoir une notification avant d'exploser le budget plutôt qu'une mauvaise surprise à la fin du mois.

AWS Cost Anomaly Detection

Utilise l'**apprentissage automatique** pour détecter les **anomalies de dépenses** :

- Si vous dépensez soudain 10x plus que normal dans une région, alerter.
- Si un service coûte étrangement cher, signaler.

C'est très utile pour détecter des ressources oubliées ou des fuites de données.

AWS Pricing Calculator

AWS Pricing Calculator permet d'estimer les coûts avant de déployer :

- Configurez votre architecture (1 instance EC2 t3.micro, 100 Go S3...)
- Voir le coût mensuel/annuel
- Comparer différentes configurations

C'est idéal pour les **POC (Proof of Concept)** et les **RFQ** (appels d'offre).

Bonnes pratiques pour maîtriser les coûts

Au-delà des outils de suivi, quelques pratiques d'architecture et d'exploitation permettent de réduire structurellement la facture plutôt que de simplement la surveiller.

Pratique	Description	Impact
Utiliser des Reserved Instances	Engagement 1 ou 3 ans pour réduction tarifaire (~30-60%)	Très utile pour charges stables 24/7
Utiliser Spot Instances	Capacité excédentaire à -70% (mais peut être interrompue)	Idéal pour tâches batch, tests
Éteindre les ressources non utilisées	Arrêter EC2 dev/test en fin de journée, supprimer RDS inutilisés	Économies rapides et simples
Auto-scaling	Adapter le nombre d'instances au trafic	Évite surprovisionnement
Optimiser le stockage	S3 Standard pour accès fréquent, S3 Glacier pour archive	Différences tarifaires importantes
Utiliser des réductions volantes	Savings Plans, commitments, consolidated billing	~30-40% de réduction possible
Monitorer régulièrement	Cost Explorer, Budgets, anomaly detection	Détection précoce des dérives

Ces pratiques se combinent plutôt que s'excluent : une architecture bien optimisée associe généralement des Reserved Instances pour la charge stable, des Spot Instances pour les tâches tolérantes aux interruptions, et un monitoring continu pour détecter les dérives que ni l'une ni l'autre ne préviennent.

Exemple concret : budget mal maîtrisé

Imaginons une startup qui lance une application web sur AWS.

Scénario optimiste :

- Petite instance EC2 dont la consommation est couverte par le solde de crédits du compte de démonstration
- 50 Go S3 (~1 € / mois)
- **Coût mensuel : ~1 €**

Scénario problématique (sans monitoring) :

- Instance EC2 mal fermée le week-end : +50 €
- Bucket S3 avec version history excessive : +100 €
- Transfert de données cross-région oublié : +200 €
- **Coût mensuel : ~350 € !**

Avec les outils AWS Budgets :

- Budget défini : 50 €/mois
- Alerte à 40 € : Action rapide
- Découverte de l'instance oubliée → suppression
- Découverte des versions excessives → nettoyage
- **Coût final : 5 €/mois**



Tip

Résultat attendu — alerte AWS Budgets reçue par email :

```
De : no-reply@notifications.aws
Objet : [AWS Budgets] Alerte – Mon Budget Mensuel (80% atteint)

Bonjour,

Votre budget "Mon Budget Mensuel" a atteint 80% de votre seuil d'alerte.

Budget : 50,00 USD
Dépensé : 40,23 USD
Prévu ce mois : 52,31 USD

Service le plus coûteux : Amazon EC2 (28,50 USD)
Recommandation : vérifiez les instances EC2 actives dans toutes les régions.

→ Accéder au Cost Explorer : https://console.aws.amazon.com/cost-management/
```

Grâce à cette alerte précoce, vous pouvez arrêter l'instance oubliée avant que la facture n'explose. Sans ce budget configuré, vous n'auriez découvert le problème qu'à la réception de la facture mensuelle.

C'est pourquoi **mettre en place un monitoring budgétaire dès le départ est essentiel.**

 [AWS Billing Documentation](#)  [AWS Cost Explorer Guide](#)  [AWS Cost Optimization Best Practices](#) 

9. Les services AWS les plus utilisés

AWS (Amazon Web Services) propose **plus de 200 services complets** couvrant le calcul, le stockage, le réseau, les bases de données, l'IA, la sécurité et bien plus encore. Sa philosophie est simple : offrir à chaque entreprise, quelle que soit sa taille, **la même puissance technologique** qu'Amazon utilise pour ses propres opérations mondiales.

Domaine	Service	Explication	Cas d'usage	Analogie
Calcul	EC2	Machines virtuelles configurables (CPU, RAM, OS)	Hébergement web, backend, batch	Louer un serveur dans le cloud
	Lambda	Exécution de code sans serveur, déclenchée par événements	Microservices, automatisation, API	Interrupteur intelligent : déclenche une action sans infrastructure
Stockage	S3	Stockage d'objets scalable et durable	Sauvegardes, fichiers, hébergement statique	Entrepôt numérique : chaque fichier est une boîte
	EBS	Stockage bloc attaché à EC2, persistant	Bases de données, stockage système	Disque dur virtuel
	EFS	Système de fichiers partagé pour EC2	Dossiers partagés, applications multi-instances	Dossier réseau partagé
Bases de données	RDS	Base relationnelle gérée (MySQL, PostgreSQL...)	ERP, CRM, applications transactionnelles	Serveur SQL géré par AWS
	DynamoDB	Base NoSQL scalable sans schéma	IoT, mobile, jeux, logs	Carnet de notes sans structure fixe
Réseau	VPC	Réseau privé isolé avec sous-réseaux et sécurité	Isolation d'environnements, segmentation	Immeuble privé avec étages et portiers

Domaine	Service	Explication	Cas d'usage	Analogie
Sécurité	CloudFront	CDN mondial pour accélérer les contenus	Sites web, vidéos, fichiers statiques	Réseau de relais pour livrer plus vite
	IAM	Gestion des identités et des accès	Contrôle des permissions, MFA, rôles	Badge d'accès pour chaque utilisateur
	KMS	Gestion des clés de chiffrement	Sécurité des données, RGPD, PCI-DSS	Coffre-fort numérique pour les clés
DevOps	CloudFormation	Déploiement d'infrastructure via des fichiers YAML/JSON	Automatisation, standardisation	Plan d'architecte pour construire automatiquement
	CodePipeline	Orchestration CI/CD pour le déploiement	Intégration continue, tests, livraison	Chaîne de montage automatisée
IA/ML	SageMaker	Plateforme de machine learning gérée	Modèles prédictifs, classification, NLP	Laboratoire de data science automatisé
Analyse	Rekognition	Analyse d'images et vidéos par IA	Détection faciale, modération, OCR	Caméra intelligente qui comprend les images
	Athena	Requêtes SQL sur des fichiers S3	Exploration de données, logs, BI	Loupe SQL sur vos fichiers
	Redshift	Entrepôt de données pour BI et reporting	Tableaux de bord, KPIs, analytique	Base de données XXL pour l'analyse

Nous n'entrons pas ici dans les détails techniques de chaque service — cela sera traité dans les jours suivants (IAM, S3, EC2, RDS, VPC...).

9.1 Automatisation et orchestration : Chef, Puppet et OpsWorks

Au-delà des services de base, AWS propose des outils pour **automatiser le déploiement et la gestion de l'infrastructure**. Ces outils sont essentiels dans une approche **DevOps** et **Infrastructure as Code (IaC)**.

AWS OpsWorks — Orchestration managée par AWS

AWS OpsWorks était une famille de services AWS destinée au déploiement et à la gestion automatisés d'applications et d'infrastructures.

Il supporte deux moteurs d'automation populaires :

- **Chef** (propriétaire de OpsWorks Chef)
- **Puppet** (partenariat AWS)

Warning

Toute la famille OpsWorks est désormais en fin de vie et désactivée. OpsWorks for Chef Automate et OpsWorks for Puppet Enterprise ont été arrêtés le 5 mai 2024 ; OpsWorks Stacks l'a été le 26 mai 2024. Cette section sert uniquement à reconnaître une infrastructure historique et à comprendre les principes de Chef et Puppet. Pour une nouvelle architecture AWS, privilégiez notamment AWS Systems Manager, les images automatisées, les services de conteneurs ou une solution de gestion de configuration encore maintenue.

Cas d'usage typiques :

- Provisionner automatiquement des serveurs EC2 avec une stack applicative complète.
- Gérer les configurations à grande échelle sans intervention manuelle.
- Assurer la conformité des serveurs (tous les serveurs web ont la même configuration).
- Automatiser les déploiements continus (CI/CD).

Ces cas d'usage reposaient concrètement sur deux moteurs d'automatisation aux philosophies différentes, que nous détaillons ci-dessous à titre historique.

Chef — Automation as Code

Chef est un outil d'**automatisation de configuration** très populaire. Il fonctionne sur un modèle de **recettes (recipes)** écrites en Ruby qui décrivent **l'état souhaité** d'une machine.

```
1 # Exemple simplifié d'une recipe Chef
2 package 'apache2' do
3   action :install
4 end
5
6 service 'apache2' do
7   action [:enable, :start]
8 end
9
10 template '/var/www/html/index.html' do
11   source 'index.html.erb'
12 end
```



Tip

Résultat attendu — exécution de `chef-client` :

```
[2024-01-15T09:12:34+00:00] INFO: Starting Chef Infra Client Run
[2024-01-15T09:12:35+00:00] INFO: Installing package apache2
[2024-01-15T09:12:42+00:00] INFO: package[apache2] installed version 2.4.57
[2024-01-15T09:12:43+00:00] INFO: service[apache2] enabled and started
[2024-01-15T09:12:43+00:00] INFO: template[/var/www/html/index.html] created
file
[2024-01-15T09:12:43+00:00] INFO: Chef Infra Client Run complete in 9.123
seconds.
```

Apache est installé, démarré et la page d'accueil est en place. Si vous relancez `chef-client`, rien ne change — c'est l'idempotence en action.

Intérêt : Au lieu de cliquer dans une UI ou d'écrire des scripts bash, vous décrivez le **résultat attendu** (Apache installé, service actif, fichiers à jour). Chef **idempotent** — si vous exécutez la recipe 10 fois, le résultat sera toujours le même.

 [Chef Documentation](#)

Puppet — Configuration Management

Puppet est un outil comparable à Chef, mais avec une approche **déclarative** différente. Au lieu de recettes, Puppet utilise un **langage déclaratif** qui décrit l'état souhaité en termes de ressources. Voici le même objectif (Apache installé et actif) exprimé en syntaxe Puppet :

```
1 # Exemple simplifié de Puppet
2 package { 'apache2':
3     ensure => present,
4 }
5
6 service { 'apache2':
7     ensure => running,
8     enable => true,
9 }
```



Résultat attendu — exécution de `puppet agent --test` :

```
Info: Caching catalog for node01.example.com
Info: Applying configuration version '1705312800'
Notice: /Stage[main]/Main/Package[apache2]/ensure: created
Notice: /Stage[main]/Main/Service[apache2]/ensure: ensure changed 'stopped' to 'running'
Notice: Applied catalog in 8.54 seconds
```

Puppet a appliqué l'état souhaité : `apache2` installé et le service actif. Tout est tracé dans les logs Puppet Master.

Intérêt : Comme Chef, il permet d'automatiser des déploiements complexes à grande échelle.

 [Puppet Documentation](#)

Ansible — Automatisation agentless et modules AWS

Ansible est aujourd'hui l'un des outils d'automatisation les plus utilisés dans les environnements Cloud, notamment en complément de **Terraform** pour la gestion de configuration.

Contrairement à Chef ou Puppet, Ansible est **agentless** : il n'installe rien sur les machines cibles — il se connecte via SSH (Linux) ou WinRM (Windows) et exécute les tâches définies dans des **playbooks** YAML. Voici un playbook typique pour configurer un serveur Apache sur une instance EC2 :

```

1  # Exemple de playbook Ansible – installation Apache sur EC2
2  ---
3  - name: Configurer un serveur web Apache sur EC2
4    hosts: ec2_instances
5    become: yes
6
7    tasks:
8      - name: Installer Apache
9        apt:
10
11          name: apache2
12
13          state: present
14
15
16      - name: Démarrer et activer le service
17
18          service:
19
20              name: apache2
21
22              state: started
23
24              enabled: yes
25
26
27      - name: Copier la page d'accueil
28
29          copy:
30
31              src: index.html
32
33              dest: /var/www/html/index.html

```



Tip

Résultat attendu — `ansible-playbook site.yml -i inventory.ini` :

```
PLAY [Configurer un serveur web Apache sur EC2] *****

TASK [Gathering Facts] *****
ok: [ec2-18-234-56-78.eu-west-3.compute.amazonaws.com]

TASK [Installer Apache] *****
changed: [ec2-18-234-56-78.eu-west-3.compute.amazonaws.com]

TASK [Démarrer et activer le service] *****
changed: [ec2-18-234-56-78.eu-west-3.compute.amazonaws.com]

TASK [Copier la page d'accueil] *****
changed: [ec2-18-234-56-78.eu-west-3.compute.amazonaws.com]

PLAY RECAP *****
ec2-18-234-56-78.eu-west-3.compute.amazonaws.com : ok=4  changed=3
unreachable=0  failed=0
```

Apache est installé et opérationnel sur l'instance EC2. La ligne `changed=3` confirme que les trois tâches ont apporté des modifications. Si vous relancez le playbook, vous verrez `changed=0` — c'est l'idempotence Ansible.

Ansible propose également une **collection dédiée AWS** (`amazon.aws`) permettant de piloter directement les ressources AWS depuis un playbook :

```

1 # Exemple : créer une instance EC2 avec le module Ansible AWS
2 - name: Lancer une instance EC2
3   amazon.aws.ec2_instance:
4     name: "mon-serveur-web"
5     instance_type: t3.micro
6     image_id: ami-0c55b159cbfafa1f0
7     region: eu-west-3
8     key_name: ma-cle-ssh
9     security_groups:
10
11     - sg-xxxxxxxxxx
12
13   tags:
14
15     Env: production
16
17     Owner: formation

```



Tip

Résultat attendu — `ansible-playbook create-ec2.yml` :

```

TASK [Lancer une instance EC2] *****
changed: [localhost]

```

```

PLAY RECAP *****
localhost : ok=1  changed=1  unreachable=0  failed=0

```

```

Instance créée : i-0a1b2c3d4e5f67890
IP publique    : 15.236.142.87
Région         : eu-west-3
Statut         : running

```

L'instance EC2 `mon-serveur-web` est lancée en `eu-west-3`. Elle est tagguée `Env: production` et visible dans la console AWS sous EC2 > Instances. Vous pouvez vous y connecter via SSH : `ssh -i ma-cle-ssh.pem ubuntu@15.236.142.87`.

Intérêt dans un contexte AWS :

- Complémentaire à **Terraform** : Terraform crée l'infrastructure (VPC, EC2, RDS...), Ansible configure les instances après leur lancement.
- Pas d'agent à installer sur les instances EC2 — idéal pour les environnements éphémères.
- Intégration native avec les services AWS via la collection `amazon.aws`.
- Utilisé dans les pipelines CI/CD avec CodePipeline ou Jenkins.

 [Ansible Documentation officielle](#)  [Collection Ansible AWS \(amazon.aws\)](#) 

CloudFormation — Infrastructure as Code AWS-native

Enfin, il existe **AWS CloudFormation**, l'outil **native AWS** pour déployer l'infrastructure via du code YAML/JSON. Ce template déclare deux ressources : une instance EC2 et un bucket S3. AWS les crée dans le bon ordre, sans commande manuelle :

```

1  # Exemple CloudFormation : créer une instance EC2 et un bucket S3
2  AWSTemplateFormatVersion: '2010-09-09'
3  Resources:
4    MyInstance:
5      Type: AWS::EC2::Instance
6      Properties:
7        ImageId: ami-0c55b159cbfafa1f0
8        InstanceType: t2.micro
9
10   MyBucket:
11     Type: AWS::S3::Bucket
12     Properties:
13       BucketName: my-app-bucket

```



Tip

Résultat attendu — après `aws cloudformation deploy --template-file stack.yaml --stack-name ma-stack` :

```
Waiting for changeset to be created...
Waiting for stack create/update to complete...

Successfully created/updated stack - ma-stack

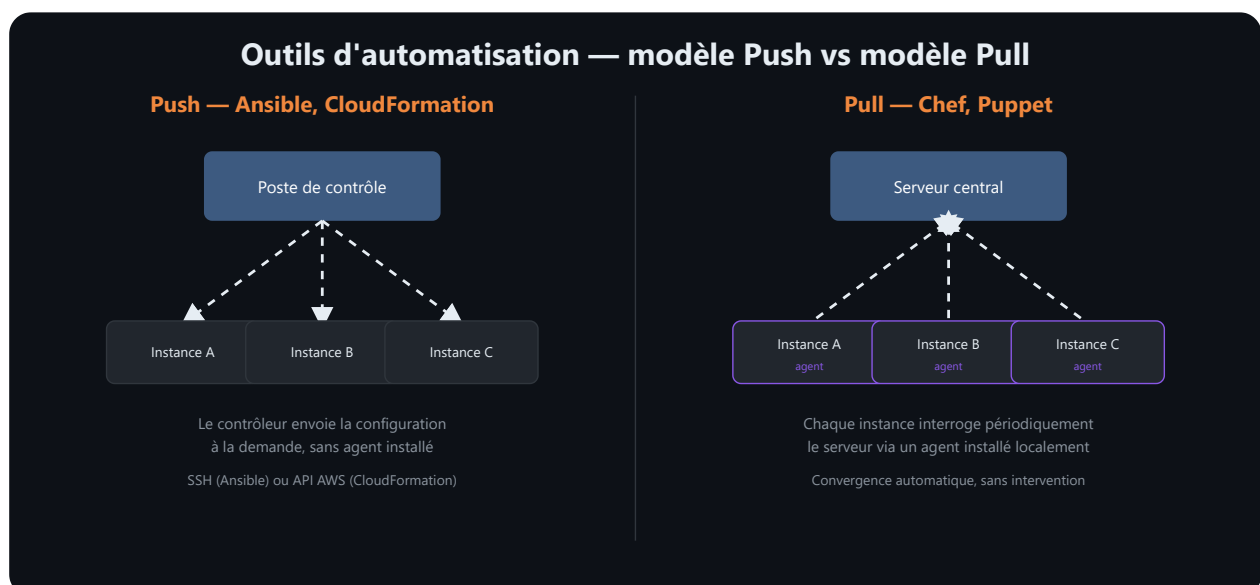
Stack ID   : arn:aws:cloudformation:eu-west-3:123456789012:stack/ma-
stack/abc12345
Status     : CREATE_COMPLETE
Resources  :
  - MyInstance  → i-0abc123def456789  (AWS::EC2::Instance)  CREATE_COMPLETE
  - MyBucket    → my-app-bucket        (AWS::S3::Bucket)     CREATE_COMPLETE
```

Les deux ressources ont été créées par CloudFormation dans le bon ordre. En cas de suppression, `aws cloudformation delete-stack --stack-name ma-stack` supprimera l'EC2 et le bucket ensemble — ce qui garantit qu'il ne reste pas de ressources orphelines.

Intérêt : CloudFormation est intégré nativement à AWS. Vous versionnez votre infrastructure comme du code et pouvez la recréer en quelques clics.

Comparaison simplifiée

Au-delà du langage et du cas d'usage, ces outils se répartissent selon deux architectures fondamentalement différentes — Chef et Puppet fonctionnent en mode Pull (agent installé sur chaque instance), tandis qu'Ansible et CloudFormation fonctionnent en mode Push (aucun agent, contrôle depuis un poste central) :



Cette distinction Push/Pull explique directement pourquoi Ansible et CloudFormation s'intègrent plus naturellement dans un contexte AWS éphémère (instances créées et détruites en continu) : sans agent à installer et maintenir, une nouvelle instance est opérationnelle dès qu'elle est joignable en SSH ou via l'API.

Outil	Modèle	Langage	Cas d'usage	Intégration AWS
Chef	Impératif (recipes)	Ruby	Configurations complexes ou parc historique	Solution Chef maintenue hors OpsWorks
Puppet	Déclaratif	Puppet DSL	Gestion de flotte à grande échelle	Solution Puppet maintenue hors OpsWorks
Ansible	Déclaratif (agentless)	YAML	Config management, post-provisioning	Via collection <code>amazon.aws</code>
CloudFormation	Déclaratif (IaC)	YAML/JSON	Déploiement d'infrastructure AWS	Native, recommandé
Terraform	Déclaratif (IaC)	HCL	Multi-cloud, plus flexible	Via AWS provider

Recommandation pratique :

- Pour **débuter avec AWS**, utiliser **CloudFormation** ou **Terraform**.
- Pour **configurer les instances après déploiement**, utiliser **Ansible** — agentless et très bien intégré à AWS.
- Pour **gérer des configurations** sur des centaines de serveurs, évaluer AWS Systems Manager ou une solution Chef/Puppet maintenue ; ne pas créer de dépendance à OpsWorks.
- Pour **Puppet**, privilégier dans les environnements d'entreprise très structurés.
- En pratique : **Terraform + Ansible** est le duo le plus courant aujourd'hui — Terraform provisionne, Ansible configure.

 [AWS CloudFormation Documentation](#)  [AWS OpsWorks for Chef Automate](#)  [Terraform AWS Provider](#)

10. Bonnes pratiques de démarrage

Info

L'environnement temporaire fourni pendant la formation permet de découvrir la console sans créer de compte AWS personnel.

10.1 Démarrer et sécuriser une session d'atelier

L'atelier fournit un compte, un rôle et des autorisations temporaires. La séquence de démarrage est la suivante :

1. Ouvrir l'atelier indiqué par la formatrice.
2. Cliquer sur **Start Lab** (le bouton de démarrage de l'environnement AWS Academy) et attendre que la console soit disponible.
3. Ouvrir la console avec le bouton **AWS** fourni par l'atelier.
4. Vérifier le rôle et la **région active** avant toute création de ressource.
5. Utiliser **AWS CloudShell** pour les commandes CLI : aucune installation ni configuration locale n'est requise.

Tip

Ce que confirme la commande `aws sts get-caller-identity` dans CloudShell :

```
1 aws sts get-caller-identity
```

```
1 {
2   "UserId": "AROAEXAMPLE:academy-session",
3   "Account": "123456789012",
4   "Arn": "arn:aws:sts::123456789012:assumed-role/LabRole/academy-session"
5 }
```

L'ARN confirme que la session utilise un rôle temporaire de l'atelier. N'exécutez pas `aws configure` et ne copiez jamais d'identifiants sur votre poste.

10.2 Repères pour la navigation dans la console AWS

Voici les éléments essentiels à repérer dans la console pendant la démonstration.

Éléments clés :

1. **Sélecteur de région** : situé en haut à droite, il permet de choisir la région AWS dans laquelle les ressources seront déployées.
2. **Barre de recherche des services** : permet d'accéder rapidement à n'importe quel service AWS (EC2, S3, VPC, IAM, RDS...).
3. **Tableau de bord (Dashboard)** : affiche les services récemment utilisés et les informations de facturation.
4. **Panneau IAM** : permet de gérer les utilisateurs, groupes, rôles et stratégies de sécurité.
5. **Billing & Cost Management** : donne accès au suivi de la consommation et à la facturation.

 [Guide de démarrage AWS Console](#)  [AWS Billing Documentation](#)  [IAM Documentation](#)

Retenons que certaines ressources sont **spécifiques à une région** et que la facturation dépend de cette localisation.

Warning

Piège fréquent — mauvaise région active : Si vous créez des ressources dans une région autre que celle attendue (ex. `us-east-1` au lieu de `eu-west-3`), vous ne les verrez pas dans votre vue habituelle et continuerez à les payer. Vérifiez toujours le sélecteur de région en haut à droite de la console avant toute création de ressource.

11. Points importants et pièges fréquents

Ce chapitre a couvert beaucoup de notions en peu de temps ; le tableau suivant recense les idées reçues les plus fréquentes chez les débutants AWS, pour clarifier une dernière fois ce qui relève du mythe et ce qui relève de la réalité technique.

Piège	Réalité
« AWS, c’est juste des serveurs dans le cloud »	AWS est une plateforme complète couvrant 200+ services — calcul, stockage, réseau, IA, sécurité, DevOps, etc.
« Je suis en sécurité, AWS gère tout »	Non. AWS gère la sécurité <i>du</i> cloud, vous gérez la sécurité <i>dans</i> le cloud (IAM, chiffrement, configuration).
« Le Cloud public n’est pas conforme RGPD »	Faux. AWS est conforme RGPD. C’est votre usage qui doit être conforme — localisation des données, consentement, droit à l’oubli, etc.
« Les coûts AWS sont imprévisibles »	Avec une bonne gouvernance (budgets, alertes, AWS Cost Explorer), les coûts sont très maîtrisables.
« Un conteneur = une VM plus légère »	Non. Un conteneur ne contient pas d’OS complet — il partage le noyau de l’hôte. C’est une architecture radicalement différente.
« Je dois mettre toutes mes données en cloud »	Non. Certaines données peuvent rester on-premise pour des raisons légales, réglementaires ou métier. Le cloud hybride existe pour ça.
« IaaS, PaaS, SaaS — pareil pareil »	Non. Chaque modèle décale les responsabilités . En IaaS, vous gérez plus ; en SaaS, AWS gère presque tout.
« Je peux utiliser n’importe quelle région »	Non. Certaines régions n’ont pas tous les services, et les données sensibles doivent rester dans des zones spécifiques (ex. RGPD en UE).

Comprendre l’infrastructure ne suffit pas — encore faut-il la sécuriser. Le Chapitre 2 est entièrement consacré à la sécurité et à la gestion des identités dans AWS : comment **IAM** structure les droits et les accès, comment appliquer les bonnes pratiques de moindre privilège, et comment mettre en place une gouvernance solide.

Travaux pratiques — Fondamentaux du cloud

Les accès sont remis au début de la formation. Utilisez uniquement l’environnement temporaire attribué et suivez l’énoncé du module, de l’atelier ou du lab sélectionné. Aucun compte AWS personnel n’est nécessaire.

AWS Academy — Cloud Foundations

Type	Référence	Intitulé
Module	Module 1	Présentation des concepts du cloud
Module	Module 2	Coûts du cloud et facturation
Module	Module 3	Présentation de l'infrastructure mondiale AWS
Contrôle	Modules 1 à 3	Contrôle des connaissances associé à chaque module

AWS Academy — Cloud Architecting

Type	Référence	Intitulé
Module	Module 1	Bienvenue dans le cours AWS Academy Cloud Architecting
Module	Module 2	Présentation de la conception d'architectures cloud

Go Deploy

Lab	Intitulé
Lab 1	Découvertes console AWS

► Parcours guidé et validation

Ressources

Documentation officielle AWS

Pour approfondir les notions abordées dans ce chapitre, voici les références officielles AWS correspondantes :

- [AWS Documentation](#) 
- [AWS Well-Architected Framework](#) 
- [AWS Free Tier](#) 
- [Régions et zones de disponibilité AWS](#) 

Le quiz ci-dessous permet de vérifier l'acquisition des notions clés de ce chapitre avant de passer au suivant.

Quiz interactif du chapitre

Choisissez une réponse : la correction expliquée apparaît immédiatement. Les propositions changent d'ordre à chaque nouvelle tentative.

Le quiz interactif reste disponible dans la version web du support.

Mini-jeu de synthèse — AWS Quest

Résolvez cinq décisions factuelles pour consolider les fondamentaux avant d'ouvrir le chapitre suivant. Le jeu est une simulation locale : il ne se connecte à aucun compte AWS.

Le quiz interactif reste disponible dans la version web du support.

Info

Le chapitre précédent a posé les bases théoriques du Cloud Computing et présenté l'écosystème AWS ; ce chapitre entre dans le concret en abordant le premier pilier opérationnel de toute architecture AWS : la sécurité des accès.

Objectifs du chapitre

À l'issue de ce chapitre, les stagiaires seront capables de :

- **Expliquer** le rôle stratégique d'IAM (users, groups, roles, policies) et rédiger une politique JSON
- **Appliquer** le modèle de responsabilité partagée et sécuriser un compte avec MFA et politiques conditionnelles
- **Utiliser** AWS STS et mettre en œuvre une fédération d'identité (IAM Identity Center, SAML, AD FS, Cognito)
- **Concevoir** une stratégie multi-comptes avec AWS Organizations et des Service Control Policies
- **Configurer** la traçabilité (CloudTrail/Config) et créer via CLI utilisateurs, groupes, rôles et policies

1. Introduction à IAM : Identity and Access Management

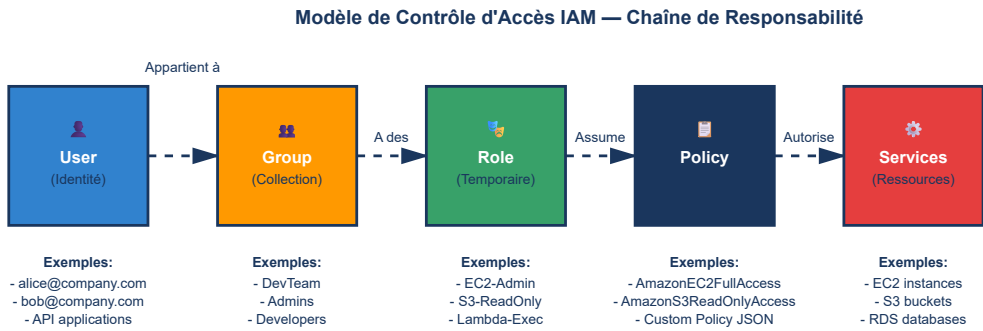
1.1 Définition et rôle stratégique

IAM (Identity and Access Management) est le service AWS qui permet de **gérer les identités, les permissions et les politiques d'accès** aux ressources AWS.

IAM est à la sécurité ce que la serrure est à une porte. Il constitue **le cœur de la gouvernance des accès** dans un environnement AWS.

- IAM contrôle **qui** peut faire **quoi** sur **quelle ressource**, et **dans quelles conditions**.
- Il permet de sécuriser les accès au niveau le plus granulaire possible.
- Il est un prérequis à toute architecture bien conçue sur AWS.

IAM fonctionne comme un contrôleur central placé entre deux mondes : d'un côté les **identités** (utilisateurs, groupes, rôles, services AWS), de l'autre les **ressources AWS** qu'elles cherchent à atteindre (S3, EC2, RDS, Lambda...). Chaque requête suit le même chemin : une identité émet un appel API, IAM évalue les **politiques JSON** qui lui sont associées, puis autorise (**Allow**) ou refuse (**Deny**) l'accès à la ressource visée.



Ce schéma résume le principe : aucune ressource AWS n'est jamais contactée directement par une identité sans passer par cette évaluation IAM — c'est ce mécanisme, invisible mais systématique, qui rend possible le contrôle d'accès au niveau le plus granulaire.

[Documentation officielle AWS IAM](#)

1.2 Concepts fondamentaux d'IAM

Utilisateur IAM

Un **utilisateur IAM** représente une identité permanente dans AWS, associée à des identifiants (login/mot de passe ou clés d'accès). Il est utilisé pour représenter une personne ou une application qui interagit avec les services AWS.

C'est comme un badge nominatif d'employé : il donne un accès personnel et identifiable au système.

Groupe IAM

Un **groupe IAM** est une collection logique d'utilisateurs qui partagent les mêmes permissions. Il permet d'appliquer des politiques communes à plusieurs utilisateurs, comme les membres d'une équipe Dev, les administrateurs ou les lecteurs.

C'est comme un département dans une entreprise (ex. IT, Marketing, Finance) : tous les membres ont les mêmes règles d'accès.

Rôle IAM

Un **rôle IAM** est une identité temporaire que l'on peut assumer pour accéder à des ressources AWS. Il est utilisé par des services AWS (comme EC2 ou Lambda), ou pour permettre l'accès entre comptes ou via une fédération d'identités.

C'est comme un badge visiteur temporaire : il donne un accès limité dans le temps à certaines zones.

Politique IAM

Une **politique IAM** est un document JSON qui définit précisément les permissions accordées ou refusées. Elle permet de contrôler les actions autorisées sur les ressources AWS.

C'est comme un règlement intérieur : il définit ce qui est autorisé ou interdit pour chaque profil.

MFA (Multi-Factor Authentication)

Le **MFA** ajoute une couche de sécurité en exigeant un second facteur d'authentification, comme un code temporaire ou une clé physique, en plus du mot de passe.

C'est comme une double serrure : il faut une clé et un code pour ouvrir la porte.

STS (Security Token Service)

Le **STS** permet de générer des identifiants temporaires pour accéder à AWS, avec une durée limitée (15 minutes à 12 heures). Il est idéal pour déléguer des accès sans exposer de credentials permanents.

C'est comme un badge temporaire valable quelques heures : il expire automatiquement après usage.

Sans STS, chaque utilisateur devrait avoir des identifiants IAM permanents dans chaque compte AWS, avec des permissions fixes. Cela rendrait la gestion des accès lourde, risquée, et peu compatible avec les standards

modernes d'authentification.

 Danger

Ne jamais utiliser le compte root pour les opérations courantes. Le compte root AWS dispose de tous les droits sans restriction — il n'est pas soumis aux politiques IAM. Toute compromission du root expose l'intégralité du compte. Créez immédiatement un utilisateur IAM administrateur, activez MFA sur le root, puis verrouillez les credentials root.

1.3 Modèle de responsabilité partagée appliqué à la sécurité

Observons le modèle de responsabilité partagée appliqué à la sécurité.

Élément	Responsabilité AWS	Responsabilité Client
Infrastructure physique	✓	✗
Réseau global	✓	✗
Contrôle d'accès utilisateur	✗	✓
Stratégies IAM	✗	✓
Configuration du chiffrement	✗	✓
Gestion des identités externes	✗	✓

Ce tableau résume la philosophie d’AWS :

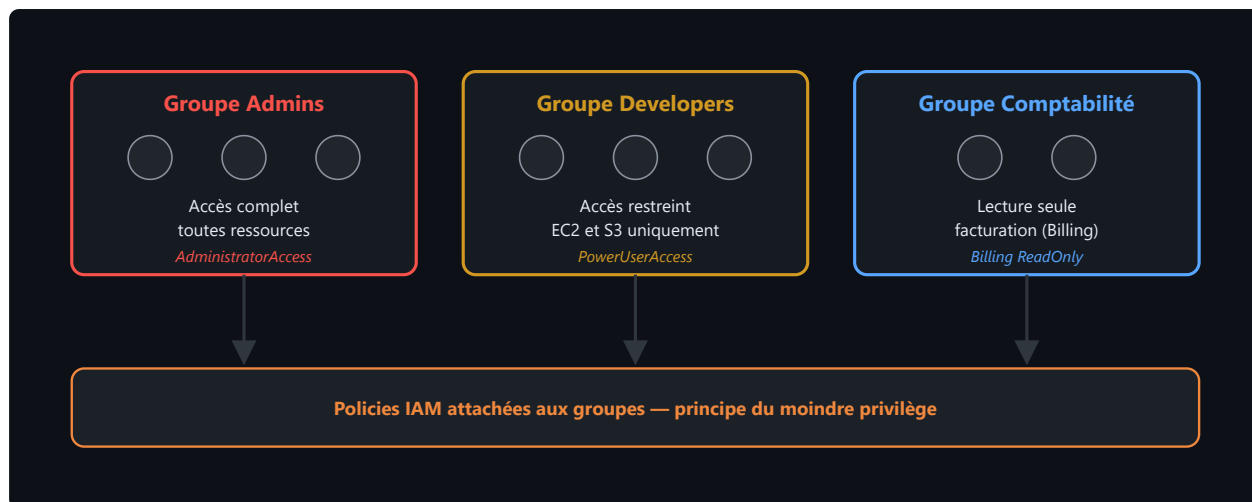
« AWS sécurise le cloud, vous sécurisez ce que vous y déployez ».

Il est important de rappeler qu’IAM ne se limite pas à la gestion d'utilisateurs. Il s'agit d'un **système d'autorisation distribué** qui s'applique aussi aux services AWS eux-mêmes, aux rôles inter-comptes et aux identités fédérées.

1.4 Exemple concret d'organisation des accès

Un administrateur met en place une **stratégie de sécurité structurée** :

- Groupe **Admins** → droits complets sur le compte AWS.
- Groupe **Developers** → accès restreint à **EC2** et **S3**.
- Groupe **Comptabilité** → lecture seule sur la facturation (Billing).



Chaque groupe reçoit une **policy JSON adaptée**, garantissant le **principe du moindre privilège** — le détail de la syntaxe JSON de ces politiques est développé juste après, en §1.5.

1.5 Structure d'une politique IAM — Exemple concret

Policy : Lecture seule sur S3

Cette politique illustre la syntaxe de base d'une policy IAM. Notez les quatre champs obligatoires : **Version**, **Statement**, **Effect**, **Action**, et **Resource**.

```

1  {
2    "Version": "2012-10-17",
3    "Statement": [
4      {
5        "Effect": "Allow",
6        "Action": [
7          "s3:GetObject",
8          "s3:ListBucket"
9        ],
10       "Resource": "*"
11     }
12   ]
13 }

```

Cette policy donne aux membres du groupe uniquement la possibilité de **lister et lire les objets S3**.

Policy : Accès restreint à un bucket spécifique

Pour restreindre l'accès à **un bucket précis** (et non à tout S3), on remplace `"Resource": "*"` par l'ARN du bucket cible.

```

1  {
2    "Version": "2012-10-17",
3    "Statement": [
4      {
5        "Effect": "Allow",
6        "Action": [
7          "s3:GetObject",
8          "s3:ListBucket"
9        ],
10       "Resource": [
11         "arn:aws:s3:::mon-bucket-secret",
12         "arn:aws:s3:::mon-bucket-secret/*"
13       ]
14     }
15   ]
16 }

```

Cette policy autorise la lecture d'objets S3 dans le bucket `mon-bucket-secret`, **mais uniquement si l'utilisateur est connecté depuis une IP comprise dans `203.0.113.0/24`**.

1.6 IAM Policy Evaluation Logic

Le **IAM Policy Evaluation Logic** est le **mécanisme interne d'AWS** qui détermine **si une action est autorisée ou refusée** lorsqu'un utilisateur ou un rôle tente d'accéder à une ressource AWS (ex : S3, EC2, RDS...).

En d'autres termes : **c'est la logique de décision** qu'AWS applique pour savoir si une requête doit être acceptée ou rejetée.

Il est utilisé à **chaque fois qu'un utilisateur, un rôle ou un service** tente d'effectuer une action sur une ressource AWS.

Exemples :

- Un utilisateur veut lire un fichier dans S3 → AWS vérifie les politiques IAM
- Un rôle Lambda veut écrire dans DynamoDB → AWS vérifie les politiques IAM
- Un service EC2 veut accéder à un secret → AWS vérifie les politiques IAM

AWS suit **trois règles fondamentales**, dans cet ordre :

1. **Deny explicite** → priorité absolue

Si une politique dit “**Effect**”: “**Deny**”, l’accès est **refusé**, même si une autre politique dit “Allow”.

2. **Allow explicite** → si aucun Deny

Si une politique dit “**Effect**”: “**Allow**”, et qu’il n’y a pas de Deny, l’accès est **autorisé**.

3. **Pas de politique = refus implicite**

Si aucune politique ne couvre l’action demandée, l’accès est **refusé par défaut**.

 [Policy Evaluation Logic](#)

 [Policy Simulator](#)

 Warning

Principe du moindre privilège (Least Privilege). Ne jamais attribuer `"Action": "*" ou "Resource": "*"` en production. Accordez uniquement les permissions strictement nécessaires à la tâche. En cas de doute, commencez par un accès minimal et élargissez progressivement selon les besoins réels.

1.7 Services IAM complémentaires

IAM ne fonctionne pas isolément : plusieurs autres services AWS s’appuient sur lui ou l’étendent pour couvrir des besoins spécifiques (multi-comptes, identifiants temporaires, utilisateurs finaux, authentification unique) que ce chapitre détaille dans les sections suivantes.

Service	Rôle
AWS Organizations	Gestion centralisée de plusieurs comptes AWS (OU, SCP, facturation consolidée).
AWS STS (Security Token Service)	Génère des identifiants temporaires sécurisés.
Amazon Cognito	Gestion d'utilisateurs finaux (authentification applicative).
IAM Identity Center (ex-SSO)	Authentification unique (SSO) sur plusieurs comptes et applications.

Retenez surtout la distinction entre Cognito (utilisateurs finaux d'une application) et IAM Identity Center (employés d'une organisation accédant à plusieurs comptes AWS) — une confusion fréquente entre ces deux services.

 [AWS Organizations](#) 

 [STS Documentation](#) 

 [Amazon Cognito](#) 

 [IAM Identity Center](#) 

2. Sécuriser les accès IAM avec MFA et politiques conditionnelles

2.1 Pourquoi MFA et politiques conditionnelles ?

Dans tout système d'information, la sécurité ne repose pas seulement sur **qui est connecté**, mais aussi sur **comment** cette connexion est sécurisée.

Sur AWS, l'**authentification multifacteur (MFA)** est une mesure essentielle pour réduire les risques liés aux accès non autorisés — en particulier sur les comptes disposant de privilèges élevés (root, admin, DevOps...).

L'activation de MFA fait partie des **bonnes pratiques fondamentales** recommandées par AWS dès la création d'un compte.

Mais MFA n'est pas seulement un bouton à cocher : combinée aux **politiques conditionnelles IAM**, elle permet de mettre en place une **sécurité contextuelle et granulaire**.

 [AWS MFA - Documentation](#) 

2.2 Comprendre MFA

Définition

MFA (Multi-Factor Authentication) est un mécanisme de sécurité qui nécessite **au moins deux méthodes d'authentification indépendantes** :

1. **Quelque chose que vous savez** → mot de passe, clé d'accès.
2. **Quelque chose que vous possédez** → application MFA (ex : Authenticator) ou clé physique (YubiKey).
3. (Optionnel) **Quelque chose que vous êtes** → biométrie.

Dans AWS, la MFA s'applique :

- au compte **root**,
- aux **utilisateurs IAM**,
- aux **rôles** via AWS CLI ou API,
- aux **accès fédérés**.

Le type de facteur utilisé pour prouver son identité varie cependant selon les moyens disponibles et le niveau de sécurité recherché.

Types de MFA supportés

AWS accepte plusieurs familles de facteurs MFA, du plus répandu (l'application mobile) au plus robuste (la clé physique) :

Type	Description	Exemples
MFA virtuelle	Application mobile (TOTP)	Google Authenticator, Authy, Microsoft Authenticator
MFA matérielle	Clé physique dédiée	YubiKey, Gemalto
Passkey / FIDO2	Authentification sans mot de passe	Clé biométrique ou physique compatible FIDO

Gemalto est un fabricant (aujourd’hui filiale de Thales) de clés de sécurité physiques concurrentes de YubiKey — les deux fonctionnent selon le même principe : un petit boîtier USB ou NFC à brancher/approcher pour valider la connexion, sans code à recopier. **FIDO2** (Fast IDentity Online 2) est le standard ouvert sur lequel reposent ces clés biométriques ou physiques : il définit comment le navigateur, l’appareil et le service en ligne dialoguent pour vérifier votre identité sans jamais transmettre de mot de passe — c’est la même technologie qui permet de se connecter avec son empreinte digitale ou Face ID sur un site compatible.

2.3 Bonnes pratiques AWS sur MFA

Au-delà de l’activation de base, quelques pratiques permettent de généraliser et de vérifier en continu le recours au MFA plutôt que de le laisser au bon vouloir de chaque utilisateur :

- Toujours activer MFA sur le **compte root** (obligatoire en production).
- Exiger MFA pour tous les utilisateurs **ayant des privilèges élevés**.
- Automatiser la vérification de MFA via **AWS Config** ou **Security Hub**.
- Interdire les actions sensibles (ex : suppression d’instances, modification de politiques) sans MFA.

Ces quatre pratiques passent d’une MFA “recommandée” à une MFA réellement appliquée et contrôlée, ce qui est la seule façon de garantir qu’aucun compte à privilèges n’y échappe.

 [AWS Security Best Practices](#)

Danger

MFA obligatoire sur le compte root et tous les comptes administrateurs. Une connexion sans MFA sur un compte à privilèges élevés est la principale cause de compromission de comptes AWS. AWS Security Hub et IAM Access Analyzer peuvent détecter automatiquement les comptes sans MFA et générer des alertes.

2.4 Audit de MFA avec AWS Config

AWS Config permet de vérifier automatiquement que les utilisateurs IAM ont bien activé MFA.

Exemple de règle : `iam-user-mfa-enabled`

 [AWS Config Rules](#) 

2.5 Politiques conditionnelles IAM

Une **policy conditionnelle** permet d'ajouter des **règles contextuelles** à une politique IAM classique.

Cela permet d'aller **au-delà des permissions statiques** en intégrant des critères comme :

- la présence ou non de MFA,
- l'adresse IP source,
- la région AWS,
- l'heure,
- le VPC Endpoint utilisé.

La clé `Condition` dans une policy JSON contrôle ces règles.

2.6 Exemples de politiques conditionnelles

Exiger MFA pour les actions sensibles

La clé `Condition` permet d'ajouter des critères supplémentaires à une policy. Voici comment bloquer **toutes les actions** si l'utilisateur n'a pas activé MFA pour sa session en cours :

```

1  {
2    "Version": "2012-10-17",
3    "Statement": [
4      {
5        "Effect": "Deny",
6        "Action": "*",
7        "Resource": "*",
8        "Condition": {
9          "BoolIfExists": {
10            "aws:MultiFactorAuthPresent": "false"
11          }
12        }
13      }
14    ]
15  }

```

Cette policy **refuse toute action** si MFA n'est pas activée pour la session.

Elle peut être attachée à :

- un groupe d'administrateurs,
- un utilisateur,
- un rôle IAM.

Remarque : cette approche est **préférée** à un simple "Allow MFA" car elle couvre tous les cas par défaut.

Restreindre les accès à une IP spécifique

Pour n'autoriser les connexions que depuis un réseau d'entreprise (VPN ou bureau), on utilise la condition

`aws:SourceIp`. Ici, `NotIpAddress` signifie "si l'IP n'est PAS dans cette plage, refuser" :

```

1  {
2    "Version": "2012-10-17",
3    "Statement": [
4      {
5        "Effect": "Deny",
6        "Action": "*",
7        "Resource": "*",
8        "Condition": {
9          "NotIpAddress": {
10             "aws:SourceIp": "203.0.113.0/24"
11          }
12        }
13      }
14    ]
15  }

```

Cette policy **refuse toute action** si la connexion ne provient pas de l'adresse IP autorisée.

Combinée à MFA, elle renforce la **sécurité d'accès des administrateurs**.

Politique combinée IP + MFA

Les conditions se combinent avec un **ET logique** : toutes doivent être vraies pour que l'accès soit autorisé. C'est la politique la plus stricte — idéale pour les comptes administrateurs :

```

1  {
2    "Version": "2012-10-17",
3    "Statement": [
4      {
5        "Effect": "Deny",
6        "Action": "*",
7        "Resource": "*",
8        "Condition": {
9          "BoolIfExists": {
10             "aws:MultiFactorAuthPresent": "false"
11
12
13          },
14          "NotIpAddress": {
15             "aws:SourceIp": "203.0.113.0/24"
16
17          }
18        }
19      }
20    ]
21  }

```

Ici, l'accès n'est autorisé que si :

- l'utilisateur se connecte **depuis l'IP autorisée** ET
- **MFA est activée.**

 [IAM Policy Elements - Condition](#)

2.7 Bonnes pratiques et points d'attention

Pour conclure cette section MFA, quelques rappels permettent d'éviter les erreurs les plus fréquentes constatées en audit de sécurité :

- MFA ne remplace pas les bonnes politiques IAM, elle **les renforce**.
- Toujours combiner MFA avec une politique conditionnelle.
- Documenter les exceptions éventuelles (services automatisés, CI/CD).
- Tester systématiquement les policies dans **Policy Simulator** avant déploiement.

Le point le plus souvent oublié est la documentation des exceptions : sans elle, un compte de service exempté de MFA pour des raisons techniques légitimes finit par ressembler, lors d'un audit, à une faille de sécurité non traitée.

2.8 STS (Security Token Service) — Identity Broker en détail

Qu'est-ce qu'une identité temporaire ?

Jusqu'à présent, nous avons parlé d'**identités permanentes** :

- Un utilisateur IAM a un **login/mot de passe** et des **clés d'accès** permanents.
- Un rôle IAM reçoit des permissions via une policy.

Mais dans une architecture moderne, il est souvent risqué de **distribuer des credentials permanents**. C'est où intervient **AWS STS (Security Token Service)**.

STS génère des identifiants temporaires (valides quelques minutes à quelques heures), qui expirent automatiquement sans intervention manuelle.

C'est comme un badge visiteur qui se détruit après 24 heures : il n'y a rien à récupérer après son expiration.

Comment fonctionne STS ?

Le processus est simple :

1. Un utilisateur ou service demande un token STS.
2. STS valide la demande (qui êtes-vous ? avez-vous le droit ?).
3. STS émet un **set de credentials temporaires** : `AccessKeyId`, `SecretAccessKey`, et `SessionToken`.
4. Ces credentials permettent d'accéder à AWS pendant leur durée de validité.

5. À l'expiration, les credentials deviennent inutilisables.

Ce mécanisme générique de STS prend tout son sens lorsqu'il est combiné à un système d'authentification externe déjà en place dans l'entreprise, via ce qu'on appelle un identity broker.

STS Identity Broker — Cas d'usage concret

Un **Identity Broker** est un système qui :

1. Authentifie un utilisateur via un système externe (AD, LDAP, SSO, application custom).
2. Contacte STS pour obtenir des credentials AWS temporaires.
3. Retourne ces credentials à l'utilisateur.

Cela permet aux utilisateurs d'accéder à AWS **sans avoir de compte IAM permanent**, et sans exposer d'accès clés stockés durablement.

Exemple : Intégration avec un système LDAP corporate

Une entreprise utilise **LDAP interne** pour gérer ses employés. Elle souhaite que ces employés accèdent à AWS **sans créer de comptes IAM**.

Approche sans STS (mauvaise) :

- Créer un compte IAM pour chaque employé.
- Distribuer des clés d'accès à chacun.
- Gérer les mots de passe dans deux systèmes (LDAP + IAM).
- Déploiement manuel, complexe, peu sécurisé.

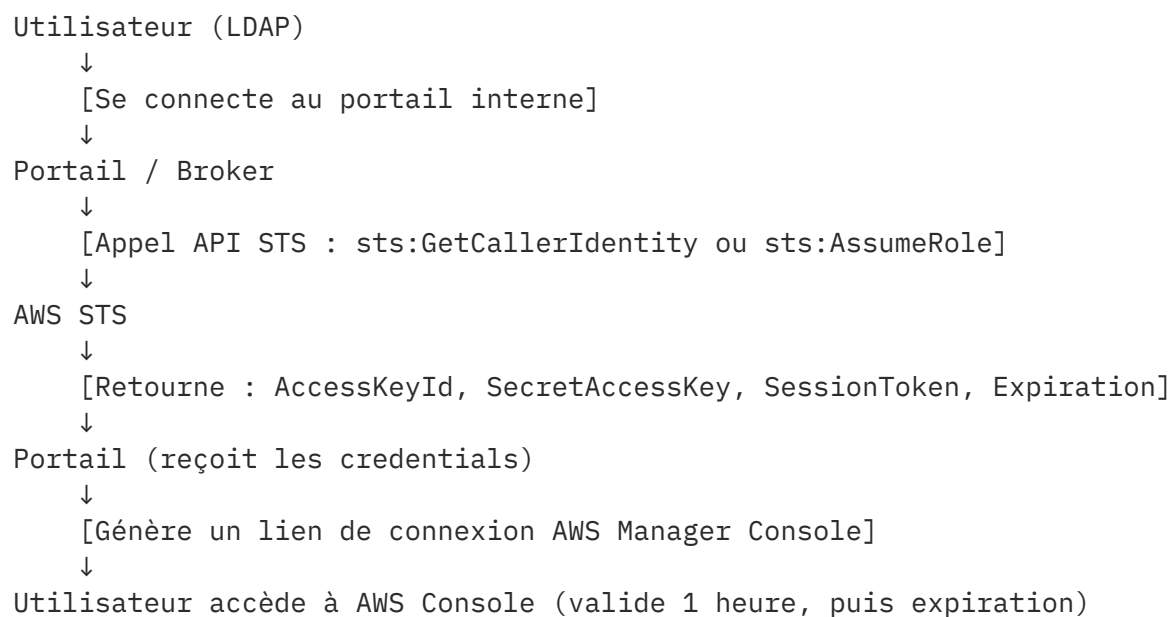
Approche avec STS Identity Broker (correcte) :

1. L'employé se connecte à un **portail interne** avec ses identifiants LDAP.
2. Le portail authentifie l'utilisateur contre LDAP.
3. Le portail contacte STS via une API : « *Cet utilisateur veut un token AWS* ».
4. STS retourne des credentials temporaires.
5. Le portail affiche à l'employé un **lien direct vers la console AWS** avec ces credentials.
6. L'employé clique, se connecte à AWS, et accède aux ressources autorisées.
7. **Aucun compte IAM permanent n'a été créé.**

Cette séquence côté fonctionnel se traduit, techniquement, par l'enchaînement d'appels suivant entre le portail et STS.

Flux technique de STS

Le schéma ci-dessous reprend l'exemple précédent sous forme de flux technique, en explicitant à quel moment intervient chaque appel API :



Ce flux illustre l'appel `AssumeRole`, mais ce n'est qu'une des opérations que STS expose selon le contexte d'authentification.

Types de requêtes STS courantes

Ce flux ne concernait que le scénario `AssumeRole` d'un identity broker ; STS expose en réalité plusieurs opérations selon le contexte d'authentification, résumées ci-dessous.

Opération STS	Cas d'usage	Durée de validité
<code>GetCallerIdentity</code>	Vérifier qui vous êtes	N/A (détection)
<code>AssumeRole</code>	Un utilisateur IAM/fédéré assume un rôle	1 heure (configurable)
<code>GetSessionToken</code>	Obtenir des credentials temporaires pour l'utilisateur courant	1 heure à 36 heures
<code>AssumeRoleWithSAML</code>	Assumer un rôle via assertion SAML (fédération)	1 heure
<code>AssumeRoleWithWebIdentity</code>	Assumer un rôle via token JWT (OAuth/OIDC)	Configurable

Les deux dernières opérations sont celles qui rendent la fédération possible : `AssumeRoleWithSAML` s'utilise avec un IdP d'entreprise (Active Directory, Okta), tandis que `AssumeRoleWithWebIdentity` s'utilise avec un fournisseur OAuth/OIDC grand public (Google, Facebook) ou un IdP compatible OIDC.

Avantages de STS Identity Broker

Au-delà du cas d'usage LDAP déjà présenté, l'utilisation d'un identity broker basé sur STS apporte des bénéfices qui s'appliquent à tout scénario de fédération :

- **Zéro compte IAM permanent** pour les utilisateurs fédérés.
- **Credentials automatiquement révoquées** à l'expiration.
- **Audit centralisé** via CloudTrail (qui a demandé un token, quand, d'où).
- **Intégration facile** avec les systèmes legacy (LDAP, SAP, Salesforce...).
- **MFA possible** au niveau du broker.

Le point sur l'audit centralisé est souvent sous-estimé : contrairement à des clés d'accès permanentes partagées, chaque appel `AssumeRole` est individuellement journalisé dans CloudTrail avec l'identité de l'appelant d'origine, ce qui facilite grandement les investigations post-incident.

Exemple de code (Python) — Simple Identity Broker

Le script suivant reprend, en Python, la logique du broker décrite plus haut : authentification externe simulée, puis appel `assume_role` pour obtenir des credentials temporaires.


```

1  import boto3
2  import sys
3
4  # Client STS
5  sts_client = boto3.client('sts')
6
7  # 1. Un utilisateur LDAP s'authentifie (simulé ici)
8  user = "alice"
9  role_arn = "arn:aws:iam::123456789012:role/FederatedUserRole"
10
11
12  try:
13
14      # 2. Appel STS pour obtenir credentials temporaires
15
16      response = sts_client.assume_role(
17
18          RoleArn=role_arn,
19
20          RoleSessionName=f"{user}-session",
21
22          DurationSeconds=3600 # 1 heure
23
24      )
25
26
27      # 3. Extraire les credentials
28
29      credentials = response['Credentials']
30
31
32
33      print(f"AccessKeyId: {credentials['AccessKeyId']}")
34
35      print("SecretAccessKey: <masquée volontairement>")

```

```

2
4     print(f"SessionToken: {credentials['SessionToken']}")
2
5     print(f"Expiration: {credentials['Expiration']}")
2
6
2
7 except Exception as e:
2
8     print(f"Erreur STS : {e}")

```



Tip

Résultat attendu — exécution du script Python STS :

```

AccessKeyId: ASIA4EXAMPLESTSTEMP
SecretAccessKey: <masquée volontairement>
SessionToken: AQoDYXdzEJr////////wEaoAK...HgC=
Expiration: 2024-01-15 11:40:00+00:00

```

Les credentials temporaires STS se distinguent par leur **AccessKeyId** commençant par **ASIA** (vs **AKIA** pour les clés permanentes). Ils expirent automatiquement à l'heure indiquée — aucune révocation manuelle nécessaire.

Ce code illustre comment un **broker** peut obtenir et distribuer des credentials temporaires.

 [AssumeRole API](#)

 [Building a custom identity broker](#)

Décision de sécurité — rôles IAM et STS

Une application EC2 doit lire un seul bucket S3. Faut-il placer des clés d'accès dans un fichier de configuration ?

► [Afficher la réponse raisonnée](#)

3. Fédération d'identité et SSO avec IAM Identity Center

3.1 Introduction à la fédération d'identité

Lorsque les entreprises grandissent, elles disposent souvent de **systèmes d'authentification déjà en place** : annuaire Active Directory, LDAP, IdP externe comme Okta ou Azure AD...

Dans ce contexte, **créer des comptes IAM manuellement pour chaque utilisateur n'est ni scalable ni sécurisé**.

C'est là qu'intervient la **fédération d'identité**, un mécanisme permettant de déléguer l'authentification à un fournisseur d'identité existant.

Sur AWS, cette fédération est gérée via **IAM Identity Center** (anciennement AWS SSO) ou via des **rôles fédérés SAML**.

Grâce à cette approche :

- les utilisateurs conservent **leurs identifiants d'entreprise**,
- aucune **gestion manuelle des mots de passe** dans AWS,
- les administrateurs appliquent des **règles centralisées de sécurité**.

 [Documentation IAM Identity Center](#)

3.2 IAM Identity Center vs IAM classique — Clarification pour débutant

Beaucoup de stagiaires confondent ces deux concepts. Voici la différence **essentielle** :

Critère	IAM classique	IAM Identity Center
Pour qui ?	Comptes AWS individuels (admins, DevOps locaux)	Organisations / Entreprises avec plusieurs comptes AWS
Authentification	Login/mot de passe IAM natif	Fédération (AD, Azure AD, Okta...) ou gestion d'utilisateurs centralisée
Gestion centralisée	Non — chaque compte gère ses propres utilisateurs	Oui — un seul endroit pour gérer tous les accès
Multi-comptes	Difficile à gérer manuellement	Natif — accès simple entre comptes
MFA	À configurer par utilisateur	Automatisé, appliqué par l'IdP
Cas d'usage	Petit équipe, POC, environnement de test	Production, conformité, grande entreprise
Exemple	Un développeur solo crée un compte IAM pour lui	Une PME avec 5 comptes AWS (Prod, Dev, Finance, etc.)

En pratique :

- Petite équipe ? → IAM classique suffit.
- Entreprise avec AD ? → IAM Identity Center connecté à l'AD.
- Application web avec utilisateurs finaux ? → Cognito (section 5).

Avant d'aller plus loin dans la mise en œuvre technique de la fédération, clarifions le vocabulaire employé.

3.3 Définitions clés

La section précédente a introduit plusieurs termes techniques propres à la fédération d'identité ; ce tableau les regroupe pour s'y référer facilement dans la suite du chapitre.

Terme	Définition
Fédération d'identité	Mécanisme qui permet à des utilisateurs authentifiés par un fournisseur d'identité externe d'accéder à AWS sans compte IAM natif.
IdP (Identity Provider)	Service qui authentifie les utilisateurs (AD FS, Azure AD, Okta, PingIdentity...).
SP (Service Provider)	Service AWS qui consomme l'identité validée par l'IdP.
SAML (Security Assertion Markup Language)	Protocole standard pour la fédération d'identité entre systèmes d'entreprise et cloud.
OAuth / OIDC (OpenID Connect)	Protocole moderne de délégation d'accès, souvent utilisé pour les applications web et mobiles.
IAM Identity Center	Service AWS permettant de gérer l'accès fédéré et le Single Sign-On (SSO).

Retenez en particulier la paire IdP/SP : AWS joue toujours le rôle de fournisseur de service (SP) qui fait confiance à un IdP externe — jamais l'inverse.

3.4 Architecture de la fédération d'identité avec AWS

1. L'utilisateur s'authentifie via l'IdP de l'entreprise (par exemple, Active Directory).
2. L'IdP émet une **assertion SAML** ou un **token OIDC**.
3. AWS IAM Identity Center valide cette assertion.
4. Un **rôle IAM fédéré** est attribué à l'utilisateur.
5. L'utilisateur accède à la console ou à l'API AWS selon ses permissions.

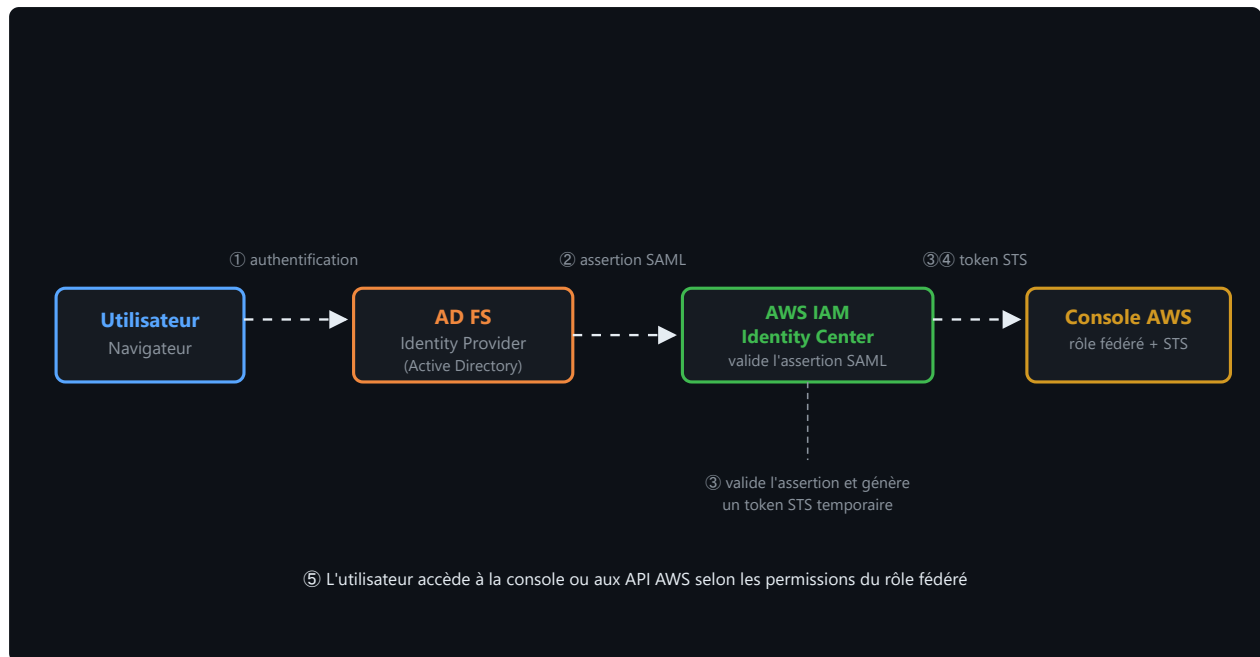
Le schéma détaillé de ce flux, avec l'implémentation concrète AD FS, est présenté juste après en §3.5.

3.5 Active Directory Federation Services (AD FS) et SAML — Implémentation pratique

AD FS est un service **Microsoft** qui joue le rôle d'**Identity Provider (IdP)** pour les entreprises utilisant **Windows Server Active Directory**.

Pour une entreprise ayant un **AD local** ou **Azure AD**, AD FS permet de créer un **pont de confiance** vers AWS via le protocole **SAML**.

Architecture générale : AD FS → AWS IAM



Étapes de mise en place (Vue d'ensemble pour débutant)

1. Configurer AD FS comme provider SAML

- En entreprise, l'administrateur AD crée une "application cloud" dans AD FS.
- Cette application est configurée pour accepter les demandes de connexion AWS.

2. Créer un fournisseur d'identité SAML dans AWS IAM

- Console AWS → IAM → Identity Providers → Create Provider.
- Type : SAML.
- Télécharger les **métadonnées SAML** d'AD FS (fichier XML).

3. Créer un rôle IAM fédéré

- Console AWS → IAM → Roles → Create Role.
- Type de confiance : **Federated Provider** (SAML).
- Sélectionner le provider créé à l'étape 2.
- Attacher des politiques (ex. : `AmazonS3ReadOnlyAccess` pour les développeurs).

4. Mapper les groupes AD aux rôles IAM

- Les groupes AD (ex. `AWS-Developers`, `AWS-Admins`) sont mappés à des rôles IAM.
- Quand un employé du groupe `AWS-Developers` se connecte, il reçoit automatiquement le rôle associé.

5. Distribuer le lien SSO aux utilisateurs

- Les utilisateurs reçoivent un **lien unique** (ex. : `https://monentreprise.com/adfs/ls/idpinitiatedsignon.aspx`).
- En cliquant, ils sont authentifiés contre l'AD et automatiquement connectés à AWS.

Exemple de Trust Policy SAML (pour IAM Role)

Le rôle IAM créé à l'étape 3 doit être associé à une trust policy qui autorise spécifiquement le provider SAML configuré à l'étape 2 à l'assumer :

```
1  {
2    "Version": "2012-10-17",
3    "Statement": [
4      {
5        "Effect": "Allow",
6        "Principal": {
7          "Federated": "arn:aws:iam::123456789012:saml-provider/ADFS"
8        },
9        "Action": "sts:AssumeRoleWithSAML",
10
11       "Condition": {
12
13         "StringEquals": {
14
15           "SAML:aud": "https://signin.aws.amazon.com/saml"
16
17         }
18       }
19     ]
20  }
```

Explication :

- **Principal** : indique qui peut assumer ce rôle (ici, le provider SAML **ADFS**).
- **Action** : l'action autorisée (**AssumeRoleWithSAML** pour les assertions SAML).
- **Condition** : vérification que la demande vient réellement d'AWS (protection).

Cette trust policy est la seule pièce technique à écrire manuellement dans toute cette architecture ; le reste (mapping des groupes, distribution du lien SSO) relève de la configuration côté AD FS et IAM Identity Center.

Avantages de cette approche

Une fois cette architecture SAML en place, les bénéfices dépassent la simple suppression des comptes IAM individuels :

- Les utilisateurs **n'ont aucun compte IAM** (zéro gestion manuelle).
- L'authentification reste **centralisée** en AD (changement de mot de passe une seule fois).
- **MFA en AD** s'applique automatiquement à AWS.
- Impossible de partager des credentials IAM (car ils n'existent pas).
- **Audit centralisé** : CloudTrail enregistre qui s'est connecté, quand, et depuis où.

Ces avantages ont cependant une contrepartie : la disponibilité d'AWS dépend désormais aussi de celle de l'infrastructure AD FS.

Points de vigilance

Cette dépendance introduit les points de vigilance suivants, à anticiper avant de généraliser la fédération SAML :

- Les **métadonnées SAML** (certificats) ont une date d'expiration. Il faut les renouveler régulièrement.
- La **confiance de certificat** doit être validée côté AWS.
- Si AD FS tombe, les utilisateurs **ne peuvent plus accéder à AWS** (prévoir un backup ou un accès d'urgence).

Warning

Single Point of Failure SAML. Si l'IdP (AD FS, Azure AD, Okta) devient indisponible, tous les accès fédérés AWS sont coupés. Prévoyez toujours un compte IAM d'urgence ("break-glass account") avec MFA, stocké en lieu sûr, pour récupérer l'accès en cas de panne de l'IdP.

 [Configuring AD FS as SAML Provider](#)

 [SAML Provider in IAM](#)

 [AssumeRoleWithSAML API](#)

3.6 Pourquoi utiliser la fédération d'identité ?

La fédération d'identité résout un problème très concret : dans une entreprise qui possède déjà un annuaire (Active Directory, LDAP, ou un fournisseur SSO cloud), créer un compte IAM distinct pour chaque employé signifie gérer deux systèmes d'identité en parallèle, avec le risque de désynchronisation que cela implique.

La gestion des identités et des mots de passe devient **centralisée** : c'est l'annuaire d'entreprise qui reste la source de vérité, et AWS ne fait que consommer les assertions d'authentification qu'il produit. Les politiques de sécurité déjà en place dans l'entreprise — complexité des mots de passe, durée de vie des sessions, restrictions d'accès — s'appliquent automatiquement à AWS sans duplication de configuration. Le nombre de comptes IAM permanents diminue fortement, puisque chaque connexion fédérée génère des identifiants temporaires plutôt qu'un compte durable : moins de comptes permanents, c'est mécaniquement moins de surface d'attaque en cas de fuite de identifiants. La traçabilité et la conformité s'en trouvent également améliorées, car chaque accès fédéré peut être rattaché à l'identité réelle de l'employé dans l'annuaire d'entreprise, plutôt qu'à un compte IAM générique partagé. Enfin, l'activation du MFA et des règles conditionnelles se fait souvent plus simplement au niveau de l'IdP (Identity Provider) qu'en la répétant pour chaque compte IAM individuel.

 [AWS Best Practices for SSO](#)

3.7 Exemple de scénario concret

Une entreprise dispose déjà d'un Active Directory local.

Elle souhaite que ses développeurs se connectent à la console AWS **avec leurs identifiants Windows**.

- Mise en place d'un **AD Connector** ou AWS Directory Service.
- Configuration d'AD FS comme IdP SAML.
- IAM Identity Center est configuré pour accepter les assertions SAML d'AD FS.
- Un rôle IAM fédéré "DeveloperRole" est mappé sur un groupe AD "AWS-Dev".
- Les développeurs accèdent à AWS en cliquant sur une URL SSO interne.

Résultat :

- Pas de création de comptes IAM individuels,
- Permissions gérées via AD,
- Accès tracé et sécurisé.

Ce scénario s'appuie sur Active Directory via AD FS, mais le même principe de fédération s'applique aussi aux IdP cloud les plus courants du marché.

3.8 Intégration avec Azure AD et Okta

IAM Identity Center permet une intégration native avec :

- **Azure AD** via SAML ou SCIM
- **Okta** via SAML ou OIDC

Cela permet de synchroniser les groupes et utilisateurs automatiquement.

 [Azure AD Integration](#)

 [Okta Integration](#)

3.9 Points de vigilance et bonnes pratiques

Pour conclure sur la fédération d'identité, voici les réflexes qui permettent d'en tirer les bénéfices tout en évitant les pièges déjà évoqués (expiration des certificats, panne de l'IdP) :

- Préférer la fédération d'identité à la multiplication des comptes IAM.
- Activer MFA au niveau de l'IdP.
- Bien cartographier les groupes AD vers les rôles IAM.
- Tenir à jour les métadonnées SAML (certificats, endpoints).
- Surveiller les connexions via CloudTrail.

 [AWS IAM : comment gérer les permissions de ses équipes ?](#)

4. Amazon Cognito : gestion d'identités applicatives

4.1 Qu'est-ce qu'Amazon Cognito ?

Amazon Cognito est un service AWS qui permet de gérer l'authentification et l'autorisation des utilisateurs dans les applications web et mobiles. Il permet de créer des **pools d'utilisateurs** (User Pools) et de connecter des **fournisseurs d'identité externes** (Google, Facebook, SAML, etc.).

4.2 Différence : IAM vs Cognito

Souvent, les stagiaires confondent IAM et Cognito. C'est normal — ce sont tous les deux des services d'identité. Mais ils n'ont **pas le même public** :

- **IAM** = gestion des identités **administratives** (accès AWS pour l'équipe IT/DevOps).
- **Amazon Cognito** = gestion des identités **applicatives** (accès à une application web ou mobile pour les utilisateurs finaux).

4.3 Cas d'usage typique

Une application web souhaite permettre à ses utilisateurs de se connecter avec leur compte Google ou via un login/mot de passe.

L'application utilise Cognito pour gérer les sessions, les jetons d'accès, et les droits d'accès aux ressources AWS.

4.3 Concepts clés

Cognito repose sur trois notions à ne pas confondre entre elles :

- **User Pool** : base d'utilisateurs gérée par Cognito (inscription, mot de passe, MFA, etc.).
- **Identity Pool** : permet d'obtenir des **credentials AWS temporaires** pour accéder à des services comme S3 ou DynamoDB.
- **Fédération d'identité** : possibilité de déléguer l'authentification à un fournisseur externe (SAML, OAuth2).

Cognito est souvent utilisé dans les architectures serverless ou mobiles. Il permet de sécuriser l'accès aux ressources AWS sans exposer de credentials statiques.

 [Amazon Cognito — Guide officiel](#)

5. Stratégie multi-comptes avec AWS Organizations

5.1 Problématique

Lorsqu'une entreprise évolue et déploie de plus en plus de workloads dans AWS, **gérer toutes les ressources dans un seul compte devient vite risqué et ingérable** :

- Risques de sécurité accrus (trop de permissions dans un même périmètre).
- Facturation difficile à segmenter.

- Difficile d'appliquer des politiques globales cohérentes.
- Problèmes de conformité et de cloisonnement des environnements.

Ces risques ne sont pas propres à AWS : ils apparaissent dès qu'une organisation fait grandir un système d'information partagé sans cloisonnement, et c'est précisément ce que vise à résoudre AWS Organizations.

5.2 Introduction à AWS Organizations

Pour répondre à ces enjeux, AWS propose **AWS Organizations**, un service qui permet de **centraliser la gouvernance de plusieurs comptes AWS** tout en conservant une séparation stricte des environnements.

La structure hiérarchique complète — Management Account, OU et SCP associées — est illustrée en §5.8 avec un exemple concret à 4 comptes.

 [Documentation officielle AWS Organizations](#)

5.3 Définition

AWS Organizations est un service qui permet de créer et de gérer plusieurs comptes AWS depuis une seule organisation centrale, d'appliquer des politiques globales et d'unifier la facturation.

Il facilite :

- la **structuration logique** des environnements (prod, dev, test...),
- l'application de **politiques de sécurité globales**,
- la **centralisation de la facturation**,
- la **gestion simplifiée des identités et des accès** à grande échelle.

Ces quatre bénéfices reposent sur quelques notions structurantes qu'il convient de définir avant d'aller plus loin.

5.4 Concepts clés d'AWS Organizations

Avant d'aller plus loin, voici le vocabulaire propre à AWS Organizations que l'on retrouvera tout au long de cette section :

Élément	Définition
Organisation	Ensemble de comptes AWS sous une gouvernance centralisée.
Management Account	Compte maître utilisé pour créer et gérer l'organisation.
Member Accounts	Comptes membres, rattachés et gérés via l'organisation.
OU (Organizational Unit)	Groupe logique de comptes AWS (par fonction ou par environnement).
SCP (Service Control Policy)	Politique globale qui définit les actions maximales autorisées dans les comptes membres.

 [AWS Organizations Concepts](#)

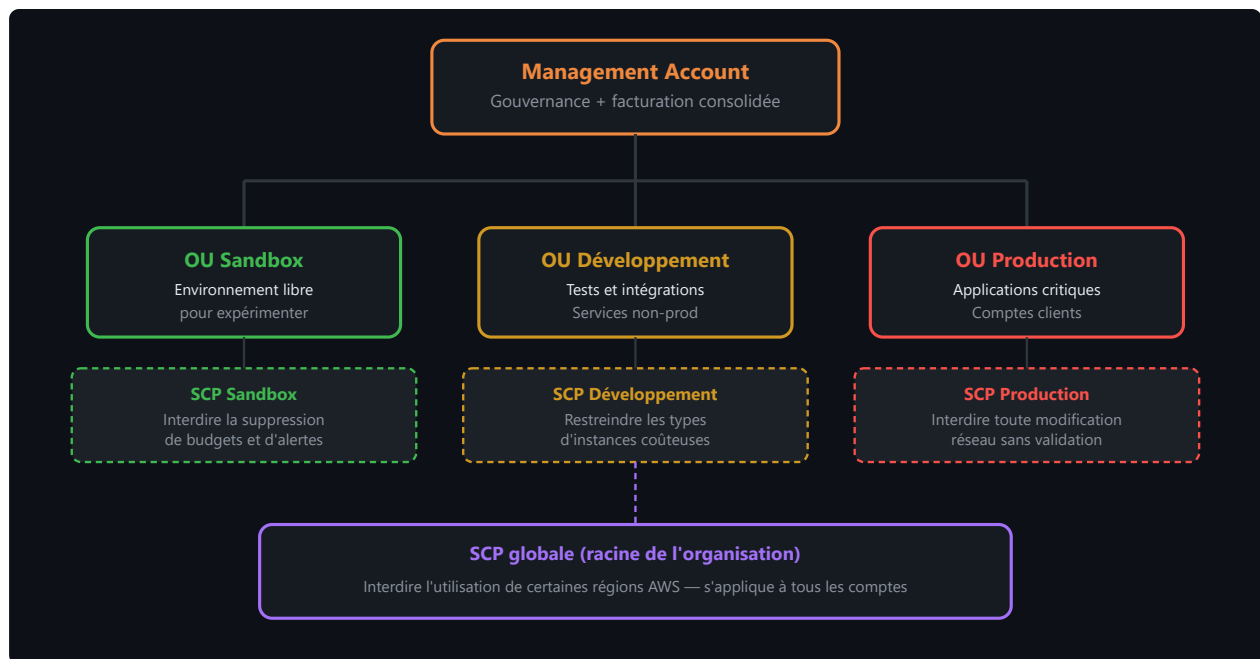
5.5 Exemple d'architecture multi-comptes

Une entreprise met en place une **stratégie à 4 comptes** :

- **Management** → Compte central de gouvernance et facturation.
- **Production** → Applications critiques.
- **Développement** → Tests et intégrations.
- **Sandbox** → Environnement libre pour expérimenter.

Ces comptes sont regroupés dans des OU distinctes et **sécurisés par des SCP** adaptées :

- SCP sur **Sandbox** → Interdire la suppression de budgets et d'alertes.
- SCP sur **Production** → Interdire toute modification réseau sans validation.
- SCP globale → Interdire l'utilisation de certaines régions AWS.



Ce schéma illustre la hiérarchie complète : le **Management Account** en racine centralise la gouvernance et la facturation consolidée de l'ensemble des comptes ; chaque OU porte sa propre SCP adaptée à son niveau de risque (Sandbox très restreinte sur la suppression de ressources de suivi budgétaire, Production verrouillée sur toute modification réseau) ; et une SCP globale attachée à la racine s'applique uniformément à tous les comptes, quelle que soit leur OU — c'est le niveau approprié pour une contrainte transverse comme l'interdiction de régions AWS non autorisées.

5.6 Service Control Policies (SCP) — Clarification

Les **SCP** sont des politiques qui **définissent la limite supérieure** des permissions dans un compte membre.

Elles **ne donnent pas de permissions** directement mais **restreignent** ce que les politiques IAM peuvent autoriser.

SCP vs IAM Policy — Différence essentielle

Beaucoup de stagiaires confondent **SCP** et **IAM Policy**. Voici la différence cruciale :

Aspect	IAM Policy	SCP
Fonction	DONNE les permissions	LIMITE les permissions (plafond)
Niveau	S'applique à un utilisateur, groupe ou rôle IAM	S'applique à tout un compte ou OU
Évaluation	"Puis-je faire cela ?"	"Le compte autorise-t-il cela ?"
Exemples	Allow s3:GetObject, Deny ec2:RunInstances	Deny iam:CreateUser, Deny *:*(sauf S3)
Cas bloqué	Un utilisateur sans policy = accès refusé	Un utilisateur avec Allow, mais SCP refuse = accès refusé

La ligne « Cas bloqué » résume l’essentiel : une SCP ne peut jamais accorder un droit, elle ne peut que le retirer — même le compte Management ne peut pas s’octroyer une permission via une SCP.

Analogie : Restaurant avec zones interdites

Pour fixer durablement cette différence, voici une analogie simple :

- **IAM Policy** = “Alice peut commander des plats du menu”.
- **SCP** = “Personne dans le restaurant ne peut avoir d’alcool” (limite absolue).

Même si Alice a l’autorisation IAM, la SCP l’empêche de commander de l’alcool.

Ordre d’évaluation des permissions

Concrètement, AWS évalue toujours les deux couches dans le même ordre lorsqu’une requête arrive :

1. SCP évalue : "Est-ce que le compte autorise cela ?"
 - Si Deny → Accès refusé (STOP)
 - Si Allow ou neutre → Continue
2. IAM Policy évalue : "Est-ce que l'utilisateur a la permission ?"
 - Si Deny explicite → Accès refusé (STOP)
 - Si Allow explicite → Accès autorisé
 - Si rien → Accès refusé (par défaut)

Résultat final = SCP AND IAM Policy

Exemple concret :

- Un utilisateur a une IAM Policy : `Allow ec2:*` (tous les droits EC2).
- Mais l'OU a une SCP : `Deny ec2:RunInstances` (interdire lancer des instances).
- **Résultat** : L'utilisateur peut faire presque tout avec EC2 **sauf lancer des instances**.

Ce principe d'intersection entre SCP et IAM Policy se retrouve dans la plupart des cas pratiques rencontrés en entreprise, dont voici quelques exemples courants.

5.7 Exemples SCP : Cas courants en entreprise

Exemple 1 : Interdire la création de ressources en dehors de `eu-west-3`

Contexte : L'entreprise doit rester conforme au RGPD (données en Europe).

```

1  {
2    "Version": "2012-10-17",
3    "Statement": [
4      {
5        "Effect": "Deny",
6        "Action": "*",
7        "Resource": "*",
8        "Condition": {
9          "StringNotEquals": {
10             "aws:RequestedRegion": "eu-west-3"
11          }
12        }
13      }
14    ]
15  }
```

Effet : Les développeurs peuvent utiliser AWS, mais **toutes les ressources doivent être en région Paris** (`eu-west-3`).
Les régions US/Asie sont bloquées.

Exemple 2 : Interdire la suppression de certaines ressources critiques

Contexte : Les stagiaires ne doivent pas pouvoir supprimer les RDS ou les VPC de production.

```
1  {
2    "Version": "2012-10-17",
3    "Statement": [
4      {
5        "Effect": "Deny",
6        "Action": [
7          "rds:DeleteDBInstance",
8          "rds:DeleteDBCluster",
9          "ec2:DeleteVpc",
10         "ec2:DeleteSubnet"
11       ],
12       "Resource": "*"
13     }
14   ]
15 }
```

Effet : Même un administrateur IAM du compte ne peut pas supprimer ces ressources (la SCP bloque).

Exemple 3 : Interdire les services coûteux (ex. Production only)

Contexte : Sur le compte Sandbox, on veut éviter les ressources chères (SageMaker, Redshift).

```

1  {
2    "Version": "2012-10-17",
3    "Statement": [
4      {
5        "Effect": "Deny",
6        "Action": [
7          "sagemaker:*",
8          "redshift:*",
9          "elasticmapreduce:*",
10         "mediaconvert:*"
11       ],
12       "Resource": "*"
13     }
14   ]
15 }

```

Effet : Sur le compte Sandbox, ces services ne peuvent pas être utilisés. Idéal pour limiter les coûts d'expérimentation.

Exemple 4 : Interdire les actions sans MFA (Politique conditionnelle)

Contexte : Les administrateurs ne peuvent utiliser la console AWS que s'ils ont MFA activée.

```

1  {
2    "Version": "2012-10-17",
3    "Statement": [
4      {
5        "Effect": "Deny",
6        "NotAction": [
7          "iam:CreateVirtualMFADevice",
8          "iam:EnableMFADevice",
9          "sts:GetSessionToken"
10      ],
11
12      "Resource": "*",
13
14      "Condition": {
15
16        "BoolIfExists": {
17
18          "aws:MultiFactorAuthPresent": "false"
19
20        }
21
22      }
23    ]
24  }

```

Effet : Tout est bloqué sauf création et activation du MFA, à moins que MFA ne soit déjà présente. Oblige à configurer MFA en premier.

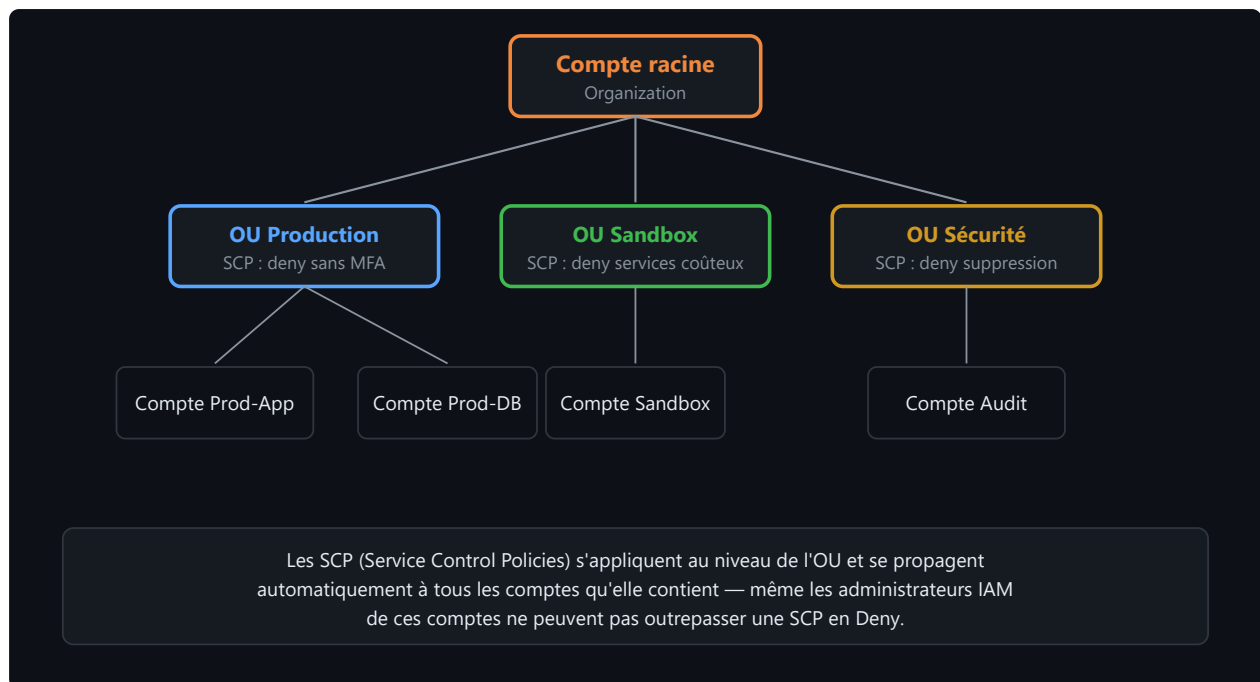
5.8 Où attacher les SCP ? — OU vs Comptes

Les SCP peuvent s'appliquer à différents niveaux :

1. À une **OU** : Affecte tous les comptes de l'OU et leurs enfants.
2. À un **compte individuel** : Affecte uniquement ce compte.
3. À la **racine** : Affecte l'organisation entière.

Bonne pratique : Préférer les OU plutôt que les comptes individuels, pour une gestion centralisée.

Exemple de structure :



5.9 Test et validation des SCP

IMPORTANT : Toujours tester les SCP dans un **compte non critique** avant de les appliquer globalement.

Utiliser le **IAM Policy Simulator** :

1. Console AWS → IAM → Policy Simulator.
2. Simuler une action (ex : `ec2:RunInstances`) avec un utilisateur.
3. Vérifier si elle est bloquée par SCP ou policy.

Cela évite les surprises en production.

 [Service Control Policies](#)

 [SCP Examples](#)

5.10 Avantages d'une stratégie multi-comptes

En reprenant les problématiques évoquées en 5.1, on voit que la stratégie multi-comptes y répond point par point :

- Séparation claire des environnements (Prod, Dev, Test).
- Meilleure sécurité grâce au cloisonnement.
- Gestion fine des coûts par compte.
- Application de politiques globales cohérentes.
- Simplification de l'audit et de la conformité.

Ce cloisonnement a toutefois un coût organisationnel : plus de comptes signifie plus de structure à maintenir (OU, SCP, budgets), ce qui justifie de s'appuyer sur les bonnes pratiques détaillées en 5.12.

5.11 Budgets et alertes multi-comptes

AWS Budgets peut être configuré au niveau de l'organisation pour :

- Suivre les dépenses par compte ou OU
- Déclencher des alertes par email ou SNS

 [AWS Budgets](#)

5.12 Bonnes pratiques recommandées par AWS

Ces recommandations d'AWS visent avant tout à limiter les risques structurels d'une organisation multi-comptes, à commencer par le compte le plus sensible de tous :

- Créer un **compte de management** dédié, jamais utilisé pour déployer des ressources.
- Utiliser **des OU logiques** (par environnement ou par métier).
- Appliquer les SCP **par OU** plutôt que par compte individuel.
- Restreindre les régions et services non utilisés pour limiter les risques.
- Intégrer AWS Organizations avec **IAM Identity Center** pour la gestion des accès à grande échelle.

 [AWS Multi-Account Strategy](#)

5.13 Points de vigilance

Pour clore cette section sur AWS Organizations, quelques rappels sur les erreurs de compréhension les plus fréquentes autour des SCP :

- Les SCP n'annulent pas les politiques IAM, elles les **limitent**.
- L'ordre d'évaluation est : SCP → IAM → Permissions effectives.
- Toujours tester les SCP dans un **compte non critique** avant de les déployer globalement.
- Ne pas donner trop de privilèges au compte Management.

Danger

Le Management Account ne doit jamais héberger de workloads applicatifs. Ce compte dispose de droits sur tous les comptes membres via les SCP. Une compromission du Management Account compromet l'ensemble de l'organisation. Restreignez l'accès au strict minimum, activez MFA, et surveillez-le via CloudTrail au niveau organisationnel.

6. Traçabilité et surveillance avec CloudTrail

6.1 Pourquoi surveiller les activités dans AWS ?

En environnement cloud, les utilisateurs peuvent créer, modifier ou supprimer des ressources **en quelques secondes**, sans passer par un processus de validation centralisé comme dans un datacenter traditionnel. Cette vitesse est un atout pour l'agilité, mais elle transforme aussi n'importe quelle erreur de configuration ou action malveillante en un risque qui se matérialise presque instantanément — d'où la nécessité de tout tracer.

L'enjeu de **sécurité** est le plus évident : sur un compte AWS, il faut pouvoir répondre à tout moment à la question *qui a fait quoi, quand et depuis où*, faute de quoi une compromission de compte peut passer inaperçue pendant des semaines. Vient ensuite l'enjeu de **conformité** : de nombreux référentiels réglementaires (ISO 27001, RGPD, PCI-DSS) exigent explicitement une traçabilité des accès et des modifications, et l'absence de journal d'audit peut à elle seule faire échouer une certification. L'**audit interne** dépend lui aussi directement de cette traçabilité : en cas d'incident (suppression accidentelle d'une base de données, fuite de données), c'est l'historique des actions qui permet de reconstituer la chronologie et d'identifier l'origine du problème en quelques minutes plutôt qu'en plusieurs jours. Enfin, cette même traçabilité sert à l'**optimisation** de la gouvernance cloud : en observant qui utilise réellement quels

services, une équipe peut ajuster ses permissions IAM au principe du moindre privilège plutôt que de les laisser trop larges par précaution.

Pour cela, AWS fournit **CloudTrail**, un service de **traçabilité des actions** dans un compte ou une organisation.

6.2 AWS CloudTrail — Définition

CloudTrail est le service AWS qui enregistre toutes les actions réalisées via la console, les API et la CLI.

Chaque événement contient :

- Date et heure de l'action
- Identité de l'utilisateur (ou rôle IAM)
- Adresse IP d'origine
- Service AWS concerné
- Action exécutée (ex. `ConsoleLogin`, `RunInstances`, `DeleteBucket`)

CloudTrail permet ainsi :

- de **visualiser l'historique des actions** dans la console,
- de **stocker les logs** dans S3,
- et de les **exploiter** dans **CloudWatch** ou **EventBridge** pour créer des alertes.

Ces capacités s'appliquent par défaut à un seul compte ; dans une organisation multi-comptes, CloudTrail offre en plus un mode de centralisation dédié.

6.3 CloudTrail et AWS Organizations

Lorsque vous utilisez **AWS Organizations**, vous pouvez :

- Activer **un seul trail** au **niveau du compte de management**
- Appliquer ce trail à **tous les comptes enfants** (OU ou organisation complète)
- Centraliser les logs dans un **seul bucket S3**.

Cela permet de suivre les activités de tous les comptes de votre organisation depuis un **point unique**.

6.4 Exemples d'événements courants enregistrés

Chaque appel API AWS génère un événement CloudTrail ; voici quelques exemples représentatifs des cas d'usage sécurité les plus courants :

Événement CloudTrail	Description	Cas d'usage
ConsoleLogin	Connexion à la console AWS	Détecter les connexions non MFA
RunInstances	Création d'une instance EC2	Suivi de consommation / sécurité
CreateBucket	Création d'un bucket S3	Audit stockage
PutUserPolicy	Modification d'une policy IAM	Traçabilité sécurité IAM
DeleteTrail	Suppression du trail CloudTrail	Détection activité critique

Le dernier événement, `DeleteTrail`, mérite une attention particulière : un attaquant qui obtient un accès administrateur tentera souvent de désactiver ou supprimer le trail pour effacer ses traces — c'est pourquoi il est recommandé de surveiller ce type d'événement via une alerte dédiée plutôt que de se contenter de sa présence dans les logs.

6.5 CloudTrail vs CloudWatch

Ces deux services sont fréquemment confondus alors qu'ils répondent à des questions différentes :

CloudTrail	CloudWatch
Enregistre les événements historiques	Surveille les métriques et ressources
Trace les actions utilisateurs/API	Crée des alertes sur seuils ou logs
Stocke les logs dans S3	Affiche en temps réel
Focus "qui a fait quoi"	Focus "ce qui se passe"

Les deux sont complémentaires :

- CloudTrail = audit et traçabilité

- CloudWatch = surveillance opérationnelle

Cette complémentarité prend tout son sens lorsque les volumes de logs CloudTrail deviennent trop importants pour être consultés manuellement, ce qui appelle une chaîne d'analyse dédiée.

6.6 Exploiter les journaux CloudTrail : de la traçabilité à l'analyse

CloudTrail ne se limite pas à enregistrer les actions : il permet aussi de les **analyser**, de les **interroger**, et de **restreindre leur accès** selon les rôles. Pour cela, AWS propose une chaîne d'outils complémentaires, chacun jouant un rôle précis dans le traitement des logs.

Étapes d'intégration : de CloudTrail à Lake Formation

1. Stockage dans S3

Les événements CloudTrail sont automatiquement archivés dans un bucket S3, sous forme de fichiers JSON. Ce stockage est durable, centralisé, et interrogeable.

2. Catalogage avec AWS Glue

Glue agit comme un annuaire technique : il décrit la structure des fichiers (colonnes, types, formats) et les rend accessibles aux outils d'analyse comme Athena. Sans Glue, Athena ne saurait pas comment lire les logs.

3. Analyse avec Amazon Athena

Athena permet d'exécuter des requêtes SQL directement sur les fichiers CloudTrail dans S3. C'est un moteur d'analyse serverless, sans base de données à déployer. On peut par exemple extraire tous les événements `DeleteBucket` du mois ou filtrer par utilisateur IAM.

4. Gouvernance avec AWS Lake Formation

Lake Formation ajoute une couche de sécurité fine : il permet de contrôler **qui peut accéder à quelles données**, jusqu'au niveau colonne. Cela garantit que seuls les rôles autorisés peuvent interroger les logs sensibles.

 [Analyser CloudTrail avec Athena](#) 

6.7 AWS Config vs CloudTrail

Si CloudTrail trace les **actions**, AWS Config trace les **états**. Ensemble, ils offrent une vision complète du comportement et de la conformité des ressources AWS.

Fonction	AWS Config	AWS CloudTrail
Ce qui est suivi	État des ressources	Actions API des utilisateurs
Exemple	"Ce bucket est-il toujours privé ?"	"Qui a modifié ce bucket ?"
Type de données	Historique de configuration	Journal d'activité
Objectif	Conformité, détection de dérive	Audit, traçabilité
Intégration	Conformance Packs, remédiation automatique	EventBridge, Athena, SIEM

 [AWS Config Documentation](#)

6.8 Cas d'usage

Un administrateur souhaite identifier tous les appels `PutUserPolicy` effectués par des utilisateurs non autorisés. Il utilise CloudTrail pour collecter les événements, Glue pour cataloguer les logs, Athena pour interroger les données, et Lake Formation pour restreindre l'accès aux résultats.

6.9 Bonnes pratiques CloudTrail

Pour conclure cette section, voici les réglages qui distinguent un CloudTrail « activé par défaut » d'un CloudTrail réellement exploitable en cas d'incident :

- Activer CloudTrail **au niveau de l'organisation**.
- Centraliser les logs dans un **bucket S3 sécurisé**.
- Activer la **chiffrement SSE-S3 ou SSE-KMS**.
- Mettre en place des **alertes EventBridge** sur les événements sensibles :
 - Connexion root
 - Connexion sans MFA
 - Suppression de logs
 - Création de ressources critiques
- Définir une **politique de rétention** et un plan d'audit régulier.

 [AWS CloudTrail Console](#)

 [Documentation CloudTrail](#)

 [ConsoleLogin event](#)

 [CloudTrail + Organizations](#)

Warning

Ne jamais désactiver CloudTrail en production. La suppression ou la désactivation d'un trail est elle-même un événement critique enregistré. Configurez des alertes EventBridge sur l'événement `DeleteTrail` et `StopLogging`. En conformité ISO 27001 ou PCI-DSS, les logs CloudTrail doivent être conservés au minimum 1 an et être immuables (activer S3 Object Lock).

6.10 Combien coûtent CloudTrail et Config ?

Ces deux services ne sont **pas gratuits au-delà d'un premier niveau minimal** — un point souvent ignoré, car “activer l'audit” semble aller de soi sans qu'on en vérifie le coût.

Service	Gratuit	Facturé
CloudTrail	1 trail de gestion (événements de gestion) par région, journalisé automatiquement, consultable 90 jours dans l'historique des événements	Trails supplémentaires : 2,00 par 100 000 événements de gestion. Événements de données (S3 objet par objet, invocations Lambda) : 0,10 par 100 000 événements — peut grimper vite sur un bucket S3 à fort trafic
		0,003 par élément de configuration enregistré, plus 0,001 à 0,0012 par évaluation de règle de conformité
AWS Config	— (pas de niveau gratuit)	

Exemple concret : un compte avec 500 ressources suivies par Config, réévaluées 4 fois par jour (2 000 évaluations/jour $\approx$ 60 000/mois) sur 10 règles de conformité coûte environ $500 \times 0,003$ (enregistrement) + $60\,000 \times 10 \times 0,001$ (évaluations) = 1,50 + 600 — **le coût des évaluations de règles domine largement**, pas l'enregistrement des ressources elles-mêmes. Limiter le nombre de règles actives et leur fréquence de déclenchement est le principal levier d'optimisation.

CloudTrail avec événements de données : ces événements ne sont pas activés par défaut et entraînent toujours des frais. Par exemple, avec un tarif hypothétique de 0,10 USD pour 100 000 événements, dix millions d'événements représenteraient 10 USD avant les autres coûts. Ce calcul sert à comprendre l'ordre de grandeur ; vérifiez le tarif de la région et la fonctionnalité choisie sur la page officielle CloudTrail Pricing avant toute estimation. Utilisez des sélecteurs d'événements précis plutôt qu'une collecte exhaustive sans objectif d'audit.

 [AWS CloudTrail Pricing](#)  [AWS Config Pricing](#)

7. Gestion pratique d'IAM avec la CLI

Info

Une activité pratique permet d'approfondir la gestion des utilisateurs, groupes et rôles IAM en CLI.

7.1 Créer un utilisateur IAM

La commande crée l'utilisateur `alice`, puis une seconde commande vérifie immédiatement qu'il a bien été enregistré dans IAM — confirmer la création après chaque commande est toujours une bonne pratique.

```
1 # Créer un nouvel utilisateur nommé "alice"
2 aws iam create-user --user-name alice
3
4 # Vérifier que l'utilisateur a été créé
5 aws iam get-user --user-name alice
```



Tip

Résultat attendu — `aws iam get-user --user-name alice` :

```
1 {
2     "User": {
3         "Path": "/",
4         "UserName": "alice",
5         "UserId": "AIDA4EXAMPLE7EXAMPLE",
6         "Arn": "arn:aws:iam::123456789012:user/alice",
7         "CreateDate": "2024-01-15T10:30:00+00:00"
8     }
9 }
```

L'utilisateur `alice` est créé. Son `UserId` commence toujours par `AIDA` pour les utilisateurs IAM. La commande `create-user` ne retourne aucun output si elle réussit.

7.2 Créer un groupe et ajouter l'utilisateur

Les groupes sont le cœur de la gestion IAM : plutôt qu'attribuer des permissions utilisateur par utilisateur, on les attache au groupe et tous les membres en héritent automatiquement.

```
1 # Créer un groupe "Developers"
2 aws iam create-group --group-name Developers
3
4 # Ajouter alice au groupe
5 aws iam add-user-to-group --group-name Developers --user-name alice
6
7 # Vérifier
8 aws iam get-group --group-name Developers
```



Tip

Résultat attendu — `aws iam get-group --group-name Developers` :

```
1  {
2      "Group": {
3          "Path": "/",
4          "GroupName": "Developers",
5          "GroupId": "AGPA4EXAMPLEGROUP",
6          "Arn": "arn:aws:iam::123456789012:group/Developers",
7          "CreateDate": "2024-01-15T10:31:00+00:00"
8      },
9      "Users": [
10         {
11             "UserName": "alice",
12             "UserId": "AIDA4EXAMPLE7EXAMPLE",
13             "Arn": "arn:aws:iam::123456789012:user/alice"
14         }
15     ],
16     "IsTruncated": false
17 }
```

Le groupe `Developers` est créé et `alice` apparaît bien dans le tableau `Users`.

7.3 Attacher une policy au groupe

On attache une policy AWS gérée au groupe `Developers`. Tous les membres actuels et futurs du groupe hériteront automatiquement de ces permissions, sans aucune action supplémentaire.

```
1 # Attacher la policy AWS gérée "AmazonS3ReadOnlyAccess" au groupe
2 aws iam attach-group-policy \
3   --group-name Developers \
4   --policy-arn arn:aws:iam::aws:policy/AmazonS3ReadOnlyAccess
5
6 # Vérifier
7 aws iam list-attached-group-policies --group-name Developers
```



Résultat attendu — `aws iam list-attached-group-policies --group-name Developers` :

```
1 {
2   "AttachedPolicies": [
3     {
4       "PolicyName": "AmazonS3ReadOnlyAccess",
5       "PolicyArn": "arn:aws:iam::aws:policy/AmazonS3ReadOnlyAccess"
6     }
7   ],
8   "IsTruncated": false
9 }
```

La policy `AmazonS3ReadOnlyAccess` est bien attachée au groupe `Developers`. Tous les membres actuels et futurs du groupe en héritent automatiquement.

7.4 Créer des clés d'accès pour un utilisateur

Les clés d'accès (`AccessKeyId` + `SecretAccessKey`) permettent d'appeler l'API AWS depuis la CLI ou un script. Elles sont générées **une seule fois** et le `SecretAccessKey` ne peut jamais être récupéré ensuite — AWS ne le stocke pas.

```
1 # Générer une paire de clés d'accès pour alice
2 aws iam create-access-key --user-name alice
3
4 # Le résultat contient AccessKeyId et SecretAccessKey
5 # Attention : SAUVEGARDEZ CES CLÉS EN LIEU SÛR
```



Tip

Résultat attendu — `aws iam create-access-key --user-name alice` :

```
1 {
2   "AccessKey": {
3     "UserName": "alice",
4     "AccessKeyId": "AKIA4EXAMPLEKEYID12",
5     "Status": "Active",
6     "SecretAccessKey": "<SECRET_TEMPORAIRE_MASQUE>",
7     "CreateDate": "2024-01-15T10:35:00+00:00"
8   }
9 }
```

Un `AccessKeyId` permanent utilise généralement le préfixe `AKIA`, tandis qu'un identifiant temporaire STS utilise `ASIA`. Le `SecretAccessKey` d'une nouvelle clé permanente ne s'affiche qu'une seule fois : ne le copiez jamais dans un support, un ticket ou un dépôt. Stockez-le dans un gestionnaire de secrets, puis préférez les rôles et identifiants temporaires dès que le cas d'usage le permet.



Danger

Ne jamais stocker ces clés en clair dans le code source, les fichiers `.env` versionnés ou les dépôts Git. Utilisez AWS Secrets Manager ou un coffre-fort (Vault, 1Password) pour les conserver. En cas de fuite, révoquez immédiatement la clé dans IAM et générez-en une nouvelle.

7.5 Créer une policy JSON personnalisée

Quand aucune policy AWS gérée ne correspond exactement à vos besoins, vous créez une policy inline. Ici, on génère d'abord le fichier JSON localement, puis on l'attache à l'utilisateur.

```
1 # Créer une policy pour accès limité à S3
2 cat > s3-readonly-policy.json << 'EOF'
3 {
4   "Version": "2012-10-17",
5   "Statement": [
6     {
7       "Effect": "Allow",
8       "Action": ["s3:GetObject", "s3:ListBucket"],
9       "Resource": ["arn:aws:s3:::mon-bucket", "arn:aws:s3:::mon-bucket/*"]
10    }
11  ]
12 }
13 EOF
14
15 # Attacher cette policy à alice
16 aws iam put-user-policy \
17   --user-name alice \
18   --policy-name S3-ReadOnly \
19   --policy-document file:///s3-readonly-policy.json
```



Tip

Résultat attendu — `aws iam list-user-policies --user-name alice` (commande de vérification) :

```
1 {
2     "PolicyNames": [
3         "S3-Readonly"
4     ],
5     "IsTruncated": false
6 }
```

La policy inline `S3-Readonly` est bien attachée directement à l'utilisateur `alice`. Une policy inline est stockée directement sur l'entité (user/group/role) et ne peut pas être réutilisée ailleurs.

7.6 Vérifier les permissions d'un utilisateur

Avant toute intervention sur un compte, il est utile de dresser l'inventaire complet des permissions effectives d'un utilisateur : politiques directes ET celles héritées via les groupes.

```
1 # Lister les politiques attachées à alice
2 aws iam list-user-policies --user-name alice
3
4 # Lister les politiques de groupe pour alice
5 aws iam list-groups-for-user --user-name alice
```



Tip

Résultat attendu — `aws iam list-groups-for-user --user-name alice` :

```
1  {
2      "Groups": [
3          {
4              "Path": "/",
5              "GroupName": "Developers",
6              "GroupId": "AGPA4EXAMPLEGROUP",
7              "Arn": "arn:aws:iam::123456789012:group/Developers",
8              "CreateDate": "2024-01-15T10:31:00+00:00"
9          }
10     ],
11     "IsTruncated": false
12 }
```

`alice` appartient au groupe `Developers`. Ses permissions effectives = ses politiques directes + les politiques de tous ses groupes d'appartenance.

7.7 Créer un rôle IAM

Un rôle n'a pas de credentials permanents : il est **assumé temporairement** par un service ou un utilisateur. Le fichier `trust-policy.json` (appelé "trust policy" ou "politique de confiance") définit **qui** a le droit d'assumer ce rôle — ici, le service EC2.

```
1  # Créer un rôle pour EC2
2  aws iam create-role \
3      --role-name EC2-S3-Access \
4      --assume-role-policy-document file:///trust-policy.json
5
6  # Contenu de trust-policy.json :
7  # {
8  #   "Version": "2012-10-17",
9  #   "Statement": [{
10
11       "Effect": "Allow",
12
13       "Principal": {"Service": "ec2.amazonaws.com"},
14
15       "Action": "sts:AssumeRole"
16   }]
17 }
```



Tip

Résultat attendu — `aws iam create-role --role-name EC2-S3-Access ...` :

```
1  {
2      "Role": {
3          "Path": "/",
4          "RoleName": "EC2-S3-Access",
5          "RoleId": "AROA4EXAMPLEROLEID1",
6          "Arn": "arn:aws:iam::123456789012:role/EC2-S3-Access",
7          "CreateDate": "2024-01-15T10:40:00+00:00",
8          "AssumeRolePolicyDocument": {
9              "Version": "2012-10-17",
10             "Statement": [{
11                 "Effect": "Allow",
12                 "Principal": {"Service": "ec2.amazonaws.com"},
13                 "Action": "sts:AssumeRole"
14             }]
15         },
16         "MaxSessionDuration": 3600
17     }
18 }
```

Le rôle `EC2-S3-Access` est créé. Son `RoleId` commence par `AROA`. La trust policy indique que seul le service EC2 (`ec2.amazonaws.com`) peut assumer ce rôle.

7.8 Attacher une policy à un rôle

Une fois le rôle créé avec sa trust policy (qui définit qui peut l'assumer), on lui attache des permissions (ce qu'il peut faire). Toute instance EC2 qui assumera ce rôle pourra lire S3 **sans credentials statiques**.

```
1 # Attacher AmazonS3ReadOnlyAccess au rôle
2 aws iam attach-role-policy \
3   --role-name EC2-S3-Access \
4   --policy-arn arn:aws:iam::aws:policy/AmazonS3ReadOnlyAccess
```



Résultat attendu — `aws iam list-attached-role-policies --role-name EC2-S3-Access` :

```
1 {
2   "AttachedPolicies": [
3     {
4       "PolicyName": "AmazonS3ReadOnlyAccess",
5       "PolicyArn": "arn:aws:iam::aws:policy/AmazonS3ReadOnlyAccess"
6     }
7   ],
8   "IsTruncated": false
9 }
```

La commande `attach-role-policy` ne retourne aucun output si elle réussit. La vérification confirme que `AmazonS3ReadOnlyAccess` est bien attachée au rôle `EC2-S3-Access`.

7.9 Activer MFA pour un utilisateur

L'activation du MFA lie un appareil virtuel (application TOTP type Google Authenticator ou Authy) à l'utilisateur. Les deux codes consécutifs (`authentication-code1` et `authentication-code2`) sont nécessaires pour synchroniser l'horloge de l'appareil avec AWS.

```
1 # Créer un appareil MFA virtuel
2 aws iam enable-mfa-device \
3   --user-name alice \
4   --serial-number arn:aws:iam::123456789012:mfa/alice-mfa \
5   --authentication-code1 123456 \
6   --authentication-code2 654321
```



Tip

Résultat attendu — `aws iam list-mfa-devices --user-name alice` :

```
1 {
2   "MFADevices": [
3     {
4       "UserName": "alice",
5       "SerialNumber": "arn:aws:iam::123456789012:mfa/alice-mfa",
6       "EnableDate": "2024-01-15T10:45:00+00:00"
7     }
8   ],
9   "IsTruncated": false
10 }
```

La commande `enable-mfa-device` ne retourne aucun output si elle réussit. La vérification confirme que le périphérique MFA virtuel est bien enregistré pour `alice`.

7.10 Politique conditionnelle : exiger MFA

Cette démo crée une policy qui bloque **absolument toutes les actions AWS** si l'utilisateur n'a pas activé MFA pour sa session. C'est une protection forte recommandée pour les comptes administrateurs — sans MFA active, même `aws s3 ls` sera refusé.

```

1  # Créer une policy refusant tout si MFA n'est pas présente
2  cat > mfa-required.json << 'EOF'
3  {
4      "Version": "2012-10-17",
5      "Statement": [{
6          "Effect": "Deny",
7          "Action": "*",
8          "Resource": "*",
9          "Condition": {
10
11              "BoolIfExists": {
12
13                  "aws:MultiFactorAuthPresent": "false"
14
15              }
16          }
17      }]
18  }
19  EOF
20
21  # Attacher au groupe Admins
22
23  aws iam put-group-policy \
24      --group-name Admins \
25      --policy-name MFA-Required \
26      --policy-document file:///mfa-required.json

```




Tip

Résultat attendu — `aws iam list-group-policies --group-name Admins` :

```
1 {
2     "PolicyNames": [
3         "MFA-Required"
4     ],
5     "IsTruncated": false
6 }
```

La commande `put-group-policy` ne retourne aucun output si elle réussit. Pour tester : connectez-vous à la console AWS sans MFA — toutes les actions retourneront `AccessDenied`. Activez MFA, reconnectez-vous, et les accès sont rétablis.

8. Points importants et pièges fréquents

Ce chapitre a couvert IAM, MFA, la fédération, Organizations et CloudTrail ; le tableau suivant recense les confusions les plus fréquentes constatées chez les stagiaires sur l'ensemble de ces sujets.

Piège courant	Réalité	Solution
"Les SCP donnent des permissions"	Les SCP limitent les permissions, elles ne les donnent pas.	Toujours combiner SCP + policies IAM.
"Un Deny peut être contourné par un Allow"	Un Deny explicite est prioritaire. Toujours.	Reconnaître que Deny > Allow dans l'évaluation.
"IAM et Cognito, c'est pareil"	IAM = accès AWS administratif. Cognito = accès application.	Utiliser IAM pour IT, Cognito pour utilisateurs finaux.
"MFA, c'est juste un code SMS"	MFA peut être TOTP, YubiKey, passkey, biométrie.	Proposer plusieurs types selon la sensibilité.
"Pas besoin de fédération si on a IAM"	La fédération centralise la gestion et réduit les comptes statiques.	Préférer la fédération en environnement d'entreprise.
"CloudTrail ralentit AWS"	CloudTrail est activé implicitement et n'impacte pas les perfs.	L'activer sans crainte pour l'audit.
"Un utilisateur sans policy n'a aucun accès"	Correct : le moindre privilège s'applique par défaut.	Toujours attacher une policy minimale.
"On peut récupérer une clé d'accès perdue"	Non. Les clés ne s'affichent qu'à la création.	Conserver les clés en lieu sûr, utiliser AWS Secrets Manager.

Ces confusions partagent un point commun : elles viennent presque toujours d'un raisonnement par analogie avec un système d'authentification classique (un seul mot de passe, un seul niveau de permission), alors qu'AWS empile plusieurs couches de contrôle qui s'évaluent selon des règles précises.

9. Choisir la bonne solution d'authentification AWS

AWS propose de nombreux services d'authentification. Voici une carte complète pour savoir lequel choisir.

Trois profils d'authentification distincts se dégagent : les développeurs/services AWS/applications internes s'appuient sur IAM Users, IAM Roles et STS ; les employés de l'entreprise (B2E) passent par IAM Identity Center (SSO), SAML 2.0 ou

Directory Service ; les utilisateurs du grand public (B2C) utilisent Cognito User Pools.

Tableau comparatif complet

Voici le détail technique de chacune de ces solutions, avec leur mode de credentials, leur durée de validité et leur coût :

Solution	Pour qui	Credentials	Durée	MFA	Coût
IAM User + Access Key	Développeurs internes, CI/CD	Permanents	Jusqu'à révocation	Oui	Gratuit
IAM Role + STS	Services AWS, scripts, cross- account	Temporaires	15 min – 36 h	Non (délégué)	Gratuit
Cognito User Pool	Utilisateurs B2C (app web/mobile)	JWT Token	Configurable	Oui (SMS, TOTP)	50 000 MAU gratuits
Cognito Identity Pool	App mobile/web → accès AWS	Credentials STS	1 h par défaut	Via User Pool	Gratuit
IAM Identity Center	Employés → multi- comptes AWS	Session token	Configurable	Oui	Gratuit
SAML 2.0	Fédération entreprise (AD, Okta)	Assertion SAML → STS	Configurable	Via IdP	Gratuit
Directory Service — Simple AD	Annuaire LDAP léger	Login AD	N/A	Non	~73 \$/mois
Directory Service — Managed AD	Active Directory Microsoft complet	Login AD	N/A	Oui	~288 \$/mois
Directory Service — AD Connector	Pont vers un AD on-premises	Login AD on- prem	N/A	Via AD	~100 \$/mois

MAU (Monthly Active Users, utilisateurs actifs mensuels) est l'unité de facturation de Cognito User Pool : AWS compte un utilisateur comme "actif" dès qu'il s'authentifie au moins une fois dans le mois, et facture au-delà des 50 000 premiers MAU gratuits — une application avec 200 000 utilisateurs connectés dans le mois ne paiera donc que sur les 150 000 dépassant le seuil gratuit.

Arbre de décision

Le tableau précédent liste les options disponibles ; l'arbre suivant permet de trancher rapidement selon le profil de l'utilisateur ou du système qui doit s'authentifier :

Qui s'authentifie ?

- Un service AWS (EC2, Lambda, ECS...)
 - → IAM Role (attaché à l'instance/fonction) + STS automatique
- Un développeur / script / CI-CD
 - Accès court terme → STS AssumeRole + profil CLI
 - Accès long terme → IAM User + Access Key (à limiter !)
- Un employé de l'entreprise
 - Accès à un seul compte AWS → IAM User (acceptable)
 - Accès à plusieurs comptes → IAM Identity Center (SSO) ☒ recommandé
 - Active Directory existant → SAML 2.0 ou AD Connector
- Un utilisateur externe (client, partenaire)
 - Authentification pure (login/mot de passe app) → Cognito User Pool
 - Authentification + accès aux services AWS → Cognito User Pool
+ Identity Pool

Cet arbre confirme une règle simple : dès qu'un utilisateur ou un service doit franchir une frontière (plusieurs comptes AWS, un IdP externe, une application publique), la solution passe presque toujours par un mécanisme de credentials temporaires plutôt que par un compte IAM permanent.

Focus : Cognito User Pool vs Identity Pool

La confusion la plus fréquente en formation :

Cognito USER POOL	Cognito IDENTITY POOL
"Qui es-tu ?"	"Qu'as-tu le droit de faire dans AWS ?"
Gère l'annuaire d'utilisateurs (login, mot de passe, attributs, MFA, réinitialisation de mdp)	Échange un token externe contre des credentials AWS temporaires (STS)
Émet des JWT : • Access Token • ID Token • Refresh Token	Accepte en entrée : • Token Cognito User Pool • Token Google / Facebook / Apple • Token SAML • Accès anonyme
Utilisé par : • Frontend (login page) • API Gateway (autoriser) • Tout service vérifiant un JWT	Utilisé pour : • Appeler S3, DynamoDB, etc. depuis une app mobile • Accès AWS sans backend

Retenez la question clé de chaque colonne : un User Pool répond à « qui es-tu ? » (authentification), un Identity Pool répond à « qu'as-tu le droit de faire dans AWS ? » (autorisation) — les deux se combinent quand une application doit à la fois authentifier ses utilisateurs et leur donner un accès direct à des services AWS comme S3.

Focus : IAM Identity Center vs SAML 2.0

Ce chapitre a présenté SAML 2.0 comme mécanisme générique de fédération (section 3) et IAM Identity Center comme service AWS dédié à la gestion du SSO ; voici comment les deux se comparent concrètement au moment de choisir :

	IAM Identity Center	SAML 2.0 direct
Configuration	Quelques clics dans la console	Configuration manuelle complexe (métadonnées XML)
Multi-comptes	✓ Natif (une entrée = tous les comptes)	✗ Un trust par compte
IdP supportés	Azure AD, Okta, Ping, SCIM	Tout IdP SAML 2.0
Portail web	✓ Inclus (aws.amazon.com/sso)	✗ À construire
Recommandé pour	Nouvelles organisations AWS	Besoins très spécifiques

IAM contrôle *qui* peut agir. Le Chapitre 3 pose la question suivante : *où* ces actions ont-elles lieu ? Vous allez concevoir le réseau privé — VPC — dans lequel vivront vos ressources AWS, avant même de déployer la première base de données ou le premier serveur.

Travaux pratiques — Sécurité et gestion des accès

AWS Academy — Cloud Foundations

Type	Référence	Intitulé
Module	Module 4	Sécurité dans le Cloud AWS
Atelier	Atelier 1	Introduction à AWS IAM — 100 points
Contrôle	Module 4	Contrôle des connaissances du module

AWS Academy — Cloud Architecting

Type	Référence	Intitulé
Module	Module 3	Sécurisation de l'accès
Module	Module 9	Sécurisation de l'accès utilisateur, aux applications et aux données
Atelier guidé	Module 3	Exploration du service AWS IAM : utilisateurs, groupes, rôles et politiques
Atelier guidé	Module 9	Sécurisation des applications à l'aide d'Amazon Cognito
Atelier guidé	Module 9	Chiffrement des données au repos avec AWS KMS

Go Deploy

Lab	Intitulé
Lab 21	Gestion des identités et des accès AWS — IAM

► Parcours guidé et validation

Ressources

Documentation officielle AWS

Pour approfondir les services présentés dans ce chapitre, voici les références officielles correspondantes :

- [AWS IAM Documentation](#) 
- [IAM Best Practices](#) 
- [AWS IAM Identity Center](#) 
- [Amazon Cognito Documentation](#) 
- [AWS CloudTrail Documentation](#) 

Le quiz ci-dessous permet de vérifier l'acquisition des notions clés de ce chapitre avant de passer au suivant.

Quiz interactif du chapitre

Choisissez une réponse : la correction expliquée apparaît immédiatement. Les propositions changent d'ordre à chaque nouvelle tentative.

Le quiz interactif reste disponible dans la version web du support.

Mini-jeu de synthèse — AWS Quest

Résolvez cinq décisions factuelles pour consolider les fondamentaux avant d'ouvrir le chapitre suivant. Le jeu est une simulation locale : il ne se connecte à aucun compte AWS.

Le quiz interactif reste disponible dans la version web du support.

Info

Les instances EC2 et les buckets S3 déployés au chapitre précédent doivent maintenant s'intégrer dans un réseau maîtrisé et s'appuyer sur des bases de données managées : ce chapitre couvre les deux piliers d'une architecture AWS mature, le réseau (VPC) et la donnée persistante (RDS, Aurora, DynamoDB).

Objectifs du chapitre

À l'issue de ce chapitre, les stagiaires seront capables de :

- **Expliquer** l'intérêt des bases managées et déployer une Amazon RDS Multi-AZ sécurisée
- **Différencier** Aurora, RDS et DynamoDB, et planifier une migration avec AWS DMS
- **Concevoir** une VPC avec subnets publics/privés, table de routage et passerelle Internet/NAT
- **Sécuriser et interconnecter** le réseau (Security Groups, NACL, VPC Peering, Transit Gateway, VPC Endpoint)
- **Configurer** Route 53 (DNS, routage avancé) et un cluster ElastiCache pour accélérer l'accès aux données

1. Bases de données dans AWS — Du service géré à la scalabilité

1.1 La révolution des bases managées

 Vidéo : [AWS Database Services Overview](#) 

Quand on parle de **bases de données dans le Cloud**, la question centrale n'est plus « Où vais-je installer un serveur ? » mais plutôt « Quel type de données vais-je stocker, et quel accès dois-je offrir ? ».

Dans un environnement traditionnel (**on-premise**), administrer une base de données impliquait :

- **Installation manuelle** du moteur (MySQL, PostgreSQL, Oracle...)
- **Configuration** des paramètres (mémoire, cache, compression...)
- **Sauvegardes régulières** et tests de restauration
- **Maintenance** des patches de sécurité
- **Réplication** pour la haute disponibilité
- **Surveillance** 24h/24 (CPU, mémoire, disque, connexions)
- **Escalade** : augmenter la taille du disque, du CPU, de la mémoire
- **Gestion des droits** d'accès pour chaque application

Autant de tâches qui détournent les équipes de leur **valeur métier réelle** : créer des applications performantes et sécurisées.

Avec **Amazon RDS** et les services managés AWS, ce paradigme s'inverse : **AWS administre l'infrastructure, vous administrez vos données.**

L'abstraction du matériel

Imaginez une **maison en location** vs. une **maison en propriété** :

- **En propriété** (on-prem) : vous gérez tout — le toit, la plomberie, l'électricité, les réparations. Si la toiture fuit, c'est à vos frais et vos équipes doivent intervenir.
- **En location** (RDS) : le propriétaire assure la structure, le toit, les murs. Vous ne vous occupez que du décor intérieur (vos données).

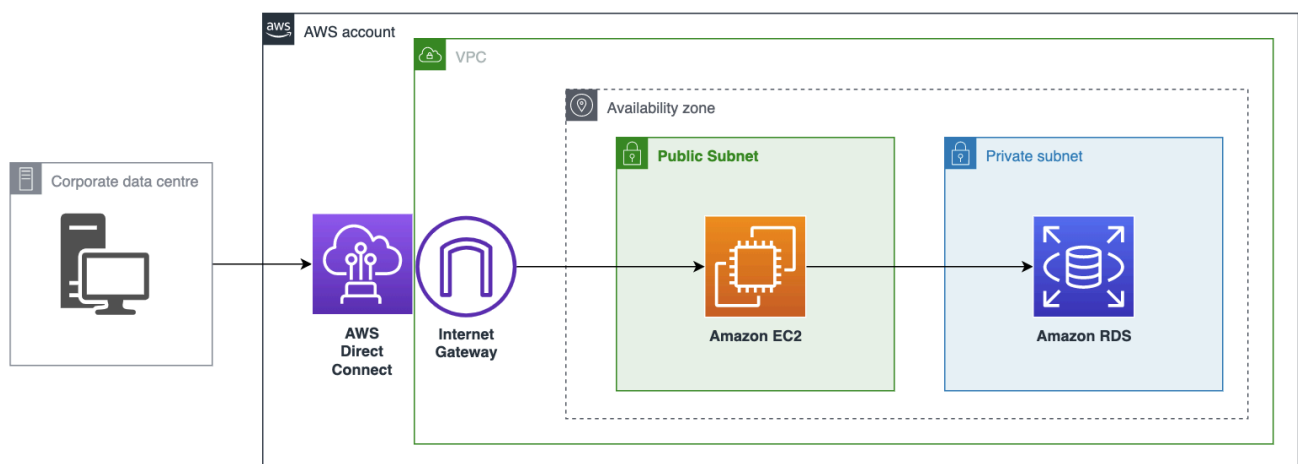
Avec RDS :

- Vous définissez simplement : quel moteur ? (MySQL, PostgreSQL, Oracle, SQL Server, Aurora) — quelle taille ? (t3.small, m5.xlarge...) — combien de stockage ? (100 Go, 5 To...)

- AWS crée l'instance, configure le stockage EBS, met en place le monitoring, gère les snapshots, applique les patches.
- Vous accédez à votre base via un endpoint standard (`mondb.xxxxx.eu-west-1.rds.amazonaws.com:3306`).

Ce changement de paradigme se concrétise par un catalogue de moteurs relationnels managés, détaillé dans la section suivante.

1.2 Amazon RDS — Bases relationnelles managées



Amazon RDS (Relational Database Service) est le service managé AWS pour les **bases de données structurées (SQL)**. Il supporte plusieurs moteurs :

Moteur	Compatibilité	Cas d'usage
MySQL	Open source, très répandu	Applications web classiques
PostgreSQL	Open source, très avancé	Applications critiques, PostGIS, données complexes
MariaDB	Fork MySQL, meilleure performance	Alternative MySQL
Oracle	Propriétaire, très coûteux en on-prem	Migrations legacy, applications critiques
SQL Server	Windows, intégration Active Directory	Environnements Microsoft
Amazon Aurora	Natif AWS, ultra-performant	Haute disponibilité, haute scalabilité

Le choix du moteur ne change rien à l'expérience de gestion : quel que soit le moteur retenu, RDS apporte le même socle de fonctionnalités managées, détaillé ci-dessous.

Caractéristiques clés

Quel que soit le moteur choisi, RDS apporte un socle commun de fonctionnalités managées qui le distingue d'une base installée manuellement sur un serveur :

Fonctionnalité	Description
Haute disponibilité	Multi-AZ : réplication synchrone sur une autre zone de disponibilité avec basculement automatique
Sauvegardes automatisées	Sauvegardes quotidiennes, conservées jusqu'à 35 jours, restauration à un instant T
Sécurité intégrée	Chiffrement au repos (AWS KMS) et en transit (SSL/TLS), isolation réseau (VPC, Security Groups), audit (CloudTrail)
Scalabilité verticale	Augmenter CPU/RAM sans interruption (dans certains cas)
Read Replicas	Jusqu'à 5 réplicas de lecture asynchrones pour répartir les lectures
Maintenance automatisée	Patches et mises à jour sans intervention manuelle

La haute disponibilité (Multi-AZ) et les Read Replicas sont souvent confondues alors qu'elles répondent à des besoins différents : la première protège contre une panne, la seconde répartit la charge de lecture — les sections suivantes détaillent chacune.

Types de stockage EBS pour RDS

RDS s'appuie sur **Amazon EBS** pour son stockage sous-jacent. Vous choisissez le type de disque selon vos besoins :

Type	Performance	Coût	Cas d'usage
General Purpose (gp3)	3 000 à 16 000 IOPS	Modéré	Production standards, dev/test
Provisioned IOPS (io1/io2)	Jusqu'à 64 000 IOPS	Élevé	Bases critiques à fort volume transactionnel
Magnetic (standard)	100-200 IOPS	Bas	Archivage, données froides (déprécié)

IOPS (Input/Output Operations Per Second) = nombre d'opérations de lecture/écriture par seconde. Un IOPS élevé garantit des temps de réponse courts pour les transactions.

Exemple concret — Plateforme de vidéo à la demande (VOD)

Supposons une plateforme qui stocke **les métadonnées** de ses contenus : titres, descriptions, dates de diffusion, catégories, droits d'accès. Les **requêtes sont complexes** (jointures sur plusieurs tables), les données sont structurées, et la **cohérence** est critique.

Avec on-prem (avant cloud) :

- Installation MySQL manuelle (4h)
- Scripts de sauvegarde réseau + test restauration (8h)
- Monitoring 24h/24 avec alertes (contrat SLA)
- Augmentation disque lors des pics → risque de downtime
- Réplication vers DataCenter secondaire (investissement)
- Coût total 5 ans : ~100 000 €, équipe IT 2 personnes

Avec RDS AWS :

- Déploiement en 3 minutes via console AWS
- Multi-AZ automatique (zéro downtime en panne)
- Snapshots automatiques (jusqu'à 35 jours)
- Augmentation CPU/stockage sans interruption
- Read Replicas pour diffuser les lectures (reports analytiques)
- Coût annuel : ~1 500 \$, gestion minimal (0,1 FTE)

Le gain ne se limite pas au coût : c'est surtout la disponibilité qui change de nature, passant d'un risque géré manuellement à un mécanisme automatique intégré au service, comme détaillé ci-dessous.

1.3 Haute disponibilité et résilience dans RDS

Multi-AZ (Availability Zones) — Résilience automatique

Multi-AZ signifie que votre base est automatiquement répliquée **de manière synchrone** sur une autre zone de disponibilité (AZ) de la **même région**.

Avant failover : **AZ 1a** héberge le RDS Primary (Master), qui accepte lectures et écritures sur `mysql.xxxxx.rds.amazonaws.com`. Il réplique de façon synchrone vers un RDS Standby en lecture seule dans **AZ 1b**, sur un stockage EBS séparé.

Pendant le failover (~30 secondes), une fois la panne d' **AZ 1a** détectée :

1. CloudWatch déclenche une alarme de détection de panne.
2. Route 53 bascule le DNS : l'endpoint redirige vers le Standby.
3. Le Standby est promu Master dans **AZ 1b**.
4. Les nouvelles écritures sont acceptées en **AZ 1b**.
5. Perte de données = 0 (réplication synchrone).

6. Downtime réseau ≈ 30-60 secondes (le temps de la reconnexion).

Après failover : AZ 1a est en panne. AZ 1b héberge désormais le RDS Primary (ex-Standby), qui accepte lectures et écritures sur le **même endpoint** `mysql.xxxxx.rds.amazonaws.com` — aucune reconfiguration applicative n'est nécessaire. Un nouveau Standby est recréé et répliqué depuis ce nouveau Primary, ce qui résorbe la panne en environ 5 minutes.

Bénéfice : zéro downtime applicatif (reconnexion auto après failover), zéro perte données. **Coût :** +50 % sur la facture RDS. **À savoir :** L'endpoint DNS ne change pas → applications se reconnectent automatiquement.

Warning

Coût Multi-AZ RDS — Attention au budget

Activer Multi-AZ **double le coût de l'instance** RDS : AWS provisionne silencieusement une instance Standby dans une autre AZ. Cette instance n'est pas accessible en lecture (contrairement aux Read Replicas) — elle sert uniquement au failover automatique.

- db.t3.micro RDS MySQL : ~12 /mois → ****~24** /mois** avec Multi-AZ
- db.r6g.large RDS MySQL : ~175 /mois → ****~350** /mois** avec Multi-AZ

Règle : activez Multi-AZ uniquement en production. Pour dev/test, une instance simple suffit.

Read Replicas — Répartition de la charge de lecture

Un **Read Replica** est une **copie asynchrone** de votre base, destinée à répartir les **lectures** (SELECT) sans surcharger la Primary.

Cas d'usage : Reporting, Analytics, Exports.

Le RDS Primary (`eu-west-1a`) gère les écritures OLTP (500 SELECT/s + 100 INSERT/s) sur son endpoint `mysql.xxxx.rds...`. Il réplique de façon asynchrone (~100ms, avec un lag possible de quelques secondes) vers trois Read Replicas en lecture seule, chacun dédié à un usage distinct :

Replica	Zone	Charge	Usage
Read Replica 1	AZ 1b	1000 GET/s	Reporting DB
Read Replica 2	AZ 1c	1000 GET/s	Analytics DB
Read Replica 3	Region 2	1000 GET/s	Disaster recovery

Limite : jusqu'à **5 Read Replicas** par base MySQL/PostgreSQL (15 sur Aurora).

Différence clé Read Replica vs Multi-AZ :

- **Multi-AZ** : synchrone, failover automatique, latence identique
- **Read Replica** : asynchrone, pas failover, peut avoir lag de 1-30s, endpoint séparé

Commande AWS CLI :

```
1  # Créer un Read Replica pour reportings
2  aws rds create-db-instance-read-replica \
3      --db-instance-identifiant formation-db-analytics \
4      --source-db-instance-identifiant formation-db \
5      --db-instance-class db.t3.small \
6      --availability-zone eu-west-1b
7
8  # Créer un Replica inter-région (DR)
9  aws rds create-db-instance-read-replica \
10
11      --db-instance-identifiant formation-db-dr \
12
13      --source-db-instance-identifiant formation-db \
14
15      --source-region eu-west-1 \
16
17      --region us-east-1 \
18
19      --db-instance-class db.t3.small
```




Tip

Résultat attendu :

```
1  {
2      "DBInstance": {
3          "DBInstanceIdentifier": "formation-db-analytics",
4          "DBInstanceClass": "db.t3.small",
5          "Engine": "mysql",
6          "DBInstanceStatus": "creating",
7          "ReadReplicaSourceDBInstanceIdentifier": "formation-db",
8          "AvailabilityZone": "eu-west-1b",
9          "MultiAZ": false,
10
11         "StorageType": "gp3",
12
13         "Endpoint": {
14
15             "Address": "formation-db-analytics.abc123.eu-west-
16 1.rds.amazonaws.com",
17
18             "Port": 3306
19         }
20     }
21 }
```

1.4 Sécurité et supervision RDS

Une base de données RDS contient généralement des données sensibles (informations clients, identifiants, données métier) : elle doit donc être protégée par du chiffrement et surveillée en continu pour détecter toute anomalie ou tentative d'accès non autorisé. RDS propose deux volets complémentaires : le **chiffrement** des données elles-mêmes, et la **supervision** de l'activité et des performances de l'instance.

Chiffrement

RDS distingue deux états dans lesquels une donnée doit être protégée : au repos (stockée sur disque) et en transit (lors de son transfert entre le client et la base).

Type	Description
Au repos	Les données sur disque EBS sont chiffrées via AWS KMS
En transit	SSL/TLS obligatoire entre client et base

Le chiffrement au repos s'appuie sur **AWS KMS** (Key Management Service) : la clé chiffre le volume de stockage, les instantanés (snapshots) et les réplicas, de façon totalement transparente pour les applications. Le chiffrement en transit, lui, impose l'usage de SSL/TLS pour toute connexion au moteur de base de données, ce qui protège les données contre l'interception sur le réseau.

 **Important** : le chiffrement **doit être activé à la création** de l'instance. Il est impossible de chiffrer après coup une instance RDS existante : la seule solution consiste à créer un instantané, puis à restaurer ce dernier vers une nouvelle instance chiffrée.

Supervision et alertes

Une fois l'instance en production, plusieurs outils AWS permettent de suivre son état de santé, ses performances et les accès qui y sont effectués. Chacun couvre un besoin différent : métriques d'infrastructure, analyse des requêtes, audit de configuration ou détail au niveau du système d'exploitation.

Outil	Rôle
CloudWatch	Métriques de base (CPU, RAM, connexions, IOPS)
Performance Insights	Requêtes coûteuses, sessions actives
CloudTrail	Audit : qui a modifié la configuration
Enhanced Monitoring	Metrics granulaires de l'OS

En pratique, **CloudWatch** suffit pour une supervision de base (alertes sur le CPU ou l'espace disque), tandis que **Performance Insights** devient indispensable dès qu'un problème de lenteur applicative nécessite d'identifier les requêtes SQL les plus coûteuses. **CloudTrail** répond à un besoin de traçabilité et de conformité (qui a modifié quoi et quand), et **Enhanced Monitoring** apporte une vue fine des ressources de l'OS sous-jacent (jusqu'à la seconde), utile pour du diagnostic système avancé.

1.5 Amazon Aurora — Performance et résilience supérieures

Amazon Aurora est le **moteur de base de données propriétaire AWS**, conçu d’emblée pour le Cloud. Compatible avec MySQL et PostgreSQL mais offre des performances bien supérieures.

Architecture distribuée

Aurora découple le **calcul** (instances) du **stockage** (volume distribué). Les écritures sont répliquées **4 fois** sur 3 AZ.

Performances

Ce découplage calcul/stockage se traduit par des gains mesurables par rapport aux moteurs open source qu’Aurora reste compatible avec :

Métrique	vs MySQL	vs PostgreSQL
Throughput max	5×	3×
Basculement automatique	30 sec	30 sec
Réplicas de lecture	15 (vs 5)	15 (vs 5)

Ces chiffres ne sont pas de simples arguments marketing : ils découlent directement de l’architecture distribuée décrite plus haut, qui permet à Aurora de répliquer les écritures et de servir davantage de réplicas sans passer par le stockage EBS classique.

Comparaison détaillée : RDS MySQL/PostgreSQL vs Aurora

Pour choisir entre les deux moteurs en connaissance de cause, voici une comparaison exhaustive critère par critère :

Aspect	RDS MySQL/PostgreSQL	Aurora
Architecture	Stockage EBS couplé à l'instance	Calcul découplé du stockage distribué
Réplication	Synchrone (Multi-AZ) / Asynchrone (Replicas)	4 copies sur 3 AZ (natif)
Failover automatique	30 sec si Multi-AZ activé	30 sec (inclus, pas surcoût)
Réplicas de lecture	5 max	15 max
Coût Multi-AZ	+50 % sur l'instance	0 € (inclus)
Scaling en écriture	Impossible (Master unique)	Impossible (Master unique)
Scaling en lecture	Avec Replicas (asynchrones)	Avec Replicas (synchrone, <1ms latence)
Serverless	Non (ne s'adapte pas)	Oui (Aurora Serverless v2)
Performance requêtes complexes	Bonne	Excellente (optimisation native AWS)
Coût stockage	Paieement par Go alloué	Paieement à l'utilisation (auto-scaling)
Certification AWS	Sujet SAA-C03	Sujet SAA-C03 (fortement recommandé)

SAA-C03 est le code de l'examen AWS Certified Solutions Architect – Associate, la certification détaillée au Chapitre 1 (section 1.3) et reprise au Chapitre 5 (section 9) — le comparatif RDS/Aurora ci-dessus fait partie des sujets fréquemment évalués dans cet examen.

Mode Serverless et Provisioned

Aurora offre **deux modèles de déploiement** :

Modèle	Principe	Avantages	Cas d'usage
Provisioned (classique)	Vous choisissez la taille (ex. <code>db.r6g.xlarge</code>)	Coût fixe, performance prévisible	Application de reporting critique, charge stable (12h/jour)
Serverless V2 (moderne)	Scaling auto de 0.5 à 1000+ ACU (Aurora Compute Units)	Paiement à l'utilisation, scaling en ≈5 sec	API imprévisible, pics aléatoires, environnements de développement

Le choix entre les deux n'est pas figé : une même charge peut démarrer en Serverless V2 pendant sa phase de croissance imprévisible, puis basculer en Provisioned une fois le trafic stabilisé et prévisible.

Créer un cluster Aurora en CLI

Info

Une activité pratique permet d'approfondir le déploiement d'un cluster Aurora.

Cette séquence crée un cluster Aurora MySQL complet avec une instance writer, une instance reader, du chiffrement activé et une fenêtre de sauvegarde automatique. Aurora est un cluster — pas une instance unique — ce qui explique les deux commandes distinctes (cluster + instance).

```

1  # 1. Créer un Aurora MySQL Cluster (2 instances : writer + reader)
2  aws rds create-db-cluster \
3      --db-cluster-identifier formation-aurora-cluster \
4      --engine aurora-mysql \
5      --engine-version 8.0.mysql_aurora.3.02.0 \
6      --master-username admin \
7      --master-user-password 'SecurePassword123!' \
8      --database-name appdb \
9      --vpc-security-group-ids sg-12345678 \
10     --db-subnet-group-name my-db-subnet-group \
11     --storage-encrypted \
12     --backup-retention-period 7 \
13     --preferred-backup-window "03:00-04:00" \
14     --preferred-maintenance-window "mon:04:00-mon:05:00" \
15     --tag-specifications 'ResourceType=cluster,Tags=[{Key=Name,Value=FormationAuroraCluster}]'
16
17  # 2. Créer les instances Aurora (Writer)
18  aws rds create-db-instance \
19      --db-instance-identifier formation-aurora-writer \
20      --db-instance-class db.r6g.large \
21      --engine aurora-mysql \
22      --db-cluster-identifier formation-aurora-cluster \
23      --publicly-accessible false

```

```
2
4
2
5 # 3. Créer instance Reader (lecture seulement)
2
6 aws rds create-db-instance \
2
7     --db-instance-identifier formation-aurora-reader \
2
8     --db-instance-class db.r6g.large \
2
9     --engine aurora-mysql \
3
0     --db-cluster-identifier formation-aurora-cluster \
3
1     --promotion-tier 2 \
3
2     --publicly-accessible false
3
3
3
4 # 4. Créer Aurora Serverless V2 (auto-scaling)
3
5 aws rds create-db-cluster \
3
6     --db-cluster-identifier formation-aurora-serverless \
3
7     --engine aurora-postgresql \
3
8     --engine-version 14.6 \
3
9     --engine-mode provisioned \
4
0     --serverlessv2-scaling-configuration 'MinCapacity=0.5,MaxCapacity=16' \
4
1     --master-username admin \
4
2     --master-user-password 'SecurePassword123!' \
```

```

4
3  --database-name appdb \
4
4  --vpc-security-group-ids sg-12345678 \
4
5  --db-subnet-group-name my-db-subnet-group
4
6
4
7  # 5. Attendre disponibilité
4
8  aws rds wait db-cluster-available \
4
9  --db-cluster-identifier formation-aurora-cluster
5
0
5
1  # 6. Récupérer les endpoints
5
2  aws rds describe-db-clusters \
5
3  --db-cluster-identifier formation-aurora-cluster \
5
4  --query 'DBClusters[0].[DBClusterEndpoint,ReaderEndpoint]'
5
5  # Résultat :
5
6  # Writer : formation-aurora-cluster.xxxx.eu-west-1.rds.amazonaws.com
(écritures)
5
7  # Reader : formation-aurora-cluster-ro.xxxx.eu-west-1.rds.amazonaws.com
(lectures)
5
8
5
9  # 7. Modifier le scaling Serverless (augmenter capacité max)
6
0  aws rds modify-db-cluster \

```



```

6
1  --db-cluster-identifiant formation-aurora-serverless \
6
2  --serverlessv2-scaling-configuration 'MinCapacity=1,MaxCapacity=32' \
6
3  --apply-immediately
6
4
6
5  # 8. Ajouter un Read Replica dans une autre région
6
6  aws rds create-db-cluster \
6
7  --db-cluster-identifiant formation-aurora-dr \
6
8  --engine aurora-mysql \
6
9  --source-region eu-west-1 \
7
0  --region us-east-1 \
7
1  --replication-source-identifiant arn:aws:rds:eu-west-
1:123456789012:cluster:formation-aurora-cluster

```



Tip

Résultat attendu :

```
1  {
2      "DBCluster": {
3          "DBClusterIdentifier": "formation-aurora-cluster",
4          "Status": "creating",
5          "Engine": "aurora-mysql",
6          "EngineVersion": "8.0.mysql_aurora.3.02.0",
7          "DBClusterEndpoint": "formation-aurora-cluster.cluster-abc123.eu-
west-1.rds.amazonaws.com",
8          "ReaderEndpoint": "formation-aurora-cluster.cluster-ro-abc123.eu-
west-1.rds.amazonaws.com",
9          "MultiAZ": true,
10         "StorageEncrypted": true,
11         "BackupRetentionPeriod": 7
12     }
13 }
```

Résultat attendu : `create-db-cluster` retourne le JSON du cluster (état `creating`). `rds wait db-cluster-available` bloque jusqu'à ce que le cluster soit prêt (peut prendre 5-10 min). `describe-db-clusters` affiche les endpoints writer et reader — deux URLs distinctes.

À retenir : Aurora est le **choix standard** pour les productions critiques AWS — meilleur ROI que RDS classique quand on inclut les coûts Multi-AZ.

Comparatif de coûts réel — RDS vs Aurora (région Paris, eu-west-3)

Tarifs indicatifs au 1er avril 2026 — à vérifier sur <https://aws.amazon.com/rds/pricing/>

Instance (calcul)

Premier poste de coût à comparer : le prix horaire de l'instance elle-même, qui varie sensiblement selon la taille choisie :

Type	RDS MySQL	Aurora MySQL	Écart
db.t3.micro	**0,017 /h** (~12 /mois)	✗ Non disponible	—
db.t3.small	0,034 /h (~25 /mois)	✗ Non disponible	—
db.t3.medium	0,068 /h (~49 /mois)	**0,073 /h** (~53 /mois)	+7 %
db.r6g.large	0,240 /h (~175 /mois)	0,260 /h (~190 /mois)	+8 %
db.r6g.2xlarge	0,960 /h (~700 /mois)	1,040 /h (~760 /mois)	+8 %

⚠ **Aurora ne propose pas de t3.micro ou t3.small.** L'entrée de gamme est le t3.medium (~53 \$/mois). Pour un usage de formation ou de très petite charge, RDS est **nettement moins cher**.

Stockage et I/O

Second poste de coût, souvent négligé lors d'un premier chiffrage : le stockage et les I/O, où Aurora inverse la tendance observée sur le calcul en devenant moins cher au Go, réplication comprise :

	RDS (gp3)	Aurora
Prix stockage	0,115 \$/Go/mois	0,10 \$/Go/mois
Minimum	20 Go (provisionné)	10 Go (auto-extensible)
Maximum	64 To	128 To
Réplication	1 copie (Multi-AZ = 2×)	6 copies dans 3 AZ (inclus)
I/O	Inclus (gp3)	0,20 \$ / million de requêtes (Standard) ou inclus (I/O-Optimized +25%)

À volume équivalent, Aurora reste donc plus cher sur le calcul mais moins cher sur le stockage — le choix final dépend du ratio entre les deux pour votre charge de travail, ce que l'exemple chiffré ci-dessous permet d'illustrer.

Aurora Serverless v2 — facturation à l'utilisation

Contrairement au mode Provisioned facturé à l’heure d’instance, Serverless v2 facture la capacité réellement consommée, exprimée en ACU (Aurora Capacity Unit) :

Facturation : ACU-heure (Aurora Capacity Unit)
1 ACU = ~2 Go de RAM + CPU proportionnel

Tarif : 0,12 \$ / ACU-heure (Paris)
Minimum : 0,5 ACU → 0,06 \$/h → ~43 \$/mois au repos
Maximum : 128 ACU → 15,36 \$/h

Avantage : scale automatiquement de 0,5 à 128 ACU en quelques secondes
Cas idéal : applications avec trafic très variable (pics journaliers, saisonnalité)

Ces tarifs unitaires restent abstraits tant qu’on ne les applique pas à un cas concret comparé côte à côte avec RDS classique.

Exemple comparatif — Application web standard, 50 Go de données

Pour donner un sens concret à ces tarifs unitaires, voici comment ils se traduisent sur un cas réel comparé côte à côte avec RDS classique :

	RDS MySQL	Aurora MySQL (Serverless v2)
Instance	db.t3.medium 49 \$/mois	0,5-4 ACU (charge variable) ~65-120 \$/mois
Stockage	50 Go × 0,115 = 5,75 \$/mois	50 Go × 0,10 = 5,00 \$/mois
I/O	Inclus	~5 \$/mois (applis légères)
Multi-AZ	+100 % instance = +49 \$/mois	Inclus (6 copies) = 0 \$
TOTAL Multi-AZ	~104 \$/mois (coût fixe, prévisible)	~75-125 \$/mois (variable, mais HA incluse)

 **Conclusion** : Aurora est plus cher à l’instance mais inclut la haute disponibilité sur 6 copies. Pour une charge faible et un budget serré : **RDS**. Pour une charge critique nécessitant de la résilience et de la scalabilité : **Aurora**.

 [AWS RDS Pricing](#)  [AWS Aurora Pricing](#)

1.6 Amazon DynamoDB — NoSQL à ultra-haute scalabilité

DynamoDB est une **base NoSQL managée** d’AWS. Elle stocke des **documents JSON** ou des **paires clé-valeur** sans schéma figé. Elle garantit des temps de réponse **en millisecondes**, même à l’échelle de millions de requêtes par seconde.

Structure d’une table DynamoDB

Contrairement à une table SQL, une table DynamoDB n’impose pas de colonnes fixes : seule la clé primaire est obligatoire, chaque type d’attribut étant identifié par un code court utilisé dans l’API et les requêtes :

Chaque attribut peut être :

- Chaîne (S)
- Nombre (N)
- Binaire (B)
- Ensemble (SS, NS, BS)
- Map (document imbriqué)
- Liste

Aucune contrainte de schéma : vous pouvez ajouter des attributs par ligne.

Cette absence de schéma figé permet de faire évoluer la structure des données au fil du temps sans migration, mais déplace la responsabilité de la cohérence des données depuis la base de données vers le code applicatif.

Modèles de tarification

DynamoDB propose deux façons de payer selon la prévisibilité de votre trafic :

Modèle	Débit garanti	Facturation	Cas d’usage
Provisionné	À définir (RCU/WCU)	Fixe + dépassement	Charge prédictible
À la demande	Illimité	Au débit réel	Charge variable

- **RCU** (Read Capacity Unit) : 1 RCU = une lecture fortement cohérente d’un item ≤ 4 Ko
- **WCU** (Write Capacity Unit) : 1 WCU = une écriture d’un item ≤ 1 Ko

Exemple chiffré — mode à la demande (région Paris) : facturé 0,32 par million de lectures et 1,60 par million d’écritures. Une application avec 500 000 lectures/jour et 50 000 écritures/jour coûte environ $(500\,000 \times 30 \times 0,32 / 1\,000\,000) + (50\,000 \times 30 \times 1,60 / 1\,000\,000) \approx 4,80 + 2,40 = \sim 7,20$ /mois rien que pour les requêtes, plus le stockage (0,25 /Go/mois). Pour une charge stable et prévisible, le mode provisionné revient souvent moins cher — mais il facture même en l’absence de trafic, contrairement au mode à la demande.

DynamoDB vs RDS

Ce choix de modèle de tarification n'est qu'un aspect parmi d'autres différences structurelles entre DynamoDB et une base relationnelle comme RDS :

Aspect	RDS (SQL)	DynamoDB (NoSQL)
Schéma	Structuré, relationnel	Flexible, semi-structuré
Requêtes	Complexes (jointures)	Simplees (accès par clé)
Scalabilité	Verticale surtout	Horizontale, ultra-massive
Latence	ms-s	ms

Cette scalabilité horizontale « ultra-massive » n'est pas magique : elle repose sur un découpage des données en partitions, et un mauvais choix de clé de partition peut annuler cet avantage en concentrant tout le trafic sur une seule partition, comme détaillé ci-dessous.

Danger

Hot Partitions DynamoDB — Erreur de conception fréquente

DynamoDB distribue vos données sur des partitions selon la **Partition Key**. Si vous choisissez une clé avec peu de valeurs distinctes (ex. : `status = "active"/"inactive"`, ou une date comme `2026-05-17`), toutes les requêtes frappent la **même partition** → throttling, latence explosive.

Symptômes : erreurs `ProvisionedThroughputExceededException`, latences P99 > 500ms.

Solutions :

- Choisir une Partition Key à **haute cardinalité** (UUID utilisateur, ID produit unique)
- Ajouter un **suffixe aléatoire** (write sharding) : `user_id#1`, `user_id#2`
- Utiliser un **Global Secondary Index** avec une clé mieux distribuée
- Passer en mode **On-Demand** pour absorber les pics sans throttling

1.7 Migration de bases de données avec AWS DMS

AWS DMS (Database Migration Service) permet la **migration sans interruption** d'une base existante vers AWS. C'est un service managé qui gère le transport, la transformation et la synchronisation de données.

Scénario réaliste de migration

Imaginez une **entreprise audiovisuelle** avec une **base Oracle on-premise** stockant les **métadonnées de contenus** (titres, droits, calendriers). Actuellement :

- **5 To de données** (10 ans d'historique)
- **Uptime critique** : 24h/24
- **Migration programmée** : 1 week-end

Sans DMS : arrêt services, export/import manuel, validation longue, risque perte données. **Avec DMS** : continuité service, réplication live, bascule maîtrisée.

Architecture de migration DMS

La source (Oracle Database on-premise/RDS, 5 To, 50 tables, actif à 1000 txn/s) passe par une tâche DMS (instance `dms.c6i.xlg`) qui exécute d'abord un Full Load (4-6 heures) vers la cible (Aurora/RDS PostgreSQL, 5 To), puis bascule en CDC (Change Data Capture) continu avec un lag inférieur à 1 seconde pour maintenir la cible synchronisée en temps réel.

Timeline type :

- J-1 : configuration DMS, full load lancé de nuit.
- J+0 : full load terminé, CDC actif, l'application reste encore on-premise.
- J+0 20h00 : validation des données, préparation du cutover.
- J+0 22h00 : cutover — redirection de l'application vers Aurora.
- J+1 : validation complète, monitoring 24h.

Cette timeline repose sur trois phases techniques distinctes, détaillées ci-dessous.

Processus de migration par étapes

Phase 1 — Full Load (copie complète) : les données transitent de la source vers la cible via un DMS Agent. Tous les types de données, tous les indices et les contraintes PRIMARY sont copiés automatiquement ; en revanche, les triggers et procédures stockées doivent être recréés manuellement.

Phase 2 — CDC (Change Data Capture) : DMS lit les logs natifs de la source (redo logs pour Oracle, binary logs pour MySQL, WAL pour PostgreSQL) et les applique sur la cible en quasi temps réel (latence sous la milliseconde).

Phase 3 — Cutover (basculement applicatif) :

- 1. Arrêter l’application (2-5 min).
- 2. Valider que le lag de réplication est proche de 0.
- 3. Rediriger les connexions vers la cible.
- 4. Vérifier les logs applicatifs.
- 5. Garder un plan de rollback armé.

La difficulté de ce cutover varie fortement selon le type de migration entrepris, comme le montre le tableau suivant.

Types de migrations DMS

La complexité d’une migration DMS dépend surtout de l’écart entre le moteur source et le moteur cible :

Type	Exemple	Complexité	Coût
Homogène (même moteur)	MySQL → RDS MySQL	★ Très facile	Bas
Hétérogène (moteurs diff)	Oracle → Aurora PostgreSQL	★ ★ ★ Moyen	Moyen
Schéma complexe	DB2 → PostgreSQL (types custom)	★ ★ ★ ★ Élevé	Élevé

L’exemple Oracle → Aurora PostgreSQL de ce chapitre se situe dans la catégorie hétérogène : DMS gère la conversion des types de données courants, mais les procédures stockées et triggers spécifiques à Oracle doivent être portés manuellement en PL/pgSQL.

Créer une tâche DMS en CLI



Info

Une activité pratique permet d’approfondir AWS Database Migration Service.

DMS fonctionne en 3 objets : un **endpoint source** (base existante), un **endpoint cible** (base AWS), et une **instance de réplication** (le moteur qui exécute la migration). On les crée dans cet ordre, puis on démarre la tâche.


```

1  # =====
2  # 1. Créer les Endpoints source et cible
3  # =====
4
5  # Endpoint SOURCE (Base existante on-prem/RDS)
6  aws dms create-endpoint \
7      --endpoint-identifier oracle-source \
8      --endpoint-type source \
9      --engine-name oracle \
10
11      --server-name oracle.company.local \
12
13      --port 1521 \
14
15      --database-name PRODDDB \
16
17      --username migration_user \
18
19      --password 'SourcePassword123!' \
20
21      --ssl-mode require \
22
23      --extra-connection-attributes 'trustServerCertificate=false'
24
25
26
27
28 # Endpoint TARGET (Aurora PostgreSQL)
29
30 aws dms create-endpoint \
31
32      --endpoint-identifier aurora-target \
33
34      --endpoint-type target \
35
36      --engine-name aurora-postgresql \
37
38      --server-name formation-aurora.xxxxx.eu-west-1.rds.amazonaws.com \

```

```

2
4  --port 5432 \
2
5  --database-name appdb \
2
6  --username migration_user \
2
7  --password 'TargetPassword123!' \
2
8  --ssl-mode require
2
9
3
0  # =====
3
1  # 2. Créer une instance DMS (compute qui exécute la migration)
3
2  # =====
3
3
3
4  aws dms create-replication-instance \
3
5  --replication-instance-identifier formation-dms-instance \
3
6  --replication-instance-class dms.c6i.xlarge \
3
7  --allocated-storage 200 \
3
8  --vpc-security-group-ids sg-12345678 \
3
9  --replication-subnet-group-identifier my-dms-subnet-group \
4
0  --multi-az \
4
1  --engine-version 3.4.7 \
4
2  --publicly-accessible false \

```

```

4
3  --tag-specifications 'ResourceType=rep,Tags=[{Key=Name,Value=FormationDMS}]'
4
4
4
5  # Attendre disponibilité (5-10 min)
4
6  aws dms wait replication-instance-available \
4
7  --filters 'Name=replication-instance-id,Values=formation-dms-instance'
4
8
4
9  # =====
5
0  # 3. Valider connexions (test avant migration)
5
1  # =====
5
2
5
3  # Tester accès source
5
4  aws dms test-connection \
5
5  --replication-instance-arn arn:aws:dms:eu-west-1:123456789012:rep:formation-
dms-instance \
5
6  --endpoint-arn arn:aws:dms:eu-west-1:123456789012:ep:oracle-source
5
7
5
8  # Tester accès cible
5
9  aws dms test-connection \
6
0  --replication-instance-arn arn:aws:dms:eu-west-1:123456789012:rep:formation-
dms-instance \

```

```

6
1  --endpoint-arn arn:aws:dms:eu-west-1:123456789012:ep:aurora-target
6
2
6
3  # =====
6
4  # 4. Créer tâche DMS (FULL LOAD + CDC)
6
5  # =====
6
6
6
7  aws dms create-replication-task \
6
8  --replication-task-identifier oracle-to-aurora-migration \
6
9  --source-endpoint-arn arn:aws:dms:eu-west-1:123456789012:ep:oracle-source \
7
0  --target-endpoint-arn arn:aws:dms:eu-west-1:123456789012:ep:aurora-target \
7
1  --replication-instance-arn arn:aws:dms:eu-west-1:123456789012:rep:formation-
dms-instance \
7
2  --migration-type cdc \
7
3  --table-mappings '{
7
4      "rules": [
7
5          {
7
6              "rule-type": "selection",
7
7              "rule-name": "include-all-tables",
7
8              "object-locator": {
7
9              "schema-name": "%",

```

```

8
0      "table-name": "%"
8
1      },
8
2      "rule-action": "include"
8
3      }
8
4  ]
8
5  }' \
8
6  --replication-task-settings '{
8
7      "TargetMetadata": {
8
8          "TargetSchema": "",
8
9          "SupportsCascadeDelete": true,
9
0          "FullLobMode": false,
9
1          "LobChunkSize": 64,
9
2          "LobMaxSize": 32
9
3      },
9
4      "FullLoadSettings": {
9
5          "TargetSchema": "",
9
6          "CreatePkAfterFullLoad": false,
9
7          "StopTaskCachedChangesNotApplied": false,
9
8          "StopTaskCachedChangesApplied": false,

```

```

9
9     "MaxFullLoadSubTasks": 8,
1
0
0     "TransactionConsistencyTimeout": 600,
1
0
1     "CommitRate": 50000
1
0
2 },
1
0
3 "Logging": {
1
0
4     "EnableLogging": true,
1
0
5     "LogComponents": [
1
0
6         {
1
0
7             "Id": "SOURCE_UNLOAD",
1
0
8             "Severity": "LOGGER_SEVERITY_DEFAULT"
1
0
9         }
1
1
0     ]
1
1
1 },

```

```

1
1
2     "ChangeProcessingDdlHandlingPolicy": {
1
1
3         "HandleSourceTableDropped": true,
1
1
4         "HandleSourceTableTruncated": true,
1
1
5         "HandleSourceTableAltered": true
1
1
6     }
1
1
7 }' \
1
1
8 --tags Key=Project,Value=Migration Key=Type,Value=Oracle2Aurora
1
1
9
1
2
0 # =====
1
2
1 # 5. Attendre tâche et monitorer statut
1
2
2 # =====
1
2
3
1
2
4 # Status : creating → modifying → ready → running → stopped

```

```

1
2
5 aws dms describe-replication-tasks \
1
2
6 --filters 'Name=replication-task-arn,Values=arn:aws:dms:eu-west-
1:123456789012:task:oracle-to-aurora-migration'

```

Tip

Résultat attendu :

```

1 {
2   "ReplicationTasks": [{
3     "ReplicationTaskIdentifier": "oracle-to-aurora-migration",
4     "Status": "running",
5     "MigrationType": "cdc",
6     "ReplicationTaskStats": {
7       "FullLoadProgressPercent": 100,
8       "ElapsedTimeMillis": 18000000,
9       "TablesLoaded": 50,
10      "TablesQueued": 0,
11      "TablesErrored": 0,
12      "TablesLoading": 0
13    }
14  }]
15 }

```



```
1 # Voir les tables migrées (status, nb rows)
2 aws dms describe-table-statistics \
3     --replication-task-arn arn:aws:dms:eu-west-1:123456789012:task:oracle-to-
aurora-migration \
4     --query 'TableStatistics[*].
[SchemaName,TableName,FullLoadRows,FullLoadErrorRows,Updates,Inserts,Deletes]'
```



Résultat attendu :

```
1 [
2     ["PRODDB", "CONTENT_META", 2500000, 0, 1250, 340, 12],
3     ["PRODDB", "RIGHTS_TABLE", 180000, 0, 45, 8, 1],
4     ["PRODDB", "CALENDAR_SLOTS", 95000, 0, 320, 120, 5],
5     ["PRODDB", "USERS", 50000, 0, 88, 15, 0]
6 ]
```

```

1  # =====
2  # 6. Commandes gestion tâche
3  # =====
4
5  # Arrêter tâche (sans supprimer)
6  aws dms stop-replication-task \
7      --replication-task-arn arn:aws:dms:eu-west-1:123456789012:task:oracle-to-
aurora-migration
8
9  # Relancer tâche
1
0  aws dms start-replication-task \
1
1      --replication-task-arn arn:aws:dms:eu-west-1:123456789012:task:oracle-to-
aurora-migration \
1
2      --start-replication-task-type resume-processing
1
3
1
4  # Redémarrer complet (FULL LOAD + CDC)
1
5  aws dms start-replication-task \
1
6      --replication-task-arn arn:aws:dms:eu-west-1:123456789012:task:oracle-to-
aurora-migration \
1
7      --start-replication-task-type cdc
1
8
1
9  # Supprimer tâche
2
0  aws dms delete-replication-task \
2
1      --replication-task-arn arn:aws:dms:eu-west-1:123456789012:task:oracle-to-
aurora-migration

```

Ces commandes de gestion suffisent pour piloter le cycle de vie d’une tâche DMS ; en pratique, quelques erreurs reviennent régulièrement lors des migrations réelles.

Pièges et bonnes pratiques DMS

Voici les erreurs les plus courantes constatées lors de migrations DMS en production, et comment les éviter :

Piège	Solution
Oublier full load avant CDC	Toujours : <code>migration-type cdc</code> (inclut full load)
Schema incomplet après migration	Procs stockées, triggers = manuels. DMS copie juste data
CDC lag important	Vérifier capacité DMS instance, réduire <code>MaxFullLoadSubTasks</code>
Incompatibilités types données	Utiliser Schema Conversion Tool (SCT) avant DMS pour préparer
Oublier transaction logs source	Source doit activer binary logs (MySQL) / redo logs (Oracle)
Cutover sans validation données	Test requêtes applicatives sur cible avant redirection

Le piège le plus coûteux reste le schéma incomplet : DMS donne l’illusion d’une migration totale alors qu’il ne copie que les données, laissant la logique métier embarquée dans la base (procédures, triggers, vues) entièrement à la charge de l’équipe de migration.

2. Amazon VPC — Concevoir un réseau privé sécurisé

2.1 Du réseau on-prem au réseau virtuel

Définition — Qu’est-ce qu’une VPC ?

Amazon VPC (Virtual Private Cloud) est un **réseau privé virtuel isolé** que vous créez dans AWS. C’est votre **datacenter logique**, entièrement contrôlé, où vous déployez vos instances EC2, vos bases RDS, vos Load Balancers, etc.

Une VPC vous offre :

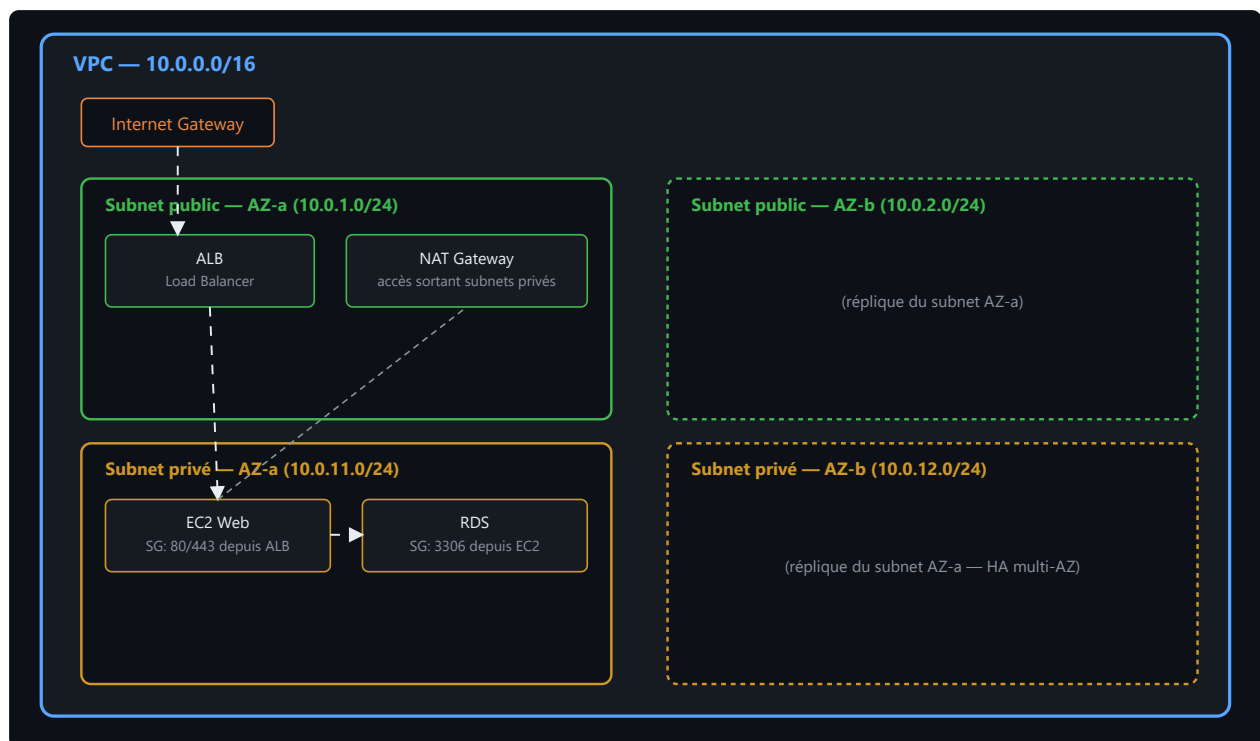
- **Isolation logique** : vos ressources ne sont pas visibles aux autres comptes AWS.

- **Contrôle total** : vous définissez les adresses IP, les routes, les pare-feux.
- **Flexibilité** : ajouter ou retirer des subnets, des passerelles à volonté.

Cette flexibilité repose sur un petit nombre de briques de base — CIDR, subnets, passerelles — que la section suivante détaille une par une.

2.2 Composants clés d'une VPC

Schéma architectural complet d'une VPC sécurisée



Flux de trafic :

1. Internet → ALB (IGW ouvre l'accès)
2. ALB → EC2 Web (Security Group + règles subnet)
3. EC2 → RDS (Security Group DB ouvre port 3306)
4. EC2 → Internet (via NAT Gateway, pour updates)

Ce schéma d'ensemble repose sur quatre briques de base, détaillées une par une dans la suite de cette section, à commencer par le plan d'adressage IP.

1. CIDR Block (Classless Inter-Domain Routing)

Une VPC commence par une **plage d'adresses IP privées**. Par exemple, `10.0.0.0/16` signifie :

`10.0.0.0/16` = 65 536 adresses disponibles (10.0.0.0 à 10.0.255.255)

Adresses réservées AWS :

`10.0.0.0` = Adresse réseau (réservée)
`10.0.0.1` = Gateway AWS (réservée)
`10.0.0.2` = DNS AWS (réservé)
`10.0.0.3` = Réserve pour l'avenir
`10.0.0.4-...` = EC2, RDS, ALB, etc.

Plages privées standards (RFC 1918) :

- `10.0.0.0/8` (la plus courante, 16 M d'adresses)
- `172.16.0.0/12` (1 M d'adresses)
- `192.168.0.0/16` (65 536 adresses)

Une fois ce CIDR défini pour toute la VPC, il reste à le découper en sous-blocs plus petits — les subnets — chacun rattaché à une zone de disponibilité précise.

2. Subnets (Sous-réseaux)

Un subnet est une **subdivision logique** d'une VPC, liée à une **zone de disponibilité (AZ)** spécifique.

Subnet public : route vers Internet Gateway → instances accessibles depuis Internet. **Subnet privé** : route vers NAT Gateway → instances qui accèdent Internet, mais non accessibles de l'extérieur.

3. Internet Gateway (IGW)

L'**Internet Gateway** est la **passerelle de sortie vers Internet**.

Coût : 0 € (sauf si vous utilisez des adresses Elastic IP).

4. NAT Gateway

Permet aux instances **privées** d'accéder à Internet **de manière sécurisée** sans être exposées.

Coût : `0.032/h` + `0.032` par Go transféré.

Coût des adresses IPv4 publiques — Point important depuis 2024

Depuis le **1er février 2024**, AWS facture **toutes les adresses IPv4 publiques**, y compris celles attachées à une instance EC2 en cours d'exécution.

Le tarif : `$0,005 / heure` par adresse IPv4 publique, soit **~3,65 \$ / mois** par IP.

Avant février 2024 :

- ✓ IP publique attachée à une instance → GRATUIT
- ✗ Elastic IP non attachée → 0,005 \$/h

Depuis février 2024 :

- ✗ IP publique attachée à EC2 → 0,005 \$/h ← NOUVEAU
- ✗ IP publique sur ELB → 0,005 \$/h ← NOUVEAU
- ✗ IP publique sur RDS → 0,005 \$/h ← NOUVEAU
- ✗ IP publique sur NAT Gateway → 0,005 \$/h ← NOUVEAU
- ✗ Elastic IP non attachée → 0,005 \$/h (inchangé)

Pourquoi cette décision ?

L'épuisement des adresses IPv4 est un problème mondial. Il ne reste plus d'adresses disponibles dans le registre IANA depuis 2011. AWS détient environ **130 millions d'adresses IPv4**, qu'il doit acheter sur le marché secondaire.

Prix de marché d'une adresse IPv4 en 2024 : environ 55 \$ l'adresse
AWS facture 0,005 \$/h × 8 760 h/an = 43,80 \$/an par IP
→ AWS récupère son investissement en ~15 mois par adresse

Impact concret :

Scénario	Nombre d'IPs	Coût mensuel
1 instance EC2 avec IP publique	1	~3,65 \$
10 instances EC2 + 1 ELB	~12	~43,80 \$
Architecture 3-tiers standard	~5-8	~18-29 \$

💡 **Bonne pratique** : utiliser IPv6 là où c'est possible (gratuit), réduire le nombre de ressources avec IP publique, et regrouper les accès via un seul Load Balancer ou une NAT Gateway.

👤 [AWS — Annonce facturation IPv4 \(2023\)](#)

AWS = plateforme 100 % API — À quoi servent vraiment les Elastic IPs ?

Vous n'avez jamais besoin d'une IP publique pour piloter AWS. Créer une instance EC2, configurer un VPC, déployer une Lambda — tout cela se fait via l'API AWS, que vous passiez par la console web, la CLI ou un SDK.

Vous (navigateur / terminal / code)

HTTPS → api.aws.amazon.com
(pas besoin d'IP publique sur vos ressources)



AWS Control Plane (infrastructure interne AWS)



Votre ressource (EC2, RDS, Lambda...)
dans votre VPC privé

Le **Control Plane** est entièrement géré par AWS et accessible depuis Internet sur ses propres IPs. Vos ressources peuvent très bien vivre dans un subnet privé sans aucune IP publique.

💡 Une instance EC2 sans IP publique est tout à fait fonctionnelle si elle n'a pas besoin d'être atteinte depuis Internet. Elle peut appeler des services AWS (S3, DynamoDB, SSM...) via des VPC Endpoints, sans jamais exposer la moindre adresse publique.

Les cas d'usage légitimes d'une Elastic IP :

Cas d'usage	Pourquoi une EIP est nécessaire
Whitelist IP chez un partenaire	Le firewall du client autorise uniquement votre IP fixe. Si l'instance redémarre avec une nouvelle IP, la connexion est bloquée.
Failover manuel rapide	En cas de panne, vous réassignez l'EIP de l'instance morte vers une instance de remplacement en quelques secondes — sans changer la DNS ni attendre la propagation.
NAT Gateway	Obligatoire : le NAT Gateway a toujours besoin d'une EIP pour sortir sur Internet au nom des instances privées.
Serveur mail (SMTP)	Les serveurs de messagerie tiers filtrent par IP source. Une IP fixe est indispensable pour la réputation mail.
VPN ou tunnel IPsec	L'extrémité du tunnel doit être connue à l'avance et ne pas changer.

Les cas où une EIP n'est PAS la bonne réponse :

Mauvais réflexe	Meilleure alternative
”Je veux accéder à mon serveur web depuis Internet”	→ Load Balancer avec DNS (pas d’IP fixe nécessaire)
“Je veux une URL stable pour mon API”	→ Route 53 + nom de domaine (DNS, pas IP)
“Je veux accéder à mon instance en SSH”	→ Session Manager (SSM) : SSH sans IP publique, sans port 22 ouvert
”Je veux que mes Lambda puissent appeler une API externe”	→ NAT Gateway (une seule EIP pour tout le subnet)

⚠ **Le réflexe à éviter** : assigner une Elastic IP à chaque instance “au cas où”. C’est la première source de surcoût inutile et de surface d’attaque élargie. Préférez toujours un accès via Load Balancer, Route 53, ou SSM Session Manager.

Architecture naïve (coûteuse et risquée) :

```

EC2-web    ← EIP ($3,65/mois, port 80/443 ouvert sur toute l'IP)
EC2-api    ← EIP ($3,65/mois, port 8080 ouvert)
EC2-admin  ← EIP ($3,65/mois, port 22 ouvert)
Total IPs  : 3 × 3,65 = 10,95 $/mois + surface d'attaque maximale

```

Architecture correcte (économique et sécurisée) :

```

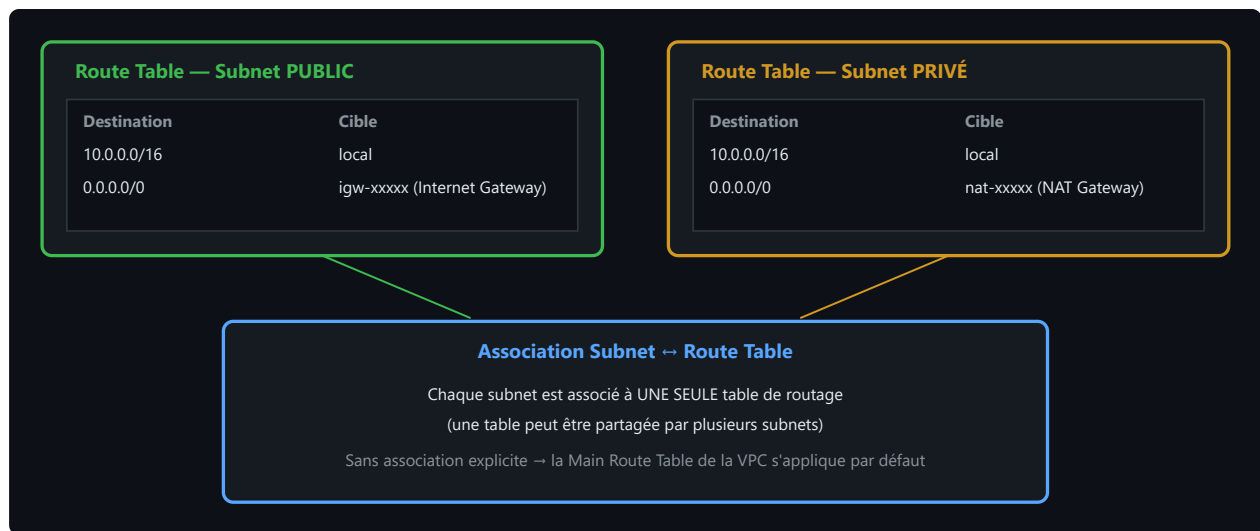
ALB        ← 1 EIP ($3,65/mois, ports 80/443 uniquement)
  - EC2-web  (privé, pas d'IP publique)
  - EC2-api  (privé, pas d'IP publique)
EC2-admin  (privé) ← SSM Session Manager (0 IP publique, 0 port ouvert)
Total IPs  : 1 × 3,65 = 3,65 $/mois + sécurité maximale

```

 [AWS Systems Manager Session Manager](#)  [Elastic IP Addresses — Documentation AWS](#)

5. Route Tables (Tables de routage)

Chaque subnet est associé à une **table de routage** qui définit comment le trafic circule.



2.3 Sécurité réseau — Security Groups et Network ACLs

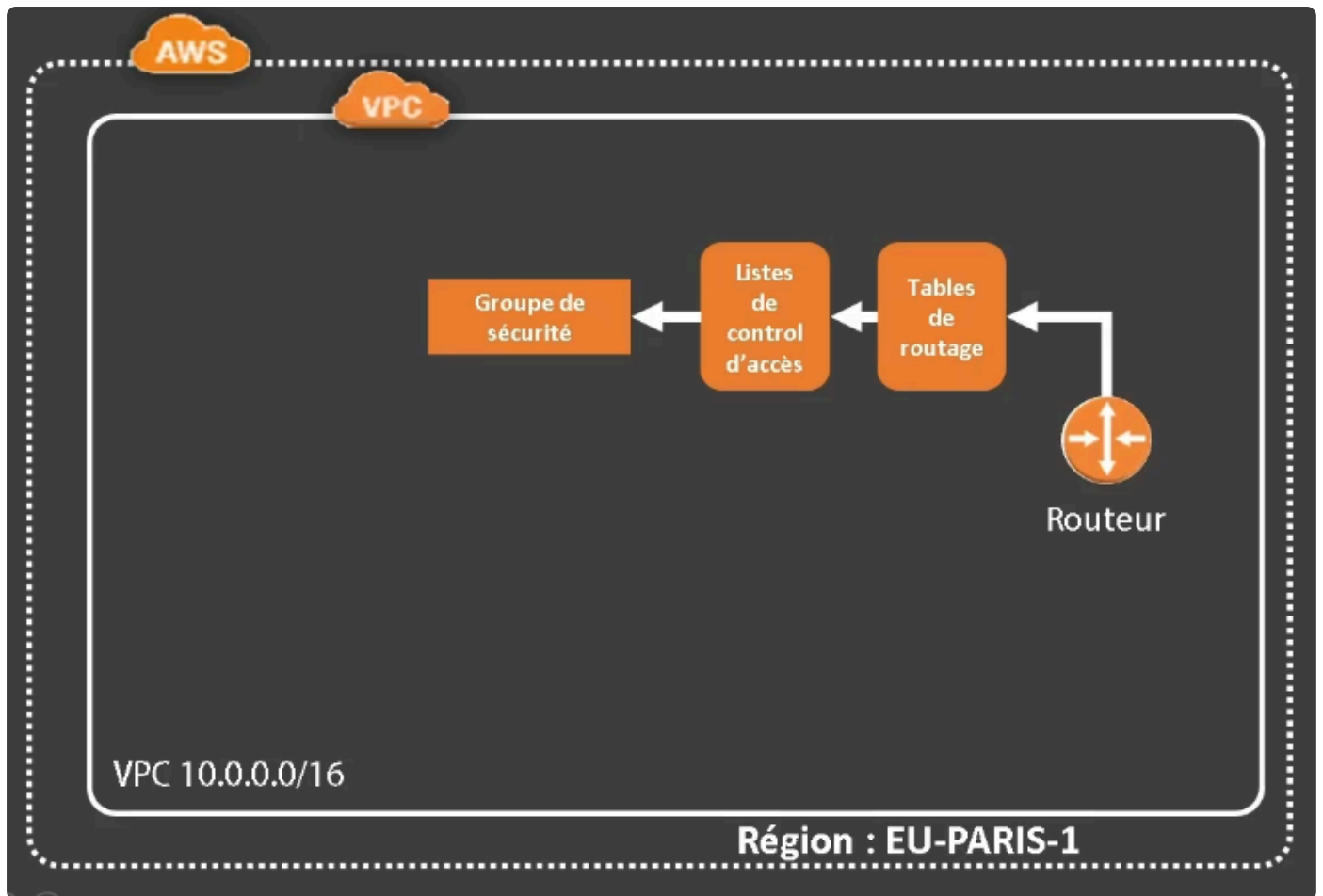
La sécurité dans une VPC repose sur **deux couches** complémentaires.

Security Groups — Pare-feu au niveau instance

Un **Security Group** est un ensemble de **règles de filtrage** appliquées à une ou plusieurs instances EC2.

Caractéristiques :

- **Stateful** : si vous autorisez les demandes entrantes, les réponses sortantes sont automatiquement autorisées.
- Changements appliqués **immédiatement**.
- Peut être modifié sur une instance en cours d'exécution.



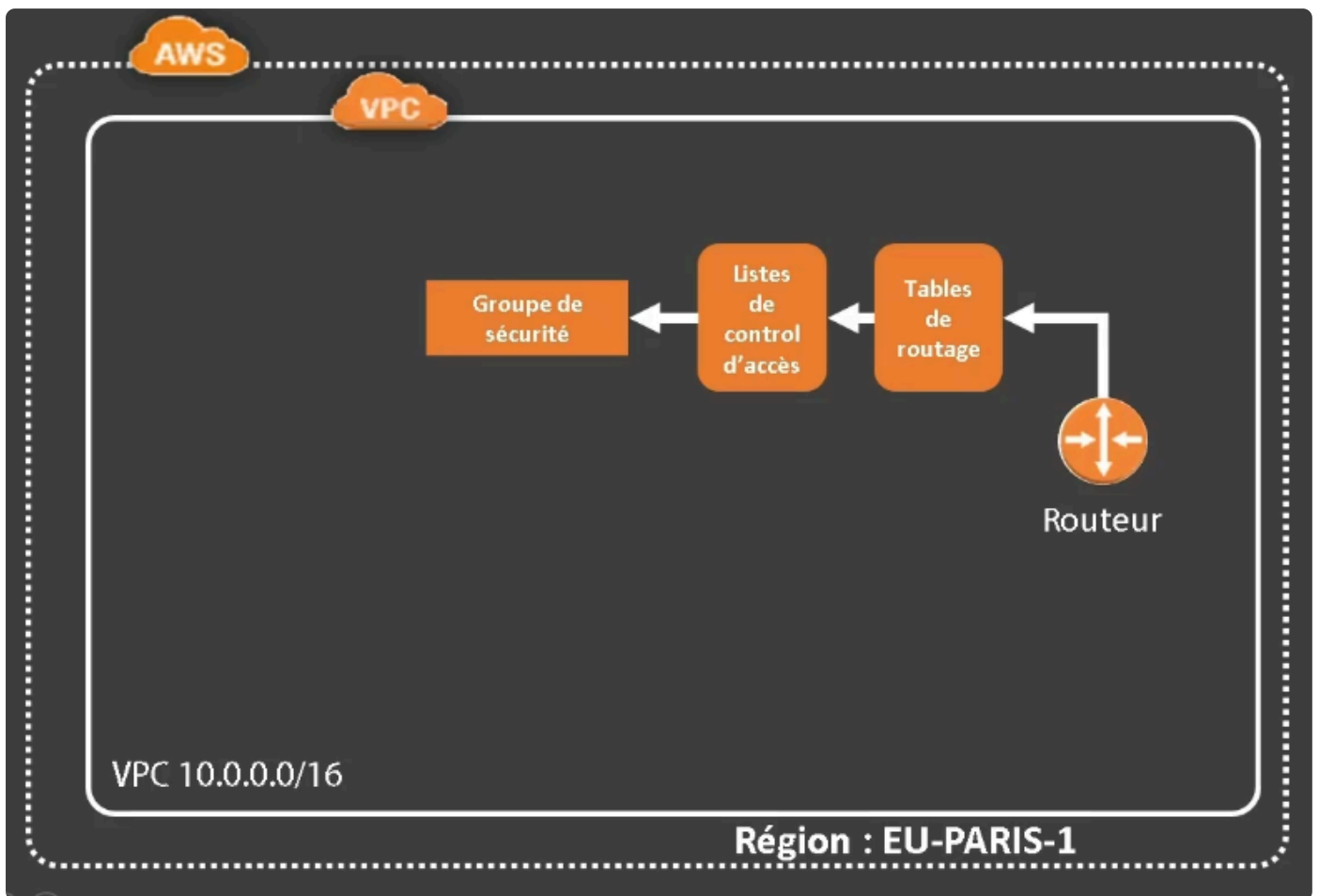
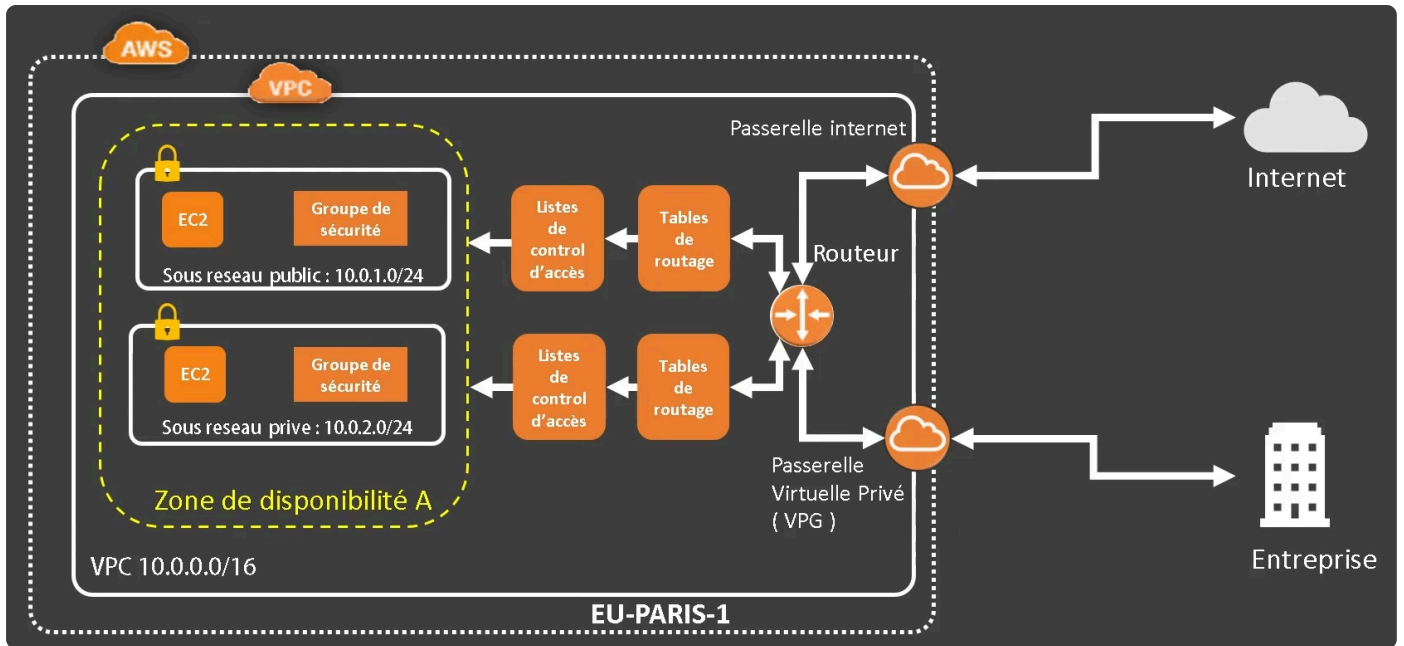
 [Groupes de sécurité : pourquoi faire ? Comment ?](#)

Network ACLs — Pare-feu au niveau subnet

Un **Network ACL** est un ensemble de règles appliquées à un **subnet entier**.

Caractéristiques :

- **Stateless** : vous devez définir **EXPLICITEMENT** les règles entrantes ET sortantes.
- Numérotées (ordre d'évaluation).
- Application par subnet.



Différences clés

Security Group et Network ACL sont souvent confondus alors qu'ils opèrent à des niveaux différents et se complètent plutôt que de se remplacer :

Aspect	Security Group	Network ACL
Portée	Instance	Subnet
État	Stateful	Stateless
Défaut	Tout refusé sauf règles	Tout refusé sauf règles

Bonne pratique : utilisez les **Security Groups** pour les **règles fines** (par instance), et les **ACLs** pour les **règles larges** (par subnet).

Comment tout s’imbrique — architecture 3-tiers dans une VPC

Ces briques (subnets, Security Groups, NAT Gateway) ne prennent tout leur sens qu’assemblées avec les ressources RDS/EC2 vues plus haut dans ce chapitre. Voici l’architecture la plus enseignée en SAA-C03 : un serveur web accessible depuis Internet, une base de données qui ne l’est jamais.

```
VPC 10.0.0.0/16
├── Subnet PUBLIC (10.0.1.0/24, AZ 1a)
│   ├── Route Table → 0.0.0.0/0 via Internet Gateway
│   ├── Application Load Balancer (ALB)
│   │   └── Security Group ALB : inbound 443 depuis 0.0.0.0/0
│   └── NAT Gateway (sort vers Internet pour le subnet privé)
├── Subnet PRIVÉ – App (10.0.10.0/24, AZ 1a)
│   ├── Route Table → 0.0.0.0/0 via NAT Gateway (pas d'IGW direct)
│   ├── Instance EC2 (serveur web)
│   │   └── Security Group EC2 : inbound 80 UNIQUEMENT depuis Security Group ALB
└── Subnet PRIVÉ – Data (10.0.20.0/24, AZ 1a)
    ├── Route Table → pas de route vers Internet (isolé)
    └── Instance RDS (déployée en section 1 de ce chapitre)
        └── Security Group RDS : inbound 3306 UNIQUEMENT depuis Security Group EC2
```

Ce que cette architecture garantit :

- Seul l’ALB est exposé sur Internet (subnet public, port 443 ouvert à tous).
- L’EC2 n’accepte du trafic que depuis l’ALB — jamais directement depuis Internet, même si son IP était devinée.
- Le RDS n’accepte du trafic que depuis l’EC2 — la base de données n’a **aucune route vers Internet**, donc même une erreur de configuration Security Group ne peut pas l’exposer.
- Chaque Security Group référence un *autre Security Group* comme source (pas une plage d’IP) — c’est la bonne pratique : si l’EC2 change d’adresse IP (redémarrage, remplacement), la règle reste valide.

Cette architecture à un seul VPC suffit pour une application isolée ; dès qu’il faut faire communiquer plusieurs VPC entre eux, un nouveau mécanisme de connexion réseau entre en jeu.

2.4 VPC Peering — Connecter plusieurs VPC

VPC Peering établit une **connexion réseau privée** entre deux VPC, permettant aux instances de communiquer comme si elles étaient dans le même réseau.

Caractéristiques

Avant de l’utiliser, il faut connaître trois propriétés structurantes du peering, en particulier la première, source de nombreuses erreurs d’architecture :

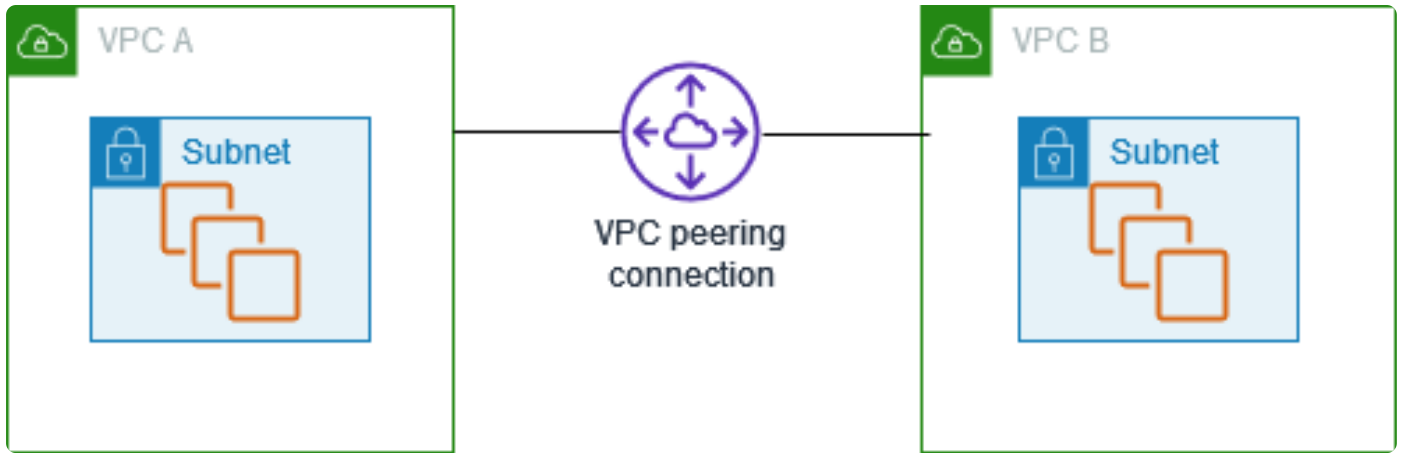
Aspect	Détail
Non-transitif	A ↔ B fonctionne. Mais A ↔ C ne passera pas par B : il faut une peering A-C explicite.
Coût	\$0.01 par million de requêtes
Inter-comptes	Deux VPC dans deux comptes AWS différents peuvent être peered
Inter-régions	Deux VPC dans deux régions différentes peuvent être peered

 **Warning**

VPC Peering est non-transitif — Piège architectural classique

Si vous avez 3 VPCs : **Prod** ↔ **Shared** et **Dev** ↔ **Shared**, cela ne signifie PAS que Prod peut parler à Dev via Shared. Le trafic ne transite JAMAIS par un VPC intermédiaire.

Pour interconnecter N VPCs avec transitivité, utilisez **AWS Transit Gateway** (hub centralisé). Avec 4 VPCs, VPC Peering crée 6 connexions à gérer — avec 10 VPCs, c’est 45 connexions. Transit Gateway réduit cela à 1 attachement par VPC.



 [Documentation VPC Peering](#)

2.5 Modèle Multi-VPC / Multi-Comptes et AWS Transit Gateway

Limitation du VPC Peering

Quand vos infrastructures deviennent **complexes** (10+ VPCs, plusieurs comptes AWS), le peering classique crée un problème :

AVEC PEERING CLASSIQUE (non-transitif) :

Pour N VPCs : $N \times (N-1) / 2$ peerings = EXPLOSION combinatoire

Exemple 4 VPCs :

VPC-Prod ↔ VPC-Dev (1)

VPC-Prod ↔ VPC-Staging (2)

VPC-Prod ↔ VPC-Shared (3)

VPC-Dev ↔ VPC-Staging (4)

VPC-Dev ↔ VPC-Shared (5)

VPC-Staging ↔ VPC-Shared (6)

↓

6 peerings manuels ! 🤖

Problèmes :

- Non-transitif : Prod ↔ Dev ↔ Shared = Prod ne voit pas Shared
- Gestion complexe : chaque nouveau VPC = 3 peerings à créer
- Pas de contrôle centralisé : règles partagées impossibles

Face à cette explosion combinatoire, AWS propose un service dédié qui remplace ce maillage de connexions point-à-point par une architecture en étoile.

AWS Transit Gateway — La solution

AWS Transit Gateway est un **hub réseau centralisé** qui connecte **toutes vos VPCs, comptes AWS et réseaux on-prem** via une seule interface.

En architecture hub-and-spoke, l’AWS Transit Gateway devient le hub centralisé (politiques et routing uniques) auquel se connectent, chacun via un seul attachement : le réseau On-Premise, VPC-Prod (`10.0.0.0/16`), VPC-Dev (`10.1.0.0/16`), VPC-Staging (`10.2.0.0/16`) et VPC-Shared (`10.3.0.0/16`) — une connexion unique par ressource, à l’échelle illimitée.

Attachements possibles :

- ✓ VPC (plusieurs)
- ✓ Comptes AWS (via RAM — Resource Access Manager)
- ✓ On-premise (via VPN ou Direct Connect)
- ✓ Transit Gateway externe (inter-régions)

Cette capacité à attacher des comptes AWS entiers (pas seulement des VPC individuels) est ce qui rend Transit Gateway indispensable dès qu’une organisation dépasse quelques comptes, comme l’illustre l’exemple concret suivant.

Architecture complète : Multi-Comptes avec Transit Gateway

L’organisation AWS regroupe plusieurs comptes, tous rattachés au même Transit Gateway (`tgw-xxx` , possédé par le Compte Prod) :

Compte	Ressource	Rattachement
PROD (123456789012)	VPC-Prod (<code>10.0.0.0/16</code>)	Attachement TGW
DEV (210987654321)	VPC-Dev (<code>10.1.0.0/16</code>)	Attachement TGW
SHARED (shared-000)	VPC-Shared (<code>10.3.0.0/16</code>)	Attachement TGW
ON-PREMISE	Network (<code>192.168.0.0/16</code>)	Attachement VPN/Direct Connect
MGMT	CloudTrail logs (audit)	—

Le Transit Gateway maintient des route tables distinctes (Production, Dev, Shared) pour contrôler quel trafic peut transiter entre quels VPC.

Avantages Transit Gateway

En reprenant les limites du VPC Peering identifiées plus haut, voici point par point ce que Transit Gateway apporte en comparaison :

Aspect	VPC Peering	Transit Gateway
Connexions N VPCs	$N(N-1)/2$ peerings 🧑‍🤝‍🧑	1 attachement par VPC ✅
Transitif	Non ($A \leftrightarrow B, B \leftrightarrow C \neq A \leftrightarrow C$)	Oui (contrôlé par routing)
Multi-comptes	Non (même compte seulement)	Oui (via Resource Access Manager)
On-premise	Non	Oui (VPN + Direct Connect)
Policies centralisées	Impossible	Oui (Network Policy)
Coût	\$0.01 par million requêtes	\$0.05 par attachement/h + data

Le seul inconvénient de Transit Gateway par rapport au Peering est donc le coût par attachement, qui reste toutefois largement compensé par l'économie de gestion dès que le nombre de VPC dépasse 3 ou 4.

Configuration AWS CLI — Transit Gateway, principe

Le Transit Gateway est créé une seule fois, puis chaque VPC s'y attache via une **attachment**. La table de routage du TGW détermine quels VPCs peuvent se parler.

```

1  # Créer le Transit Gateway
2  aws ec2 create-transit-gateway \
3      --description "Formation Transit Gateway Hub" \
4      --tag-specifications 'ResourceType=transit-gateway,Tags=
[{'Key=Name,Value=FormationTGW}]'
5  # Résultat : TransitGatewayId = tgw-0123456789abcdef
6
7  # Attacher un VPC au TransitGateway
8  aws ec2 create-transit-gateway-vpc-attachment \
9      --transit-gateway-id tgw-0123456789abcdef \
10
11      --vpc-id vpc-prod-12345 \
12
13      --subnet-ids subnet-prod-1a subnet-prod-1b

```




Tip

Résultat attendu :

```
1 {"TransitGatewayVpcAttachment": {"State": "pending", "TransitGatewayId":  
"tgw-0123456789abcdef", "VpcId": "vpc-prod-12345"}}
```



Info

Pour aller plus loin — hors périmètre de cette formation : partager un Transit Gateway entre plusieurs comptes AWS (via Resource Access Manager) et l'étendre à un réseau on-premise (VPN Site-to-Site) relèvent du niveau Advanced Networking Specialty. Le principe reste le même — un attachement par ressource, une table de routage centralisée — mais la mise en œuvre cross-account est un sujet à part entière.

Pièges Transit Gateway

Avant de déployer un Transit Gateway en production, quatre points méritent une attention particulière :

Piège	Solution
TGW par défaut permet tout	Créer des route tables TGW restrictives par environnement
Coût : \$0.05/attachement/h	Budget pour 20 VPCs = ~\$72/mois ($0.05 \times 20 \times 720$ h)
Association subnet obligatoire	Au moins 1 subnet par AZ pour la résilience
CIDR overlap interdit	VPCs partagés doivent avoir CIDRs différents

Le premier piège est le plus insidieux : un Transit Gateway nouvellement créé route tout le trafic entre tous les attachements par défaut, ce qui annule instantanément le cloisonnement Prod/Dev que l'architecture est censée garantir tant que des route tables restrictives ne sont pas explicitement configurées.

2.6 VPC Endpoints — Accès privé aux services AWS

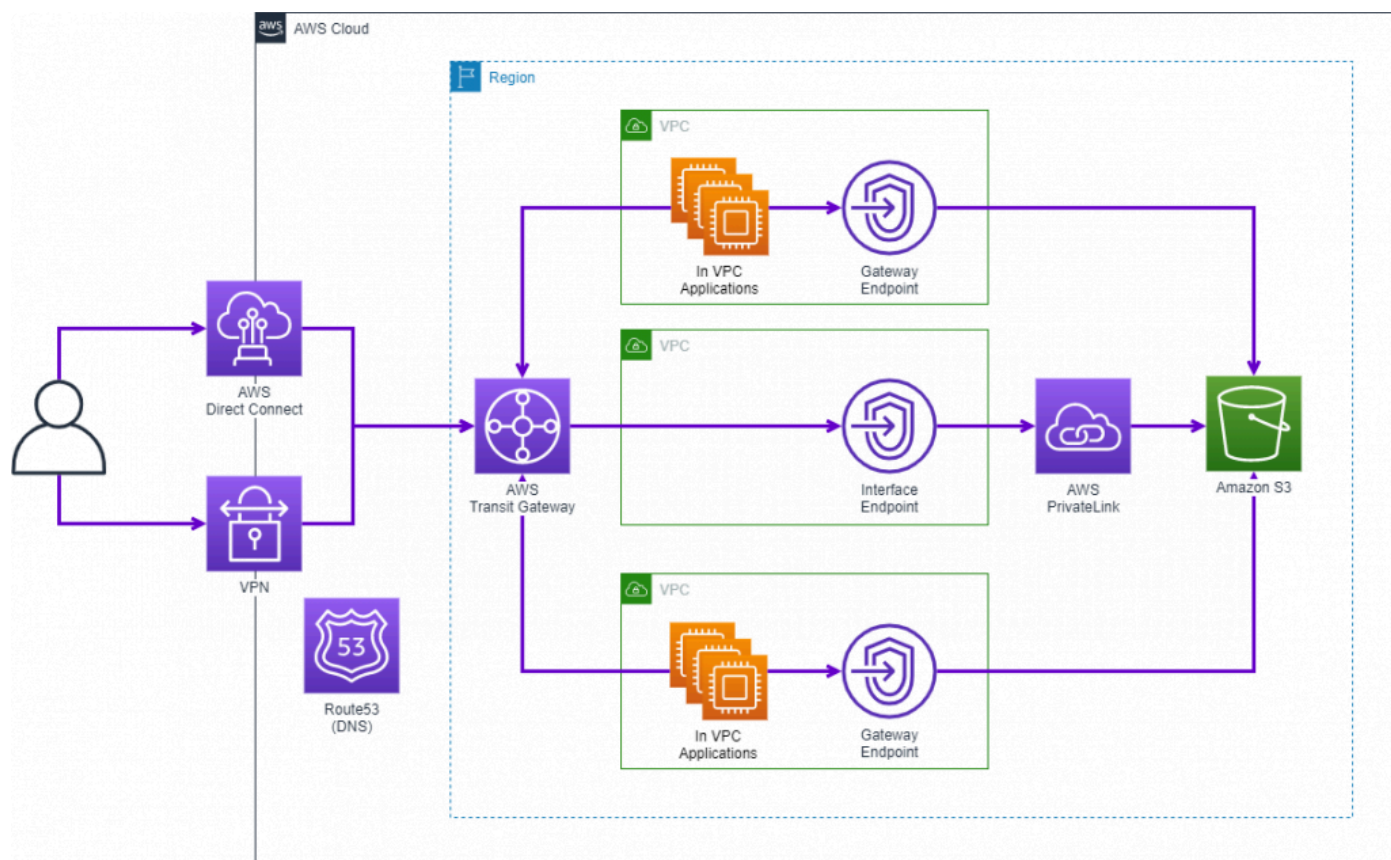
Définition

Un **VPC Endpoint** est une **passerelle privée** qui permet à vos ressources d'accéder à des **services AWS** sans passer par Internet.

Deux types

AWS propose deux implémentations de VPC Endpoint selon le service ciblé, avec un fonctionnement et un modèle de coût différents :

Type	Services	Fonctionnement	Coût
Gateway Endpoint	S3, DynamoDB	Interface simple (ajoute une route)	Gratuit
Interface Endpoint	EC2, Secrets Manager, Lambda, etc.	Interface élastique privée (ENI)	\$0.007/h + transfert



 [Documentation VPC Endpoints](#)

2.7 ENI (Elastic Network Interfaces) — Les cartes réseau d’AWS

Qu’est-ce qu’une ENI ?

Une **ENI** est une **carte réseau virtuelle** attachée à une instance EC2. Elle gère vos **adresses IP**, vos **adresses MAC**, vos **Security Groups** et vos **routes réseau**.

Dans une infrastructure **on-prem**, vous aviez des **cartes réseau physiques** (NIC) dans vos serveurs. Sur AWS, c’est exactement la même chose, mais **virtuelle et reconfigurable**.

Anatomie d’une ENI

Voici les attributs concrets que porte une ENI, illustrés sur l’interface principale d’une instance EC2 :

Instance EC2 (t3.medium)

- Primary ENI (eth0) [obligatoire]
 - Primary IP privée : 10.0.1.42 (CIDR subnet)
 - Secondary IP privées : 10.0.1.43, 10.0.1.44 (optionnel)
 - Elastic IP publique : 203.0.113.12 (optionnel)
 - MAC Address : 02:c1:1f:a0:2b:4d (auto-générée)
 - Security Group : sg-12345678
 - Source/Dest Check : ☒ activée (drop trafic non-destiné)

 [Comment conserver son IP sur AWS ?](#)

Cas d’usage : Multiple ENIs sur une même instance

Certains scénarios nécessitent **plusieurs ENIs** sur une instance :

SCÉNARIO : Serveur pare-feu / VPN / Load Balancer

Instance (m5.xlarge) : 4 ENIs possibles

- eth0 (Primary) : connectée à VPC public → Internet
 - 10.0.1.10
- eth1 (Secondary) : connectée à VPC privée → Apps internes
 - 10.1.1.10
- eth2 (Secondary) : connectée à VPC client (peering) → Client network
 - 10.2.1.10
- eth3 (Secondary) : Management/monitoring
 - 10.0.2.10 (subnet privé)

Résultat : machine routeur/pare-feu multi-réseaux ! 🔥

Voyons maintenant comment créer et attacher concrètement une ENI supplémentaire à une instance existante via l’AWS CLI.

Configuration ENI via AWS CLI

On crée ici une ENI indépendante puis on l’attache à une instance existante. Cela permet d’ajouter une interface réseau secondaire sans recréer l’instance — utile pour un serveur bastion ou un routeur multi-réseaux.

```

1  # =====
2  # 1. Créer une ENI seule (détachée)
3  # =====
4
5  aws ec2 create-network-interface \
6      --subnet-id subnet-12345678 \
7      --description "Secondary management interface" \
8      --private-ip-addresses PrivateIpAddress=10.0.2.100,Primary=true \
9      --groups sg-mgmt-12345678 \
10
11      --tag-specifications 'ResourceType=network-interface,Tags=
[{{Key=Name,Value=ENI-Management}}]'
12
13
14
15  # Résultat : NetworkInterfaceId = eni-0a1b2c3d4e5f6g7h8
16
17
18
19  # =====
20
21
22  # 2. Attacher une ENI existante à une instance
23
24  # =====
25
26
27
28
29
30
31  aws ec2 attach-network-interface \
32
33      --network-interface-id eni-0a1b2c3d4e5f6g7h8 \
34
35
36      --instance-id i-0123456789abcdef0 \
37
38
39      --device-index 1  # eth1 (0=primary/eth0, 1=eth1, etc.)
40
41
42
43  # =====

```

```

2
4 # 3. Allouer une Elastic IP et l'attacher à une ENI
2
5 # =====
2
6
2
7 # Allouer Elastic IP
2
8 aws ec2 allocate-address \
2
9     --domain vpc \
3
0     --network-interface-id eni-0a1b2c3d4e5f6g7h8 \
3
1     --private-ip-address 10.0.2.100
3
2
3
3 # Résultat : PublicIp = 203.0.113.50
3
4
3
5 # =====
3
6 # 4. Ajouter une IP privée secondaire à une ENI
3
7 # =====
3
8
3
9 aws ec2 assign-private-ip-addresses \
4
0     --network-interface-id eni-0a1b2c3d4e5f6g7h8 \
4
1     --private-ip-addresses 10.0.2.101 10.0.2.102
4
2

```

```

4
3 # =====
4
4 # 5. Changer de Security Group sur une ENI
4
5 # =====
4
6
4
7 aws ec2 modify-network-interface-attribute \
4
8   --network-interface-id eni-0a1b2c3d4e5f6g7h8 \
4
9   --groups sg-new-12345678 sg-management-67890
5
0
5
1 # =====
5
2 # 6. Détacher une ENI (reste dans la VPC, peut être réattachée)
5
3 # =====
5
4
5
5 aws ec2 detach-network-interface \
5
6   --attachment-id eni-attach-12345678
5
7
5
8 # =====
5
9 # 7. Voir toutes les ENI d'une instance
6
0 # =====
6
1

```

```

6
2  aws ec2 describe-instances \
6
3  --instance-ids i-0123456789abcdef0 \
6
4  --query 'Reservations[0].Instances[0].NetworkInterfaces[*].
[NetworkInterfaceId,Attachment.DeviceIndex,PrivateIpAddresses[0].PrivateIpAddress,P
rivateIpAddresses[0].Association.PublicIp]'
6
5
6
6  # Résultat (exemple) :
6
7  # [
6
8  #   ["eni-primary", 0, "10.0.1.42", "203.0.113.50"],      ← eth0 + Elastic IP
6
9  #   ["eni-secondary", 1, "10.0.2.100", null]              ← eth1 (privée)
7
0  # ]

```



Tip

Résultat attendu :

```

1  [
2    ["eni-0a1b2c3d4e5f00001", 0, "10.0.1.42", "203.0.113.50"],
3    ["eni-0a1b2c3d4e5f00002", 1, "10.0.2.100", null]
4  ]

```

Cas d'usage réels : ENI multiples

Au-delà du scénario pare-feu/VPN déjà illustré, plusieurs cas d'usage concrets justifient l'ajout d'ENI supplémentaires sur une instance :

Scénario	Interfaces	Bénéfice
Routeur/Pare-feu	3-4 ENIs	Connexion à plusieurs VPCs/subnets sans NAT
Haute disponibilité	Primary ENI + failover	IP privée = même, instance change (failover transparent)
Serveur DNS interne	Primary + management	Trafic DNS sur une interface, logs/monitoring sur autre
Load Balancer maison	Multiple NICs	Distribution load par interface réseau
Serveur VPN/bastion	2+ interfaces	Accès de plusieurs subnets via une machine unique

Le cas de la haute disponibilité mérite une attention particulière : en détachant une ENI d’une instance défaillante et en la rattachant à une instance de secours, l’IP privée (et donc la configuration réseau côté clients) reste inchangée, ce qui rend le failover transparent pour le reste de l’infrastructure.

Piège : Source/Destination Check

Par défaut, AWS bloque tout paquet dont l’IP source ou destination ne correspond pas à l’interface. C’est intentionnel pour la sécurité, mais cela empêche un routeur ou un pare-feu de forwarder le trafic. Il faut donc **désactiver cette vérification** sur les instances qui jouent un rôle de routage.

```

1  # ⚠ Par défaut, une ENI REFUSE le trafic non-destiné à elle-même.
2  # Exemple : routeur firewall doit FOWARDER le trafic.
3
4  # Désactiver Source/Destination check
5  aws ec2 modify-network-interface-attribute \
6    --network-interface-id eni-12345678 \
7    --no-source-dest-check
8
9  # Résultat : routeur peut maintenant forwarder (forwarding activé)
10
11
12  # Réactiver (sécurité)
13
14  aws ec2 modify-network-interface-attribute \
15    --network-interface-id eni-12345678 \
16    --source-dest-check

```

Tip

Résultat attendu :

```

# modify-network-interface-attribute : aucun output si succès

# Vérification : aws ec2 describe-network-interface-attribute --network-
interface-id eni-12345678 --attribute sourceDestCheck
{
  "NetworkInterfaceId": "eni-12345678",
  "SourceDestCheck": {
    "Value": false
  }
}

```

La vérification Source/Destination est désactivée (`Value: false`). L'interface peut désormais forwarder du trafic dont elle n'est pas la destination finale — comportement requis pour une instance jouant le rôle de routeur ou de pare-feu.

Diagnostic réseau — routage et Security Groups

Une instance possède une IP publique et un Security Group autorisant HTTPS, mais reste inaccessible depuis Internet. Que faut-il encore vérifier ?

► [Afficher la réponse raisonnée](#)

3. Amazon Route 53 — DNS Intelligent

3.1 Fondamentaux du DNS

Qu'est-ce que le DNS ?

Le **DNS (Domain Name System)** est un système **mondial décentralisé** qui traduit des **noms lisibles** (`www.example.com`) en **adresses IP** (`93.184.216.34`).

3.2 Amazon Route 53 — Service DNS managé

Définition

Amazon Route 53 est le **service DNS managé** d'AWS. Il permet de **résoudre les noms de domaine**, de **diriger le trafic intelligemment**, et d'**assurer la haute disponibilité**.

Le nom « Route 53 » vient du **port 53**, utilisé par le protocole DNS.

Capacités

Route 53 combine plusieurs fonctionnalités :

Fonctionnalité	Description
Registrar	Acheter et gérer des domaines (.com, .fr, .io, etc.)
Résolution DNS	Traduire noms → IP
Health Checks	Vérifier si une ressource est disponible
Routage intelligent	Diriger le trafic selon latence, géolocalisation, poids, failover
Alias Records	Lier un domaine à une ressource AWS (ALB, CloudFront, S3)

Les fonctions Registrar et résolution DNS existent chez tout fournisseur DNS ; ce qui distingue Route 53, ce sont les Health Checks et le routage intelligent, détaillés dans les sections suivantes.

Types d'enregistrements DNS courants

La résolution DNS elle-même s'appuie sur plusieurs types d'enregistrements normalisés, dont voici les plus courants :

Type	Exemple	Rôle
A	example.com → 93.184.216.34	IPv4
AAAA	example.com → 2606:2800:...	IPv6
CNAME	www.example.com → example.com	Alias
MX	example.com → mail.example.com	Serveur mail
TXT	example.com → v=spf1...	Enregistrement texte
NS	example.com → ns1.route53...	Serveurs DNS

Les enregistrements **A/AAAA** et **CNAME** sont ceux que vous manipulerez le plus souvent pour pointer un domaine vers une ressource AWS ; **Alias Records** (vu ci-dessus) est une extension propriétaire Route 53 qui se comporte comme un CNAME mais peut cibler la racine d'un domaine, ce qu'un CNAME classique interdit.

3.3 Politiques de routage — Diriger le trafic intelligemment

Route 53 n'est pas un simple DNS classique. C'est un **routeur de trafic applicatif**.

Politique Simple

Cas de base : un domaine pointe vers **une seule adresse IP**.

Politique Pondérée

Distribuer le trafic en pourcentage entre plusieurs ressources.

Cas d’usage : déploiement progressif, A/B testing, migration progressive.

Politique Latence

Router les utilisateurs vers la ressource **la plus rapide** (latence réseau minimale).

Cas d’usage : applications globales, réduction latence.

Politique Failover (Basculement)

En cas de panne détectée, router vers une ressource de secours.

Cas d’usage : haute disponibilité, reprise après sinistre.

3.4 Health Checks et Monitoring

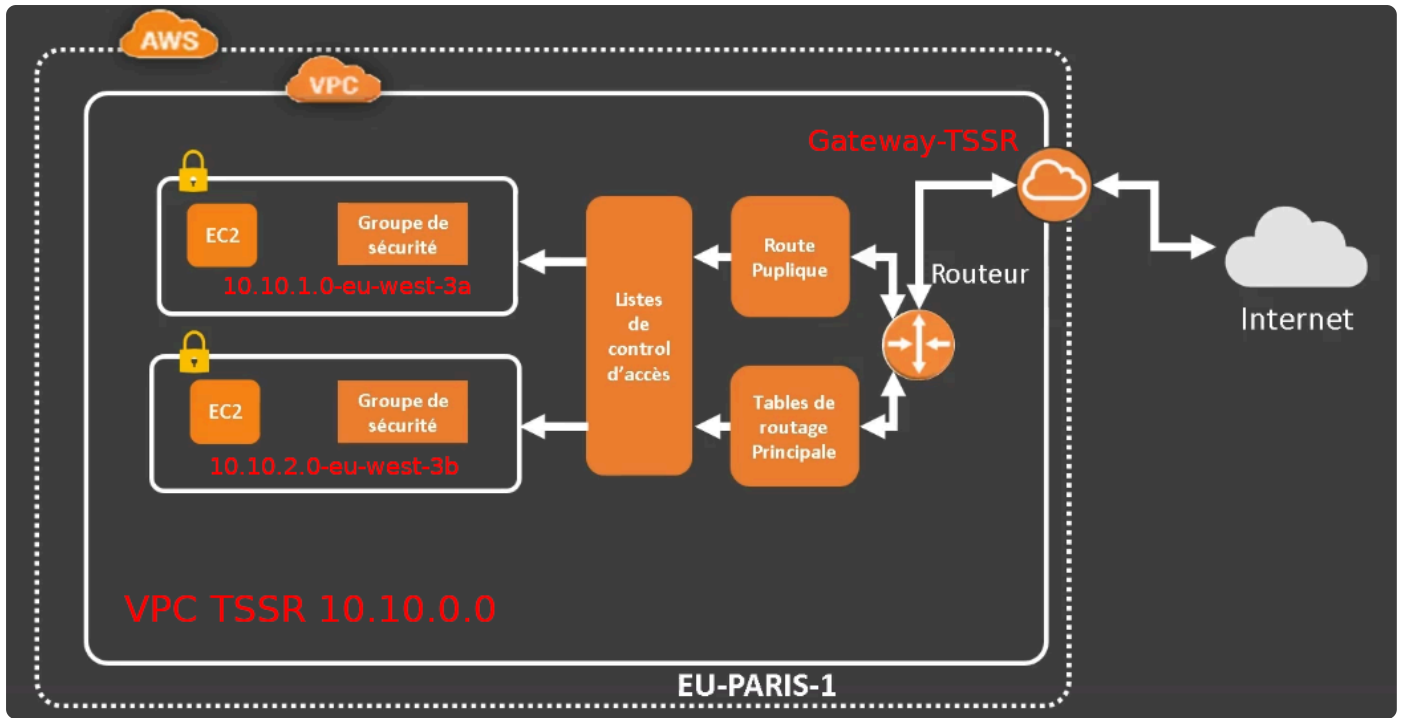
Route 53 peut **monitorer la santé** des ressources et basculer automatiquement.

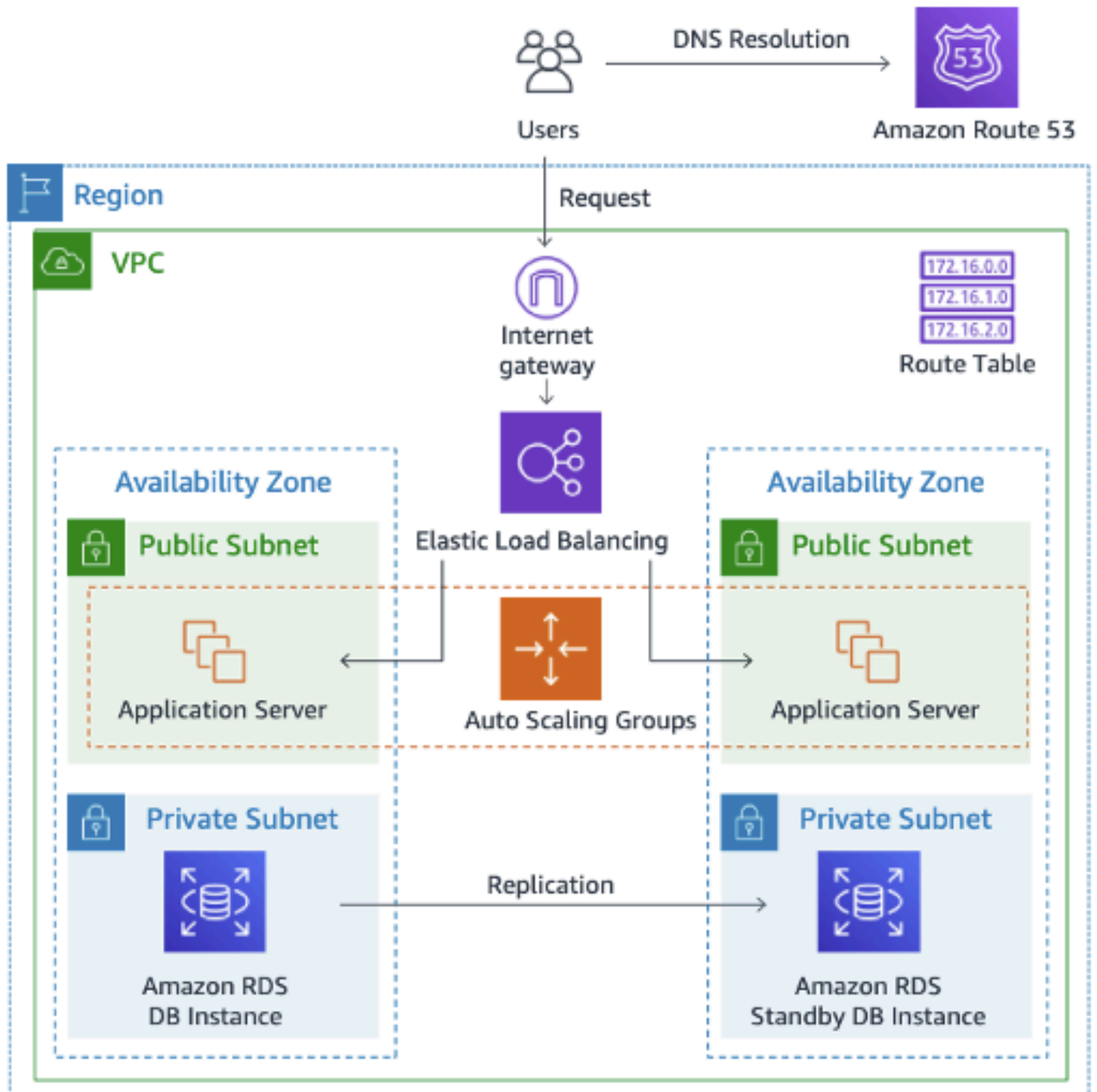
Types de Health Checks

Quatre types de vérification sont disponibles selon le niveau de granularité souhaité :

Type	Description	Fréquence
HTTP/HTTPS	Effectue une requête GET, attend 2xx/3xx	Toutes les 30s
TCP	Établit une connexion TCP	Toutes les 10s
Calculated	Combine plusieurs health checks	Toutes les 30s
CloudWatch	Déclenché par une alarme CloudWatch	Variable

C’est ce Health Check qui alimente la politique de routage Failover vue en 3.3 : dès qu’il détecte une ressource en panne (délai selon la fréquence choisie), Route 53 bascule automatiquement le trafic vers la ressource de secours.





4. Amazon ElastiCache — Mise en cache distribuée

4.1 Pourquoi une couche cache ?

Imaginez une **base de données RDS** qui reçoit **1 000 requêtes par seconde** pour lire les **mêmes 10 utilisateurs**. Chaque requête demande 5–10 ms à la base. Résultat : **goulot d'étranglement**, latence élevée, coût RDS énorme.

Solution : placez un **cache rapide** devant la base. Les 1 000 requêtes frappent le cache (**< 1 ms**) au lieu de la base.

Amazon ElastiCache est le service managé AWS pour placer un **cache distribuée** haute performance devant vos applications.

4.2 Deux moteurs : Redis vs Memcached

Redis (Remote Dictionary Server)

Redis est le moteur le plus riche fonctionnellement des deux, avec des structures de données avancées et une option de persistance :

```
Cas d'usage : Sessions utilisateur, panier e-commerce, rankings, pubsub
Structure : Chaînes, listes, ensembles, hashes, streams, géo-spatial
Persistance : RDB + AOF (journalisation)
Clustering : Oui, avec failover automatique
Transactions : MULTI/EXEC
TTL (durée de vie clé) : Oui
```

Redis peut perdre les données stockées en mémoire lors d'un redémarrage ou d'une panne — la **persistance** est le mécanisme qui les sauvegarde sur disque pour pouvoir les recharger ensuite. AWS ElastiCache pour Redis combine deux méthodes complémentaires : **RDB** (Redis Database, un instantané complet du contenu de la mémoire pris à intervalles réguliers, rapide à recharger mais qui peut perdre les toutes dernières écritures) et **AOF** (Append Only File, un journal qui enregistre chaque commande d'écriture au fil de l'eau, plus lourd mais qui permet de ne perdre quasiment aucune donnée en cas de panne).

Analogie : un **dictionnaire magique ultra-rapide** qui se souvient des modifications.

Memcached

À l'inverse, Memcached vise la simplicité et la performance brute plutôt que la richesse fonctionnelle :

```
Cas d'usage : Cache objet simple (résultats DB, pages HTML)
Structure : Chaînes et blobs uniquement
Persistance : Non (tout volatil)
Clustering : Oui, mais pas de failover
Transactions : Non
TTL : Oui
```

Analogie : un **panier à oublier** ultra-simple — parfait pour des données éphémères.

4.3 Cas d'usage typiques

En croisant les caractéristiques des deux moteurs avec des besoins concrets, un choix se dégage assez naturellement pour chaque scénario :

Cas d'usage	Moteur	Raison
Session utilisateur	Redis	Besoin de persistance, expiration TTL
Panier e-commerce	Redis	Structures complexes (hash), transactions
Leaderboard	Redis	Opérations set triées (<code>ZSET</code>)
Cache HTML statique	Memcached	Simple, volatil, très rapide
Résultats requête DB	Redis	Contrôle TTL par clé, publish/subscribe
Real-time counters	Redis	Opérations atomiques (<code>INCR</code>)

Redis domine ce tableau non par supériorité générale, mais parce que la plupart des cas d'usage cache modernes ont besoin d'au moins une fonctionnalité (TTL fin, structures, persistance) que Memcached n'offre pas ; Memcached reste pertinent dès que la simplicité et le débit brut priment sur ces fonctionnalités.

4.4 Architecture ElastiCache

Cluster Mode Disabled (simple, old-school)

L'application (EC2, Lambda) envoie ses requêtes `GET cache_key` vers le nœud ElastiCache Redis primary (`eu-west-1a`), qui réplique de façon asynchrone vers un nœud Replica standby en lecture seule (`eu-west-1b`).

Limitation : une seule shard, donc un seul nœud — le CPU de ce nœud unique plafonne la capacité totale.

Cluster Mode Enabled (production, sharding)

L'application (EC2, Lambda) hache chaque clé pour la router vers l'une des trois shards : Shard 1 (clés 1-3), Shard 2 (clés 4-6) ou Shard 3 (clés 7-10). Chaque shard a son propre primary node, répliqué vers un Replica correspondant (`eu-west-1b`).

Bénéfice : parallélisation et scalabilité linéaire — ajouter des shards augmente la capacité totale, contrairement au mode Cluster Disabled.

4.5 Commandes Redis essentielles

Voici un aide-mémoire des commandes Redis les plus utilisées, organisées par type de structure de données, pour s'entraîner en local avant de déployer un cluster sur AWS :

```

1  # Installation (macOS via Homebrew)
2  brew install redis
3
4  # Lancer serveur Redis local (développement)
5  redis-server
6
7  # Client Redis (dans un autre terminal)
8  redis-cli
9
1
0  # ——— CHAÎNES (Strings) ———
1
1  SET nom "Alice"                # Stocker
1
2  GET nom                        # Récupérer → "Alice"
1
3  APPEND nom " Dupont"          # Ajouter → "Alice Dupont"
1
4  STRLEN nom                    # Longueur → 12
1
5  INCR compteur                 # Incrémenter (atomique)
1
6  DECR compteur                 # Décrémenter
1
7
1
8  # ——— LISTES (Lists) ———
1
9  RPUSH queue "tache1"          # Ajouter à droite
2
0  RPUSH queue "tache2" "tache3" # Multiple
2
1  LPOP queue                    # Retirer de gauche
2
2  LLEN queue                   # Longueur
2
3  LRANGE queue 0 -1             # Tout afficher

```

```

2
4
2
5 # —— HASHES (Objets) ——
2
6 HSET user:100 nom "Alice"          # Stocker champ
2
7 HSET user:100 email "alice@ex.com" age 28
2
8 HGET user:100 nom                  # Récupérer → "Alice"
2
9 HGETALL user:100                   # Tous les champs
3
0 HDEL user:100 age                  # Supprimer champ
3
1
3
2 # —— ENSEMBLES TRIÉS (Sorted Sets, ZSET) ——
3
3 ZADD leaderboard 100 "Alice"        # Score 100 → Alice
3
4 ZADD leaderboard 150 "Bob" 200 "Charlie"
3
5 ZRANGE leaderboard 0 -1             # Ordre croissant
3
6 ZREVRANGE leaderboard 0 -1          # Ordre décroissant (top)
3
7 ZRANK leaderboard "Alice"           # Position → 0 (première)
3
8
3
9 # —— EXPIRATION ——
4
0 SET session:user123 "data"
4
1 EXPIRE session:user123 3600         # Expirer dans 1 heure
4
2 TTL session:user123                 # Temps restant → 3599

```

```

4
3
4
4 # ——— TRANSACTIONS ———
4
5 MULTI
4
6 SET clé1 "valeur1"
4
7 SET clé2 "valeur2"
4
8 EXEC                                # Atomique : tout ou rien
4
9
5
0 # ——— PUBLISH/SUBSCRIBE ———
5
1 SUBSCRIBE channel:notifications    # S'abonner
5
2 PUBLISH channel:notifications "Hello" # Diffuser

```

Ces commandes s'exécutent en local pour se familiariser avec Redis ; la section suivante montre comment provisionner l'équivalent managé sur AWS.

4.6 Créer un cluster Redis en CLI

Info

Une activité pratique permet d'approfondir le déploiement d'un cache Redis.

On crée ici le type de cluster le plus simple : un nœud Redis unique, sans réplication. En production, on ajouterait un groupe de réplication (`create-replication-group`) avec un nœud primaire et des replicas, mais ce modèle suffit pour comprendre les concepts.

```

1  # 1. Créer un cluster Redis (simple, mode Cluster Mode Disabled)
2  aws elasticache create-cache-cluster \
3      --cache-cluster-id formation-redis-simple \
4      --cache-node-type cache.t3.micro \
5      --engine redis \
6      --engine-version 7.0 \
7      --num-cache-nodes 1 \
8      --vpc-security-group-ids sg-12345678 \
9      --cache-subnet-group-name my-subnet-group \
10     --tags Key=Name,Value=FormationRedis Key=Env,Value=Dev
1
1
1
2  # 2. Attendre que le cluster soit disponible
1
3  aws elasticache wait cache-cluster-available \
1
4      --cache-cluster-id formation-redis-simple
1
5
1
6  # 3. Récupérer l'endpoint (adresse:port)
1
7  aws elasticache describe-cache-clusters \
1
8      --cache-cluster-id formation-redis-simple \
1
9      --query 'CacheClusters[0].CacheNodes[0].Endpoint'
2
0  # Résultat exemple : formation-redis-
simple.abc123.ng.0001.euw1.cache.amazonaws.com:6379

```



Tip

Résultat attendu :

```
1  {
2      "CacheClusters": [{
3          "CacheClusterId": "formation-redis-simple",
4          "CacheClusterStatus": "available",
5          "Engine": "redis",
6          "EngineVersion": "7.0.7",
7          "CacheNodeType": "cache.t3.micro",
8          "CacheNodes": [{
9              "CacheNodeId": "0001",
10
11              "CacheNodeStatus": "available",
12
13              "Endpoint": {
14
15                  "Address": "formation-redis-
simple.abc123.ng.0001.euw1.cache.amazonaws.com",
16
17                  "Port": 6379
18              }
19          }]
20      }]
21  }
```

```

1 # 4. Créer un Replication Group (Multi-AZ avec failover auto)
2 aws elasticache create-replication-group \
3   --replication-group-id formation-redis-ha \
4   --replication-group-description "Redis avec failover" \
5   --engine redis \
6   --engine-version 7.0 \
7   --cache-node-type cache.t3.micro \
8   --num-cache-clusters 2 \
9   --automatic-failover-enabled \
10  --multi-az-enabled \
11  --vpc-security-group-ids sg-12345678 \
12  --cache-subnet-group-name my-subnet-group
13
14 # 5. Décrire le replication group
15 aws elasticache describe-replication-groups \
16   --replication-group-id formation-redis-ha
17
18 # 6. Créer un cluster Memcached (simple)
19 aws elasticache create-cache-cluster \
20   --cache-cluster-id formation-memcached \
21   --cache-node-type cache.t3.micro \
22   --engine memcached \
23   --engine-version 1.6.17 \

```



```

2
4  --num-cache-nodes 3 \
2
5  --vpc-security-group-ids sg-12345678 \
2
6  --cache-subnet-group-name my-subnet-group
2
7
2
8  # 7. Supprimer un cluster (attention : perte de données)
2
9  aws elasticache delete-cache-cluster \
3
0  --cache-cluster-id formation-redis-simple

```

Le Replication Group (commande 4) est la configuration à privilégier en production : contrairement au cluster simple créé en commande 1, il offre un failover automatique en cas de panne du nœud primaire.

4.7 Bonne pratique : Cache-Aside Pattern

Le pattern le plus courant pour intégrer un cache :

Requête application :

1. Cache.GET(clé) ?
 - Si HIT → retourner valeur (< 1 ms) 
 - Si MISS → aller à 2
2. Requête base de données
RDS.SELECT(clé) → résultat
3. Stocker en cache
Cache.SET(clé, résultat, TTL=3600) # Expire en 1 heure
4. Retourner résultat application

Avantage : logique simple, contrôle du cache

Risque : cache stale (données anciennes) pendant TTL

Ce pattern place la responsabilité du remplissage du cache côté application plutôt que côté base de données — c'est ce choix architectural qui explique pourquoi il faut définir un TTL cohérent avec la fréquence de mise à jour réelle des données en base.

4.8 Combien coûte ElastiCache ?

ElastiCache se facture à l'heure d'instance active, comme EC2 et RDS — pas de coût “à la requête” contrairement à DynamoDB. Le prix dépend du type de nœud et du nombre de nœuds (primary + replicas).

Type de nœud	RAM	Usage typique	Prix indicatif (région Paris)
cache.t3.micro	0,5 Go	Dev/test, très petite charge	~0,020 /h (~15 /mois)
cache.t3.medium	3,09 Go	Sessions utilisateur, petite prod	~0,080 /h (~58 /mois)
cache.r6g.large	13,07 Go	Cache applicatif de production	~0,226 /h (~165 /mois)

Ce qui fait varier la facture :

- **Nombre de nœuds** : un cluster Redis en haute disponibilité (primary + replica) double le coût du nœud seul — exactement comme Multi-AZ sur RDS.
- **Cluster Mode Enabled** (sharding) : chaque shard supplémentaire est un nœud facturé en plus — utile pour la scalabilité, mais le coût grimpe linéairement avec le nombre de shards.
- **Transfert de données** : gratuit entre ElastiCache et EC2 dans la même AZ, facturé au-delà (inter-AZ, inter-région).

Repère utile : ElastiCache n'est rentable que si le cache réduit suffisamment la charge sur RDS pour permettre une instance RDS plus petite, ou évite d'ajouter des Read Replicas RDS payants. Sur une charge de lecture très répétitive (mêmes clés interrogées des milliers de fois), le calcul est presque toujours favorable ; sur des requêtes peu répétées, le cache n'apporte rien et n'est qu'un coût supplémentaire.

 [Amazon ElastiCache Pricing](#)

5. Points importants et pièges fréquents

5.1 Pièges RDS et Bases de données

Ce chapitre a couvert beaucoup de mécanismes RDS/Aurora en détail ; voici les confusions les plus fréquentes qui reviennent en certification comme en production :

Piège	Réalité	Conséquence	Solution
RDS n'est pas auto-scalable en stockage	Faut augmenter manuellement (RDS) ou activer auto-scaling (Aurora)	Saturation disque → downtime	Vérifier "Storage autoscaling" dans RDS config
Multi-AZ RDS ≠ haute disponibilité lue	Multi-AZ = résilience (failover), pas scalabilité lecture	Bottleneck en lecture malgré Multi-AZ	Ajouter Read Replicas (asynchrones)
Chiffrement RDS doit être activé à la création	On ne peut pas l'activer après coup	Recréer l'instance = downtime	Checker "Encrypt at rest" lors création
Read Replica ≠ Multi-AZ	Replica = asynchrone, pour lectures. Multi-AZ = synchrone, failover	Confondre les deux gâche design	Multi-AZ pour haute dispo, Replicas pour scalabilité lecture
Snapshot RDS = backup manuel	Snapshots manuels ne s'auto-suppriment pas	Surcoûts stockage	Supprimer manuellement ou appliquer cycle vie

Les deux pièges les plus fréquemment testés en examen sont la confusion Multi-AZ/Read Replica et l'impossibilité de chiffrer après coup — gardez ces deux réflexes en tête avant de passer au réseau.

5.2 Pièges VPC et Réseau

Le réseau concentre le plus grand nombre de pièges de ce chapitre, du fait de la diversité des composants (Peering, Security Groups, NACL, Transit Gateway, ENI) :

Piège	Réalité	Conséquence	Solution
VPC Peering n'est pas transitif	$A \leftrightarrow B$ et $B \leftrightarrow C$ ne signifie pas $A \leftrightarrow C$	Communication A-C bloquée	Créer peering A-C explicitement ou utiliser Transit Gateway
Security Group $\neq$ Network ACL	SG = stateful (instance), NACL = stateless (subnet)	Oublier une règle sortante NACL bloque tout	Vérifier ACL entrée ET sortie
IGW coûte 0 €	NAT Gateway coûte \$0.032/h + transfert	Budget explosif avec gros trafic privé	Minimiser NAT, utiliser VPC Endpoints S3
Security Group par défaut refuse tout	Sauf trafic sortant (autorisé par défaut)	Instances isolées jusqu'à ouverture ingress	Ajouter règles ingress explicites
Security Group : ALLOW vs DENY	SG = whitelist (ALLOW seulement), pas de DENY	Penser NACL pour bloquer IPs spécifiques	NACL pour explicite DENY
Associer route table incorrect	Associer table RT publique à subnet privé = accès internet non sécurisé	Instances "privées" exposées à Internet	Vérifier subnet $\leftrightarrow$ route table
ENI : Source/Dest Check activé par défaut	ENI refuse le trafic non-destiné à elle (sécurité)	Routeur/pare-feu ne peut pas forwarder	Désactiver source-dest-check pour routeurs
Attach ENI = Device index critique	Device index 0 = primary (obligatoire), 1+ = secondary	Erreur lors attach bloque l'instance	Vérifier device index disponible avant attach
Transit Gateway $\neq$ gratuit	0.05/attachement/h + 0.02 par Go data	Coût pour 20 VPCs = ~\$72/mois	Budget transit gateway si 10+ VPCs

Piège	Réalité	Conséquence	Solution
Transit Gateway routing par défaut = tous allowed	TGW fait transiter tous les paquets par défaut	Communication imprévue entre VPCs	Restreindre via Route Tables TGW explicites
VPC CIDR overlap interdit dans Transit Gateway	Tous les VPCs attachés doivent avoir CIDR différents	Adresses en collision = paquets perdus	Planifier CIDR par VPC avant TGW

Le point commun à la majorité de ces pièges réseau est une confusion entre deux mécanismes qui se ressemblent en apparence (SG/NACL, Peering/Transit Gateway, IGW/NAT) mais répondent à des besoins différents — relire leur définition respective lève la plupart de ces confusions.

5.3 Pièges Route 53 et DNS

Côté DNS, les pièges sont moins nombreux mais tout aussi coûteux en cas d'erreur, notamment sur le TTL :

Piège	Réalité	Conséquence	Solution
DNS Route 53 a un TTL	Les réponses sont cachées pendant le TTL	Changement DNS peut prendre 24h	Baisser TTL avant changement (300s)
Health Check ≠ Failover automatique	Health check détecte panne, failover redirection	Juste détecter ne suffit pas	Configurer failover + health check
Alias Records ≠ CNAME	Alias = pointeur AWS (gratuit, flexible), CNAME = alias DNS classique	Confondre risque problèmes CNAME root	Toujours Alias pour AWS resources (ALB, CloudFront)

Warning

TTL Route 53 trop court = coût de requêtes élevé

Un TTL de **30 secondes** sur un enregistrement très consulté (ex. : API avec 1 000 req/s) génère des **résolutions DNS répétées**. Route 53 facture **\$0.40 par million de requêtes** pour les zones publiques.

Exemple : 1 000 req/s × 2 résolutions/min (TTL=30s) × 3 600 s/h = ~~432 000 requêtes DNS/h~~ → ****\$4/jour**** juste pour le DNS.

Règle :

- TTL **300s** (5 min) pour la plupart des enregistrements stables
- TTL **60s** maximum pendant une migration ou un failover planifié
- Remonter le TTL à **300-3600s** après stabilisation

5.4 Pièges DynamoDB

DynamoDB fonctionne différemment d'une base relationnelle, ce qui génère des erreurs de conception typiques chez les habitués du SQL :

Piège	Réalité	Conséquence	Solution
DynamoDB provisionné vs on-demand	Mode provisionné = moins cher si prévisible	Charge imprévisible = throttling ou surcoûts	Choisir on-demand si variable, provisionné si stable
Partition key ≠ Sort key	Partition = hash (required), Sort = range (optional)	Query sans sort key = full table scan	Bien concevoir partition + sort key
DynamoDB TTL n'est pas immédiat	TTL supprime dans 24-48h après expiration	Données restent visibles brièvement	Ne pas compter sur TTL pour sécurité
Global Secondary Index (GSI) coûte	GSI = throughput supplémentaire à provisionner	Surcoûts si GSI mal utilisés	Bien planifier projections, ne créer que GSI utiles

Le piège de la partition/sort key est le plus structurant : mal le concevoir dès le départ oblige souvent à recréer la table entière, DynamoDB ne permettant pas de modifier ces clés après coup.

5.5 Pièges ElastiCache

Côté cache, les pièges tiennent surtout à la confusion entre les deux moteurs et à une mauvaise gestion de la fraîcheur des données :

Piège	Réalité	Conséquence	Solution
Redis vs Memcached	Redis = persistance, structures complexes. Memcached = volatil, simple	Choisir Memcached pour données durables = perte	Redis pour sessions, Memcached pour cache éphémère
Cache-Aside pattern = possibilité cache stale	TTL peut garder données anciennes 1h	Utilisateurs voient données obsolètes	Réduire TTL ou implémenter invalidation manuelle
ElastiCache dans VPC ≠ accessible depuis EC2 autre subnet	Besoin Security Group + route table	EC2 ne peut pas accéder cache	Vérifier SG ElastiCache permet EC2, même VPC
Cluster mode disabled : une seule shard	Pas de sharding = un seul nœud max CPU	Bottleneck CPU même avec plusieurs replicas	Cluster mode enabled pour scalabilité

Le troisième piège rappelle qu'ElastiCache reste soumis aux mêmes règles réseau qu'EC2 ou RDS : être dans le même VPC ne suffit pas, il faut aussi que les Security Groups autorisent explicitement le flux entre l'application et le cache.

5.6 Pièges Architecture Générale

Ces derniers pièges dépassent un service unique et concernent des choix d'architecture transverses vus dans ce chapitre :

Piège	Réalité	Conséquence	Solution
Aurora Serverless = scaling pas instantané	Scaling automatique peut durer 30-60s	Latence pics pendant scaling	Pas idéal real-time, mieux RDS provisionné
VPC Endpoint S3 évite la NAT	Mais doit configurer politiques explicites	S3 accès reste privé mais règles complexes	Créer endpoint + bucket policy restrictive
VPC Endpoint = interface privée (coût)	Interface endpoint = ENI = \$0.007/h	Nombreux endpoints = facture élevée	Gateway endpoint pour S3/DynamoDB (gratuit)

Ce dernier piège illustre un principe général de ce chapitre : sur AWS, chaque mécanisme d'automatisation ou de connectivité a un modèle de coût propre, qu'il faut vérifier avant de le généraliser à grande échelle plutôt que de supposer qu'il est gratuit par défaut.

6. Construire une VPC en CLI

Info

Une activité pratique permet d'approfondir la construction d'un VPC complet.

6.1 Créer une VPC complète avec AWS CLI

Cette séquence construit une VPC de zéro, pièce par pièce : VPC → subnets public/privé → Internet Gateway → NAT Gateway → tables de routage. C'est l'ordre obligatoire — chaque ressource dépend de la précédente.

```
1  # =====
2  # ÉTAPE 1 : Créer la VPC avec CIDR /16
3  # =====
4
5  aws ec2 create-vpc \
6    --cidr-block 10.0.0.0/16 \
7    --tag-specifications 'ResourceType=vpc,Tags=[{Key=Name,Value=FormationVPC},
{Key=Env,Value=Training}]'
8
9  # Récupérer l'ID VPC depuis la sortie précédente
1
0  VPC_ID="vpc-12345678"
1
1
1
2  # Valider la VPC créée
1
3  aws ec2 describe-vpcs --vpc-ids $VPC_ID
```



Tip

Résultat attendu :

```
1  {
2      "Vpcs": [{
3          "VpcId": "vpc-12345678",
4          "CidrBlock": "10.0.0.0/16",
5          "State": "available",
6          "IsDefault": false,
7          "Tags": [{"Key": "Name", "Value": "FormationVPC"}, {"Key": "Env",
"Value": "Training"}]
8      }]
9  }
```

```

1  # =====
2  # ÉTAPE 2 : Créer subnets PUBLICS (pour ALB, NAT Gateway)
3  # =====
4
5  # Subnet public en AZ 1a (10.0.1.0/24 = 256 IPs)
6  aws ec2 create-subnet \
7      --vpc-id $VPC_ID \
8      --cidr-block 10.0.1.0/24 \
9      --availability-zone eu-west-1a \
10
11      --tag-specifications 'ResourceType=subnet,Tags=[{Key=Name,Value=SubnetPublic-
12  AZa},{Key=Type,Value=Public}]'
13
14  SUBNET_PUBLIC_AZa="subnet-public-a"
15
16
17
18  # Subnet public en AZ 1b (10.0.2.0/24 = 256 IPs)
19
20  aws ec2 create-subnet \
21
22      --vpc-id $VPC_ID \
23
24      --cidr-block 10.0.2.0/24 \
25
26      --availability-zone eu-west-1b \
27
28      --tag-specifications 'ResourceType=subnet,Tags=[{Key=Name,Value=SubnetPublic-
29  AZb},{Key=Type,Value=Public}]'
30
31  SUBNET_PUBLIC_AZb="subnet-public-b"
32
33
34

```

```

2
3  ## =====
2
4  # ÉTAPE 3 : Créer subnets PRIVÉS (pour EC2, RDS)
2
5  ## =====
2
6
2
7  # Subnet privé en AZ 1a (10.0.10.0/24)
2
8  aws ec2 create-subnet \
2
9      --vpc-id $VPC_ID \
3
0      --cidr-block 10.0.10.0/24 \
3
1      --availability-zone eu-west-1a \
3
2      --tag-specifications 'ResourceType=subnet,Tags=
[{'Key=Name,Value=SubnetPrivate-AZa},{Key=Type,Value=Private}]'
3
3
3
4  SUBNET_PRIVATE_AZa="subnet-private-a"
3
5
3
6  # Subnet privé en AZ 1b (10.0.20.0/24) pour haute dispo
3
7  aws ec2 create-subnet \
3
8      --vpc-id $VPC_ID \
3
9      --cidr-block 10.0.20.0/24 \
4
0      --availability-zone eu-west-1b \

```

```

4
1  --tag-specifications 'ResourceType=subnet,Tags=
[{{Key=Name,Value=SubnetPrivate-AZb}},{{Key=Type,Value=Private}}]'
4
2
4
3  SUBNET_PRIVATE_AZb="subnet-private-b"
4
4
4
5  # Lister tous les subnets créés
4
6  aws ec2 describe-subnets --filters "Name=vpc-id,Values=$VPC_ID"

```

Tip

Résultat attendu :

```

1  {
2      "Subnets": [
3          {"SubnetId": "subnet-public-a", "CidrBlock": "10.0.1.0/24",
"AvailabilityZone": "eu-west-1a", "State": "available", "Tags": [{{"Key":
"Name", "Value": "SubnetPublic-AZa"}}]},
4          {"SubnetId": "subnet-public-b", "CidrBlock": "10.0.2.0/24",
"AvailabilityZone": "eu-west-1b", "State": "available", "Tags": [{{"Key":
"Name", "Value": "SubnetPublic-AZb"}}]},
5          {"SubnetId": "subnet-private-a", "CidrBlock": "10.0.10.0/24",
"AvailabilityZone": "eu-west-1a", "State": "available", "Tags": [{{"Key":
"Name", "Value": "SubnetPrivate-AZa"}}]},
6          {"SubnetId": "subnet-private-b", "CidrBlock": "10.0.20.0/24",
"AvailabilityZone": "eu-west-1b", "State": "available", "Tags": [{{"Key":
"Name", "Value": "SubnetPrivate-AZb"}}]}
7      ]
8  }

```

```

1  # =====
2  # ÉTAPE 4 : Créer Internet Gateway (accès Internet pour VPC)
3  # =====
4
5  aws ec2 create-internet-gateway \
6      --tag-specifications 'ResourceType=internet-gateway,Tags=
[{'Key=Name,Value=FormationIGW}]'
7
8  IGW_ID="igw-12345678"
9
1
0  # Attacher IGW à la VPC
1
1  aws ec2 attach-internet-gateway \
1
2      --internet-gateway-id $IGW_ID \
1
3      --vpc-id $VPC_ID
1
4
1
5  # Valider l'attachement
1
6  aws ec2 describe-internet-gateways --internet-gateway-ids $IGW_ID

```



Tip

Résultat attendu :

```

1  {
2      "InternetGateways": [{
3          "InternetGatewayId": "igw-12345678",
4          "State": "available",
5          "Attachments": [{"State": "available", "VpcId": "vpc-12345678"}],
6          "Tags": [{"Key": "Name", "Value": "FormationIGW"}]
7      }]
8  }

```

```

1  # =====
2  # ÉTAPE 5 : Créer Elastic IPs et NAT Gateways
3  # =====
4
5  # Allouer une Elastic IP pour NAT Gateway AZ 1a
6  aws ec2 allocate-address \
7      --domain vpc \
8      --tag-specifications 'ResourceType=elastic-ip,Tags=[{Key=Name,Value=NAT-EIP-
AZa}]'
9
10
11 ALLOCATION_ID_AZa="eipalloc-12345678"
12
13
14
15
16 # Allouer une Elastic IP pour NAT Gateway AZ 1b
17
18 aws ec2 allocate-address \
19
20     --domain vpc \
21
22     --tag-specifications 'ResourceType=elastic-ip,Tags=[{Key=Name,Value=NAT-EIP-
AZb}]'
23
24
25
26
27 ALLOCATION_ID_AZb="eipalloc-87654321"
28
29
30
31 # Créer NAT Gateway en AZ 1a (place dans subnet PUBLIC)
32
33 aws ec2 create-nat-gateway \
34
35     --subnet-id $SUBNET_PUBLIC_AZa \
36
37     --allocation-id $ALLOCATION_ID_AZa \

```

```

2
3   --tag-specifications 'ResourceType=nat-gateway,Tags=[{Key=Name,Value=NAT-GW-
AZa}]'
2
4
2
5 NAT_GW_ID_AZa="nat-12345678"
2
6
2
7 # Créer NAT Gateway en AZ 1b (pour HA)
2
8 aws ec2 create-nat-gateway \
2
9   --subnet-id $SUBNET_PUBLIC_AZb \
3
0   --allocation-id $ALLOCATION_ID_AZb \
3
1   --tag-specifications 'ResourceType=nat-gateway,Tags=[{Key=Name,Value=NAT-GW-
AZb}]'
3
2
3
3 NAT_GW_ID_AZb="nat-87654321"
3
4
3
5 # Attendre que les NAT Gateways soient disponibles
3
6 aws ec2 wait nat-gateway-available --nat-gateway-ids $NAT_GW_ID_AZa
$NAT_GW_ID_AZb
3
7
3
8 # =====
3
9 # ÉTAPE 6 : Créer et configurer ROUTE TABLES PUBLIQUES
4
0 # =====

```



```

4
1
4
2 # Créer route table pour subnets PUBLICS
4
3 aws ec2 create-route-table \
4
4     --vpc-id $VPC_ID \
4
5     --tag-specifications 'ResourceType=route-table,Tags=[{Key=Name,Value=RT-
Public},{Key=Type,Value=Public}]'
4
6
4
7 RT_PUBLIC="rtb-public-12345"
4
8
4
9 # Ajouter route par défaut vers IGW (0.0.0.0/0 → IGW)
5
0 aws ec2 create-route \
5
1     --route-table-id $RT_PUBLIC \
5
2     --destination-cidr-block 0.0.0.0/0 \
5
3     --gateway-id $IGW_ID
5
4
5
5 # Associer subnet PUBLIC AZa avec route table PUBLIC
5
6 aws ec2 associate-route-table \
5
7     --subnet-id $SUBNET_PUBLIC_AZa \
5
8     --route-table-id $RT_PUBLIC
5
9

```

```

6
0 # Associer subnet PUBLIC AZb avec route table PUBLIC
6
1 aws ec2 associate-route-table \
6
2     --subnet-id $SUBNET_PUBLIC_AZb \
6
3     --route-table-id $RT_PUBLIC
6
4
6
5 # =====
6
6 # ÉTAPE 7 : Créer et configurer ROUTE TABLES PRIVÉES
6
7 # =====
6
8
6
9 # Créer route table pour subnets PRIVÉS en AZ 1a
7
0 aws ec2 create-route-table \
7
1     --vpc-id $VPC_ID \
7
2     --tag-specifications 'ResourceType=route-table,Tags=[{Key=Name,Value=RT-
Private-AZa},{Key=Type,Value=Private}] '
7
3
7
4 RT_PRIVATE_AZa="rtb-private-a"
7
5
7
6 # Ajouter route par défaut vers NAT Gateway AZ 1a
7
7 aws ec2 create-route \
7
8     --route-table-id $RT_PRIVATE_AZa \

```

```

7
9  --destination-cidr-block 0.0.0.0/0 \
8
0  --nat-gateway-id $NAT_GW_ID_AZa
8
1
8
2  # Associer subnet PRIVÉ AZa avec sa route table
8
3  aws ec2 associate-route-table \
8
4  --subnet-id $SUBNET_PRIVATE_AZa \
8
5  --route-table-id $RT_PRIVATE_AZa
8
6
8
7  # Créer route table pour subnets PRIVÉS en AZ 1b
8
8  aws ec2 create-route-table \
8
9  --vpc-id $VPC_ID \
9
0  --tag-specifications 'ResourceType=route-table,Tags=[{Key=Name,Value=RT-
Private-AZb},{Key=Type,Value=Private}]'
9
1
9
2  RT_PRIVATE_AZb="rtb-private-b"
9
3
9
4  # Ajouter route vers NAT Gateway AZ 1b
9
5  aws ec2 create-route \
9
6  --route-table-id $RT_PRIVATE_AZb \
9
7  --destination-cidr-block 0.0.0.0/0 \

```

```

9
8  --nat-gateway-id $NAT_GW_ID_AZb
9
9
1
0
0  # Associer subnet PRIVÉ AZb
1
0
1  aws ec2 associate-route-table \
1
0
2  --subnet-id $SUBNET_PRIVATE_AZb \
1
0
3  --route-table-id $RT_PRIVATE_AZb
1
0
4
1
0
5  # =====
1
0
6  # RÉSUMÉ : Vérifier la VPC complète
1
0
7  # =====
1
0
8
1
0
9  echo "✅ VPC FormationVPC créée avec succès !"
1
1
0  echo "=====

```

```

1
1
1 echo "VPC ID : $VPC_ID"
1
1
2 echo "Subnets publics : $SUBNET_PUBLIC_AZa, $SUBNET_PUBLIC_AZb"
1
1
3 echo "Subnets privés : $SUBNET_PRIVATE_AZa, $SUBNET_PRIVATE_AZb"
1
1
4 echo "Internet Gateway : $IGW_ID"
1
1
5 echo "NAT Gateways : $NAT_GW_ID_AZa (AZa), $NAT_GW_ID_AZb (AZb)"
1
1
6 echo "_____ "
1
1
7
1
1
8 # Lister la configuration complète
1
1
9 aws ec2 describe-vpcs --vpc-ids $VPC_ID --query 'Vpcs[0]'
1
2
0 aws ec2 describe-route-tables --filters "Name=vpc-id,Values=$VPC_ID"

```



Tip

Résultat attendu :

```
1  {
2      "VpcId": "vpc-12345678",
3      "CidrBlock": "10.0.0.0/16",
4      "State": "available",
5      "Tags": [{ "Key": "Name", "Value": "FormationVPC" }]
6  }
```

6.2 Créer et configurer un RDS Multi-AZ

Cette démo crée une instance RDS MySQL en production-ready : Multi-AZ activé (réplication synchrone vers une AZ de secours), chiffrement KMS, logs CloudWatch, et authentification IAM. Chaque option a son impact sur la résilience et la sécurité.

```

1  # 1. Créer une instance RDS MySQL avec Multi-AZ ET chiffrement
2  aws rds create-db-instance \
3      --db-instance-identifier formation-db \
4      --db-instance-class db.t3.micro \
5      --engine mysql \
6      --engine-version 8.0.35 \
7      --master-username admin \
8      --master-user-password 'MonMotDePasseSecurise123!' \
9      --allocated-storage 20 \
10     --storage-type gp3 \
11     --storage-encrypted \
12     --kms-key-id arn:aws:kms:eu-west-1:123456789012:key/12345678-1234-1234-1234-123456789012 \
13     --vpc-security-group-ids sg-12345678 \
14     --db-subnet-group-name my-db-subnet-group \
15     --multi-az \
16     --backup-retention-period 7 \
17     --backup-window "03:00-04:00" \
18     --maintenance-window "mon:04:00-mon:05:00" \
19     --enable-cloudwatch-logs-exports '["error","general","slowquery"]' \
20     --enable-iam-database-authentication \
21     --tag-specifications 'ResourceType=db,Tags=[{Key=Name,Value=FormationDB},{Key=Env,Value=Training}]'

```

```

2
3 # 2. Attendre que l'instance soit disponible (peut prendre 5-10 min)
2
4 aws rds wait db-instance-available \
2
5     --db-instance-identifiant formation-db
2
6
2
7 # 3. Récupérer l'endpoint RDS (adresse pour connexion)
2
8 aws rds describe-db-instances \
2
9     --db-instance-identifiant formation-db \
3
0     --query 'DBInstances[0].Endpoint.Address' \
3
1     --output text
3
2 # Résultat : formation-db.abc123.eu-west-1.rds.amazonaws.com

```



Tip

Résultat attendu :

```
formation-db.abc123.eu-west-1.rds.amazonaws.com
```

```

1 # 4. Vérifier la configuration Multi-AZ
2 aws rds describe-db-instances \
3     --db-instance-identifiant formation-db \
4     --query 'DBInstances[0].
[DBInstanceIdentifier,MultiAZ,AvailabilityZone,PreferredBackupWindow]'

```




Résultat attendu :

```
1  [  
2    "formation-db",  
3    true,  
4    "eu-west-1a",  
5    "03:00-04:00"  
6  ]
```

```

1  # 5. Modifier l'instance pour augmenter la classe (scaling vertical)
2  aws rds modify-db-instance \
3      --db-instance-identifier formation-db \
4      --db-instance-class db.t3.small \
5      --apply-immediately
6
7  # 6. Créer un Read Replica pour analytics/reporting
8  aws rds create-db-instance-read-replica \
9      --db-instance-identifier formation-db-analytics \
10
11      --source-db-instance-identifier formation-db \
12
13      --db-instance-class db.t3.micro \
14
15      --availability-zone eu-west-1b \
16
17      --storage-type gp3
18
19
20  # 7. Créer un Read Replica inter-région pour Disaster Recovery
21
22  aws rds create-db-instance-read-replica \
23
24      --db-instance-identifier formation-db-dr \
25
26      --source-db-instance-identifier formation-db \
27
28      --source-region eu-west-1 \
29
30      --region us-east-1 \
31
32      --db-instance-class db.t3.micro \
33
34      --tag-specifications 'ResourceType=db,Tags=[{Key=Name,Value=FormationDB-DR}]'

```

```

2
4 # 8. Promouvoir un Read Replica en base indépendante (cas failover manual)
2
5 # ⚠ Une fois promu, il n'est plus répliqué = devient Master indépendant
2
6 aws rds promote-read-replica \
2
7     --db-instance-identifier formation-db-analytics
2
8
2
9 # 9. Créer une sauvegarde manuelle (snapshot) avec timestamp
3
0 SNAPSHOT_ID="formation-db-backup-$(date +%Y%m%d-%H%M%S) "
3
1 aws rds create-db-snapshot \
3
2     --db-instance-identifier formation-db \
3
3     --db-snapshot-identifier $SNAPSHOT_ID \
3
4     --tags Key=Name,Value="Manual backup" Key=Version,Value="pre-update"
3
5
3
6 # 10. Lister tous les snapshots
3
7 aws rds describe-db-snapshots \
3
8     --db-instance-identifier formation-db \
3
9     --query 'DBSnapshots[*].
[DBSnapshotIdentifier,SnapshotCreateTime,DBInstanceIdentifier,AllocatedStorage]'

```



Résultat attendu :

```
1  [  
2      ["formation-db-backup-20260315-143022", "2026-03-15T14:30:22.000Z",  
"formation-db", 20],  
3      ["formation-db-backup-20260316-031500", "2026-03-16T03:15:00.000Z",  
"formation-db", 20]  
4  ]
```

```

1  # 11. Restaurer une base à partir d'un snapshot (crée une nouvelle instance)
2  aws rds restore-db-instance-from-db-snapshot \
3      --db-instance-identifier formation-db-restored \
4      --db-snapshot-identifier formation-db-backup-20240324 \
5      --db-instance-class db.t3.micro
6
7  # 12. Copier un snapshot vers une autre région (pour DR)
8  aws rds copy-db-snapshot \
9      --source-db-snapshot-identifier arn:aws:rds:eu-west-
1:123456789012:snapshot:formation-db-backup \
1
0      --target-db-snapshot-identifier formation-db-backup-us \
1
1      --source-region eu-west-1 \
1
2      --region us-east-1
1
3
1
4  # 13. Supprimer une instance RDS (avec snapshot final)
1
5  aws rds delete-db-instance \
1
6      --db-instance-identifier formation-db \
1
7      --final-db-snapshot-identifier formation-db-final-snapshot \
1
8      --delete-automated-backups
1
9
2
0  # 14. Supprimer une instance SANS snapshot (⚠ DANGER)
2
1  aws rds delete-db-instance \
2
2      --db-instance-identifier formation-db-analytics \
2
3      --skip-final-snapshot

```

La base RDS étant provisionnée, il reste à verrouiller son accès réseau : c'est le rôle des Security Groups configurés ci-dessous.

6.3 Configurer Security Groups (pare-feu instance)

Les Security Groups fonctionnent en chaîne dans une architecture 3-tiers : l'ALB accepte le trafic public, les instances EC2 n'acceptent que le trafic de l'ALB, et RDS n'accepte que le trafic des instances EC2. Cette démo crée les 3 SG et configure leurs règles d'entrée dans cet ordre.

```

1  # =====
2  # Créer Security Groups pour architecture 3-tiers
3  # =====
4
5  # SG 1 : ALB (Application Load Balancer)
6  aws ec2 create-security-group \
7      --group-name sg-alb \
8      --description "Security Group pour Application Load Balancer" \
9      --vpc-id $VPC_ID \
1
0      --tag-specifications 'ResourceType=security-group,Tags=[{Key=Name,Value=SG-
ALB}]'
1
1
1
2  SG_ALB="sg-alb-12345"
1
3
1
4  # SG 2 : EC2 Web (Applications)
1
5  aws ec2 create-security-group \
1
6      --group-name sg-web \
1
7      --description "Security Group pour serveurs web" \
1
8      --vpc-id $VPC_ID \
1
9      --tag-specifications 'ResourceType=security-group,Tags=[{Key=Name,Value=SG-
Web}]'
2
0
2
1  SG_WEB="sg-web-12345"
2
2

```

```

2
3  # SG 3 : RDS Database
2
4  aws ec2 create-security-group \
2
5      --group-name sg-db \
2
6      --description "Security Group pour base de données RDS" \
2
7      --vpc-id $VPC_ID \
2
8      --tag-specifications 'ResourceType=security-group,Tags=[{Key=Name,Value=SG-
DB}]'
2
9
3
0  SG_DB="sg-db-12345"
3
1
3
2  # =====
3
3  # RÈGLES INGRESS SG-ALB : Accepter trafic Internet
3
4  # =====
3
5
3
6  # HTTP (port 80) depuis Internet
3
7  aws ec2 authorize-security-group-ingress \
3
8      --group-id $SG_ALB \
3
9      --protocol tcp \
4
0      --port 80 \
4
1      --cidr 0.0.0.0/0

```



```
4
2
4
3 # HTTPS (port 443) depuis Internet
4
4 aws ec2 authorize-security-group-ingress \
4
5     --group-id $SG_ALB \
4
6     --protocol tcp \
4
7     --port 443 \
4
8     --cidr 0.0.0.0/0
4
9
5
0 # =====
5
1 # RÈGLES INGRESS SG-Web : Accepter depuis ALB
5
2 # =====
5
3
5
4 # Port 80 (HTTP) depuis SG-ALB uniquement
5
5 aws ec2 authorize-security-group-ingress \
5
6     --group-id $SG_WEB \
5
7     --protocol tcp \
5
8     --port 80 \
5
9     --source-security-group-id $SG_ALB
6
0
```

```

6
1  # Port 443 (HTTPS) depuis SG-ALB
6
2  aws ec2 authorize-security-group-ingress \
6
3      --group-id $SG_WEB \
6
4      --protocol tcp \
6
5      --port 443 \
6
6      --source-security-group-id $SG_ALB
6
7
6
8  # Port 22 (SSH) depuis IP admin (YOUR_IP remplacer)
6
9  aws ec2 authorize-security-group-ingress \
7
0      --group-id $SG_WEB \
7
1      --protocol tcp \
7
2      --port 22 \
7
3      --cidr YOUR_IP/32  # Exemple : 203.0.113.0/32
7
4
7
5  # =====
7
6  # RÈGLES INGRESS SG-DB : Accepter depuis EC2 Web
7
7  # =====
7
8
7
9  # Port 3306 (MySQL) depuis SG-WEB uniquement

```

```

8
0  aws ec2 authorize-security-group-ingress \
8
1  --group-id $SG_DB \
8
2  --protocol tcp \
8
3  --port 3306 \
8
4  --source-security-group-id $SG_WEB
8
5
8
6  # Port 3306 depuis autre subnet privé (si applicable)
8
7  aws ec2 authorize-security-group-ingress \
8
8  --group-id $SG_DB \
8
9  --protocol tcp \
9
0  --port 3306 \
9
1  --cidr 10.0.0.0/8
9
2
9
3  # =====
9
4  # RÈGLES EGRESS (sortante) : Vérifier et modifier si nécessaire
9
5  # =====
9
6
9
7  # Par défaut, tout trafic sortant est autorisé (EGRESS)
9
8  # Lister les règles de sortie

```

```
9
9  aws ec2 describe-security-groups \
1
0
0  --group-ids $SG_WEB \
1
0
1  --query 'SecurityGroups[0].IpPermissionsEgress'
1
0
2
1
0
3  # Révoquer trafic EGRESS par défaut (si vous voulez contrôle strict)
1
0
4  aws ec2 revoke-security-group-egress \
1
0
5  --group-id $SG_WEB \
1
0
6  --protocol -1 \
1
0
7  --cidr 0.0.0.0/0
1
0
8
1
0
9  # Autoriser seulement EGRESS vers RDS DB
1
1
0  aws ec2 authorize-security-group-egress \
1
1
1  --group-id $SG_WEB \
```

```

1
1
2  --protocol tcp \
1
1
3  --port 3306 \
1
1
4  --destination-security-group-id $SG_DB
1
1
5
1
1
6  # Autoriser seulement EGRESS vers Internet (DNS, HTTPS)
1
1
7  aws ec2 authorize-security-group-egress \
1
1
8  --group-id $SG_WEB \
1
1
9  --protocol tcp \
1
2
0  --port 443 \
1
2
1  --cidr 0.0.0.0/0
1
2
2
1
2
3  aws ec2 authorize-security-group-egress \
1
2
4  --group-id $SG_WEB \

```

```
1
2
5  --protocol udp \
1
2
6  --port 53 \
1
2
7  --cidr 0.0.0.0/0
1
2
8
1
2
9  # =====
1
3
0  # OPÉRATIONS : Modifier et supprimer règles
1
3
1  # =====
1
3
2
1
3
3  # Lister toutes les règles d'un SG
1
3
4  aws ec2 describe-security-groups --group-ids $SG_WEB
```



Tip

Résultat attendu :

```
1  {
2      "SecurityGroups": [{
3          "GroupId": "sg-web-12345",
4          "GroupName": "sg-web",
5          "Description": "Security Group pour serveurs web",
6          "IpPermissions": [
7              {"IpProtocol": "tcp", "FromPort": 80, "ToPort": 80,
"UserIdGroupPairs": [{"GroupId": "sg-alb-12345"}]},
8              {"IpProtocol": "tcp", "FromPort": 443, "ToPort": 443,
"UserIdGroupPairs": [{"GroupId": "sg-alb-12345"}]},
9              {"IpProtocol": "tcp", "FromPort": 22, "ToPort": 22, "IpRanges":
[{"CidrIp": "203.0.113.0/32"}]}
10         ]
11     }]
12 }
```

```

1  # Révoquer une règle ingress (exemple : SSH)
2  aws ec2 revoke-security-group-ingress \
3      --group-id $SG_WEB \
4      --protocol tcp \
5      --port 22 \
6      --cidr YOUR_IP/32
7
8  # Ajouter une règle avec description
9  aws ec2 authorize-security-group-ingress \
10
11      --group-id $SG_WEB \
12
13      --protocol tcp \
14
15      --port 80 \
16
17      --source-security-group-id $SG_ALB \
18
19      --group-rule-description "HTTP from ALB"
20
21
22
23
24
25
26 # Supprimer un Security Group (doit d'abord être détaché)
27
28
29
30
31
32
33 # =====
34
35 # RÉSUMÉ : Architecture complète
36
37
38 # =====
39
40
41
42
43 echo "✅ Security Groups configurés !"

```



```

2
4 echo "_____ "
2
5 echo "SG-ALB : $SG_ALB"
2
6 echo "  INGRESS : 80 (HTTP), 443 (HTTPS) from 0.0.0.0/0"
2
7 echo "  EGRESS : All (par défaut)"
2
8 echo ""
2
9 echo "SG-Web : $SG_WEB"
3
0 echo "  INGRESS : 80, 443 from SG-ALB; 22 from YOUR_IP"
3
1 echo "  EGRESS : Restreint (DB, DNS, Internet)"
3
2 echo ""
3
3 echo "SG-DB : $SG_DB"
3
4 echo "  INGRESS : 3306 from SG-Web"
3
5 echo "  EGRESS : Aucun (DB ne sort pas)"
3
6 echo "_____ "

```

Le réseau et la base de données étant sécurisés, il reste à rendre l'application joignable par son nom de domaine : c'est l'objet de la dernière section pratique de ce chapitre.

6.4 Créer une Zone Route 53 et des Enregistrements

Une Hosted Zone Route 53 est le conteneur DNS pour un domaine. On crée la zone, puis on y ajoute les enregistrements (A, CNAME, MX...) via des "change batches" JSON. Chaque modification est atomique — soit tout réussit, soit tout échoue.

```
1 # 1. Créer une zone Route 53
2 aws route53 create-hosted-zone \
3   --name example.com \
4   --caller-reference $(date +%s) \
5   --hosted-zone-config PrivateZone=false
```



Résultat attendu :

```
1 {
2   "HostedZone": {
3     "Id": "/hostedzone/Z1234567890ABC",
4     "Name": "example.com.",
5     "Config": {"PrivateZone": false},
6     "ResourceRecordSetCount": 2
7   },
8   "DelegationSet": {
9     "NameServers": [
10     "ns-123.awsdns-15.com",
11     "ns-456.awsdns-57.net",
12     "ns-789.awsdns-31.co.uk",
13     "ns-1012.awsdns-62.org"
14   ]
15 }
16 }
```

```

1 ZONE_ID="Z1234567890ABC"
2
3 # 2. Créer un enregistrement A (IPv4)
4 aws route53 change-resource-record-sets \
5     --hosted-zone-id $ZONE_ID \
6     --change-batch '{
7         "Changes": [{
8             "Action": "CREATE",
9             "ResourceRecordSet": {
10
11                 "Name": "www.example.com",
12
13                 "Type": "A",
14
15                 "TTL": 300,
16
17                 "ResourceRecords": [{"Value": "93.184.216.34"}]
18             }
19         }]
20     }'
21
22 # 3. Créer un Health Check HTTP
23
24 aws route53 create-health-check \
25
26     --health-check-config '{
27
28         "Type": "HTTP",
29
30         "IPAddress": "93.184.216.34",
31
32         "Port": 80,

```

```

2
4     "ResourcePath": "/health",
2
5     "FullyQualifiedDomainName": "www.example.com",
2
6     "RequestInterval": 30,
2
7     "FailureThreshold": 3
2
8     }'

```

```

0 # 4. Créer un enregistrement avec failover
3

```

```

1 aws route53 change-resource-record-sets \
3
2     --hosted-zone-id $ZONE_ID \
3
3     --change-batch '{
3
4         "Changes": [
3
5             {
3
6                 "Action": "CREATE",
3
7                 "ResourceRecordSet": {
3
8                     "Name": "app.example.com",
3
9                     "Type": "A",
4
0                     "SetIdentifier": "Primary",
4
1                     "Failover": "PRIMARY",
4
2                     "TTL": 60,

```

```

4
3     "ResourceRecords": [{ "Value": "93.184.216.34" }],
4
4     "HealthCheckId": "health-check-id-12345"
4
5     }
4
6 },
4
7 {
4
8     "Action": "CREATE",
4
9     "ResourceRecordSet": {
5
0         "Name": "app.example.com",
5
1         "Type": "A",
5
2         "SetIdentifier": "Secondary",
5
3         "Failover": "SECONDARY",
5
4         "TTL": 60,
5
5         "ResourceRecords": [{ "Value": "93.184.216.35" }],
5
6     }
5
7 }
5
8 ]
5
9 { '

```



Résultat attendu :

```

1  # change-resource-record-sets (enregistrement A) :
2  {
3      "ChangeInfo": {
4          "Id": "/change/C1234567890ABC",
5          "Status": "PENDING",
6          "SubmittedAt": "2026-03-15T14:30:00.000Z"
7      }
8  }
9
1
0  # create-health-check :
1
1  {
1
2      "HealthCheck": {
1
3          "Id": "health-check-id-12345",
1
4          "HealthCheckConfig": {
1
5              "IPAddress": "93.184.216.34",
1
6              "Port": 80,
1
7              "Type": "HTTP",
1
8              "ResourcePath": "/health",
1
9              "FullyQualifiedDomainName": "www.example.com",
2
0              "RequestInterval": 30,
2
1              "FailureThreshold": 3
2
2          },
2
3          "HealthCheckVersion": 1

```

```
2
4     }
2
5 }
```

Le statut **PENDING** est normal — Route 53 propage les changements DNS dans les 60 secondes en général. Une fois propagé, le statut passe à **INSYNC**. Les enregistrements A avec failover sont actifs : si le health check échoue sur le Primary, Route 53 bascule automatiquement vers le Secondary.

Le réseau et les données sont maintenant en place. Le Chapitre 4 y installe le calcul et le stockage — instances EC2, buckets S3, et les conteneurs (ECS/ECR/Fargate) comme alternative moderne à la VM classique.

Travaux pratiques — Amazon VPC et bases de données

AWS Academy — Cloud Foundations

Type	Référence	Intitulé
Module	Module 5	Mise en réseau et diffusion de contenu
Module	Module 8	Bases de données
Atelier	Atelier 2	Création de votre VPC et lancement d'un serveur web — 100 points
Atelier	Atelier 5	Création d'un serveur de base de données RDS — 100 points

Type	Référence	Intitulé
Module	Module 6	Ajout d'une couche de base de données
Module	Module 7	Création d'un environnement réseau
Module	Module 8	Connexion de réseaux
Atelier guidé	Module 6	Création d'une base de données Amazon RDS
Atelier Défi	Module 6	Migration d'une base de données vers Amazon RDS
Atelier guidé	Module 7	Création d'un cloud privé virtuel — VPC
Atelier Défi	Module 7	Création d'un environnement réseau VPC pour le café
Atelier guidé	Module 8	Création d'une connexion d'appairage de VPC

Go Deploy

Lab	Intitulé
Lab 14	Créer un Virtual Private Cloud — VPC
Lab 15	Sécurité en couches dans un VPC
Lab 16	VPC Peering
Lab 17	Elastic IPs
Lab 18	NAT Gateways
Lab 19	Introduction à AWS Elastic Load Balancing
Lab 20	Accéder à S3 depuis une instance EC2 privée avec un VPC Endpoint
Lab 23	Introduction aux VPC Flow Logs
Lab 27	Migrer des bases de données avec AWS DMS
Lab 29	Introduction à Amazon DynamoDB
Lab 30	Déployer RDS Multi-AZ et une Read Replica, puis simuler un basculement

► Parcours guidé et validation

Ressources

Documentation officielle AWS

Pour approfondir les services réseau et bases de données présentés dans ce chapitre, voici les références officielles correspondantes :

- [Amazon RDS Documentation](#) 
- [Amazon DynamoDB Documentation](#) 
- [Amazon VPC Documentation](#) 
- [Amazon Route 53 Documentation](#) 

- [Amazon ElastiCache Documentation](#) 
- [AWS Transit Gateway](#) 

Le quiz ci-dessous permet de vérifier l'acquisition des notions clés de ce chapitre avant de passer au suivant.

Quiz interactif du chapitre

Choisissez une réponse : la correction expliquée apparaît immédiatement. Les propositions changent d'ordre à chaque nouvelle tentative.

Le quiz interactif reste disponible dans la version web du support.

Mini-jeu de synthèse — AWS Quest

Résolvez cinq décisions factuelles pour consolider les fondamentaux avant d'ouvrir le chapitre suivant. Le jeu est une simulation locale : il ne se connecte à aucun compte AWS.

Le quiz interactif reste disponible dans la version web du support.

Info

Après avoir sécurisé les accès avec IAM au chapitre précédent, les stagiaires disposent des bases nécessaires pour créer et protéger de vraies ressources AWS : ce chapitre aborde les deux briques les plus utilisées du Cloud AWS, le stockage objet S3 et le calcul EC2.

Objectifs du chapitre

À l'issue de ce chapitre, les stagiaires seront capables de :

- **Créer et administrer** un bucket Amazon S3 (chiffrement, versioning, lifecycle, CLI)
- **Choisir et lancer** une instance EC2 adaptée (type, AMI, stockage, Security Groups, CLI)
- **Comparer** les modèles de tarification EC2 et utiliser AWS Compute Optimizer
- **Déployer** un conteneur sur ECS/ECR/Fargate comme alternative à l'instance EC2 classique
- **Concevoir** une architecture haute disponibilité combinant Elastic Load Balancing et Auto Scaling

1. Introduction aux services de stockage AWS

 Vidéo : [Introduction to Amazon S3](#)

1.1 Pourquoi plusieurs services de stockage ?

Le stockage est au cœur de toute infrastructure cloud. AWS propose plusieurs types de stockage, mais **Amazon Simple Storage Service (S3)** est le service le plus emblématique : fiable, scalable et économique.

Créé en 2006, S3 a révolutionné la façon dont les entreprises stockent leurs données en passant d'un modèle de serveur à un modèle **d'espace de stockage à la demande**.

Avant de plonger dans les services techniques, il est essentiel de comprendre **pourquoi AWS propose plusieurs modèles de stockage** et dans quels contextes les utiliser.

Dans une entreprise traditionnelle, le stockage repose sur :

- des **disques durs internes** (pour les postes ou serveurs locaux),
- des **baies NAS/SAN** (pour le stockage partagé),
- parfois des sauvegardes sur bande ou sur site distant.

AWS transpose ces modèles dans le Cloud et les **rend flexibles, évolutifs et disponibles à la demande**.

1.2 Les trois modèles de stockage AWS

AWS reprend ces trois familles de stockage traditionnelles et leur donne chacune un service managé équivalent :

Type de stockage	Service AWS	Cas d'usage typique	Analogie utilisateur
Objet	Amazon S3	Sauvegarde, site statique, logs, Data Lake	Dropbox / Google Drive
Bloc	Amazon EBS	Disque de VM, base de données, stockage persistant	Disque dur local
Fichier	Amazon EFS/FSx	Partage réseau, systèmes distribués	NAS / Partage Windows

À retenir : Chaque type de stockage a ses propres performances, coûts et scénarios d'usage. Nous allons détailler S3 et EBS en priorité, car ce sont les services les plus utilisés par les administrateurs AWS en début de carrière.

 [Documentation Amazon S3](#)

2. Amazon S3 : Le stockage objet scalable

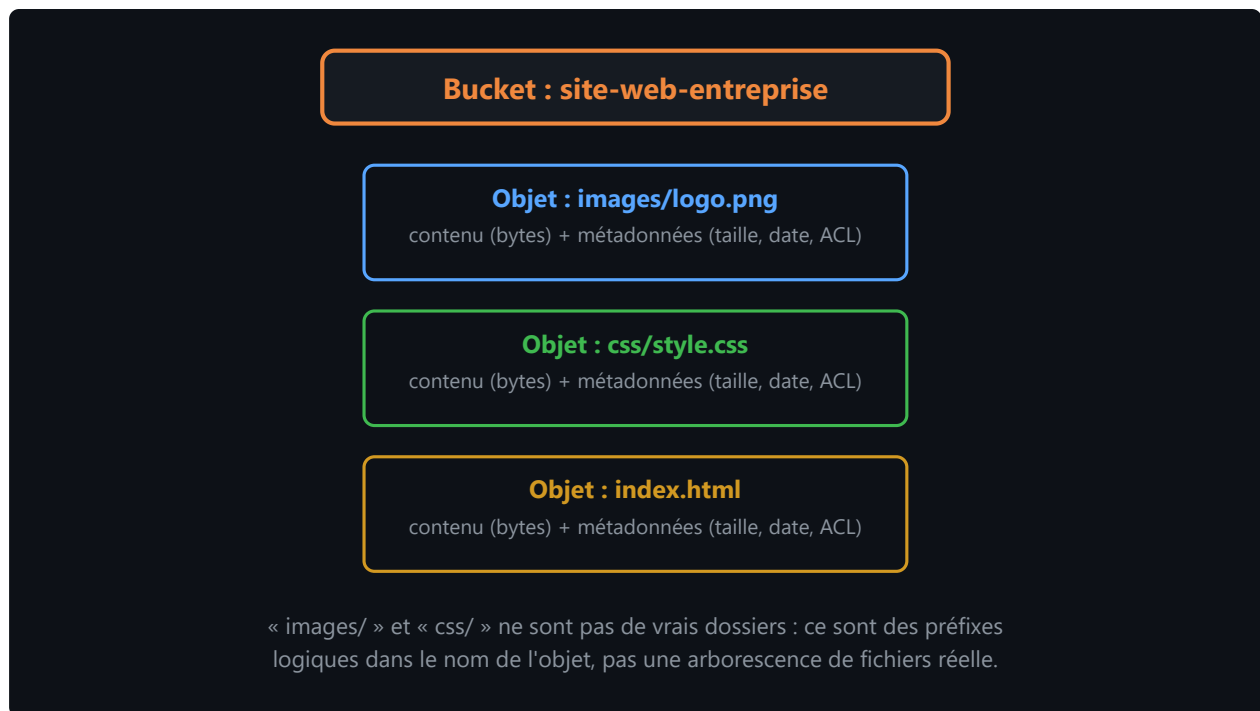
2.1 Qu'est-ce qu'Amazon S3 ?

Amazon S3 (Simple Storage Service) est le service de stockage le plus utilisé sur AWS. Il permet de stocker des fichiers (appelés **objets**) dans le cloud, avec une capacité quasiment illimitée, une très haute disponibilité (accès garanti) et une durabilité exceptionnelle (les données ne sont pas perdues).

Mais attention : **S3 ne fonctionne pas comme un disque dur classique**. C'est un système de **stockage objet**, ce qui signifie que chaque fichier est stocké avec des informations supplémentaires (appelées **métadonnées**) dans un conteneur appelé **bucket**.

2.2 L'armoire de rangement : analogie avec S3

Imaginez une **armoire de rangement** :



- L'armoire, c'est le **bucket** : un conteneur dans lequel vous rangez vos fichiers.
- Chaque fichier est un **objet** : il contient le contenu (ex. une image) + des étiquettes (métadonnées) comme son nom, sa date, ses droits d'accès.
- Les "dossiers" que vous voyez dans S3 ne sont pas réels : ce sont juste des **préfixes logiques** dans le nom du fichier (ex. `images/logo.png`).

Cette absence de vraie hiérarchie de dossiers surprend souvent les nouveaux utilisateurs ; l'exemple suivant montre concrètement comment ces préfixes s'utilisent en pratique.

2.3 Exemple concret

Vous créez un bucket nommé `site-web-entreprise`. Vous y déposez :

- `images/logo.png`
- `css/style.css`
- `js/app.js`

Ces fichiers peuvent ensuite être **consultés via Internet**, sans serveur web, si vous configurez le bucket en mode **site statique**.

 [S3 Static Website Hosting](#) 

2.4 Structure interne de S3

Reprenons l'analogie de l'armoire avec le vocabulaire technique exact que vous rencontrerez dans la console et l'API S3 :

Élément	Description
Bucket	Conteneur global (nom unique dans AWS), lié à une région
Objet	Fichier + métadonnées (nom, taille, type, permissions)
Préfixe	Partie du nom qui simule un dossier (ex. <code>images/</code>)
Key	Identifiant unique de l'objet au sein du bucket
Métadonnées	Informations sur l'objet (format, date, ACL, tags)

Retenez surtout la **Key** : c'est cet identifiant unique, et non un chemin de dossier réel, que l'API S3 utilise pour retrouver un objet — ce qui explique pourquoi renommer un “dossier” entier sur S3 revient en réalité à recopier tous les objets sous une nouvelle Key.

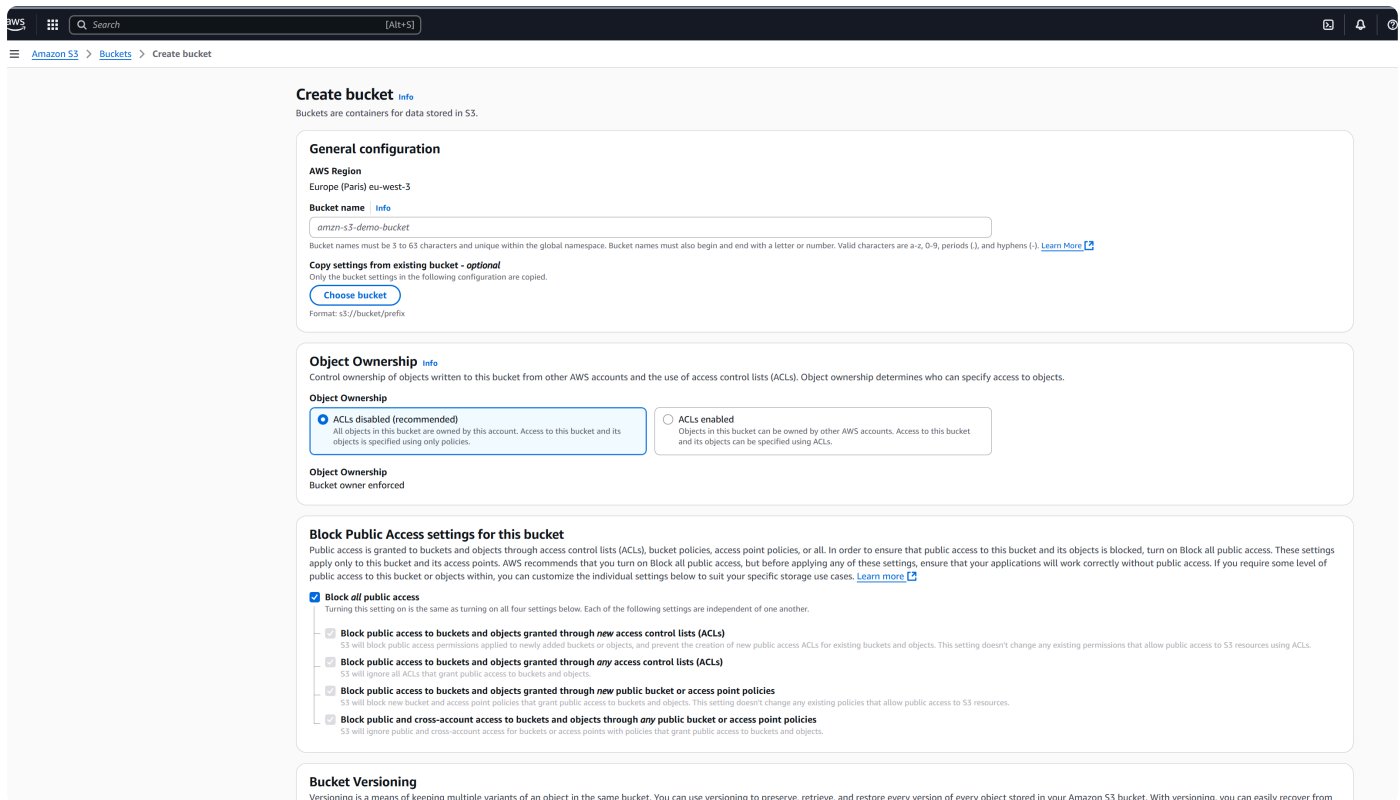
2.5 Caractéristiques clés de S3

Ces éléments structurels s'accompagnent de propriétés de service qui expliquent la popularité de S3 :

Caractéristique	Description
Durabilité	99,999999999% (11 neuf) grâce à la réplication automatique sur plusieurs zones.
Disponibilité	Haute disponibilité (jusqu'à 99,99% selon la classe).
Évolutivité	Pas de limite pratique en nombre d'objets.
Sécurité	Contrôle fin via IAM, ACL, politiques de bucket et chiffrement.
Coût à l'usage	Payez uniquement pour le stockage et les requêtes.

La distinction entre durabilité (99,999999999 %, la probabilité de ne jamais perdre un objet) et disponibilité (99,99 %, la probabilité qu'il soit accessible à un instant donné) est un point souvent confondu : un objet peut être temporairement inaccessible (disponibilité) sans jamais être perdu (durabilité).

2.6 Console S3 — Création d'un bucket



Create bucket [info](#)
Buckets are containers for data stored in S3.

General configuration

AWS Region
Europe (Paris) eu-west-3

Bucket name [info](#)

Copy settings from existing bucket - optional
Only the bucket settings in the following configuration are copied.
[Choose bucket](#)
Format: s3://bucket/prefix

Object Ownership [info](#)
Control ownership of objects written to this bucket from other AWS accounts and the use of access control lists (ACLs). Object ownership determines who can specify access to objects.

Object Ownership

☒ **ACLs disabled (recommended)**
All objects in this bucket are owned by this account. Access to this bucket and its objects is specified using only policies.

☐ **ACLs enabled**
Objects in this bucket can be owned by other AWS accounts. Access to this bucket and its objects can be specified using ACLs.

Object Ownership
Bucket owner enforced

Block Public Access settings for this bucket
Public access is granted to buckets and objects through access control lists (ACLs), bucket policies, access point policies, or all. In order to ensure that public access to this bucket and its objects is blocked, turn on Block all public access. These settings apply only to this bucket and its access points. AWS recommends that you turn on Block all public access, but before applying any of these settings, ensure that your applications will work correctly without public access. If you require some level of public access to this bucket or objects within, you can customize the individual settings below to suit your specific storage use cases. [Learn more](#)

☒ **Block all public access**
Turning this setting on is the same as turning on all four settings below. Each of the following settings are independent of one another.

- ☒ **Block public access to buckets and objects granted through new access control lists (ACLs)**
S3 will block public access permissions applied to newly added buckets or objects, and prevent the creation of new public access ACLs for existing buckets and objects. This setting doesn't change any existing permissions that allow public access to S3 resources using ACLs.
- ☒ **Block public access to buckets and objects granted through any access control lists (ACLs)**
S3 will ignore all ACLs that grant public access to buckets and objects.
- ☒ **Block public access to buckets and objects granted through new public bucket or access point policies**
S3 will block new bucket and access point policies that grant public access to buckets and objects. This setting doesn't change any existing policies that allow public access to S3 resources.
- ☒ **Block public and cross-account access to buckets and objects through any public bucket or access point policies**
S3 will ignore public and cross-account access for buckets or access points with policies that grant public access to buckets and objects.

Bucket Versioning
Versioning is a means of keeping multiple variants of an object in the same bucket. You can use versioning to preserve, retrieve, and restore every version of every object stored in your Amazon S3 bucket. With versioning, you can easily recover from

3. Protéger et optimiser les données S3

Amazon S3 propose plusieurs mécanismes pour **sécuriser vos fichiers, préserver leur historique, et réduire les coûts de stockage**. Ces options sont souvent méconnues, mais elles sont essentielles pour bien gérer vos données dans le cloud.

3.1 Chiffrement : protéger les fichiers contre les accès non autorisés

Quand vous stockez un fichier dans S3, vous pouvez demander à AWS de le **chiffrer automatiquement**. Cela signifie que même si quelqu'un accède physiquement au disque, il ne pourra pas lire le contenu sans la clé.

Types de chiffrement et gestion des clés

S3 propose trois mécanismes complémentaires, deux pour les données au repos et un pour les données en transit :

Acronyme	Signification complète	Description pédagogique
SSE-S3	<i>Server-Side Encryption with Amazon S3-managed keys</i>	Le chiffrement est géré automatiquement par AWS S3 . Vous n'avez rien à configurer.
SSE-KMS	<i>Server-Side Encryption with AWS Key Management Service</i>	Le chiffrement utilise AWS KMS , un service de gestion de clés. Vous définissez et contrôlez les clés.
HTTPS/TLS	<i>HyperText Transfer Protocol Secure / Transport Layer Security</i>	Ce protocole sécurise les échanges entre votre navigateur ou application et AWS.

SSE-S3 et SSE-KMS protègent la donnée stockée sur disque, tandis que HTTPS/TLS protège la donnée pendant son trajet réseau — les deux catégories sont à activer simultanément, l'une ne remplaçant pas l'autre.

3.2 Détails des types de chiffrement

SSE (Server-Side Encryption)

Chiffrement effectué **côté serveur**, c'est-à-dire par AWS une fois que les données sont reçues.

SSE-S3 : AWS chiffre les objets S3 avec une clé gérée par le service S3 lui-même.

- **Avantage** : aucune configuration requise.
- **Niveau de sécurité** : standard, suffisant pour de nombreux cas d'usage.

SSE-KMS : AWS chiffre les objets S3 avec une clé gérée par **AWS KMS**, que vous pouvez créer, activer/désactiver, auditer.

- **Avantage** : contrôle granulaire sur les clés.
- **Complexité** : nécessite configuration, permissions IAM, et gestion des quotas KMS.

C'est précisément ce service KMS, mentionné dans SSE-KMS, qui mérite d'être détaillé séparément puisqu'il ne se limite pas à S3.

KMS (Key Management Service)

Service AWS permettant de **créer, stocker et gérer** des clés de chiffrement.

- Utilisé dans **SSE-KMS**, mais aussi pour chiffrer des volumes EBS, des secrets, etc.
- Permet la **rotation automatique**, l'audit via CloudTrail, et l'intégration avec IAM.

Le chiffrement au repos protège la donnée stockée, mais elle circule aussi sur le réseau avant d'arriver sur S3 — c'est là qu'intervient le troisième mécanisme.

HTTPS / TLS

Protocole de **sécurisation des communications réseau**.

- **HTTPS** est HTTP + TLS.
- **TLS (Transport Layer Security)** : protocole de chiffrement qui protège les données en transit.
- **Activé par défaut** dans la console AWS et les SDK/API.

Ces trois mécanismes sont donc à considérer ensemble plutôt qu'isolément, chacun couvrant une phase différente du cycle de vie de la donnée.

3.3 À retenir sur le chiffrement

En résumé, le choix entre les deux mécanismes de chiffrement au repos dépend surtout de la sensibilité des données, tandis que le chiffrement en transit ne se discute pas :

- **SSE-S3** : simple, automatique, suffisant pour les données non sensibles.
- **SSE-KMS** : recommandé pour les données sensibles ou les environnements réglementés.
- **HTTPS/TLS** : toujours activé pour sécuriser les échanges réseau.

Info

SSE-KMS et coûts KMS — Chaque requête de chiffrement/déchiffrement via KMS est facturée (environ 0,03 \$ pour 10 000 requêtes). Pour un bucket avec de nombreuses petites opérations, cela peut s'accumuler. Activez le **Bucket Key** (option KMS) pour réduire le nombre d'appels KMS jusqu'à 99% en utilisant une clé de données par bucket plutôt que par objet.

3.4 Versioning : garder l'historique des fichiers

Le **versioning** permet de conserver toutes les versions d'un fichier, même si vous le modifiez ou le supprimez par erreur.

Exemple :

Concrètement, le versioning fonctionne ainsi :

- Vous téléversez `rapport.pdf`
- Vous le modifiez et téléversez une nouvelle version

- Vous pouvez toujours revenir à la version précédente

Chaque nouvelle version occupe un espace de stockage supplémentaire — S3 ne remplace jamais un fichier, il en ajoute une nouvelle version, ce qui a un impact direct sur la facturation à surveiller. Au-delà de ce point d'attention, le versioning répond à plusieurs besoins concrets :

- Éviter les pertes accidentelles
- Respecter des exigences réglementaires
- Tracer les modifications

 [S3 Versioning Guide](#)

3.5 Lifecycle Policies : automatiser le nettoyage et l'archivage

Les **politiques de cycle de vie** permettent de définir des règles pour :

- Supprimer automatiquement les fichiers après X jours
- Déplacer les fichiers vers une classe de stockage moins coûteuse
- Archiver dans Glacier pour la conformité

Cela vous aide à **réduire les coûts** sans perdre vos données.

Exemple concret : Un fichier log commence en **Standard** (accès rapide), est déplacé en **Standard-IA** après 30 jours (moins accédé, moins cher), puis archivé en **Glacier** après 90 jours (rarement consulté).

 [S3 Lifecycle Rules](#)

3.6 Classes de stockage : choisir le bon niveau selon l'usage

Amazon S3 propose plusieurs **classes de stockage**, selon la fréquence d'accès et le niveau de disponibilité souhaité.

Classe	Disponibilité	Coût	Temps d'accès	Cas d'usage
Standard	Multi-AZ	💰💰	Millisecondes	Fichiers actifs, souvent consultés
Standard-IA	Multi-AZ	💰	Millisecondes	Sauvegardes, fichiers rarement lus
One Zone-IA	Mono-AZ	💰	Millisecondes	Données non critiques
Intelligent-Tiering	Automatique	💰	Variable	Accès imprévisible
Glacier / Deep Archive	Multi-AZ	💰 très bas	Minutes à heures	Archivage long terme, conformité

Moins une donnée est accessible rapidement, moins elle coûte. C’est un **levier puissant pour optimiser votre budget**.

S3 Intelligent-Tiering : Optimisation automatique

S3 Intelligent-Tiering est une classe de stockage qui déplace automatiquement les objets entre quatre niveaux d'accès selon votre modèle de consultation réel.

Niveau	Délai avant bascule	Économie
Frequent Access	0-30 jours	coût = Standard
Infrequent Access	30-90 jours	~40%
Archive Instant	90-180 jours	~70%
Deep Archive	180+ jours	~95%

Avantages :

- Aucune configuration manuelle requise.
- Frais de gestion faibles (0,0025 \$ par objet par mois).
- Idéal pour les données dont la fréquence d'accès est **imprévisible**.

Cas d'usage :

- Logs d'applications avec accès sporadique.
- Archives de données sans schéma d'accès défini.
- Données de machine learning exploratoires.

Différence avec Lifecycle :

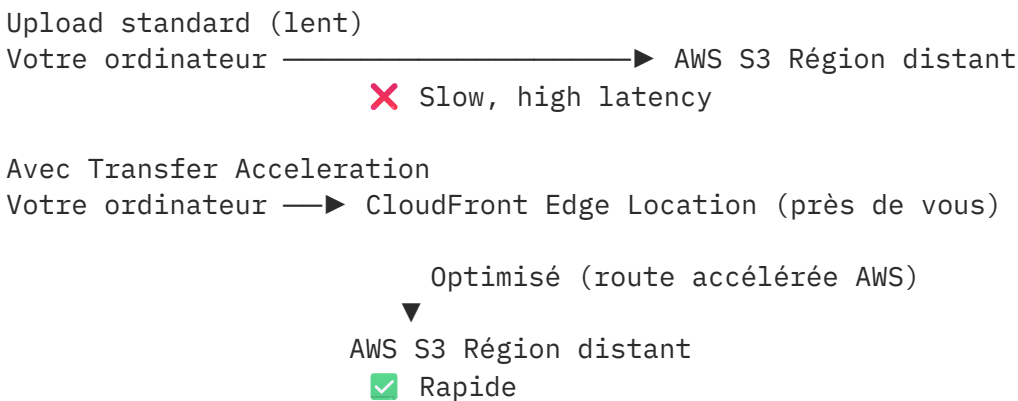
- Lifecycle : vous définissez les règles (ex. “après 90 jours, archiver en Glacier”).
- Intelligent-Tiering : AWS observe votre accès réel et adapte automatiquement.

Ces optimisations de classe de stockage concernent la donnée déjà présente dans S3 ; un autre levier d'optimisation, indépendant, concerne la vitesse à laquelle cette donnée y arrive.

S3 Transfer Acceleration : Optimisation des uploads volumineux

S3 Transfer Acceleration améliore les **vitesse d'upload** vers S3 en utilisant le réseau CloudFront d'AWS.

Fonctionnement :



Activation :

```
1 # Activer Transfer Acceleration
2 aws s3api put-bucket-accelerate-configuration \
3     --bucket mon-bucket \
4     --accelerate-configuration Status=Enabled
5
6 # Upload avec accélération
7 aws s3 cp mon-fichier-gros.zip \
8     s3://mon-bucket/uploads/ \
9     --region eu-west-1
```



Tip

Résultat attendu :

```
# put-bucket-accelerate-configuration : aucun output si succès

# s3 cp retourne la progression :
upload: ./mon-fichier-gros.zip to s3://mon-bucket/uploads/mon-fichier-gros.zip
```

Transfer Acceleration est activé sur le bucket. Les uploads utilisent désormais les Edge Locations CloudFront pour rejoindre le bucket S3, ce qui réduit la latence depuis les clients distants.

Coûts :

- Frais supplémentaires par Go transféré (environ \$0.04/Go).
- À justifier uniquement pour uploads volumineux ou latence critique.

Transfer Acceleration ne concerne que le sens montant (upload) ; le scénario bien plus fréquent en production consiste à distribuer efficacement du contenu déjà présent dans S3 vers un grand nombre de visiteurs.

S3 comme origine CloudFront — distribuer du contenu statique à grande échelle

Transfer Acceleration optimise l'**upload** vers S3. Le cas d'usage inverse — beaucoup plus fréquent en production — est de distribuer efficacement du contenu **depuis** S3 vers des millions de visiteurs : c'est le rôle de **CloudFront** utilisé comme CDN devant un bucket S3.

Sans CloudFront (chaque visiteur télécharge depuis S3 directement) :

Visiteur Tokyo → S3 eu-west-3 (Paris) ❌ Latence élevée, coût egress à chaque fois

Visiteur New York → S3 eu-west-3 (Paris) ❌ Latence élevée, coût egress à chaque fois

Visiteur Paris → S3 eu-west-3 (Paris) ✅ Rapide (mais N requêtes = N factures egress)

Avec CloudFront devant S3 :

Visiteur Tokyo → Edge Location Tokyo (cache HIT après 1er accès, < 50 ms)

Visiteur New York → Edge Location New York (cache HIT après 1er accès, < 50 ms)

Visiteur Paris → Edge Location Paris (cache HIT après 1er accès, < 20 ms)

↓
▼ (uniquement au premier accès, cache MISS)
S3 eu-west-3 (Paris) — origine unique

Pourquoi c'est la bonne pratique, pas juste une option :

- **Coût réduit** : le trafic sortant de S3 vers CloudFront est gratuit (les deux services AWS communiquent via le réseau interne). Seul le trafic CloudFront → visiteur final est facturé, à un tarif généralement inférieur à l'égress S3 direct.
- **Bucket privé possible** : avec une **Origin Access Control (OAC)**, le bucket S3 n'a besoin d'aucun accès public — seul CloudFront peut le lire. Les visiteurs ne touchent jamais directement S3.
- **Cache réduit la charge S3** : un fichier consulté 100 000 fois par jour ne génère qu'une poignée de requêtes S3 réelles (une par Edge Location, tant que le cache est valide), le reste est servi depuis le cache CloudFront.

Configuration minimale (CLI) :

```
1 # Créer la distribution CloudFront avec S3 comme origine et OAC
2 aws cloudfront create-distribution \
3   --origin-domain-name mon-bucket.s3.eu-west-3.amazonaws.com \
4   --default-root-object index.html
```



Résultat attendu (extrait) :

```
1 {
2   "Distribution": {
3     "Id": "E1A2B3C4D5E6F7",
4     "DomainName": "d1111111abcdef8.cloudfront.net",
5     "Status": "InProgress"
6   }
7 }
```

La distribution devient **Deployed** après quelques minutes de propagation sur le réseau mondial CloudFront. Le bucket S3 reste privé — seule cette distribution CloudFront (via son OAC) est autorisée à le lire, configuré automatiquement dans la bucket policy.

 [Amazon CloudFront — Restricting access to S3](#)

Warning

Coûts Transfer Acceleration : Cette fonctionnalité engendre des frais supplémentaires (~0,04 \$/Go pour les transferts vers des Edge Locations). N'activez Transfer Acceleration que si vous uploadez régulièrement des fichiers volumineux (> 100 Mo) depuis des clients géographiquement éloignés de la région S3. Pour les petits fichiers ou les accès locaux, le gain est négligeable et le surcoût inutile.

Cas d'usage :

- Uploads de fichiers vidéo ou binaires depuis un client distant.
- Synchronisations multi-sites hautes performances.
- Distributions de fichiers volumineux vers plusieurs régions AWS.

Chiffrement, classes de stockage et distribution du contenu reposent tous sur un même socle : la capacité à définir précisément qui peut faire quoi sur un bucket, ce que permettent les bucket policies.

3.7 Stratégies de compartiment (Bucket Policies) : Contrôle d'accès granulaire

Les **bucket policies** sont des documents JSON qui définissent **qui** peut accéder à **quoi** dans un bucket S3.

Structure d'une bucket policy

Une bucket policy est un document JSON attaché directement au bucket (et non à un utilisateur IAM). Elle contrôle qui peut accéder à quoi, y compris des accès publics ou inter-comptes AWS.


```

1  {
2    "Version": "2012-10-17",
3    "Statement": [
4      {
5        "Sid": "AllowPublicRead",
6        "Effect": "Allow",
7        "Principal": "*",
8        "Action": "s3:GetObject",
9        "Resource": "arn:aws:s3:::mon-bucket/*"
1     },
1     {
2       "Sid": "DenyEncryptedUploads",
3       "Effect": "Deny",
4       "Principal": "*",
5       "Action": "s3:PutObject",
6       "Resource": "arn:aws:s3:::mon-bucket/*",
7       "Condition": {
8         "StringNotEquals": {
9           "s3:x-amz-server-side-encryption": "AES256"
10        }
11      }
12    ]
13  }

```

```
2
4 }
```

Éléments clés :

- **Sid** : identifiant lisible de la règle (ex. "AllowPublicRead")
- **Effect** : `Allow` ou `Deny`
- **Principal** : qui a l'accès (`*` = tout le monde, ou un ARN spécifique)
- **Action** : quelle opération S3 (`s3:GetObject` , `s3:PutObject` , `s3:DeleteObject` , etc.)
- **Resource** : sur quel objet (ARN format)
- **Condition** : contextes additionnels (IP, SSL, chiffrement, etc.)

Les trois cas d'usage suivants montrent comment combiner ces éléments pour répondre à des besoins concrets, du plus permissif au plus restrictif.

Cas d'usage 1 : Site web statique public

Pour héberger un site web HTML/CSS sur S3, le bucket doit autoriser la lecture publique. Voici la policy à appliquer — notez `"Principal": "*"` qui signifie "tout le monde sans authentification" :

```
1 {
2   "Version": "2012-10-17",
3   "Statement": [
4     {
5       "Sid": "PublicReadGetObject",
6       "Effect": "Allow",
7       "Principal": "*",
8       "Action": "s3:GetObject",
9       "Resource": "arn:aws:s3:::mon-site-web/*"
10    }
11  ]
12 }
```

Effet : Tous les utilisateurs peuvent **lire** les fichiers du bucket (parfait pour un site statique).

Danger

Bucket public : risque de fuite de données — L'utilisation de `"Principal": "*"` rend l'ensemble des objets du bucket accessibles sur Internet **sans authentification**. Ne l'appliquez **jamais** à un bucket contenant des données sensibles (fichiers clients, logs internes, clés, backups). Depuis 2023, AWS bloque par défaut les accès publics sur les nouveaux buckets — cette policy nécessite de désactiver explicitement ce blocage.

Cas d'usage 2 : Restreindre à une adresse IP spécifique

Pour un bucket contenant des données sensibles, on limite l'accès au réseau de l'entreprise via la condition `aws:SourceIp`. Tout accès depuis une IP extérieure sera refusé, même avec des credentials IAM valides.

```
1  {
2    "Version": "2012-10-17",
3    "Statement": [
4      {
5        "Sid": "RestrictToCompanyIP",
6        "Effect": "Allow",
7        "Principal": "*",
8        "Action": ["s3:GetObject", "s3:PutObject"],
9        "Resource": "arn:aws:s3:::mon-bucket-prive/*",
10
11       "Condition": {
12
13         "IpAddress": {
14
15           "aws:SourceIp": "203.0.113.0/24"
16
17         }
18       }
19     ]
20   }
```

Effet : Seules les IPs du réseau 203.0.113.0/24 peuvent accéder au bucket.

Cas d'usage 3 : Forcer le chiffrement pour tous les uploads

Cette policy est une protection anti-erreur : elle refuse tout upload qui ne spécifie pas le chiffrement côté serveur (SSE-S3). Même si un développeur oublie de configurer le chiffrement dans son code, AWS rejette la requête.

```
1  {
2    "Version": "2012-10-17",
3    "Statement": [
4      {
5        "Sid": "DenyUnencryptedObjectUploads",
6        "Effect": "Deny",
7        "Principal": "*",
8        "Action": "s3:PutObject",
9        "Resource": "arn:aws:s3:::mon-bucket-sensible/*",
10
11        "Condition": {
12
13          "StringNotEquals": {
14
15            "s3:x-amz-server-side-encryption": "AES256"
16
17          }
18
19        }
20      }
21    ]
22  }
```

Effet : Toute tentative d'upload sans chiffrement SSE-S3 sera **rejetée**.

Appliquer une bucket policy via CLI

Une fois le fichier JSON rédigé, voici comment l'appliquer, le vérifier, et le supprimer si nécessaire :

```

1  # Créer un fichier policy.json (voir ci-dessus)
2  # Appliquer la policy
3  aws s3api put-bucket-policy \
4      --bucket mon-bucket \
5      --policy file://policy.json
6
7  # Vérifier la policy
8  aws s3api get-bucket-policy --bucket mon-bucket
9
10 # Supprimer la policy
11
12 aws s3api delete-bucket-policy --bucket mon-bucket

```



Tip

Résultat attendu :

```

# put-bucket-policy : aucun output si succès

# get-bucket-policy retourne :
{
  "Policy": "{\n\"Version\": \"2012-10-17\", \"Statement\":\n[\n{\n\"Sid\": \"AllowPublicRead\", \"Effect\": \"Allow\", \"Principal\": \"*\", \"Action\": \"s3:GetObject\", \"Resource\": \"arn:aws:s3:::mon-bucket/*\"}\n]}\n}"
}

# delete-bucket-policy : aucun output si succès

```

Point important : Les bucket policies s'ajoutent aux **ACL (Access Control Lists)**, il faut les deux pour une sécurité complète.

3.8 Bonnes pratiques S3

Pour conclure cette section, voici une synthèse des réflexes à adopter systématiquement lors de la création d'un bucket en production, en reprenant chaque mécanisme vu plus haut :

- Activez le **versioning** dès que vous stockez des fichiers importants.
- Utilisez **SSE-S3** pour un chiffrement simple et automatique.
- Créez des **règles de cycle de vie** pour archiver ou supprimer les fichiers inutilisés.

- Choisissez la **classe de stockage** adaptée à chaque type de données.
- Appliquez des **bucket policies** pour restreindre l'accès selon le principe du moindre privilège.
- Utilisez **Intelligent-Tiering** si l'accès est imprévisible.
- Activez **Transfer Acceleration** pour les uploads volumineux critiques.

Passons maintenant à la pratique : la section suivante applique concrètement ces bonnes pratiques via l'AWS CLI.

4. Gestion de S3 en CLI

Info

Une activité pratique permet d'approfondir la manipulation de S3 en CLI.

4.1 Créer un bucket S3

Le nom d'un bucket S3 doit être **unique à l'échelle mondiale** (personne d'autre dans le monde ne peut avoir le même nom). La syntaxe utilise `$(date +%s)` pour ajouter un timestamp et éviter les conflits.

```
1 # Créer un bucket (le nom doit être unique à l'échelle mondiale AWS)
2 aws s3api create-bucket \
3     --bucket mon-bucket-formation-$(date +%s) \
4     --region eu-west-1
5
6 # Vérifier la création
7 aws s3api list-buckets \
8     --query 'Buckets[].Name' \
9     --output table
```

Explications :

- `create-bucket` : commande pour créer un nouveau bucket
- `--bucket` : nom du bucket (doit respecter les règles : minuscules, pas d'espaces)
- `--region` : région AWS cible
- `--query` : filtre la réponse pour n'afficher que les noms
- `--output table` : affiche le résultat sous forme de tableau



Tip

Résultat attendu :

```
# create-bucket retourne :
{
  "Location": "/mon-bucket-formation-1715936400"
}

# list-buckets --output table affiche :
-----
|               ListBuckets               |
+-----+
| mon-bucket-formation-1715936400         |
| autre-bucket-existant                   |
+-----+
```

4.2 Télécharger un fichier dans S3

On crée d'abord un fichier local de test, puis on le pousse dans S3. La commande `s3 cp` fonctionne dans les deux sens : local → S3 ou S3 → local.

```
1 # Créer un fichier d'exemple
2 echo "Ceci est un contenu test pour S3" > test.txt
3
4 # Télécharger le fichier
5 aws s3 cp test.txt s3://mon-bucket-formation/documents/test.txt
6
7 # Vérifier le téléchargement
8 aws s3 ls s3://mon-bucket-formation/documents/ --recursive
```

Explications :

- `s3 cp` : copie un fichier local vers S3
- `s3://` : préfixe pour les chemins S3
- `s3 ls` : liste les contenus d'un bucket S3
- `--recursive` : affiche les fichiers dans les sous-dossiers aussi



Tip

Résultat attendu :

```
upload: ./test.txt to s3://mon-bucket-formation/documents/test.txt

# s3 ls retourne :
2024-05-17 14:23:05          34 test.txt
```



Warning

Coûts S3 : requêtes et transfert — Chaque opération `s3 cp` ou `s3 ls` génère des requêtes facturées (PUT, GET, LIST). Le tarif standard est ~0,005 pour 1 000 requêtes PUT et ~0,004 pour 1 000 requêtes GET. Les transferts de données **sortants d’AWS vers Internet** sont facturés (~0,09 \$/Go). Préférez regrouper les petits fichiers et minimisez les téléchargements hors AWS pour maîtriser les coûts.

4.3 Activer le versioning sur un bucket

Le versioning conserve toutes les versions d’un fichier. Si on écrase `test.txt`, l’ancienne version reste accessible. Une fois activé, le versioning **ne peut pas être désactivé** (seulement suspendu).

```
1 # Activer le versioning
2 aws s3api put-bucket-versioning \
3     --bucket mon-bucket-formation \
4     --versioning-configuration Status=Enabled
5
6 # Vérifier le versioning
7 aws s3api get-bucket-versioning \
8     --bucket mon-bucket-formation
```




Tip

Résultat attendu :

```
# put-bucket-versioning : aucun output si succès

# get-bucket-versioning retourne :
{
  "Status": "Enabled"
}
```



Warning

Versioning et coûts de stockage — Une fois activé, le versioning conserve **toutes les versions** de chaque objet — y compris celles écrasées ou supprimées. Si vous modifiez fréquemment des fichiers volumineux, le stockage total peut rapidement multiplier les coûts. Combinez toujours le versioning avec une **lifecycle policy** qui expire les anciennes versions (ex. : supprimer les versions antérieures à 90 jours).

4.4 Créer une politique de cycle de vie

Automatiser le déplacement des données vers des classes de stockage moins chères est une bonne pratique FinOps. Ici, on définit deux règles : les logs passent en STANDARD_IA à 30 jours, puis en GLACIER à 90 jours ; les fichiers temporaires sont supprimés après 1 an.

```

1 # Créer un fichier JSON pour la politique
2 cat > lifecycle-policy.json << 'EOF'
3 {
4     "Rules": [
5         {
6             "Id": "ArchiveAfter30Days",
7             "Status": "Enabled",
8             "Prefix": "logs/",
9             "Transitions": [
10                {
11                    "Days": 30,
12                    "StorageClass": "STANDARD_IA"
13                },
14                {
15                    "Days": 90,
16                    "StorageClass": "GLACIER"
17                }
18            ]
19        },
20        {
21            "Id": "DeleteAfter365Days",
22            "Status": "Enabled",
23            "Prefix": "temp/",

```

```

2
4     "Expiration": {
2
5         "Days": 365
2
6     }
2
7 }
2
8 ]
2
9 }
3
0 EOF
3
1
3
2 # Appliquer la politique
3
3 aws s3api put-bucket-lifecycle-configuration \
3
4     --bucket mon-bucket-formation \
3
5     --lifecycle-configuration file://lifecycle-policy.json

```

Explications :

- `put-bucket-lifecycle-configuration` : applique une politique de cycle de vie
- `Transitions` : changement automatique de classe de stockage selon l'âge
- `Expiration` : suppression automatique après X jours
- `Prefix` : la règle s'applique uniquement aux fichiers commençant par ce préfixe



Tip

Résultat attendu :

```
# put-bucket-lifecycle-configuration : aucun output si succès

# get-bucket-lifecycle-configuration retourne :
{
  "Rules": [
    {
      "ID": "ArchiveAfter30Days",
      "Status": "Enabled",
      "Prefix": "logs/",
      "Transitions": [
        { "Days": 30, "StorageClass": "STANDARD_IA" },
        { "Days": 90, "StorageClass": "GLACIER" }
      ]
    },
    {
      "ID": "DeleteAfter365Days",
      "Status": "Enabled",
      "Prefix": "temp/",
      "Expiration": { "Days": 365 }
    }
  ]
}
```

4.5 Activer le chiffrement par défaut

Le chiffrement par défaut garantit que **tout objet** ajouté au bucket est automatiquement chiffré, sans que le développeur ait à y penser. C'est une exigence fréquente en entreprise pour la conformité.

```

1  # Créer une configuration de chiffrement
2  cat > encryption-config.json << 'EOF'
3  {
4      "Rules": [
5          {
6              "ApplyServerSideEncryptionByDefault": {
7                  "SSEAlgorithm": "AES256"
8              }
9          }
10     ]
11 }
12 EOF
13
14 # Appliquer le chiffrement
15 aws s3api put-bucket-encryption \
16     --bucket mon-bucket-formation \
17     --server-side-encryption-configuration file://encryption-config.json

```

Explications :

- `put-bucket-encryption` : active le chiffrement automatique
- `AES256` : chiffrement SSE-S3 (clés gérées par AWS)
- Toute nouvelle donnée sera automatiquement chiffrée



Tip

Résultat attendu :

```
# put-bucket-encryption : aucun output si succès

# get-bucket-encryption retourne :
{
  "ServerSideEncryptionConfiguration": {
    "Rules": [
      {
        "ApplyServerSideEncryptionByDefault": {
          "SSEAlgorithm": "AES256"
        },
        "BucketKeyEnabled": false
      }
    ]
  }
}
```

4.6 Bloquer l'accès public au bucket

Par défaut depuis 2023, AWS bloque l'accès public sur les nouveaux buckets. Cette commande permet de le confirmer explicitement — et de le vérifier sur des buckets existants qui pourraient être mal configurés.

```
1 # Bloquer tout accès public (meilleure pratique pour les données sensibles)
2 aws s3api put-public-access-block \
3     --bucket mon-bucket-formation \
4     --public-access-block-configuration \
5     "BlockPublicAcls=true,IgnorePublicAcls=true,BlockPublicPolicy=true,RestrictPublicB
6     ckets=true"
7 # Vérifier la configuration
8 aws s3api get-public-access-block \
9     --bucket mon-bucket-formation
```

Explications :

- `put-public-access-block` : active les protections contre l'accès accidentellement public
- `BlockPublicAcls=true` : empêche d'ajouter des ACL publiques
- `IgnorePublicAcls=true` : ignore les ACL publiques existantes

- `BlockPublicPolicy=true` : empêche les politiques publiques
- `RestrictPublicBuckets=true` : restreint tous les accès publics



Résultat attendu :

```
# put-public-access-block : aucun output si succès

# get-public-access-block retourne :
{
  "PublicAccessBlockConfiguration": {
    "BlockPublicAcls": true,
    "IgnorePublicAcls": true,
    "BlockPublicPolicy": true,
    "RestrictPublicBuckets": true
  }
}
```



Bucket accessible publiquement — Si l'un des quatre paramètres est à `false`, le bucket peut être exposé publiquement. Des milliers de fuites de données AWS ont été causées par des buckets S3 mal configurés. Vérifiez systématiquement cette configuration sur vos buckets de production, en particulier ceux créés avant 2023 ou migrés depuis un autre compte AWS.

4.7 Télécharger un fichier depuis S3

La commande `s3 sync` est particulièrement utile pour les sauvegardes : elle compare le contenu local et S3, et ne transfère que les fichiers qui ont changé (plus rapide et économique qu'un `cp` complet).

```

1  # Télécharger un fichier spécifique
2  aws s3 cp s3://mon-bucket-formation/documents/test.txt ./test-local.txt
3
4  # Télécharger un dossier entier en récursif
5  aws s3 cp s3://mon-bucket-formation/documents/ ./documents/ --recursive
6
7  # Synchroniser un bucket avec un dossier local (bidirectionnel)
8  aws s3 sync s3://mon-bucket-formation/ ./backup-local/
9
10 # Afficher la taille d'un objet
11
12 aws s3api head-object \
13     --bucket mon-bucket-formation \
14     --key documents/test.txt \
15     --query 'ContentLength' \
16     --output text

```

Explications :

- `s3 cp` : copie des fichiers depuis S3 vers local
- `--recursive` : copie tous les fichiers d'un "dossier"
- `s3 sync` : synchronise des répertoires (idéal pour sauvegarde)
- `head-object` : récupère les métadonnées sans télécharger le fichier



Tip

Résultat attendu :

```
# s3 cp retourne :  
download: s3://mon-bucket-formation/documents/test.txt to ./test-local.txt  
  
# s3 sync retourne :  
download: s3://mon-bucket-formation/images/logo.png to ./backup-  
local/images/logo.png  
download: s3://mon-bucket-formation/css/style.css to ./backup-  
local/css/style.css  
download: s3://mon-bucket-formation/documents/test.txt to ./backup-  
local/documents/test.txt  
  
# head-object retourne (taille en octets) :  
34
```



Warning

Coûts de transfert sortant (egress) — Le téléchargement de données depuis S3 vers Internet est facturé (~0,09 \$/Go pour les premiers 10 To). Les transferts entre services AWS dans la même région sont gratuits. Si vous synchronisez régulièrement des données volumineuses vers des machines hors AWS (postes développeurs, serveurs on-premise), anticipez ce coût dans votre budget.

5. Amazon EC2 : La couche de calcul AWS

5.1 Introduction à EC2

 Vidéo : [Lancer sa première machine virtuelle Windows avec EC2](#)

Après avoir stocké nos données avec Amazon S3, nous allons voir comment **les traiter, les héberger ou les exécuter** grâce à **Amazon Elastic Compute Cloud (EC2)**.

EC2 est l'un des premiers services historiques d'AWS (2006). Il permet de **louer de la puissance de calcul à la demande**, avec une flexibilité inégalée par rapport aux serveurs physiques traditionnels.

 [Documentation officielle Amazon EC2](#)

Amazon EC2 (Elastic Compute Cloud) est le service AWS qui permet de créer des **machines virtuelles** dans le cloud, appelées **instances EC2**.

5.2 Pourquoi utiliser EC2 ?

EC2 reprend le principe familier d'un serveur physique — un système d'exploitation, du CPU, de la RAM, du stockage, une carte réseau — mais en supprime toutes les contraintes matérielles, ce qui explique son adoption massive comme brique de calcul de base sur AWS.

Le **lancement est rapide** : là où commander, recevoir et configurer un serveur physique prenait des semaines, une instance EC2 est prête à l'emploi en quelques clics ou quelques lignes de CLI, avec un système d'exploitation déjà installé. Le service est aussi **flexible** : vous choisissez la puissance de calcul, le système d'exploitation, le type de stockage attaché et la configuration réseau, et vous pouvez faire évoluer ces choix a posteriori si les besoins changent — un projet peut commencer sur une petite instance et migrer vers une plus puissante sans réinstallation. Le modèle est **économique** parce que la facturation suit la consommation réelle plutôt qu'un investissement matériel figé : vous payez à l'heure ou à la seconde pour ce qui tourne, et vous pouvez arrêter une instance dès qu'elle n'est plus utile pour cesser d'être facturé. Enfin, EC2 est nativement **connecté** au reste de l'écosystème AWS : une instance peut lire et écrire dans un bucket S3, s'authentifier via un rôle IAM sans stocker de clé d'accès, et vivre dans un VPC dont vous contrôlez entièrement le découpage réseau — cette intégration native évite d'avoir à recoller manuellement des briques hétérogènes comme sur une infrastructure on-premise.

5.3 Les composants essentiels d'une instance EC2

Avant de lancer une instance, voici les briques que vous devrez assembler — chacune sera détaillée dans les sections suivantes :

Composant	Rôle dans l'architecture EC2
Instance EC2	Machine virtuelle hébergée chez AWS
AMI	Image système (Linux, Windows, etc.) utilisée comme modèle
Type d'instance	Détermine la puissance (CPU, RAM, réseau)
Security Group	Pare-feu virtuel qui contrôle les accès réseau
EBS	Disque dur virtuel attaché à l'instance
EFS	Système de fichiers partagé entre plusieurs instances
Key Pair	Clé SSH utilisée pour se connecter à l'instance en toute sécurité

Lors du lancement d'une instance, vous devez choisir chacun de ces éléments.

5.4 Les services de conteneurs AWS — ECS, ECR et Fargate

Une instance EC2 reste une machine virtuelle complète. Elle démarre un système d'exploitation entier, avec son propre noyau, avant même de pouvoir exécuter la moindre application. Pour héberger dix applications indépendantes, il faudrait donc dix machines virtuelles — ou une seule, mal isolée, où toutes les applications se partagent le même système.

Le **conteneur** répond à ce problème autrement. Au lieu de virtualiser un système d'exploitation entier, il isole uniquement le processus applicatif et ses dépendances — bibliothèques, variables d'environnement, fichiers — dans un espace cloisonné qui partage le noyau Linux de la machine hôte. Résultat : dix conteneurs peuvent tourner sur une seule instance EC2. Ils démarrent en quelques secondes, pas en quelques minutes. Et ils s'exécutent identiquement sur un poste de développement, en intégration continue, ou en production, une fois empaquetés. C'est cette portabilité qui a fait le succès de Docker, le format de conteneur devenu standard de facto.

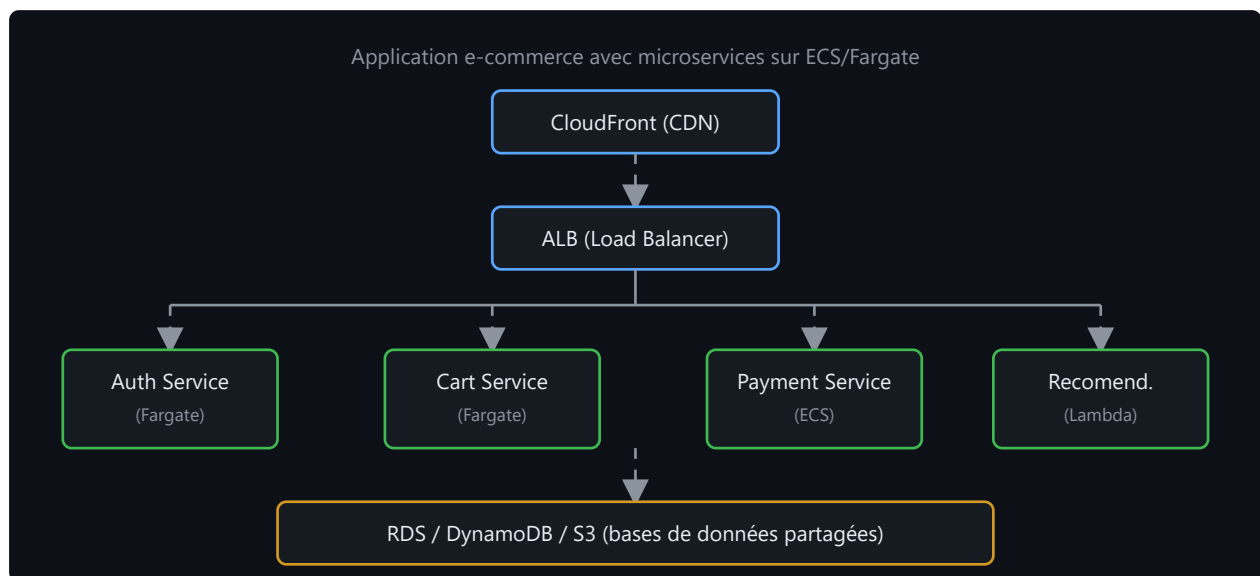
AWS propose trois services complémentaires pour faire tourner des conteneurs en production. Ils ne sont pas concurrents entre eux — chacun a un rôle distinct :

- **Amazon ECR (Elastic Container Registry)** — un entrepôt d'images. De la même manière que S3 stocke des fichiers, ECR stocke des **images de conteneurs** : le paquet figé qui contient l'application et tout ce qu'il lui faut pour s'exécuter. C'est l'équivalent AWS de Docker Hub, privé et intégré à IAM. On y pousse ("push") une image depuis son poste ou sa chaîne CI/CD ; les services d'exécution la récupèrent ("pull") au moment de démarrer un conteneur.

- **Amazon ECS (Elastic Container Service)** — l’orchestrateur. Il décide où et comment exécuter les conteneurs à partir des images stockées dans ECR : démarrer le nombre demandé, redémarrer automatiquement en cas de panne, répartir la charge, arrêter proprement lors d’une mise à jour. C’est une alternative plus simple d’usage que Kubernetes (dont l’équivalent AWS géré est **EKS**, hors périmètre de cette formation), pensée nativement pour l’écosystème AWS.
- **AWS Fargate** — un mode d’exécution pour ECS (et EKS) qui retire la dernière brique manuelle. Plus d’instance EC2 à provisionner ni à administrer pour héberger les conteneurs : AWS alloue la capacité de calcul à la demande, facturée au conteneur réellement exécuté plutôt qu’à l’instance entière. C’est le pendant serverless d’ECS, comme Lambda l’est pour du code sans conteneur.

Info

À retenir : ECR stocke l’image, ECS orchestre son exécution, Fargate retire la gestion des serveurs sous-jacents. On peut très bien utiliser ECS **sans** Fargate (type de lancement “EC2”, où l’on gère soi-même les instances qui hébergent les conteneurs) — Fargate est une option, pas une obligation.



Le schéma ci-dessus illustre le flux complet : une image applicative est construite puis poussée vers un dépôt ECR, un **cluster ECS** définit la capacité disponible (EC2 ou Fargate), une **définition de tâche** (task definition) décrit quelle image lancer avec quelles ressources CPU/mémoire, et un **service ECS** maintient en permanence le nombre de tâches demandé — en relançant automatiquement toute tâche qui s’arrête de manière inattendue. C’est ce mécanisme de service qui rapproche ECS d’Auto Scaling pour des instances EC2 (vu en section 11) : dans les deux cas, AWS surveille l’état réel et le compare à l’état désiré pour corriger tout écart sans intervention humaine.

Passons à la pratique en créant un dépôt ECR, en y poussant une image, puis en la déployant sur un cluster ECS en mode Fargate.

```
1 # Créer un dépôt ECR pour stocker l'image de l'application
2 aws ecr create-repository \
3     --repository-name demo-app \
4     --image-scanning-configuration scanOnPush=true
5 # --image-scanning-configuration scanOnPush=true : ECR analyse automatiquement
6 # chaque image poussée à la recherche de vulnérabilités connues (CVE)
```



Tip

Résultat attendu :

```
1 {
2     "repository": {
3         "repositoryArn": "arn:aws:ecr:eu-west-
3:123456789012:repository/demo-app",
4         "repositoryUri": "123456789012.dkr.ecr.eu-west-
3.amazonaws.com/demo-app",
5         "imageScanningConfiguration": { "scanOnPush": true }
6     }
7 }
```

Le `repositoryUri` retourné est l'adresse à utiliser pour pousser l'image — il faut d'abord authentifier Docker auprès d'ECR, puis étiqueter l'image locale avec cette adresse avant de la transférer.

```
1 # S'authentifier auprès d'ECR : récupère un jeton temporaire et le transmet à
Docker
2 aws ecr get-login-password --region eu-west-3 | \
3     docker login --username AWS --password-stdin 123456789012.dkr.ecr.eu-west-
3.amazonaws.com
4
5 # Étiqueter l'image locale avec l'adresse complète du dépôt ECR
6 docker tag demo-app:latest 123456789012.dkr.ecr.eu-west-3.amazonaws.com/demo-
app:latest
7
8 # Pousser l'image vers ECR
9 docker push 123456789012.dkr.ecr.eu-west-3.amazonaws.com/demo-app:latest
```

 Warning

Erreur fréquente : le jeton d'authentification `get-login-password` expire au bout de 12 heures — un `docker push` qui échoue avec une erreur 401 après une pause déjeuner ne signifie pas que les droits IAM sont mauvais, mais qu'il faut simplement relancer la commande de login.

Une fois l'image disponible dans ECR, il reste à créer le cluster ECS (le regroupement logique de capacité) puis à y déployer un service Fargate qui référence cette image :

```

1  # Créer un cluster ECS – avec Fargate, ce cluster ne contient aucune instance
EC2 à gérer
2  aws ecs create-cluster --cluster-name demo-cluster
3
4  # Enregistrer une définition de tâche (task definition) qui référence l'image
ECR
5  # Le fichier task-definition.json décrit : l'image, le CPU/mémoire alloués,
6  # et le mode réseau (awsvpc, obligatoire avec Fargate)
7  aws ecs register-task-definition --cli-input-json file://task-definition.json
8
9  # Créer le service qui maintient en permanence 2 tâches actives sur ce cluster
1
0  aws ecs create-service \
1
1  --cluster demo-cluster \
1
2  --service-name demo-service \
1
3  --task-definition demo-app \
1
4  --desired-count 2 \
1
5  --launch-type FARGATE \
1
6  --network-configuration "awsvpcConfiguration={subnets=[subnet-
0123456789abcdef0],securityGroups=[sg-0123456789abcdef0],assignPublicIp=ENABLED}"
1
7  # --launch-type FARGATE : AWS fournit la capacité de calcul, aucune instance
EC2 à créer
1
8  # --desired-count 2 : ECS relance automatiquement toute tâche qui s'arrête pour
revenir à 2

```



Tip

Résultat attendu (extrait) :

```
1  {
2      "service": {
3          "serviceName": "demo-service",
4          "status": "ACTIVE",
5          "desiredCount": 2,
6          "runningCount": 0,
7          "launchType": "FARGATE"
8      }
9  }
```

Le `runningCount` remonte progressivement à 2 en quelques dizaines de secondes, le temps que Fargate provisionne la capacité et démarre les conteneurs.

Cette bascule vers les conteneurs pose une question légitime : pourquoi ne pas tout faire en Lambda (vu en section 12), qui est déjà serverless ? La réponse tient à la nature de la charge : Lambda est pensé pour des exécutions courtes et déclenchées par un événement (une requête HTTP, un fichier déposé sur S3), avec une durée d'exécution plafonnée à 15 minutes et un environnement recréé à chaque invocation. ECS/Fargate convient mieux à une application qui tourne en continu (un serveur web, une API qui doit rester disponible en permanence, un traitement de longue durée) ou qui a besoin d'un contrôle plus fin sur son environnement d'exécution (dépendances système précises, taille mémoire supérieure aux limites Lambda). Les deux approches sont complémentaires plutôt qu'exclusives dans une architecture réelle.

[Amazon ECS — Documentation officielle](#) [Amazon ECR — Documentation officielle](#) [Conteneurs sur AWS — chaîne AWS France](#) — rechercher “ECS Fargate” pour les démonstrations les plus récentes

6. Choisir le bon type d'instance, AMI, stockage et sécurité

6.1 Types d'instances EC2

AWS propose plusieurs familles d'instances, selon le type de charge à traiter :

Famille	Usage recommandé	Exemple d'application
t / t4g	Usage général, burst occasionnel	Serveur web, environnement de test
m	Charges équilibrées CPU/Mémoire	Application métier, ERP
c	Calcul intensif	Simulation scientifique, traitement d'image
r	Mémoire importante	Base de données en mémoire, Redis
g / p	GPU (accélération graphique/IA)	Intelligence artificielle, machine learning
d / h	Stockage rapide SSD	Big Data, traitement de logs volumineux

Pour un TP ou un test, on utilise souvent `t3.micro` (gratuit dans le Free Tier AWS).

Warning

Types d'instances coûteux — Les familles `g`, `p` (GPU) et `x` (mémoire très haute) peuvent coûter plusieurs dizaines de dollars **par heure**. Par exemple, une instance `p3.16xlarge` (GPU ML) dépasse 24 \$/h. Ne lancez ces types que si votre workload le justifie, et pensez à les **arrêter immédiatement** après utilisation. En formation ou développement, restez sur des types `t3.micro` ou `t3.small`.

Explication des suffixes de type :

- `t3` : type t (général), génération 3
- `micro`, `small`, `medium` : taille croissante
- `xlarge` ou `2xlarge` : très puissants, pour les charges importantes
- le `g` que l'on trouve dans `t4g`, `m6g`, `c6g`, etc. signale que l'instance tourne sur un processeur **Graviton**, la puce ARM conçue par AWS elle-même (au lieu d'un processeur x86 classique Intel/AMD). Les instances Graviton offrent généralement un meilleur rapport performance/prix (jusqu'à 20-40 % moins cher à performance équivalente), mais nécessitent que votre application soit compilée pour l'architecture ARM — la plupart des

langages interprétés (Python, Node.js, Java) et des images Docker officielles la supportent nativement, mais un vieux binaire compilé spécifiquement pour x86 ne fonctionnera pas dessus sans recompilation.

Le type d'instance ne détermine que la puissance matérielle disponible ; le logiciel qui tourne dessus au démarrage dépend d'un second choix indépendant, l'AMI.

6.2 AMI (Amazon Machine Image)

L'AMI est le **système d'exploitation** de votre machine EC2.

 Vidéo : [AMI — Amazon Machine Images](#)

Types d'AMI

On distingue trois origines possibles pour une AMI, selon qui l'a créée et publiée :

- **Public AMI** : proposées par AWS (Linux, Windows, Ubuntu, etc.)
- **Custom AMI** : créées par vous (ex. avec des logiciels préinstallés)
- **Marketplace AMI** : proposées par des éditeurs tiers (ex. WordPress, SAP)

L'AMI détermine ce que contient votre machine au démarrage.

Classification des AMI

Les AMI peuvent être classifiées dans les grandes catégories suivantes :

AMI persistantes (basées sur EBS)

- L'ensemble du filesystem est stocké sur EBS (Elastic Block Store).
- EBS fonctionne de manière similaire à un **NAS (Network Attached Storage)** et permet le partage des données sur le réseau.
- Ces volumes ne sont associés à aucun type de matériel spécifique, ce qui les rend très pratiques pour transférer des données entre zones ou d'une région à une autre.
- Les AMI persistantes sont configurées avec un ou plusieurs volumes EBS.

AMI volatiles (basées sur S3)

- Contrairement aux AMI persistantes, les AMI volatiles stockent leurs données via le service AWS S3 (Simple Storage Service).
- Ces AMI **ne peuvent pas être transférées** depuis une zone de disponibilité ou région vers une autre.

	Persistante	Volatile
Filesystem root	Sur volume EBS	Sur la machine hôte elle-même
Taille espace disque	Jusqu'à 16 TB	Jusqu'à 10 GB
Persistance	Données persistantes après l'arrêt de l'instance	Données persistantes seulement pendant la vie de l'instance
Temps de démarrage	Sous 1 minute	Jusqu'à 5 minutes
Coûts	Le coût de l'instance plus le coût des volumes EBS sont facturés	Seulement le coût de l'instance est facturé

Comparaison des AMI de type persistant et volatile

Il est souvent utile en entreprise de créer ses propres AMI afin de pouvoir déployer plus rapidement des instances EC2 correspondant aux besoins spécifiques. Vous pouvez enregistrer le disque contenant cette AMI après lancement de la machine EC2 et après avoir ajouté les spécificités de l'ensemble de vos machines.

6.3 Stockage associé à EC2

EBS — Elastic Block Store

EBS est le disque virtuel « par défaut » d'EC2, attaché à une seule instance à la fois :

- **Disque attaché** à une instance EC2.
- **Persiste** même si l'instance est arrêtée.
- Permet les **snapshots** (sauvegardes incrémentales).
- Idéal pour le stockage de données d'application.
- Peut être attaché/détaché dynamiquement.

Cas d'usage : serveur web avec base de données, ERP, systèmes de fichiers importants.

EFS — Elastic File System

Là où un volume EBS ne peut être monté que sur une seule instance à la fois, EFS lève cette limite en proposant un système de fichiers partagé :

- **Système de fichiers partagé** entre plusieurs instances.
- Montable sur **plusieurs instances EC2 simultanément**.
- Idéal pour les architectures distribuées.

- Escalabilité automatique sans gestion de capacité.
- Compatible avec NFS (Network File System).

Ce partage simultané entre plusieurs instances est précisément ce qu'EBS ne permet pas — c'est le critère de choix déterminant entre les deux services.

Cas d'usage : cluster d'applications, déploiement multi-serveurs, stockage partagé.

Avantages :

- Haute disponibilité multi-AZ.
- Performance prédictible et constante.
- Paiement à l'usage (pas de provisionnement anticipé).

EFS couvre bien les usages Linux/NFS génériques, mais certains environnements ont des besoins plus spécifiques (compatibilité Windows, calcul haute performance) auxquels répond un troisième service.

FSx — Managed File Systems

Amazon FSx propose des systèmes de fichiers managés, avec deux options principales :

FSx for Windows File Server :

- Compatible **Active Directory** et **SMB** (partages Windows).
- Idéal pour les environnements Windows d'entreprise.
- Partages de fichiers compatibles avec les domaines Windows.

SMB (Server Message Block) est le protocole de partage de fichiers natif de Windows — c'est exactement ce que vous utilisez quand vous accédez à un dossier partagé via `\\serveur\dossier` sur un réseau d'entreprise. FSx for Windows reproduit ce protocole nativement dans AWS, ce qui permet à des applications Windows existantes de continuer à fonctionner sans modification.

FSx for Lustre :

- Optimisé pour le **high-performance computing (HPC)** et machine learning.
- Très haute performance pour les grandes quantités de données.

Lustre est un système de fichiers distribué open source conçu à l'origine pour les supercalculateurs : il répartit un même fichier sur plusieurs serveurs de stockage pour permettre à des milliers de machines de le lire et l'écrire simultanément à très haut débit. C'est ce qui en fait le choix de référence pour l'entraînement de modèles de machine learning sur de gros volumes de données ou les simulations scientifiques (météo, génomique, calcul financier).

Comparatif EFS vs FSx :

Critère	EFS	FSx (Windows)	FSx (Lustre)
Protocole	NFS (Linux/Unix)	SMB (Windows)	Lustre (HPC)
OS Support	Linux/Unix	Windows	Linux/HPC
Active Directory	Non	✅ Oui	Non
Performance	Modérée, extensible	Haute	Très haute (HPC)
Coût	Bas à moyen	Moyen-élevé	Élevé
Idéal pour	Linux distribué	Partages Windows d'entreprise	Calcul scientifique/IA

Conseil pratique :

- EFS : première option pour Linux si pas besoin de domaine.
- FSx for Windows : environnement Windows avec Active Directory.
- FSx for Lustre : uniquement si performance HPC requise.

👤 [EBS Documentation](#) 📄 [EFS Documentation](#) 📄 [FSx Documentation](#) 📄

6.4 Sécurité EC2

Security Groups : pare-feu virtuel

Les **Security Groups** sont des pare-feux virtuels qui :

- Autorisent ou bloquent le trafic,
- Sont **stateful** (les réponses sont automatiquement autorisées),
- Peuvent être appliqués à plusieurs instances.

Ce caractère stateful est essentiel à comprendre avant de lire des règles concrètes : il signifie qu'une règle entrante suffit à autoriser la réponse correspondante, sans avoir à écrire une règle sortante symétrique.

Exemple de règles

Voici un jeu de règles typique pour un serveur web accessible publiquement mais administrable uniquement depuis une IP de confiance :

Direction	Port	Source/Destination
Inbound	22 (SSH)	192.168.1.100/32
Inbound	80 (HTTP)	0.0.0.0/0
Inbound	443 (HTTPS)	0.0.0.0/0
Outbound	Tout	par défaut

Règle entrante exemple :

- Autoriser le port **22 (SSH)** uniquement depuis une IP précise.
- Autoriser les ports **80 et 443 (HTTP/HTTPS)** depuis n'importe où.

Règle sortante (par défaut) :

- Autoriser tout trafic sortant.

 [Security Groups](#)

Key Pairs : accès SSH sécurisé

Les **Key Pairs** servent à sécuriser l'accès SSH :

- **Clé publique** enregistrée dans AWS et stockée dans l'instance au démarrage.
- **Clé privée** conservée localement par l'administrateur.

Flux de connexion :

1. Vous lancez une instance EC2 et sélectionnez une Key Pair.
2. AWS injecte la clé publique dans `~/.ssh/authorized_keys` de l'instance.
3. Depuis votre ordinateur, vous utilisez votre clé privée pour vous connecter en SSH.
4. L'authentification par clé est plus sécurisée qu'un mot de passe (impossible à craquer par brute force).

Security Groups et Key Pairs suffisent pour la grande majorité des instances ; certains secteurs réglementés exigent cependant des garanties matérielles supplémentaires.

6.5 Options de conformité EC2

Les environnements réglementés (santé, finance, RGPD) ont besoin de **garanties de conformité**. AWS fournit plusieurs mécanismes :

Dedicated Instances

Le premier niveau d'isolation matérielle, le plus simple à activer, consiste à louer du matériel non partagé sans en avoir le contrôle direct :

- Instance EC2 qui s'exécute sur **matériel physique dédié**.
- Pas de partage avec d'autres clients AWS.
- Idéal pour les **exigences légales** ou de conformité.
- Coût plus élevé que le partage de matériel.

Quand la conformité exige non seulement un matériel non partagé mais aussi un contrôle explicite du placement des instances, on passe au niveau supérieur.

Dedicated Hosts

Ce second niveau va plus loin que Dedicated Instances en donnant une visibilité complète sur le serveur physique sous-jacent :

- **Serveur physique entier** réservé pour votre compte.
- Contrôle total : vous décidez quelles instances y tournent.
- Utile pour les **licences logicielles** (ex. Windows, SQL Server avec licensing par socket/processeur).
- Exigences réglementaires très strictes.

Ces deux options d'isolation matérielle n'ont cependant rien à voir avec le type de stockage attaché à l'instance, sujet traité séparément ci-dessous.

Instance Store (éphémère)

À l'opposé de la conformité matérielle, ce type de stockage répond à un besoin de performance brute, au prix d'une contrainte forte :

- Stockage **très rapide** mais **temporaire** sur l'hyperviseur physique.
- **Attention** : données perdues à l'arrêt/redémarrage de l'instance.
- Idéal pour cache, données temporaires, haute performance.
- À éviter pour données persistantes.

Danger

Instance Store : perte de données garantie à l'arrêt — Contrairement à EBS, le stockage instance store **n'est pas persistant**. Toutes les données écrites dessus sont définitivement perdues si l'instance est arrêtée, terminée ou si l'hôte physique tombe en panne. Ne stockez jamais de données de production, de bases de données ou de fichiers importants sur instance store sans sauvegarde préalable vers S3 ou EBS.

Encrypted EBS Volumes

Dernier levier de conformité, orthogonal aux précédents puisqu'il porte sur la donnée plutôt que sur le matériel :

- Les volumes EBS peuvent être chiffrés avec **AWS KMS**.
- Le chiffrement est **transparent** pour l'application.
- Utile pour la conformité HIPAA, PCI-DSS, ISO 27001.

Cas d'usage conformité :

- Données médicales → Dedicated Instance + EBS chiffré + Audit CloudTrail.
- Données financières → Dedicated Host + KMS + VPC isolé.
- Données RGPD → Région EU + Versioning S3 + Chiffrement.

Choix de service — S3 ou EBS

Où placer des images partagées par plusieurs instances EC2 et accessibles par URL ?

► Afficher la réponse raisonnée

7. AWS Compute Optimizer — Dimensionnement optimal

7.1 Qu'est-ce que AWS Compute Optimizer ?

AWS Compute Optimizer est un service qui **analyse vos patterns d'utilisation** des instances EC2 et recommande des types plus optimisés en coût et performance.

	Instance actuelle	Recommandation
Type	t3.large (trop puissante)	t3.small (plus économique)
CPU utilisation	5%	—
Mémoire	12%	—
Coût mensuel	80 \$	~48 \$ (économie 60%)
Performance	—	identique
Risque	—	très faible (marges CPU)

Compute Optimizer applique cette analyse automatiquement à partir des métriques CloudWatch collectées.

7.2 Fonctionnement

1. **Collecte** : Compute Optimizer récupère les métriques CloudWatch (CPU, mémoire, réseau) sur **14 jours minimum**.
2. **Analyse** : Machine Learning compare votre utilisation réelle avec les capacités des autres types.
3. **Recommandation** : Propose des types économiquement viables.
4. **Confiance** : Indique un score de confiance (low, medium, high).

Ce score de confiance conditionne directement la nature de la recommandation reçue, qui se décline en quatre catégories.

7.3 Types de recommandations

Compute Optimizer ne se limite pas à proposer une taille plus petite : il distingue quatre types de recommandations selon ce que révèle l'analyse de vos métriques :

Recommandation	Bénéfice	Risque	Exemple
Downsizer	Économies importantes	Risque d'augmenter le CPU > 100%	t3.large → t3.small
Upgrade	Meilleure performance	Légère augmentation de coût	m5.large → m5.xlarge
Switch Family	Meilleure performance/\$	Changement d'architecture	t3.large → m6i.large
Aucune recommandation	Instance bien dimensionnée	N/A	✅ Garder tel quel

Voyons maintenant comment activer le service et récupérer concrètement ces recommandations via la CLI.

7.4 Activation et utilisation

Compute Optimizer analyse l'utilisation réelle de vos instances (CPU, mémoire, réseau) sur 14 jours et suggère le type le mieux adapté. La commande suivante affiche ces recommandations sous forme de tableau comparatif.

```

1  # Vérifier les recommandations Compute Optimizer
2  aws compute-optimizer get-ec2-instance-recommendations \
3      --region eu-west-1 \
4      --query 'instanceRecommendations[].{
5          Instance:instanceArn,
6          Current:currentInstanceType,
7          Recommended:recommendationOptions[0].instanceType,
8          Savings:recommendationOptions[0].savingsOpportunity.estimatedMonthlySavings.value,
9          ConfidenceLevel:currentInstanceType
10     }' \
11      --output table

```



Tip

Résultat attendu :

```
-----
|                                     GetEc2InstanceRecommendations
|
+-----+-----+-----+-----+-----+
| Instance | Current | Recommended | Savings |
ConfidenceLevel |
+-----+-----+-----+-----+-----+
| arn:aws:ec2:eu-west-1:123:instance | t3.large | t3.small | 47.82 |
t3.large |
| arn:aws:ec2:eu-west-1:123:instance | m5.xlarge | m5.large | 62.40 |
m5.xlarge |
+-----+-----+-----+-----+-----+
-----
```

Si aucune recommandation n'apparaît, Compute Optimizer manque encore de données (il lui faut au minimum 30h d'activité sur les instances).

7.5 Cas d'usage

Au-delà de la simple réduction de facture, Compute Optimizer s'intègre dans plusieurs démarches organisationnelles :

- **Optimisation de coûts** : identifier toutes les instances surdimensionnées.
- **Gouvernance cloud** : politiques de rightsizing automatisées.
- **Migration** : recommandations pour basculer vers une architecture nouvelle.
- **Audit FinOps** : justification des dépenses EC2.

Avantage clé : Compute Optimizer s'appuie sur **12-14 jours de données réelles**, pas sur des hypothèses théoriques.

8. Options de tarification AWS EC2

AWS propose plusieurs modèles de tarification pour s'adapter aux besoins techniques et budgétaires des entreprises. Le choix dépend du niveau de prévisibilité des workloads, du budget disponible, et de la tolérance aux interruptions.

8.1 On-Demand (À la demande)

C'est le modèle par défaut, sans aucun engagement, sur lequel viennent se comparer tous les autres :

- **Païement à l'heure** ou à la seconde pour la capacité de calcul utilisée, sans engagement à long terme.
- **Idéal pour** les charges de travail à court terme, les tests et le développement.
- **Pas de paiement anticipé** ni d'engagement minimum.
- **Prix plus élevé** que les autres options mais offre une flexibilité maximale.
- **Recommandé pour** les applications ne pouvant pas être interrompues et ayant des charges de travail imprévisibles.

Exemple : Vous avez un pic de trafic imprévu. Vous lancez des instances On-Demand pour répondre à la demande, puis les arrêtez après le pic.

8.2 Savings Plans

Pour une charge stable et prévisible, on peut aller plus loin que le simple On-Demand en échangeant un engagement contre une réduction substantielle :

- **Engagement de consommation horaire en dollars** (\$/heure) sur une période de 1 ou 3 ans.
- **Réductions** pouvant atteindre **72%** par rapport au tarif à la demande.
- **Deux types principaux :**
 - **Compute Savings Plans :** Flexibilité maximale couvrant EC2, Fargate et Lambda, avec support multi-familles d'instances, tailles et régions.
 - **EC2 Instance Savings Plans :** Réductions plus importantes mais limité à une famille d'instances dans une région spécifique.
- **Options de paiement** flexibles impactant le taux de réduction :
 - **No Upfront :** Aucun paiement initial.
 - **Partial Upfront :** Paiement partiel initial.
 - **Full Upfront :** Paiement total initial offrant les meilleures réductions.

8.3 Instances Spot (À prix réduit)

Ce modèle exploite un principe totalement différent des deux précédents : au lieu d'un engagement, c'est l'acceptation d'un risque d'interruption qui est rémunérée par une forte réduction :

- **Utilisation de la capacité EC2 inutilisée** d'AWS.
- **Réductions jusqu'à 90%** par rapport au prix à la demande.
- **Les instances peuvent être interrompues** avec un préavis de 2 minutes si AWS a besoin de la capacité.

- **Idéal pour :**
 - Les charges de travail tolérantes aux interruptions.
 - Le calcul haute performance (HPC).
 - Les jobs batch (traitement par lots).
 - Les workloads flexibles en termes de début et de fin.

Best practices pour la résilience :

- Utiliser les **groupes d'auto-scaling** pour gérer les interruptions automatiquement.
- Implémenter via **EC2 Spot Fleet** ou **EC2 Spot Instances Requests**.
- Concevoir l'application pour tolérer les interruptions.

Warning

Instances Spot : interruption en 2 minutes — AWS peut récupérer vos instances Spot avec seulement **2 minutes de préavis** lorsque la capacité est nécessaire. Ne jamais utiliser des instances Spot pour des workloads critiques sans tolérance aux interruptions (bases de données de production, serveurs web sans état de session externalisé). Toujours prévoir un mécanisme de sauvegarde ou de checkpoint des données en cours de traitement.

8.4 Reserved Instances (RI)

Plus ancien que les Savings Plans, ce modèle d'engagement reste pertinent dans certains cas, notamment grâce à sa revente possible :

- **Engagement** sur une instance spécifique pour **1 ou 3 ans**.
- **Réductions jusqu'à 75%** par rapport au prix à la demande.
- **Différences principales** avec les Savings Plans :
 - Les RI sont liées à une instance spécifique (type, taille, région, zone).
 - Les Savings Plans sont basés sur un engagement de consommation en dollars (plus flexibles).
 - Les RI peuvent être vendues sur le **AWS RI Marketplace**, pas les Savings Plans.

8.5 Comparatif synthétique

Après ce tour des quatre modèles, voici une vue d'ensemble pour choisir rapidement selon vos priorités :

Critère	On-Demand	Reserved	Spot	Savings Plans
Engagement	Aucun	1 ou 3 ans	Aucun	1 ou 3 ans
Réduction potentielle	✗	✓ (75%)	✓ ✓ ✓ (90%)	✓ ✓ (72%)
Flexibilité	✓ ✓ ✓	✗	✓ ✓	✓ ✓
Risque d'interruption	✗	✗	✓ ✓ ✓	✗
Idéal pour	Dev/Test	Prod stable	Batch/CI	Prod optimisée

Recommandation : Pour la plupart des entreprises, **Savings Plans** offre le meilleur compromis entre réduction (72%) et flexibilité.

8.6 Cas métier : Choisir la meilleure option tarifaire

Cas 1 : Site e-commerce avec trafic prévisible

Ce premier cas combine deux modèles vus plus haut pour traiter séparément la charge stable et les pics ponctuels :

- **Charge :** trafic stable, pics prévisibles en fin d'année.
- **Infrastructure :** 10 instances t3.large en continu, +20 during soldes.
- **Recommandation :** **Savings Plans (Compute)** pour les 10 instances permanentes + **Spot** pour les 20 supplémentaires pendant les soldes.
- **Économie :** ~72% sur la base, ~90% sur les renforts = **78% global**.

Coût mensuel sans optimisation (tout On-Demand) :
 $10 \times 730h \times 0,096\$ \text{ (t3.large)} = 699 \$$

Coût optimisé (Savings Plans + Spot) :
 $10 \times (0,096\$ \times 0,28) + 20 \times (0,096\$ \times 0,10) = 26,88 \$ + 19,2 \$ = 46,08 \$$

Économie : $652,92 \$ / \text{mois} = 7,835 \$ / \text{an}$

Ce calcul montre qu'un mix Savings Plans + Spot peut réduire la facture de plus des trois quarts sans sacrifier ni la disponibilité de la base ni la flexibilité des renforts saisonniers. Tous les profils de charge ne se prêtent cependant pas à cette optimisation.

Cas 2 : Environnement de développement/test

À l'inverse du cas précédent, ce scénario montre qu'il ne faut pas systématiquement chercher une réduction tarifaire :

- **Charge** : variable, utilisation heures de travail uniquement.
- **Infrastructure** : 2-4 instances selon le sprint en cours.
- **Recommandation** : **On-Demand** uniquement (pas d'engagement, flexibilité totale).
- **Économie** : aucune, mais coûts minimaux et liberté maximale.

Ici, la simplicité et l'absence d'engagement priment sur l'économie potentielle, car le nombre d'instances change trop souvent pour justifier un engagement Savings Plans ou RI.

Cas 3 : Job batch nocturne de traitement

Ce troisième cas est le profil idéal pour les instances Spot, tolérantes aux interruptions par nature :

- **Charge** : lance chaque nuit des instances pour 4h, puis arrêt.
- **Infrastructure** : 50 instances c5.2xlarge pour le parallélisme.
- **Recommandation** : **Spot instances** avec **Spot Fleet** (demande auto-scaling de remplacement).
- **Économie** : ~90% vs On-Demand = 81 /j au lieu de 810 , soit 8,100\$/mois.

```

1  # Configuration Spot Fleet pour job batch
2  aws ec2 request-spot-fleet \
3      --spot-fleet-request-config '{
4          "IamFleetRole": "arn:aws:iam::xxxxx:role/fleet",
5          "SpotPrice": "0.10",
6          "TargetCapacity": 50,
7          "LaunchSpecifications": [{
8              "ImageId": "ami-xxxxx",
9              "InstanceType": "c5.2xlarge",
10             "KeyName": "ma-cle"
11         }]
12     }'
```



Tip

Résultat attendu :

```
{
  "SpotFleetRequestId": "sfr-0a1b2c3d4e5f6789a",
  "SpotFleetRequestState": "submitted"
}
```

Cas 4 : Application critiques 24/7 avec charge non prévisible

Ce dernier cas est le plus délicat, car il combine une exigence de continuité de service avec une charge impossible à anticiper précisément :

- **Charge** : pas de pattern clair, augmentations soudaines.
- **Infrastructure** : 4-30 instances selon la demande.
- **Recommandation** : **Savings Plans (mélange)** pour la charge de base + **On-Demand** pour les pics.
- **Avantage** : si charge explose au-delà des prévisions, On-Demand absorbe sans coupure.

Ces quatre cas illustrent qu'il n'existe pas de modèle tarifaire universellement meilleur : le bon choix résulte toujours d'un arbitrage entre prévisibilité de la charge et tolérance aux interruptions. Pour chiffrer précisément votre propre scénario, AWS met à disposition un outil de simulation dédié.

8.7 Outil : AWS Pricing Calculator

Cet outil web permet d'estimer le coût d'une architecture avant de la déployer, en combinant plusieurs services et modèles tarifaires :

```
URL : https://calculator.aws/
1. Sélectionner la région
2. Ajouter EC2 : type, nombre, durée
3. Sélectionner option (On-Demand, Reserved, Spot)
4. Voir l'estimation mensuelle/annuelle
5. Exporter en PDF pour justifier budgets
```

 [EC2 Pricing](#)

9. Lancer une instance EC2 en CLI

Info

Une activité pratique permet d'approfondir le lancement d'instances EC2 en CLI.

9.1 Créer une Key Pair

La Key Pair est l'équivalent d'une clé SSH. AWS génère la paire (privée + publique), conserve la clé publique, et vous remet la clé privée **une seule fois**. Le `chmod 600` est obligatoire — SSH refuse de se connecter si la clé est trop permissive.

```
1 # Créer une nouvelle paire de clés
2 aws ec2 create-key-pair \
3     --key-name ma-cle-formation \
4     --query 'KeyMaterial' \
5     --output text > ~/.ssh/ma-cle-formation.pem
6
7 # Définir les permissions correctes (très important pour SSH)
8 chmod 600 ~/.ssh/ma-cle-formation.pem
9
10 # Vérifier les clés existantes
11
12 aws ec2 describe-key-pairs \
13     --query 'KeyPairs[].KeyName' \
14     --output table
```

Explications :

- `create-key-pair` : génère une nouvelle paire de clés
- `--key-name` : identifiant de la clé
- `--output text` : affiche juste le contenu privé
- `chmod 600` : permissions restrictives (propriétaire seul peut lire)



Tip

Résultat attendu :

```
# La clé privée est écrite dans ~/.ssh/ma-cle-formation.pem (aucun output CLI)

# describe-key-pairs --output table affiche :
-----+-----
|          DescribeKeyPairs          |
+-----+-----+
|          ma-cle-formation          |
+-----+-----+
```



Warning

Clé privée : une seule chance de la télécharger — AWS ne conserve jamais la clé privée. Si vous perdez le fichier `.pem`, vous devrez créer une nouvelle Key Pair et relancer une instance — il est impossible de récupérer l'accès SSH autrement. Sauvegardez systématiquement votre clé dans un gestionnaire de secrets (AWS Secrets Manager, HashiCorp Vault) ou dans un endroit sécurisé.

9.2 Lancer une instance EC2 simple

On récupère d'abord l'ID de la dernière AMI Ubuntu 20.04 disponible, puis on lance une instance `t3.micro` (éligible au Free Tier). La variable `$AMI_ID` évite de copier-coller un ID d'image qui change régulièrement.

```

1  # Récupérer l'AMI Ubuntu la plus récente
2  AMI_ID=$(aws ec2 describe-images \
3      --owners 099720109477 \
4      --filters "Name=name,Values=ubuntu/images/hvm-ssd/ubuntu-focal-20.04-amd64-
server-*" \
5      --query 'sort_by(Images, &CreationDate)[-1].[ImageId]' \
6      --output text)
7
8  # Lancer l'instance
9  aws ec2 run-instances \
1
10     --image-id $AMI_ID \
11
12     --instance-type t3.micro \
13
14     --key-name ma-cle-formation \
15
16     --security-groups sg-formation \
17
18     --region eu-west-1 \
19
20     --query 'Instances[0].[InstanceId,PublicIpAddress,State.Name]' \
21
22     --output table

```

Explications :

- `describe-images` : recherche d'une AMI Ubuntu récente
- `--image-id` : l'AMI à utiliser (système d'exploitation)
- `--instance-type t3.micro` : type d'instance (gratuit dans Free Tier)
- `--key-name` : la clé SSH pour se connecter
- `--security-groups` : pare-feu assigné à l'instance



Tip

Résultat attendu :

```
-----
|                               RunInstances                               |
+-----+-----+-----+-----+
| InstanceId | PublicIpAddr | State |
+-----+-----+-----+-----+
| i-0abc123def45678 | None | pending |
+-----+-----+-----+-----+

# Après ~30 secondes, l'état passe à "running" :
+-----+-----+-----+-----+
| i-0abc123def45678 | 54.171.23.45 | running |
+-----+-----+-----+-----+
```

9.3 Se connecter à l'instance

La première commande récupère l'IP publique de l'instance via CLI (plutôt que de la copier depuis la console), la seconde s'y connecte en SSH avec la clé créée précédemment.

```
1 # Obtenir l'adresse IP publique
2 INSTANCE_ID="i-xxxxx"
3 IP=$(aws ec2 describe-instances \
4     --instance-ids $INSTANCE_ID \
5     --query 'Reservations[0].Instances[0].PublicIpAddress' \
6     --output text)
7
8 # Se connecter en SSH
9 ssh -i ~/.ssh/ma-cle-formation.pem ubuntu@$IP
1
0
1
1 # Une fois connecté, les commandes Linux habituelles s'exécutent normalement
```

Explications :

- `describe-instances` : récupère les infos de l'instance
- `PublicIpAddress` : adresse IP pour accès depuis Internet
- `-i ~/.ssh/ma-cle-formation.pem` : utilise votre clé privée

- `ubuntu` : utilisateur par défaut dans les AMI Ubuntu



Tip

Résultat attendu :

```
# describe-instances retourne l'IP :
54.171.23.45

# Connexion SSH réussie :
Warning: Permanently added '54.171.23.45' (ED25519) to the list of known
hosts.
Welcome to Ubuntu 20.04.6 LTS (GNU/Linux 5.15.0-1053-aws x86_64)

* Documentation:  https://help.ubuntu.com
* Management:    https://landscape.canonical.com
* Support:       https://ubuntu.com/advantage

ubuntu@ip-10-0-1-45:~$
```

Warning

Sécurité SSH : limitez l'accès au port 22 — Ouvrir le port 22 à `0.0.0.0/0` (toutes les IPs) expose votre instance aux scanners automatisés et tentatives de brute force. Dans votre Security Group, restreignez toujours l'accès SSH à votre adresse IP publique uniquement (`monip/32`). Utilisez `curl ifconfig.me` pour connaître votre IP courante.

9.4 Créer une image (AMI) à partir d'une instance

Créer une AMI depuis une instance configurée permet de la **cloner** rapidement : toutes vos installations et configurations sont préservées. C'est la base du déploiement automatisé avec Auto Scaling.

```

1  # Créer une AMI personnalisée
2  aws ec2 create-image \
3      --instance-id i-xxxxx \
4      --name "Ma-Formation-AMI" \
5      --description "AMI avec mes configurations de formation" \
6      --no-reboot \
7      --output table
8
9  # Vérifier la création
1
0  aws ec2 describe-images \
1
1      --owners self \
1
2      --query 'Images[].[ImageId,Name,State]' \
1
3      --output table

```

Explications :

- `create-image` : crée une nouvelle AMI à partir d'une instance
- `--no-reboot` : ne redémarre pas l'instance (accélère le processus)
- `--owners self` : affiche uniquement vos AMI personnalisées



Tip

Résultat attendu :

```
# create-image retourne :
{
  "ImageId": "ami-0a1b2c3d4e5f67890"
}

# describe-images --output table affiche :
-----
|                               DescribeImages                               |
+-----+-----+-----+
|      ImageId      |      Name      |      State      |
+-----+-----+-----+
| ami-0a1b2c3d4e5f6 | Ma-Formation-AMI | pending         |
+-----+-----+-----+

# Après quelques minutes :
| ami-0a1b2c3d4e5f6 | Ma-Formation-AMI | available       |
```

9.5 Créer un snapshot EBS

Un snapshot EBS est une sauvegarde du disque d'une instance. Contrairement à une AMI (qui inclut le système complet), le snapshot est juste le contenu du volume — utile pour restaurer des données sans recréer toute une instance.

```

1  # Lister les volumes EBS attachés
2  aws ec2 describe-volumes \
3      --filters "Name=attachment.instance-id,Values=i-xxxxx" \
4      --query 'Volumes[].[VolumeId,Size,State]' \
5      --output table
6
7  # Créer un snapshot d'un volume
8  aws ec2 create-snapshot \
9      --volume-id vol-xxxxx \
10
11      --description "Snapshot de sauvegarde formation" \
12
13      --tag-specifications 'ResourceType=snapshot,Tags=[{Key=Name,Value=backup-
formation}]' \
14
15      --output table
16
17
18
19
20  # Vérifier l'état du snapshot
21
22  aws ec2 describe-snapshots \
23
24
25      --owner-ids self \
26
27
28      --query 'Snapshots[].[SnapshotId,VolumeSize,State,Progress]' \
29
30
31      --output table

```

Explications :

- `describe-volumes` : liste les disques EBS attachés à une instance
- `create-snapshot` : crée une sauvegarde du disque
- `--tag-specifications` : ajoute des étiquettes pour l'organiser
- Le snapshot est **incrémental** : seuls les changements sont sauvegardés



Tip

Résultat attendu :

```
# describe-volumes --output table :
```

```
-----  
|          DescribeVolumes          |  
+-----+-----+-----+  
|   VolumeId   | Size |   State   |  
+-----+-----+-----+  
| vol-0a1b2c3d4e | 20  | in-use    |  
+-----+-----+-----+
```

```
# create-snapshot retourne :
```

```
{  
  "SnapshotId": "snap-0a1b2c3d4e5f67890",  
  "VolumeId": "vol-0a1b2c3d4e",  
  "State": "pending",  
  "Progress": ""  
}
```

```
# describe-snapshots après quelques minutes :
```

```
-----  
|          DescribeSnapshots          |  
+-----+-----+-----+-----+  
|   SnapshotId   | Size |   State   | Progress |  
+-----+-----+-----+-----+  
| snap-0a1b2c3d4e5 | 20  | completed | 100%    |  
+-----+-----+-----+-----+
```

Warning

Coûts des snapshots EBS — Les snapshots sont stockés dans S3 (managé par AWS) et facturés ~0,05 /Go-mois.

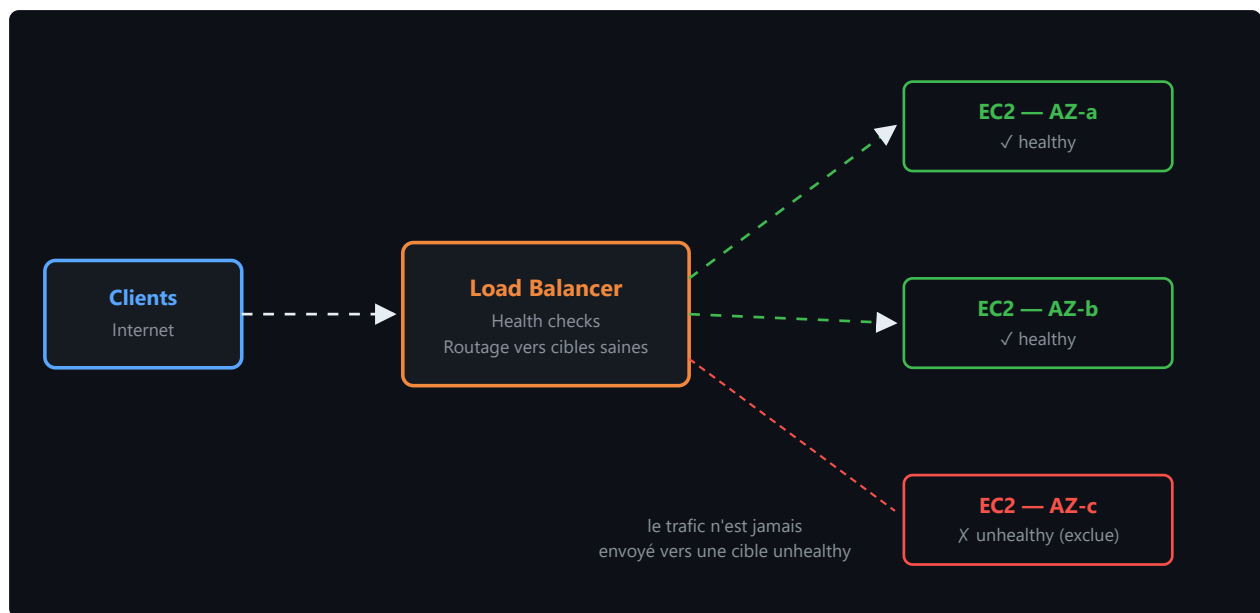
Un snapshot de 100 Go coûte ~5 /mois. Les snapshots sont **incrémentiels** (seules les modifications depuis le dernier snapshot sont stockées), mais les premiers snapshots peuvent être volumineux. Planifiez une politique de rétention et supprimez les snapshots obsolètes pour maîtriser les coûts.

10. Elastic Load Balancing (ELB) — Répartition du trafic

10.1 Pourquoi un Load Balancer ?

Un **Load Balancer** agit comme un répartiteur de trafic. Il reçoit les requêtes des clients et les distribue vers les instances EC2 disponibles, selon des règles de routage et de santé (**health checks**).

Architecture simple



10.2 Types de Load Balancer AWS

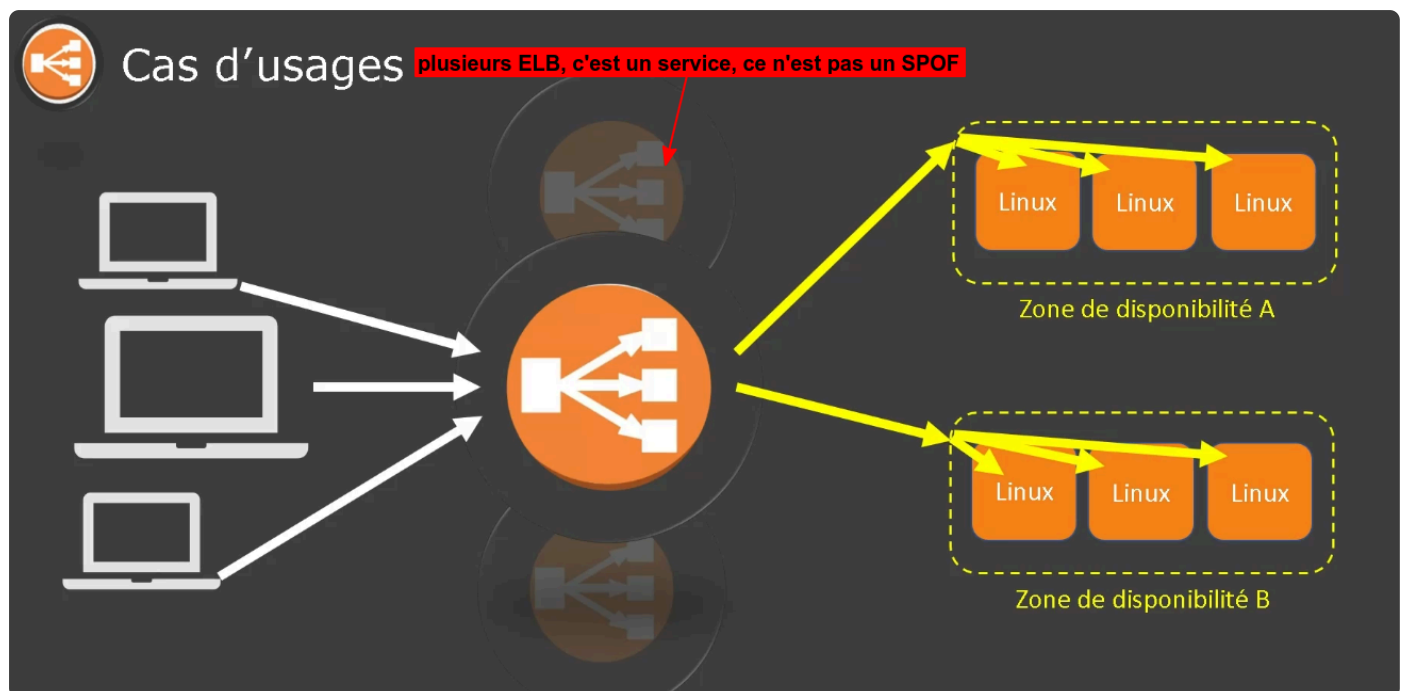
AWS propose trois types de Load Balancer, qui n'opèrent pas au même niveau du modèle réseau OSI :

Type	Cas d'usage typique	Protocole	Niveau OSI
ALB (Application Load Balancer)	Applications web, microservices	HTTP/HTTPS	Couche 7 (Application)
NLB (Network Load Balancer)	Faible latence, TCP	TCP/UDP	Couche 4 (Transport)
GLB (Gateway Load Balancer)	Appliances réseau (firewall, inspection)	IP	Couche 3 (Réseau)

ALB (Application Load Balancer) : conçu pour les applications web. Il fonctionne au niveau **HTTP/HTTPS** (couche 7 du modèle OSI) et permet un **routing avancé** (par URL, en-tête, hostname, etc.).

NLB (Network Load Balancer) : adapté aux applications nécessitant une **faible latence**. Il fonctionne au niveau **TCP** (couche 4), idéal pour les bases de données ou les services temps réel.

GLB (Gateway Load Balancer) : utilisé pour intégrer des **appliances réseau** comme des pare-feu ou des outils d'inspection. Il fonctionne au niveau **IP**.



10.3 Fonctionnement du Load Balancer

Concrètement, le Load Balancer répète un cycle simple pour maintenir le trafic dirigé uniquement vers des instances fonctionnelles :

- Le Load Balancer **vérifie l'état** des instances via des **health checks** (tests de disponibilité).
- Il **répartit les requêtes** vers les instances **saines** uniquement.
- Il **s'adapte automatiquement** à l'ajout ou la suppression d'instances via **Auto Scaling**.

Health Check - Exemple :

Chaque 30 secondes :

1. LB envoie requête GET `http://instance:80/health`
2. Instance répond HTTP 200 OK
3. Instance est marquée "saine"

Si pas de réponse ou erreur 5xx :

1. Instance marquée "défaillante"
2. Pas plus de trafic envoyé vers elle

11. Auto Scaling — Adaptation dynamique des ressources

11.1 Qu'est-ce qu'Auto Scaling ?

Un **Auto Scaling Group (ASG)** est un groupe d'instances EC2 géré automatiquement. Il peut être associé à un Load Balancer pour garantir que :

- Les nouvelles instances sont automatiquement **enregistrées** auprès du Load Balancer.
- Les instances défaillantes sont **retirées** du pool.
- Le trafic est toujours dirigé vers les **ressources disponibles**.

Cette intégration avec le Load Balancer n'est qu'une face d'Auto Scaling ; l'autre face concerne les règles qui décident du nombre d'instances à faire tourner à un instant donné.

11.2 Politiques de scaling — Fondamentaux

Les politiques définissent **quand et comment** ajouter ou retirer des instances, dans deux directions symétriques.

Scale-out (Agrandissement)

Voici la séquence déclenchée lorsque la charge augmente :

```
Charge CPU dépasse 70% pendant 5 min
      ▼
Ajouter 2 instances supplémentaires
      ▼
Attendre que les instances démarrent
      ▼
Health check OK : instances intégrées au LB
```

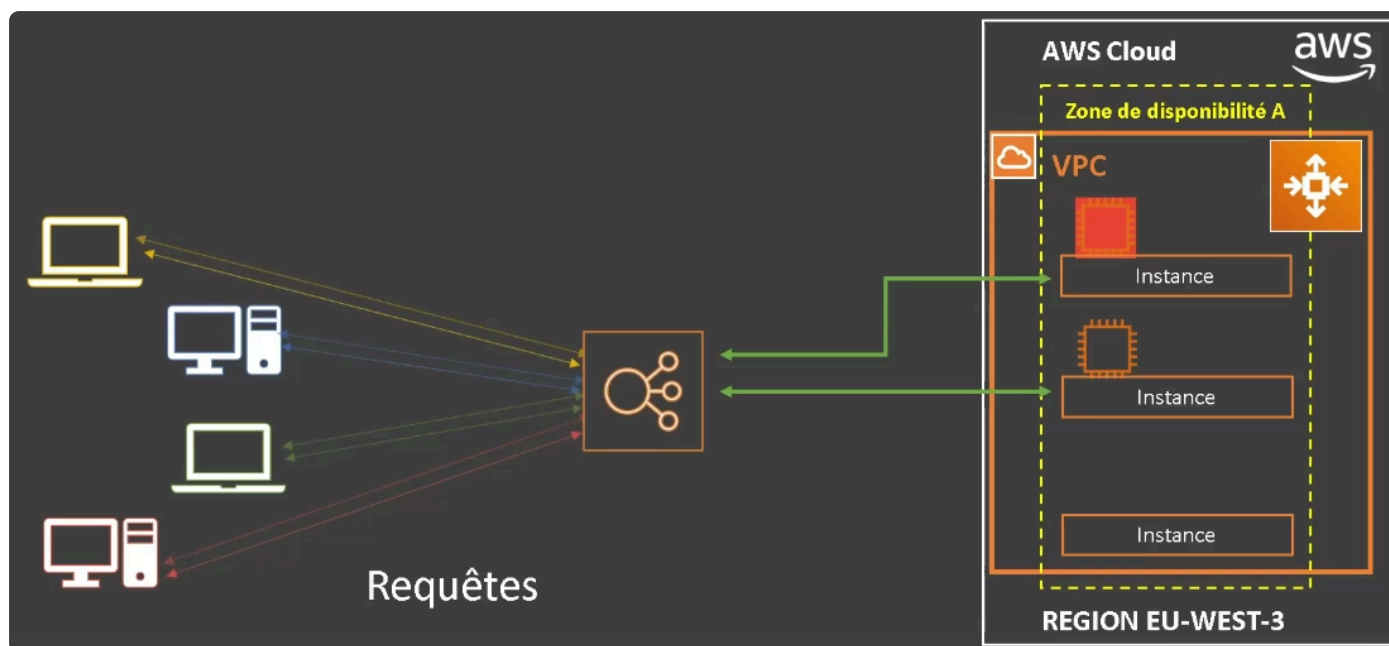
À l'opposé de cette montée en charge, un mécanisme symétrique retire les instances devenues inutiles pour éviter de payer une capacité surdimensionnée.

Scale-in (Réduction)

Et voici la séquence inverse, déclenchée lorsque la charge redescend :

```
Charge CPU chute à 30% pendant 10 min
▼
Retirer 1 instance
▼
Attendre que les requêtes actuelles finissent
▼
Fermer l'instance, libérer les ressources
```

Notez l'asymétrie des délais (5 min pour scale-out, 10 min pour scale-in) : c'est volontaire, mieux vaut réagir vite à une surcharge et prudemment à une baisse, pour éviter d'osciller sans arrêt entre ajout et retrait d'instances.



11.3 Configuration d'Auto Scaling

Un ASG typique comporte :

Min Size : 2 instances · Max Size : 10 instances · Desired Capacity : 4 instances Launch Template : my-ami-config · Load Balancer : my-alb

Scaling Policies : Target CPU 70% · Scale out +2 instances/5 min · Scale in -1 instance/10 min

Paramètres clés :

- **Min Size** : minimum d'instances (au moins 2 pour la haute disponibilité)
- **Max Size** : limite supérieure pour éviter les coûts explosifs
- **Desired Capacity** : nombre d'instances cible en ce moment
- **Launch Template** : modèle (AMI, type, security group, etc.) pour les nouvelles instances

Cette configuration de base repose entièrement sur le CPU pour déclencher le scaling ; Auto Scaling peut en réalité observer bien d’autres signaux pour piloter ses décisions.

11.4 Métriques CloudWatch et politiques de scaling avancées

Auto Scaling peut se baser sur **plusieurs métriques CloudWatch**, pas seulement CPU.

Métriques disponibles

Selon la nature de l’application, une métrique différente du CPU peut être plus pertinente pour déclencher le scaling :

Métrique	Source	Cas d’usage typique
CPU Utilization	CloudWatch	Charge générale, serverless
NetworkIn / NetworkOut	CloudWatch	Applications réseau intensives
ALB Target Count	CloudWatch + ELB	Nombre de requêtes traitées
Request Count Per Target	CloudWatch + ELB	Répartition de charge par instance
Target Response Time	CloudWatch + ELB	Dégradation de performance
Memory Utilization	CloudWatch Agent	Applications mémoire-intensives
Queue Depth (SQS)	SQS	Traitement asynchrone

Rien n’empêche de combiner plusieurs de ces métriques sur un même Auto Scaling Group, comme le montre l’exemple suivant avec CPU et débit réseau.

Exemple de politique de scaling multi-métriques

On crée ici deux politiques indépendantes sur le même ASG : l’une réagit au CPU, l’autre au débit réseau. AWS les évalue en parallèle et déclenche le scaling dès que l’une d’elles est satisfaite.

```

1  # Politique 1 : Scale out si CPU > 75% pendant 2 minutes
2  aws autoscaling put-scaling-policy \
3      --auto-scaling-group-name mon-asg \
4      --policy-name cpu-scale-out \
5      --policy-type TargetTrackingScaling \
6      --target-tracking-configuration '{
7          "TargetValue": 75.0,
8          "PredefinedMetricSpecification": {
9              "PredefinedMetricType": "ASGAverageCPUUtilization"
10
11          },
12          "ScaleOutCooldown": 120,
13          "ScaleInCooldown": 300
14      }'
15
16  # Politique 2 : Scale out si Network > 1 Gbps
17
18  aws autoscaling put-scaling-policy \
19      --auto-scaling-group-name mon-asg \
20      --policy-name network-scale-out \
21      --policy-type TargetTrackingScaling \
22      --target-tracking-configuration '{
23          "TargetValue": 70.0,
24          "CustomizedMetricSpecification": {
25              "MetricName": "NetworkOut",

```

```

2
4         "Namespace": "AWS/EC2",
2
5         "Statistic": "Average"
2
6     }
2
7 }

```



Tip

Résultat attendu :

```

# put-scaling-policy (cpu-scale-out) retourne :
{
  "PolicyARN": "arn:aws:autoscaling:eu-west-
1:123456789012:scalingPolicy:a1b2c3d4:autoScalingGroupName/mon-
asg:policyName/cpu-scale-out",
  "Alarms": [
    {
      "AlarmName": "TargetTracking-mon-asg-AlarmHigh-cpu-scale-out",
      "AlarmARN": "arn:aws:cloudwatch:eu-west-
1:123456789012:alarm:TargetTracking-mon-asg-AlarmHigh"
    }
  ]
}

# put-scaling-policy (network-scale-out) retourne de même avec un ARN
différent

```

Cooldown Periods (délais entre actions)

Les deux politiques créées ci-dessus configurent chacune un délai de stabilisation après une action de scaling, pour éviter les décisions trop rapprochées :

- **ScaleOutCooldown** (120-300s) : attend avant la prochaine augmentation.
 - Évite les oscillations rapides (scaling de “ping-pong”).
 - Laisse le temps aux instances de démarrer.
- **ScaleInCooldown** (300-900s) : plus long que scale-out.
 - Garantit l'équilibre avant réduction.
 - Préserve la performance en cas de pics rapides.

Exemple réaliste :


```
T=0s    CPU = 80% → Déclenche scale-out (+2 instances)
T=120s  Instances démarrent (cool-down scale-out)
T=180s  CPU = 60% → Pourrait déclencher scale-in MAIS...
T=240s  Attendre le cooldown scale-in
T=540s  CPU toujours < 30% → Scale-in (-1 instance)
```

Conseil : Définir des cooldowns asymétriques (court pour scale-out, long pour scale-in) pour favoriser la disponibilité.

 [Auto Scaling EC2](#)

11.5 Avantages combinés Load Balancer + Auto Scaling

Utilisés ensemble, Load Balancer et Auto Scaling forment le duo de base d'une architecture web résiliente sur AWS :

- **Résilience** : les instances défaillantes sont automatiquement **remplacées**.
- **Scalabilité** : le nombre d'instances s'adapte à la **charge** en temps réel.
- **Performance** : le trafic est réparti de manière **optimale** entre les ressources disponibles.
- **Économie** : vous payez uniquement pour les ressources utilisées.
- **Sécurité** : le Load Balancer peut gérer les **certificats SSL/TLS** pour sécuriser les communications.

Cette combinaison reste néanmoins bâtie sur des instances EC2 qui tournent en continu, même au repos. Le modèle serverless présenté dans la section suivante pousse l'élasticité un cran plus loin.

12. AWS Lambda — Le calcul sans serveur

12.1 Pourquoi Lambda, quand on a déjà EC2 et Auto Scaling ?

Vous venez de voir comment EC2 et Auto Scaling permettent d'adapter dynamiquement une flotte de serveurs à la charge. Mais même avec Auto Scaling, une instance EC2 minimale **tourne en permanence** — vous la payez même quand elle ne traite aucune requête.

AWS Lambda pousse le modèle serverless plus loin : au lieu de faire tourner un serveur en continu, vous déployez une **fonction** — un bloc de code — qu'AWS exécute uniquement quand un événement le déclenche (requête HTTP, fichier déposé sur S3, message dans une file, tâche planifiée...). Entre deux exécutions, **aucune ressource ne tourne, donc rien n'est facturé**.

	EC2 (même avec Auto Scaling)	Lambda
Ce que vous gérez	OS, runtime, mises à jour, capacité	Uniquement votre code
Facturation	À l'heure/seconde tant que l'instance tourne	À l'exécution (durée × mémoire allouée)
Charge nulle	Coût minimal non nul (au moins 1 instance)	0 \$ — aucune exécution, aucun coût
Démarrage	Minutes (boot instance) ou secondes (déjà démarrée)	Millisecondes à quelques secondes (cold start)
Durée d'exécution max	Illimitée	15 minutes par exécution

Cette limite de 15 minutes n'est pas anodine : elle signale que Lambda est pensé pour des traitements courts et événementiels, pas pour remplacer un serveur applicatif de longue durée.

12.2 Fonctionnement d'une fonction Lambda

Une fonction Lambda est un paquet de code (Python, Node.js, Java, Go, etc.) associé à une configuration : mémoire allouée (128 Mo à 10 Go), timeout maximal, et un ou plusieurs **triggers** — les événements qui la déclenchent.

Événement déclencheur		Fonction Lambda		Résultat
Requête HTTP (API Gateway)	→	Exécute le code	→	Réponse HTTP
Fichier déposé sur S3	→	(runtime + mémoire	→	Traitement
Message dans une file SQS	→	alloués à la demande)	→	Traitement
Planification (EventBridge) exécutée	→		→	Tâche

Le CPU alloué est proportionnel à la mémoire configurée — une fonction à 1 769 Mo de RAM obtient l'équivalent d'un vCPU complet. AWS gère entièrement l'infrastructure sous-jacente : vous ne choisissez ni AMI, ni type d'instance, ni Security Group pour la fonction elle-même.

12.3 Créer et invoquer une fonction Lambda en CLI

Info

Une activité pratique permet d'approfondir la création et l'invocation de fonctions Lambda.

Cette séquence crée une fonction Lambda Python minimale, l'invoque manuellement, puis vérifie les logs d'exécution dans CloudWatch.

```

1 # 1. Écrire le code de la fonction (fichier local)
2 cat > lambda_function.py << 'EOF'
3 def lambda_handler(event, context):
4     nom = event.get('nom', 'monde')
5     return {
6         'statusCode': 200,
7         'body': f'Bonjour, {nom} ! Fonction exécutée avec succès.'
8     }
9 EOF
1
0
1
1 # 2. Empaqueter le code en ZIP (format attendu par Lambda)
1
2 zip function.zip lambda_function.py
1
3
1
4 # 3. Créer le rôle IAM que la fonction va assumer (permissions d'exécution)
1
5 aws iam create-role \
1
6     --role-name formation-lambda-role \
1
7     --assume-role-policy-document '{
1
8         "Version": "2012-10-17",
1
9         "Statement": [{
2
0             "Effect": "Allow",
2
1             "Principal": {"Service": "lambda.amazonaws.com"},
2
2             "Action": "sts:AssumeRole"
3
3         }]

```

```

2
4     }'
2
5
2
6 # 4. Attacher la policy minimale pour écrire les logs CloudWatch
2
7 aws iam attach-role-policy \
2
8     --role-name formation-lambda-role \
2
9     --policy-arn arn:aws:iam::aws:policy/service-role/AWSLambdaBasicExecutionRole
3
0
3
1 # 5. Créer la fonction Lambda
3
2 aws lambda create-function \
3
3     --function-name formation-bonjour \
3
4     --runtime python3.12 \
3
5     --role arn:aws:iam::123456789012:role/formation-lambda-role \
3
6     --handler lambda_function.lambda_handler \
3
7     --zip-file fileb://function.zip \
3
8     --memory-size 128 \
3
9     --timeout 10
4
0
4
1 # 6. Invoquer la fonction avec un événement de test
4
2 aws lambda invoke \

```

```
4
3  --function-name formation-bonjour \
4
4  --payload '{"nom": "Formation AWS"}' \
4
5  --cli-binary-format raw-in-base64-out \
4
6  reponse.json
4
7
4
8  cat reponse.json
```



Résultat attendu :

```
1  {
2    "StatusCode": 200,
3    "ExecutedVersion": "$LATEST"
4  }
```

Contenu de `reponse.json` :

```
1  {"statusCode": 200, "body": "Bonjour, Formation AWS ! Fonction exécutée
avec succès."}
```

La fonction s'est exécutée en quelques centaines de millisecondes. Aucune instance EC2 n'a été provisionnée — Lambda a alloué l'environnement d'exécution le temps de traiter cette seule invocation, puis l'a libéré.

```
1  # 7. Consulter les logs d'exécution (CloudWatch Logs, créés automatiquement)
2  aws logs tail /aws/lambda/formation-bonjour --follow
```

Info

Le rôle IAM est la seule “sécurité réseau” de Lambda par défaut. Contrairement à EC2, une fonction Lambda n’a pas de Security Group tant qu’elle n’est pas explicitement rattachée à un VPC (`--vpc-config`). Une Lambda simple qui n’a besoin que d’appeler d’autres services AWS (S3, DynamoDB) via leurs API n’a généralement pas besoin d’être dans un VPC — le rôle IAM suffit à contrôler ce qu’elle a le droit de faire.

12.4 Combien coûte Lambda ?

Lambda facture deux choses : le **nombre d’invocations** et la **durée × mémoire** consommée.

Élément	Tarif (région Paris)	Free Tier mensuel
Invocations	0,20 \$ par million	1 million gratuit
Durée de calcul	0,0000166667 \$ par Go-seconde	400 000 Go-secondes gratuites

Exemple concret : une fonction à 512 Mo (0,5 Go) qui s’exécute 200 ms, appelée 2 millions de fois par mois :

Invocations : $2\,000\,000 \times 0,20 \$ / 1\,000\,000 = 0,40 \$$
Durée : $2\,000\,000 \times 0,2\text{ s} \times 0,5\text{ Go} \times 0,0000166667 \$ = 3,33 \$$
TOTAL $\approx 3,73 \$/\text{mois}$

À comparer à une instance EC2 t3.micro tournant en continu pour traiter le même trafic (~12 \$/mois de base, même si elle est idle 80 % du temps) : Lambda devient très avantageux dès que la charge est **intermittente** plutôt que constante. À l’inverse, pour un service qui reçoit du trafic 24h/24 à haut volume, une instance EC2 dimensionnée correctement (ou plusieurs derrière un ELB) redevient souvent moins chère que des millions d’invocations Lambda.

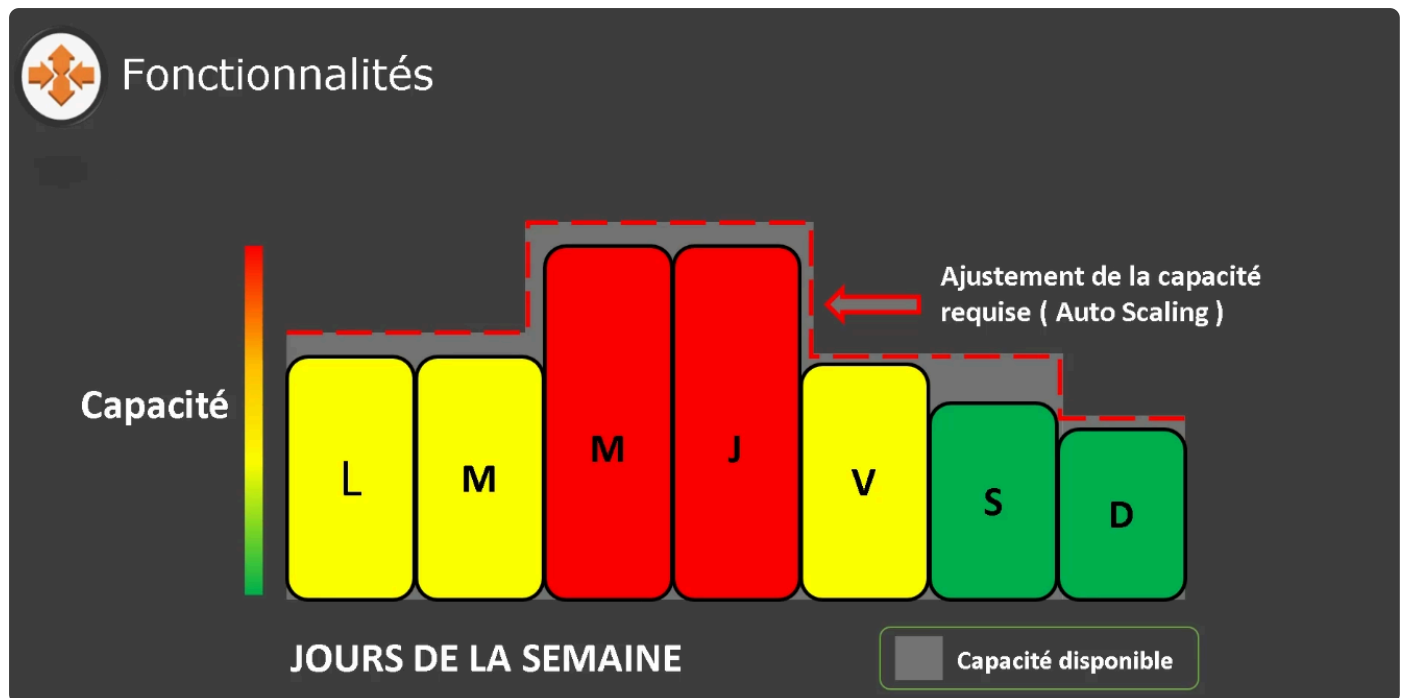
 [AWS Lambda Pricing](#)  [AWS Lambda Documentation](#)

13. Architecture complète : Illustration e-commerce

Scénario réaliste — montée en charge pendant les soldes, puis retour à la normale :

13.1 Avant les soldes (charge normale)

Les clients normaux passent par l'Application Load Balancer, qui répartit le trafic sur trois instances EC2 (EC2-1, EC2-2, EC2-3). L'Auto Scaling Group est configuré avec Min=2, Max=10, Desired=3.



13.2 Pendant les soldes (pic de trafic)

Le trafic est multiplié par 5, le CPU moyen passe à 85% — cela déclenche le scale-out : 4 instances supplémentaires sont ajoutées derrière l'Application Load Balancer, portant le total à 7 instances actives (Desired Capacity = 7).

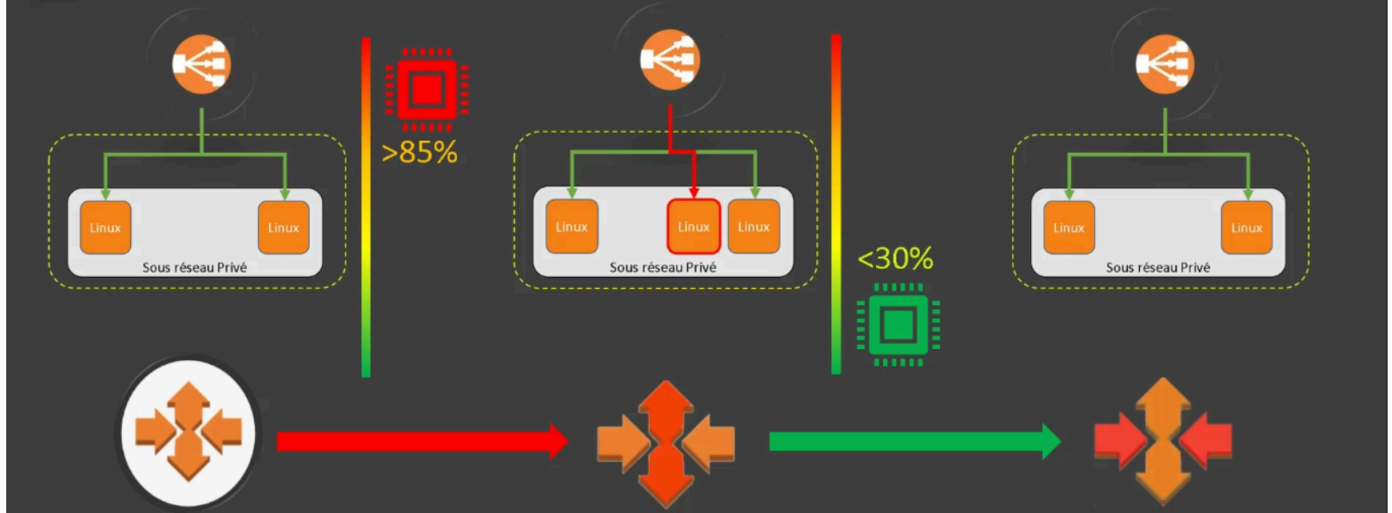


Cas D'usages

Nominale

Augmentation

Réduction



Configuration de lancement

Groupe AutoScaling

Politique d'Autoscaling

Quoi ?

- Image AMI (Linux Windows)

Où ?

- VPC et sous réseau

Quand ?

- Planifié

13.3 Après les soldes (retour à la normale)

Une fois le pic absorbé, le mécanisme symétrique de scale-in ramène progressivement l'infrastructure à son niveau normal :

Trafic revient à la normale

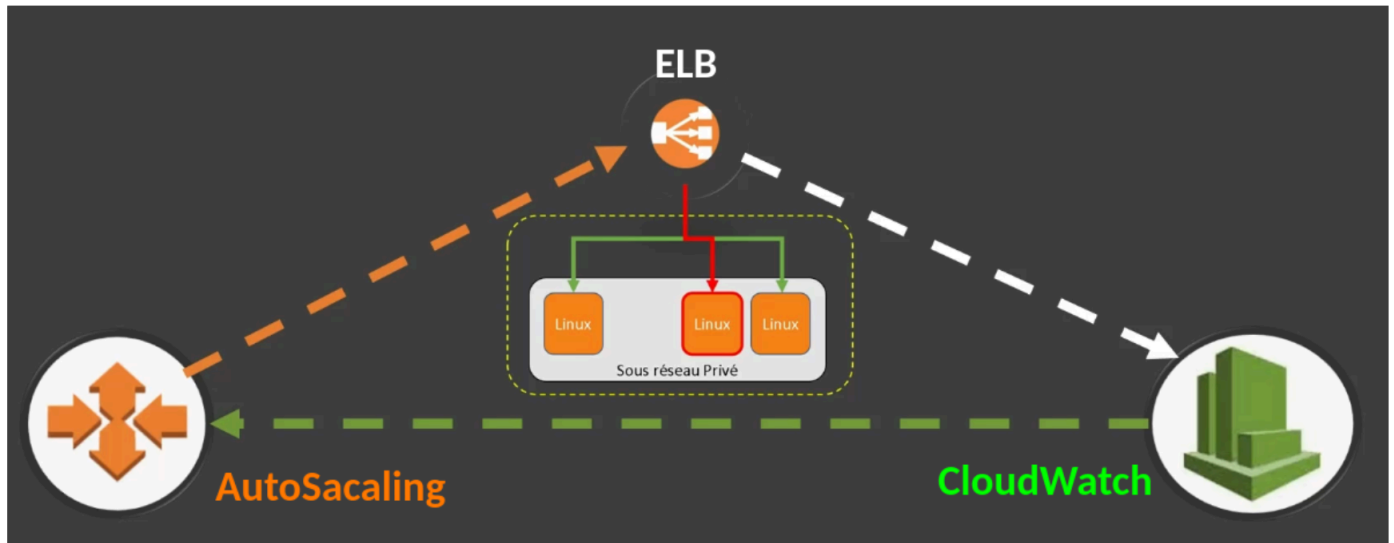
CPU chute à 40%

Déclenchement du scale-in

Retirer 4 instances

Desired Capacity = 3

Coûts réduits



14. Bonnes pratiques — Architecture hautement disponible

14.1 Architecture résiliente S3

Multi-région :

- S3 est déjà multi-AZ au sein d'une région.
- Pour une résilience maximale, activer la **réplication cross-région** (CRR).
- Utile pour respecter des exigences de conformité (RGPD, etc.).

Versioning + Lifecycle :

- Toujours activer le versioning sur les buckets de production.
- Combiner avec des règles de cycle de vie pour éviter les coûts exponentiels.
- Exemple : garder 30 versions actives, archiver le reste en Glacier.

Monitoring et alertes :

- CloudWatch pour surveiller les métriques S3.
- CloudTrail pour auditer les accès et modifications.
- S3 Access Analyzer pour vérifier les politiques d'accès.

Ces principes de résilience S3 se retrouvent, sous une forme adaptée, côté calcul EC2.

14.2 Architecture résiliente EC2

Load Balancer + Auto Scaling minimum :

- Toujours au minimum 2 instances (haute disponibilité).
- Répartir sur **plusieurs zones de disponibilité (AZ)**.
- Configurer les health checks correctement.

Sécurité en couches :

- Security Groups : bloquer les ports inutiles.
- IAM Roles : donner uniquement les permissions nécessaires.
- Subnets privés pour les instances sans accès Internet.

Sauvegardes :

- Snapshots EBS réguliers (quotidiens ou hebdomadaires).
- Externaliser les sauvegardes sur S3.
- Tester la restauration régulièrement !

Une fois la résilience assurée pour S3 et EC2, le levier suivant consiste à s'assurer que cette infrastructure ne coûte pas plus que nécessaire.

14.3 Optimisation des coûts

S3 Coûts :

- Utiliser **Intelligent-Tiering** si l'accès est imprévisible.
- Activer les **lifecycle policies** agressives pour archiver.
- Monitorer la bande passante (les téléchargements hors AWS coûtent cher).

Warning

Attention aux coûts cachés S3 — Le prix du stockage S3 Standard (~0,023 /Go-mois) est souvent bien inférieur aux coûts de **transfert sortant** (~0,09 /Go) et aux **frais de requêtes** (PUT, COPY, LIST, GET). Pour un Data Lake avec des millions d'objets, les requêtes LIST peuvent représenter une part significative de la facture. Activez **Cost Explorer** avec les tags S3 pour identifier les sources de dépenses.

EC2 Coûts :

- Utiliser **Savings Plans** pour les workloads stables (72% d'économie).

- **Spot** pour les job batch ou CI/CD tolérants aux interruptions.
- AWS Compute Optimizer pour dimensionner correctement.
- Éteindre les ressources de dev/test en fin de journée.

Estimateur AWS :

- Utiliser le **Pricing Calculator** pour estimer les coûts futurs.
- Vérifier les coûts inattendus via la **Cost Explorer**.

Coût maîtrisé et résilience assurée, il reste un dernier axe à travailler pour une architecture réellement mature : la performance à grande échelle.

14.4 Performance et scalabilité

S3 Performance :

- Utiliser des **préfixes intelligents** pour éviter les goulots d'étranglement (ex. `2024/03/24/log-xxxxx`).
- Activer **S3 Transfer Acceleration** pour les uploads volumineux (MultiPart Upload).
- CloudFront comme CDN pour la distribution worldwide.

EC2 Performance :

- **Monitoring continu** : CPU, mémoire, réseau, disque.
- **Auto Scaling** sur **multiple métriques** : CPU, mémoire, débit réseau.
- Usar **Read Replicas** pour les bases de données.
- **Connexion pooling** pour les applications critiques.

Ces bonnes pratiques de résilience, coût et performance forment la base d'une architecture AWS mature ; certains secteurs d'activité imposent en plus des garanties de conformité réglementaire spécifiques.

15. Conformité et sécurité pour les données sensibles

Les environnements soumis à des réglementations (RGPD, HIPAA, PCI-DSS) nécessitent des garanties strictes.

15.1 Frameworks de conformité AWS

Chaque référentiel impose ses propres exigences, qui s'appuient néanmoins sur un socle commun de services AWS déjà présentés dans ce chapitre :

Framework	Objectif	Services AWS applicables
RGPD	Protection des données personnelles UE	Encryption, Data Residency, CloudTrail
HIPAA	Confidentialité des données santé	Dedicated Instance, Encrypted EBS, Audit logs
PCI-DSS	Sécurité des données cartes bancaires	VPC isolé, Encryption, Firewall
ISO 27001	Gestion de la sécurité informatique	IAM, KMS, CloudTrail, Monitoring

Ces quatre référentiels se recoupent largement sur les mécanismes techniques à activer ; ce qui change surtout, c'est le niveau d'exigence et le périmètre des données couvertes.

15.2 Bonnes pratiques de conformité pour S3

Quel que soit le référentiel visé, la mise en œuvre concrète repose sur les mêmes briques techniques : chiffrement, isolation réseau et traçabilité. Côté stockage S3, cela se traduit par la liste de contrôle suivante :

- ☒ Chiffrement : SSE-KMS (clés maîtrisées)
- ☒ Versioning : actif (trace des modifications)
- ☒ Bucket Policy : restreint à IP/domaine
- ☒ Logging : S3 Access Logs dans un bucket séparé
- ☒ CloudTrail : audit API dans le compte AWS
- ☒ MFA Delete : protection contre la suppression
- ☒ Block Public : tous les accès publics bloqués
- ☒ Lifecycle : archivage des données obsolètes
- ☒ Réplication CRR : backup multi-région

Cette liste de contrôle S3 couvre le stockage ; il reste à sécuriser de la même façon les instances de calcul qui traitent ces données.

15.3 Bonnes pratiques de conformité pour EC2

Côté calcul, la liste de contrôle est différente mais suit la même logique de défense en profondeur :

- ☒ Dedicated Instance : pas de partage d'hôte physique
- ☒ EBS chiffré : SSE-KMS pour tous les volumes
- ☒ Security Group : minimaliste (moindre privilège)
- ☒ IAM Role : permissions spécifiques au rôle

- ☒ CloudWatch Agent : logs applicatifs
- ☒ VPC privé : pas d'accès Internet direct
- ☒ Snapshots EBS : conservés X années
- ☒ Patch Management : système à jour
- ☒ Monitoring : alertes sur anomalies

Ces deux listes de contrôle restent abstraites tant qu'elles ne sont pas assemblées dans une architecture concrète, ce que montre l'exemple RGD suivant.

15.4 Exemple : Architecture RGD multi-région

Pour voir comment ces éléments S3 et EC2 s'assemblent concrètement, voici un exemple d'architecture répondant aux exigences RGD avec une région de secours :

```
Région EU (Ireland)
- VPC Privé
  - Subnets privés (applications)
  - Subnet public (NAT Gateway seulement)
  - Security Group très restrictif
- S3 Bucket
  - Versioning activé
  - SSE-KMS (clé EU managée)
  - Bucket Policy : IP/IAM restrictifs
  - CloudTrail logging
- EC2 Instances
  - Dedicated Instance
  - EBS chiffré KMS
  - Snapshots quotidiens → S3
  - CloudWatch + CloudTrail

Région EU (Frankfurt) – Backup
- S3 Réplication CRR du bucket EU-Ireland
  - Préserve 7 ans (conformité)
```

Cette architecture répond à l'exigence RGD de résidence des données (tout reste en UE) tout en assurant une continuité d'activité via la réplication vers une seconde région européenne. Mettre en œuvre ces mesures ne suffit cependant pas : il faut aussi pouvoir le prouver à un auditeur.

15.5 Audit et certification

AWS Artifact : plateforme d'AWS pour les certifications de conformité. Les rapports téléchargés (SOC 2, ISO 27001) servent à prouver à vos clients ou auditeurs qu'AWS respecte les normes de sécurité.

```
1  # Dans la console AWS → Security, Identity & Compliance → Artifact
2  # Télécharger :
3  # - AWS Compliance Summary
4  # - SOC 2 Type II reports
5  # - ISO 27001 certificates
6  # Utile pour les audits externes
```

CloudTrail pour l'audit — ces commandes permettent de lister les trails actifs et de rechercher les actions effectuées sur une ressource précise (ici, un bucket S3) :

```
1  aws cloudtrail describe-trails --region eu-west-1
2  aws cloudtrail list-events \
3      --region eu-west-1 \
4      --max-results 50 \
5      --lookup-attributes AttributeKey=ResourceName,AttributeValue=mon-bucket
```



Tip

Résultat attendu :

```
# describe-trails retourne :
{
  "trailList": [
    {
      "Name": "management-events-trail",
      "S3BucketName": "my-cloudtrail-logs-bucket",
      "IncludeGlobalServiceEvents": true,
      "IsMultiRegionTrail": true,
      "HomeRegion": "eu-west-1",
      "TrailARN": "arn:aws:cloudtrail:eu-west-1:123456789012:trail/management-events-trail",
      "LogFileValidationEnabled": true
    }
  ]
}

# list-events retourne des événements du type :
{
  "Events": [
    {
      "EventId": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
      "EventName": "PutObject",
      "ReadOnly": "false",
      "EventTime": "2024-05-17T14:23:05+00:00",
      "Username": "stagiaire-demo",
      "Resources": [
        {
          "ResourceType": "AWS::S3::Object",
          "ResourceName": "mon-bucket/documents/rapport.pdf"
        }
      ]
    }
  ]
}
```

16. Points importants et pièges fréquents

Ce chapitre a couvert S3, EC2, la tarification, le Load Balancing, Auto Scaling et Lambda ; voici les idées reçues les plus fréquentes chez les débutants sur l'ensemble de ces sujets.

Piège	Réalité	Conséquence
S3 a une structure de dossiers	Non ! C'est du stockage objet, les "dossiers" sont juste des préfixes dans les noms	Impossible de renommer les dossiers, penser en clés, pas en hiérarchies
Versioning S3 ne prend pas de place supplémentaire	Faux ! Chaque version est stockée complètement	Les coûts explosent vite si vous versionnez des fichiers volumineux
Les instances EC2 garderont leurs données après arrêt	Seulement si vous utilisez EBS persistant	Les données en instance store (stockage éphémère) sont perdues à l'arrêt
On-Demand est la meilleure option tarifaire	Non, c'est la plus chère	Reserved / Spot / Savings Plans peuvent économiser 70-90%
Auto Scaling remplace les instances défaillantes instantanément	Non, il faut le temps de démarrage (2-5 min)	Configurer les health checks correctement et accepter un délai
Toute IP EC2 est durable	Non, les IPs publiques changent à l'arrêt/redémarrage	Utiliser Elastic IP pour les IPs stables ou les DNS
Un Security Group "ouvert" (0.0.0.0/0) sur tous les ports est OK si la machine n'a rien à cacher	Non ! C'est une faille de sécurité	Les scanners de ports peuvent découvrir la machine, minimiser l'exposition
EBS et S3 sont interchangeables	Non ! EBS est un disque (bloc), S3 est du stockage objet	Choisir le bon service selon le cas d'usage

Réseau, données, calcul et stockage sont posés. Reste à ne plus tout faire à la main : le Chapitre 5 automatise ce déploiement avec CloudFormation, pour que la même architecture puisse être recréée à l'identique en une commande.

17. Ressources

Documentation officielle AWS

Pour approfondir les services de stockage et de calcul présentés dans ce chapitre, voici les références officielles correspondantes :

- [AWS Compute Optimizer](#) 
- [CloudTrail Documentation](#) 
- [AWS Compliance](#) 
- [Artifact Console](#) 
- [S3 Intelligent-Tiering](#) 
- [S3 Transfer Acceleration](#) 
- [Bucket Policies Guide](#) 
- [AWS Lambda Documentation](#) 

Le quiz ci-dessous permet de vérifier l'acquisition des notions clés de ce chapitre avant de passer au suivant.

Quiz interactif du chapitre

Choisissez une réponse : la correction expliquée apparaît immédiatement. Les propositions changent d'ordre à chaque nouvelle tentative.

Le quiz interactif reste disponible dans la version web du support.

Mini-jeu de synthèse — AWS Quest

Résolvez cinq décisions factuelles pour consolider les fondamentaux avant d'ouvrir le chapitre suivant. Le jeu est une simulation locale : il ne se connecte à aucun compte AWS.

Le quiz interactif reste disponible dans la version web du support.

Info

Le réseau et les bases de données mis en place au chapitre précédent constituent une architecture fonctionnelle mais encore déployée manuellement ; ce dernier chapitre refferme la formation en automatisant ce déploiement et en donnant les clés pour évaluer et faire évoluer une architecture AWS dans la durée.

Objectifs du chapitre

À l'issue de ce chapitre, les stagiaires seront capables de :

- Définir RTO/RPO et mettre en œuvre une stratégie de sauvegarde (AWS Backup, snapshots EBS/RDS)
- Expliquer l'intérêt de l'Infrastructure as Code et écrire/déployer un template CloudFormation
- Utiliser AWS Systems Manager et déployer une application avec Elastic Beanstalk
- Superviser une infrastructure avec CloudWatch et appliquer le Well-Architected Framework
- Découpler une architecture (SQS, SNS, API Gateway, Step Functions) et situer les certifications AWS

1. RTO/RPO et Récupération de Sauvegarde

1.1 Définitions essentielles

Avant d'automatiser une infrastructure, il faut comprendre deux concepts critiques pour la **continuité de service** :

RTO (Recovery Time Objective) — Temps d'Indisponibilité Acceptable

RTO = combien de temps maximum l'application peut-elle rester indisponible avant que l'impact métier devienne intolérable ?

Exemples concrets :

Service	RT0 typical	Raison
-----	-----	-----
Site e-commerce (Amazon)	5 minutes	Chaque minute sans vente = perte
Application interne (RH)	8 heures	Métier critique mais moins urgent
Service vidéo (Netflix)	30 minutes	Perte d'utilisateurs, mais pas urgent
Système bancaire	15 minutes	Réglementation stricte
API partenaire (non-vital)	4 heures	Impact mineur sur le business

Le RTO répond à la question du temps ; un second indicateur, tout aussi structurant, répond à la question de la donnée perdue.

RPO (Recovery Point Objective) — Quantité de Données Perdable

RPO = combien de données suis-je prêt à perdre en cas de sinistre ?

Exemples concrets :

Application	RPO	Raison
-----	-----	-----
E-commerce actif	5 minutes	Transactions en temps réel = critique
Logs d'application	1 jour	Données historiques, non urgentes
Données de client (CRM)	1 heure	Important pour relancer les clients
Backups archivés	30 jours	Archive long terme, peu critique

RTO et RPO ne se choisissent pas indépendamment : ils s'associent en stratégies cohérentes, dont le coût grimpe à mesure que les deux délais se raccourcissent.

Relation RTO ↔ RPO

Voici trois stratégies types, du plus réactif (et coûteux) au plus économique (et lent à restaurer) :

Scénario : Serveur RDS tombe en panne à 10:00

Stratégie 1 (RTO court, RPO court)

- Sauvegarde automatique toutes les 10 minutes
- Multi-AZ activé (failover < 2 minutes)
- RTO = 2 minutes, RPO = 10 minutes
- Coût : ★★★★★ (cher)

Stratégie 2 (RTO moyen, RPO moyen)

- Sauvegarde quotidienne (minuit)
- Backup lisible rapidement (1 heure pour restaurer)
- RTO = 1 heure, RPO = 24 heures
- Coût : ★★ (raisonnable)

Stratégie 3 (RTO long, RPO long)

- Sauvegarde hebdomadaire
- Pas de failover automatique
- RTO = 8 heures, RPO = 7 jours
- Coût : ★ (très bon marché)

Ces trois stratégies sont génériques ; voyons maintenant comment elles se traduisent concrètement en services AWS.

1.2 Stratégies AWS pour Atteindre RTO/RPO

Chaque technologie AWS vue dans les chapitres précédents se positionne différemment sur l'échelle RTO/RPO/coût :

Technologie	RTO	RPO	Coût	Cas d'usage
Multi-AZ	< 2 min	≈ 0 min	Moyen	Haute dispo critique
Snapshots EBS	15-30 min	1 jour	Faible	Backup régulier
AWS Backup	1-4 heures	1-24 heures	Faible-Moyen	Backup centralisé
Read Replicas (RDS)	5-10 min	≈ 0 min	Moyen	Failover rapide BD
AWS Glacier	1-12 heures	Sans limite	Très faible	Archive long terme
Lambda + S3	10-60 min	1 heure	Très faible	Backup custom

Dans la pratique, ces technologies se combinent plutôt qu'elles ne s'excluent : Multi-AZ pour l'instantané, Snapshots ou AWS Backup pour la profondeur d'historique, Glacier pour l'archivage réglementaire à long terme. AWS Backup a justement pour rôle de centraliser la gestion de plusieurs de ces mécanismes.

1.3 AWS Backup — Service Centralisé de Sauvegarde

AWS Backup est un **service managé** pour centraliser et automatiser les sauvegardes de ressources AWS.

Ressources sauvegardables par AWS Backup

Plutôt que de configurer une sauvegarde différente pour chaque service, AWS Backup couvre en un seul endroit la plupart des ressources vues dans les chapitres précédents :

Compute & Storage:

- ✓ Amazon EC2 instances
- ✓ Amazon EBS volumes
- ✓ Amazon EFS (Elastic File System)

Database:

- ✓ Amazon RDS databases (MySQL, PostgreSQL, Oracle, SQL Server)
- ✓ Amazon DynamoDB tables
- ✓ Amazon Aurora databases

Backup Store:

- ✓ AWS Storage Gateway
- ✓ VMware vSphere

Cette couverture large est précisément ce qui justifie l'existence d'AWS Backup : centraliser en un seul plan de sauvegarde des ressources qui, sans lui, nécessiteraient chacune leur propre mécanisme (snapshots EBS manuels, exports DynamoDB, etc.).

Déployer AWS Backup via CLI

On crée d'abord un **Backup Vault** (le coffre où seront stockées les sauvegardes), puis un **Backup Plan** (la planification) avec ses règles de rétention, et enfin une **Backup Selection** (les ressources à sauvegarder). Ces trois objets forment un système complet.

```

1  # 1. Créer un Backup Vault (conteneur pour les sauvegardes)
2  aws backup create-backup-vault \
3      --backup-vault-name mon-vault-production \
4      --region eu-west-3
5
6  # Output : BackupVaultArn
7
8  # 2. Créer un Backup Plan (planification automatique)
9  # Sauvegarder toutes les instances EC2 chaque jour à minuit
1
0  cat > backup-plan.json << 'EOF'
1
1  {
1
2      "BackupPlanName": "SauvegardeQuotidienne",
1
3      "Rules": [
1
4          {
1
5              "RuleName": "Sauvegarde Quotidienne",
1
6              "TargetBackupVaultName": "mon-vault-production",
1
7              "ScheduleExpression": "cron(0 0 ? * * *)",
1
8              "StartWindowMinutes": 60,
1
9              "CompletionWindowMinutes": 120,
2
0              "Lifecycle": {
2
1                  "DeleteAfterDays": 30,
2
2                  "MoveToColdStorageAfterDays": 7
2
3              }

```

```

2
4     }
2
5 ]
2
6 }
2
7 EOF
2
8
2
9 # 3. Créer le plan
3
0 aws backup create-backup-plan \
3
1     --backup-plan file://backup-plan.json \
3
2     --region eu-west-3

```



Tip

Résultat attendu :

```

{
  "BackupPlanId": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
  "BackupPlanArn": "arn:aws:backup:eu-west-3:123456789012:backup-
plan:a1b2c3d4-e5f6-7890-abcd-ef1234567890",
  "CreationDate": "2026-03-24T10:15:00.000Z",
  "VersionId": "NDEzMWV1NzgtMTk2My00NjMxLWJlOTYt"
}

```



```

1 # 4. Assigner des ressources au plan
2 aws backup create-backup-selection \
3     --backup-plan-id mon-plan-123 \
4     --backup-selection file://selection.json \
5     --region eu-west-3
6
7 # 5. Vérifier les backups créés
8 aws backup list-recovery-points-by-resource \
9     --resource-arn "arn:aws:ec2:eu-west-3:123456789:instance/i-12345678" \
10
11     --region eu-west-3
12
13
14
15
16 # 6. Restaurer une instance EC2 à partir d'un backup
17
18 aws backup start-restore-job \
19
20     --recovery-point-arn "arn:aws:backup:eu-west-3:123456789:recovery-point:..." \
21
22     --iam-role-arn "arn:aws:iam::123456789:role/AWSBackupDefaultRole" \
23
24     --metadata key1=value1,key2=value2 \
25
26     --region eu-west-3

```



Tip

Résultat attendu :

```
# list-recovery-points-by-resource :
{
  "RecoveryPoints": [
    {
      "RecoveryPointArn": "arn:aws:backup:eu-west-3:123456789012:recovery-point:abc123",
      "CreationDate": "2026-03-24T00:05:12.000Z",
      "Status": "COMPLETED",
      "BackupSizeInBytes": 8589934592,
      "BackupVaultName": "mon-vault-production"
    }
  ]
}

# start-restore-job :
{
  "RestoreJobId": "restore-job-0abc123def456"
}
```

Warning

Coûts AWS Backup : Le stockage des sauvegardes est facturé selon le volume stocké (~0,05 /Go/mois en stockage chaud). Le déplacement vers Glacier (cold storage) réduit le coût à ~0,01 /Go/mois après 7 jours. Vérifiez régulièrement les coûts dans **Cost Explorer** et ajustez les politiques de rétention.

Bonnes pratiques AWS Backup

Pour conclure sur AWS Backup, voici les réflexes qui distinguent une politique de sauvegarde réellement fiable d'une simple case cochée :

- ✓ Planifier les sauvegardes en dehors des heures de pic
- ✓ Utiliser des Backup Vaults séparés pour Prod/Staging/Dev
- ✓ Tester régulièrement les restaurations (RTO réel)
- ✓ Définir une rétention appropriée (30j prod, 7j dev)
- ✓ Combiner avec CloudWatch Events pour alertes

Le point le plus souvent négligé est le test de restauration : une sauvegarde jamais restaurée n'est qu'une hypothèse — c'est seulement en la testant qu'on connaît le RTO réel, et pas seulement celui annoncé sur le papier.

1.4 Snapshots EBS et RDS — Sauvegardes Point-in-Time

Snapshot EBS = photo point-in-time d'un volume EC2

Un snapshot EBS capture l'état d'un disque à un instant donné. Il est stocké dans S3 (géré par AWS) et peut servir à restaurer un volume ou à déplacer une instance vers une autre région.

```
1 # Créer un snapshot EBS manuel
2 aws ec2 create-snapshot \
3   --volume-id vol-12345678 \
4   --description "Sauvegarde avant migration" \
5   --region eu-west-3
6
7 # Lister les snapshots
8 aws ec2 describe-snapshots \
9   --owner-ids self \
10  --region eu-west-3
11
12 # Créer un volume à partir du snapshot
13 aws ec2 create-volume \
14   --snapshot-id snap-12345678 \
15   --availability-zone eu-west-3a \
16   --region eu-west-3
```



Tip

Résultat attendu :

```
# create-snapshot :
{
  "SnapshotId": "snap-0abc123def456789a",
  "VolumeId": "vol-12345678",
  "State": "pending",
  "StartTime": "2026-03-24T09:00:00.000Z",
  "Progress": "0%",
  "Description": "Sauvegarde avant migration"
}

# describe-snapshots (après quelques minutes) :
{
  "Snapshots": [
    {
      "SnapshotId": "snap-0abc123def456789a",
      "State": "completed",
      "Progress": "100%",
      "VolumeSize": 20
    }
  ]
}
```

Snapshot RDS = sauvegarde complète de base de données

```
1 # Créer un snapshot RDS manuel
2 aws rds create-db-snapshot \
3   --db-instance-identifier production-db \
4   --db-snapshot-identifier prod-db-backup-2026-03-24 \
5   --region eu-west-3
6
7 # Lister les snapshots
8 aws rds describe-db-snapshots \
9   --region eu-west-3
10
11
12 # Restaurer une BD à partir d'un snapshot
13
14 aws rds restore-db-instance-from-db-snapshot \
15   --db-instance-identifier production-db-restored \
16   --db-snapshot-identifier prod-db-backup-2026-03-24 \
17   --region eu-west-3
```



Tip

Résultat attendu :

```
# create-db-snapshot :
{
  "DBSnapshot": {
    "DBSnapshotIdentifier": "prod-db-backup-2026-03-24",
    "DBInstanceIdentifier": "production-db",
    "Status": "creating",
    "Engine": "mysql",
    "AllocatedStorage": 100,
    "SnapshotCreateTime": "2026-03-24T09:30:00.000Z"
  }
}

# describe-db-snapshots (après quelques minutes) :
{
  "DBSnapshots": [
    {
      "DBSnapshotIdentifier": "prod-db-backup-2026-03-24",
      "Status": "available",
      "PercentProgress": 100
    }
  ]
}
```

Résultat attendu : `create-db-snapshot` retourne un JSON avec le `DBSnapshotIdentifier` et le statut `creating`. Quelques minutes plus tard, `describe-db-snapshots` affiche le statut `available`. La restauration (`restore-db-instance-from-db-snapshot`) crée une **nouvelle instance** RDS — pas un remplacement de l'existante.

2. Pourquoi automatiser dans le Cloud ?

2.1 Le déploiement manuel : une source d'erreurs

Dans un environnement traditionnel, déployer une infrastructure peut prendre **des jours, voire des semaines**. Les équipes IT doivent :

- Commander du matériel physique

- Installer les systèmes d'exploitation
- Configurer le réseau et les accès
- Mettre à jour les pare-feu et les politiques de sécurité
- Documenter tous les changements (quand c'est fait...)

Le problème : chaque déploiement manuel génère des **incohérences**. Deux administrateurs ne font jamais exactement la même chose. Certains oublis passent inaperçus :

Infrastructure créée manuellement :

Admin Alice crée un VPC le 10 janvier
 → Configure 2 subnets, 1 IGW, 1 NAT Gateway
 → Oublie de documenter les CIDR utilisés
 → Quitte l'entreprise 6 mois après

Admin Bob doit déployer l'infrastructure
 → Cherche la documentation (introuvable)
 → Recrée un nouveau VPC similaire MAIS différent
 → Incompatibilité lors du peering : PERTE DE TEMPS

Résultat : deux "mêmes" infrastructures qui ne sont PAS identiques

Ce scénario, banal dans beaucoup d'organisations, illustre pourquoi le Cloud a besoin d'un mécanisme qui élimine la dépendance à la mémoire humaine et à une documentation à jour.

2.2 L'automatisation dans le Cloud : standardisation et rapidité

Dans le cloud AWS, l'**automatisation** permet de :

- **Standardiser** les environnements → Prod et Dev sont identiques
- **Réduire** les délais de déploiement → Quelques minutes au lieu de semaines
- **Limiter** les erreurs humaines → Le code est testé avant déploiement
- **Faciliter** la montée en charge → Spawner 100 instances avec un clic

Infrastructure automatisée avec CloudFormation :

Étape 1 : Écrire un template YAML (1-2 heures)
 Étape 2 : Déployer en Prod exactement identique au Dev (2 minutes)
 Étape 3 : Déployer en Test (2 minutes)
 Étape 4 : Dupliquer pour un client différent (2 minutes)

Avantage : zéro divergence entre les environnements
 zéro oubli de configuration
 traçabilité complète (versionning Git du template)

CloudFormation n'est cependant qu'un outil parmi plusieurs dans la boîte à outils AWS dédiée à l'automatisation.

2.3 Les outils AWS pour l'automatisation

AWS propose plusieurs outils complémentaires pour automatiser une infrastructure, chacun adapté à un niveau différent :

Outil	Fonction	Cas d'usage
CloudFormation	Déploiement d'infrastructures as code (IaC)	Créer VPC, EC2, RDS, S3 en une seule opération
Quick Starts	Templates CloudFormation préconfigurés par AWS	Déployer rapidement une architecture éprouvée (VPC + ELB + RDS)
Systems Manager	Gestion centralisée et automatisation opérationnelle	Exécuter des scripts, patcher les serveurs, inventorier les ressources
Elastic Beanstalk	Déploiement PaaS simplifié d'applications web	Déployer une app Node.js sans gérer l'infrastructure
CLI / SDK	Automatisation par scripts et développement	Orchestrer plusieurs services AWS via Python/Bash

Référence : [AWS Automation Overview](#) Référence : [AWS CloudFormation Documentation](#)

2.4 AWS Quick Starts — Templates Éprouvés

AWS Quick Starts sont des **templates CloudFormation préconfigurés et validés par AWS** pour déployer des architectures complètes en quelques clics.

Qu'est-ce qu'un Quick Start ?

Un Quick Start n'est rien d'autre qu'un template CloudFormation déjà écrit et testé par AWS, que vous n'avez qu'à paramétrer :

Quick Start = CloudFormation template complet + documentation + bonnes pratiques

Exemplar :

- Déployer une architecture WordPress hautement disponible (VPC + ALB + Auto Scaling + RDS Multi-AZ + CloudFront)
- Sans avoir à écrire 500 lignes YAML
- Basé sur des best practices AWS
- Testé et validé en production

Ce principe s'applique à de nombreuses architectures de référence, comme le montrent les exemples courants ci-dessous.

Quick Starts courants (exemples)

Ce principe s'applique à de nombreuses architectures de référence, comme le montrent ces exemples courants :

Quick Start	Qu'il déploie	Temps
WordPress on AWS	VPC, ALB, Auto Scaling, RDS, CloudFront	10-15 min
Kubernetes on AWS	EKS cluster complet avec worker nodes	20-30 min
SQL Server on EC2	Instance EC2 + RDS SQL Server + backup	15 min
Jenkins on AWS	Jenkins Master + Agent instances + monitoring	10 min
Hadoop on AWS	Cluster EMR complet multi-node	20 min

EKS (Elastic Kubernetes Service, déjà présenté au Chapitre 1 section 5) est le service AWS de Kubernetes managé.

Hadoop est un framework open source historique de traitement de données massives (Big Data) : il répartit le calcul et le stockage sur un grand nombre de machines pour traiter des volumes trop importants pour un seul serveur. **EMR** (Elastic MapReduce) est le service managé AWS qui déploie et exploite des clusters Hadoop (ainsi que des outils compatibles comme Spark) sans que vous ayez à installer et administrer vous-même les serveurs du cluster.

Accéder aux Quick Starts

Les Quick Starts sont accessibles depuis la console CloudFormation ou directement via CLI. Voici comment les trouver et les utiliser :

```
1 # 1. Via la console AWS → CloudFormation → Quick Starts
2 #   https://aws.amazon.com/quickstarts/
3
4 # 2. Chercher un Quick Start par domaine :
5 #   - VPC / Networking
6 #   - Databases & Analytics
7 #   - Business Applications
8 #   - DevOps Tools
9
1
0 # 3. Cliquer sur "Launch" → remplir les paramètres → Create Stack
```

Une fois le Quick Start repéré et lancé, les bénéfices concrets se retrouvent dans le résultat final plutôt que dans la procédure elle-même.

Avantages des Quick Starts

Ces bénéfices se résument en six points, qui expliquent pourquoi il est rarement judicieux de repartir de zéro quand une architecture de référence existe déjà :

- ✓ Gain de temps : architecture complète en 15 min au lieu de 3-4 heures
- ✓ Bonnes pratiques : design conforme au Well-Architected Framework
- ✓ Haute disponibilité : Multi-AZ, load balancing, failover automatique
- ✓ Maintenance : AWS met à jour les templates régulièrement
- ✓ Support : documentation et troubleshooting fournis
- ✓ Flexibilité : vous pouvez modifier les templates après déploiement

Voyons maintenant ces avantages à l'œuvre sur un cas concret : le déploiement d'un site WordPress complet.

Exemple : Déployer WordPress via Quick Start

Plutôt que de créer manuellement VPC, ALB, RDS et Auto Scaling, on lance une seule commande CloudFormation qui déploie toute l'architecture WordPress en ~10 minutes :

```

1  # 2. Chercher "WordPress"
2  # 3. Cliquer sur "Launch Quick Start"
3  # 4. Remplir les paramètres (taille VPC, taille RDS, DNS, etc.)
4  # 5. Vérifier les options VPC et subnet
5  # 6. Cliquer "Create Stack"
6  # 7. Attendre ~10 minutes
7  # 8. CloudFormation affiche l'URL du site WordPress
8
9  # Vous pouvez aussi utiliser AWS CLI :
1
0  aws cloudformation create-stack \
1
1  --stack-name wordpress-production \
1
2  --template-url "https://s3.amazonaws.com/quickstart-
reference/wordpress/latest/templates/wordpress.yaml" \
1
3  --parameters \
1
4  ParameterKey=KeyName,ParameterValue=ma-clé-ssh \
1
5  ParameterKey=InstanceType,ParameterValue=t3.small \
1
6  ParameterKey=DBInstanceClass,ParameterValue=db.t3.micro \
1
7  --region eu-west-3

```



Tip

Résultat attendu :

```
1 {  
2     "StackId": "arn:aws:cloudformation:eu-west-  
3:123456789012:stack/wordpress-production/a1b2c3d4-e5f6-7890-abcd-ef1234567890"  
3 }
```

La stack `wordpress-production` est en cours de création. Suivez la progression dans la console CloudFormation → Events, ou via `aws cloudformation describe-stack-events --stack-name wordpress-production`. Après ~10 minutes, le statut passe à `CREATE_COMPLETE` et l'URL WordPress est disponible dans les outputs de la stack.

3. AWS CloudFormation — Infrastructure as Code

3.1 Qu'est-ce que CloudFormation ?

AWS CloudFormation est un service qui permet de **modéliser et déployer des ressources AWS** sous forme de code.

Plutôt que de créer manuellement une VPC, des sous-réseaux, des groupes de sécurité ou des instances EC2 dans la console, on les **décrit dans un template JSON ou YAML** et CloudFormation s'occupe du reste.

Analogie : CloudFormation est comme une RECETTE DE CUISINE pour construire une infrastructure

Recette classique :

Ingrédients : 1 VPC, 2 subnets, 1 IGW, 3 EC2

Étapes :

1. Créer la VPC avec CIDR 10.0.0.0/16
2. Créer subnet public 10.0.1.0/24
3. Créer subnet privé 10.0.2.0/24
4. Ajouter une IGW et l'attacher à la VPC
5. Créer 3 instances EC2 dans le subnet public
6. Configurer les groupes de sécurité

Avantage : la recette peut être réutilisée 100 fois identiquement et versionnée dans Git

Concrètement, cette « recette » se traduit par un cycle de vie en cinq étapes, de la rédaction du template à la suppression de la stack.

3.2 Fonctionnement simplifié de CloudFormation

1. **Rédiger un template** (JSON/YAML) décrivant les ressources à créer
2. **Créer une stack** dans CloudFormation (via console ou CLI)
3. **AWS provisionne** toutes les ressources **dans le bon ordre** (résout les dépendances automatiquement)
4. **Mettre à jour** la stack pour faire évoluer l'infrastructure (ajout, suppression, modification de ressources)
5. **Supprimer** la stack si plus besoin (CloudFormation supprime toutes les ressources associées)

Cette étape 3 (« AWS provisionne dans le bon ordre ») repose sur une propriété centrale de CloudFormation :

l'**idempotence**. Relancer un déploiement avec un template inchangé ne doit rien recréer, puisque l'état réel du compte correspond déjà à l'état déclaré dans le template — CloudFormation compare avant d'agir, il n'exécute pas bêtement la liste des ressources à chaque appel. C'est ce qui rend un déploiement CloudFormation reproductible sans effet de bord : l'exécuter une fois ou dix fois produit le même résultat final.



3.3 Exemple simple de template CloudFormation (YAML)

Voici un template minimaliste créant une VPC, un subnet, et une instance EC2 :

```

1  # Version du format CloudFormation (toujours 2010-09-09)
2  AWSTemplateFormatVersion: '2010-09-09'
3
4  # Description brève du template
5  Description: |
6      Infrastructure AWS simple pour débutants
7      Crée une VPC, un subnet public, une IGW et une instance EC2
8
9  # Paramètres (permet de rendre le template réutilisable)
10
11 # Ici, on paramètre le type d'instance EC2 pour pouvoir changer facilement
12
13 Parameters:
14
15     InstanceType:
16
17         Type: String
18
19         Default: t2.micro
20
21         Description: Type d'instance EC2 (t2.micro, t2.small, etc.)
22
23         AllowedValues:
24
25             - t2.micro
26
27             - t2.small
28
29             - t3.micro
30
31
32 # Les ressources à créer
33
34 Resources:
35
36     # Ressource 1 : Créer une VPC

```

```

2
4  # Chaque ressource a un identifiant logique (MonVPC) et un type AWS
2
5  MonVPC:
2
6      # Type de ressource AWS
2
7      Type: AWS::EC2::VPC
2
8      # Propriétés spécifiques
2
9      Properties:
3
0          # CIDR block : plage d'adresses IP pour cette VPC
3
1          CidrBlock: 10.0.0.0/16
3
2          # Activer le hostname DNS
3
3          EnableDnsHostnames: true
3
4          # Tags pour identifier facilement
3
5          Tags:
3
6              - Key: Name
3
7                Value: MonVPC-Formation
3
8
3
9  # Ressource 2 : Créer un subnet public dans la VPC
4
0  # !Ref est une fonction intrinsèque qui référence une autre ressource
4
1  MonSubnetPublic:
4
2      Type: AWS::EC2::Subnet

```

Properties:

`# !Ref MonVPC = ID logique de la VPC créée au-dessus`

`VpcId: !Ref MonVPC`

`# Plage d'adresses pour ce subnet (doit être dans le CIDR de la VPC)`

`CidrBlock: 10.0.1.0/24`

`# Zone de disponibilité (peut varier, AWS en assigne une par défaut)`

`AvailabilityZone: eu-west-1a`

`# Assigner automatiquement une IP publique aux instances`

`MapPublicIpOnLaunch: true`

Tags:

`- Key: Name`

`Value: MonSubnet-Public`

`# Ressource 3 : Créer une Internet Gateway (permet l'accès à Internet)`

MonIGW:

`Type: AWS::EC2::InternetGateway`

Properties:

Tags:

`- Key: Name`


```

6
2      Value: MonIGW
6
3
6
4  # Ressource 4 : Attacher l'IGW à la VPC
6
5  # CloudFormation comprend automatiquement que cette ressource dépend de
MonVPC et MonIGW
6
6  AttachIGW:
6
7      Type: AWS::EC2::VPCGatewayAttachment
6
8      Properties:
6
9          # Référence à la VPC créée
7
0      VpcId: !Ref MonVPC
7
1          # Référence à l'IGW créé
7
2      InternetGatewayId: !Ref MonIGW
7
3
7
4  # Ressource 5 : Groupe de sécurité (pare-feu simplifié)
7
5  MonSecurityGroup:
7
6      Type: AWS::EC2::SecurityGroup
7
7      Properties:
7
8          # Description obligatoire
7
9      GroupDescription: Autorise SSH et HTTP
8
0      # Associer au VPC

```

```
8
1  VpcId: !Ref MonVPC
8
2  # Règles de trafic entrant
8
3  SecurityGroupIngress:
8
4      # Permettre SSH depuis n'importe quelle IP (⚠ À ÉVITER EN PROD)
8
5      - IpProtocol: tcp
8
6        FromPort: 22
8
7        ToPort: 22
8
8        CidrIp: 0.0.0.0/0
8
9        Description: SSH access
9
0      # Permettre HTTP
9
1      - IpProtocol: tcp
9
2        FromPort: 80
9
3        ToPort: 80
9
4        CidrIp: 0.0.0.0/0
9
5        Description: HTTP access
9
6  Tags:
9
7      - Key: Name
9
8        Value: MonSG-Formation
9
9
```

```

1
0
0   # Ressource 6 : Instance EC2
1
0
1   MonInstance:
1
0
2   Type: AWS::EC2::Instance
1
0
3   Properties:
1
0
4       # AMI ID (Ubuntu 24.04 LTS en eu-west-1)
1
0
5       ImageId: ami-0d71ea30463e0ff8d
1
0
6       # Type d'instance (utilise le paramètre défini au-dessus)
1
0
7       InstanceType: !Ref InstanceType
1
0
8       # Placer dans le subnet créé
1
0
9       SubnetId: !Ref MonSubnetPublic
1
1
0       # Associer le security group
1
1
1       SecurityGroupIds:
1
2       - !Ref MonSecurityGroup

```

```
1
1
3     # Script à exécuter au démarrage (user data)
1
1
4     UserData:
1
1
5     Fn::Base64: |
1
1
6     #!/bin/bash
1
1
7     # Mises à jour système
1
1
8     apt-get update
1
1
9     apt-get install -y nginx curl
1
2
0     # Démarrer Nginx
1
2
1     systemctl start nginx
1
2
2     systemctl enable nginx
1
2
3     # Créer une page de test
1
2
4     echo "<h1>Bonjour du serveur créé par CloudFormation</h1>" >
/var/www/html/index.html
```

```
1
2
5     Tags:
1
2
6         - Key: Name
1
2
7         Value: MonServeur-Formation
1
2
8
1
2
9 # Outputs : affiche les résultats après déploiement
1
3
0 # Utile pour récupérer les IP, URLs, etc.
1
3
1 Outputs:
1
3
2     # Affiche l'ID de la VPC créée
1
3
3     VPCId:
1
3
4     Description: ID de la VPC
1
3
5     # !Ref récupère l'ID physique de la ressource
1
3
6     Value: !Ref MonVPC
1
3
7     # Export permet à d'autres stacks de référencer cette valeur
```

```

1
3
8   Export:
1
3
9       Name: MonVPC-Id
1
4
0
1
4
1   # Affiche l'IP publique de l'instance
1
4
2   PublicIP:
1
4
3       Description: Adresse IP publique de l'instance EC2
1
4
4       # !GetAtt récupère un attribut spécifique d'une ressource
1
4
5       Value: !GetAtt MonInstance.PublicIp
1
4
6   Export:
1
4
7       Name: MonServeur-PublicIP
1
4
8
1
4
9   # Affiche l'URL HTTP pour accéder au serveur
1
5
0   WebServerURL:

```

```
1
5
1   Description: URL pour accéder au serveur web
1
5
2   # !Sub remplace les variables ${...} par leurs valeurs
1
5
3   Value: !Sub 'http://${MonInstance.PublicIp}'
```

Ce template minimaliste illustre la structure de base (Parameters, Resources, Outputs) mais reste volontairement simplifié à des fins pédagogiques. Voici maintenant une version plus proche de ce qu'on rencontre réellement en production.

3.3 (suite) — Exemple Production : VPC + Serveur Web Apache

Voici un template **production-ready** déployant une **VPC complète avec serveur web Apache** et un Security Group. C'est le template utilisé dans les ateliers Dawan.

```

1  AWSTemplateFormatVersion: '2010-09-09'
2  Description: |
3      Déploiement d'une VPC + serveur web Apache via CloudFormation
4      - VPC avec CIDR 10.0.0.0/16
5      - 1 subnet public (10.0.1.0/24)
6      - Internet Gateway + route vers l'extérieur
7      - Security Group (SSH + HTTP)
8      - Instance EC2 t3.micro avec Apache2 automatiquement installé
9      - Output : URL publique du serveur web
1
0
1
1  # Paramètres pour rendre le template réutilisable
1
2  Parameters:
1
3      KeyPairName:
1
4          Type: AWS::EC2::KeyPair::KeyName
1
5          Description: Nom de la paire de clés EC2 existante (pour SSH)
1
6
1
7      InstanceType:
1
8          Type: String
1
9          Default: t3.micro
2
0          AllowedValues:
2
1              - t3.micro
2
2              - t3.small
2
3              - t2.micro

```



```

2
4     Description: Type d'instance EC2
2
5
2
6 # Les ressources à créer
2
7 Resources:
2
8     # VPC
2
9     FormationVPC:
3
0         Type: AWS::EC2::VPC
3
1         Properties:
3
2             CidrBlock: 10.0.0.0/16
3
3             EnableDnsSupport: true
3
4             EnableDnsHostnames: true
3
5             Tags:
3
6                 - Key: Name
3
7                   Value: Formation-VPC-WebLab
3
8
3
9     # Subnet public
4
0     PublicSubnet:
4
1         Type: AWS::EC2::Subnet
4
2         Properties:

```

```

4
3     VpcId: !Ref FormationVPC
4
4     CidrBlock: 10.0.1.0/24
4
5     AvailabilityZone: eu-west-3a
4
6     MapPublicIpOnLaunch: true
4
7     Tags:
4
8         - Key: Name
4
9           Value: Formation-Subnet-Public
5
0
5
1  # Internet Gateway
5
2  InternetGateway:
5
3      Type: AWS::EC2::InternetGateway
5
4      Properties:
5
5          Tags:
5
6              - Key: Name
5
7                Value: Formation-IGW
5
8
5
9  # Attacher IGW à la VPC
6
0  AttachGateway:
6
1      Type: AWS::EC2::VPCGatewayAttachment

```

```

6
2   Properties:
6
3       VpcId: !Ref FormationVPC
6
4       InternetGatewayId: !Ref InternetGateway
6
5
6
6   # Table de routage publique
6
7   PublicRouteTable:
6
8       Type: AWS::EC2::RouteTable
6
9       Properties:
7
0           VpcId: !Ref FormationVPC
7
1       Tags:
7
2           - Key: Name
7
3             Value: Formation-PublicRouteTable
7
4
7
5   # Route vers l'IGW (tout ce qui va en dehors du VPC)
7
6   PublicRoute:
7
7       Type: AWS::EC2::Route
7
8       DependsOn: AttachGateway
7
9       Properties:
8
0           RouteTableId: !Ref PublicRouteTable

```

```

8
1     DestinationCidrBlock: 0.0.0.0/0
8
2     GatewayId: !Ref InternetGateway
8
3
8
4     # Associer la route table au subnet
8
5     SubnetRouteTableAssociation:
8
6         Type: AWS::EC2::SubnetRouteTableAssociation
8
7     Properties:
8
8         SubnetId: !Ref PublicSubnet
8
9         RouteTableId: !Ref PublicRouteTable
9
0
9
1     # Security Group (pare-feu)
9
2     WebServerSecurityGroup:
9
3         Type: AWS::EC2::SecurityGroup
9
4     Properties:
9
5         GroupDescription: Permettre HTTP et SSH
9
6         VpcId: !Ref FormationVPC
9
7     SecurityGroupIngress:
9
8         # SSH (port 22) - accès depuis n'importe où (⚠ en production : limiter
à votre IP)
9
9         - IpProtocol: tcp

```

```
1
0
0      FromPort: 22
1
0
1      ToPort: 22
1
0
2      CidrIp: 0.0.0.0/0
1
0
3      Description: SSH access
1
0
4      # HTTP (port 80) - serveur web public
1
0
5      - IpProtocol: tcp
1
0
6      FromPort: 80
1
0
7      ToPort: 80
1
0
8      CidrIp: 0.0.0.0/0
1
0
9      Description: HTTP web server
1
1
0      Tags:
1
1
1      - Key: Name
1
1
2      Value: Formation-WebServerSG
```

```

1
1
3
1
1
1
4   # Instance EC2
1
1
5   WebServerInstance:
1
1
6       Type: AWS::EC2::Instance
1
1
7       Properties:
1
1
8           ImageId: ami-04a92520784b94538   # Amazon Linux 2023 (eu-west-3)
1
1
9           InstanceType: !Ref InstanceType
1
2
0           KeyName: !Ref KeyPairName
1
2
1           SubnetId: !Ref PublicSubnet
1
2
2           SecurityGroupIds:
1
2
3               - !Ref WebServerSecurityGroup
1
2
4           # Script pour configurer Apache2
1
2
5           UserData:

```

```
1
2
6 Fn::Base64: !Sub |
1
2
7 #!/bin/bash
1
2
8 # Mise à jour du système
1
2
9 yum update -y
1
3
0
1
3
1 # Installer Apache HTTP Server
1
3
2 yum install -y httpd
1
3
3
1
3
4 # Activer et démarrer Apache
1
3
5 systemctl enable httpd
1
3
6 systemctl start httpd
1
3
7
1
3
8 # Créer une page HTML de test
```

```
1
3
9   cat > /var/www/html/index.html << 'ENDHTML'
1
4
0   <!DOCTYPE html>
1
4
1   <html>
1
4
2   <head>
1
4
3       <title>CloudFormation - Formation AWS</title>
1
4
4       <style>
1
4
5           body { font-family: Arial, sans-serif; margin: 40px; }
1
4
6           h1 { color: #FF9900; }
1
4
7       </style>
1
4
8   </head>
1
4
9   <body>
1
5
0       <h1>Bienvenue sur votre serveur web CloudFormation!</h1>
```



```

1
5
1      <p>Cette instance EC2 a été déployée automatiquement via
CloudFormation.</p>
1
5
2      <p><strong>Informations serveur :</strong></p>
1
5
3      <ul>
1
5
4          <li>Instance ID : ${AWS::StackId}</li>
1
5
5          <li>Région : ${AWS::Region}</li>
1
5
6          <li>Instance Type : ${InstanceType}</li>
1
5
7      </ul>
1
5
8  </body>
1
5
9  </html>
1
6
0  ENDHTML
1
6
1
1
6
2  Tags:

```

```

1
6
3     - Key: Name
1
6
4     Value: Formation-WebServer
1
6
5
1
6
6 # Outputs - affiche les résultats utiles après déploiement
1
6
7 Outputs:
1
6
8     WebServerURL:
1
6
9     Description: URL publique du serveur web
1
7
0     Value: !Sub 'http://${WebServerInstance.PublicDnsName}'
1
7
1     Export:
1
7
2     Name: !Sub '${AWS::StackName}-WebServerURL '
1
7
3
1
7
4     WebServerPublicIP:
1
7
5     Description: Adresse IP publique de l'instance

```

```

1
7
6     Value: !Sub '${WebServerInstance.PublicIp}'
1
7
7
1
7
8     SSHCommand:
1
7
9     Description: Commande SSH pour se connecter au serveur
1
8
0     Value: !Sub 'ssh -i /chemin/vers/cle.pem ec2-
user@${WebServerInstance.PublicDnsName}'
1
8
1
1
8
2     SecurityGroupId:
1
8
3     Description: ID du Security Group
1
8
4     Value: !Ref WebServerSecurityGroup

```

Ce template complet est prêt à l'emploi ; voyons maintenant comment le déployer concrètement via la CLI.

Déployer ce template

Info

Une activité pratique permet d'approfondir le déploiement de ce template CloudFormation.

```

1  # 1. Créer la stack depuis le fichier YAML local
2  aws cloudformation create-stack \
3      --stack-name formation-webserver-atelier \
4      --template-body file:///vpc-webserver.yaml \
5      --parameters \
6          ParameterKey=KeyPairName,ParameterValue=ma-clé-ssh \
7          ParameterKey=InstanceType,ParameterValue=t3.micro \
8      --region eu-west-3
9
1
0  # Attendre que la stack soit créée (statut CREATE_COMPLETE)
1
1  aws cloudformation wait stack-create-complete \
1
2      --stack-name formation-webserver-atelier \
1
3      --region eu-west-3
1
4
1
5  # 2. Récupérer l'URL publique du serveur
1
6  aws cloudformation describe-stacks \
1
7      --stack-name formation-webserver-atelier \
1
8      --query 'Stacks[0].Outputs[?OutputKey=='WebServerURL'].OutputValue' \
1
9      --output text \
2
0      --region eu-west-3
2
1
2
2  # Output : http://ec2-12-34-56-78.eu-west-3.compute.amazonaws.com
2
3  # → Ouvrir cette URL dans un navigateur pour voir le serveur Apache

```

```

2
4
2
5 # 3. Supprimer toute l'infrastructure quand vous avez terminé
2
6 aws cloudformation delete-stack \
2
7     --stack-name formation-webserver-atelier \
2
8     --region eu-west-3
2
9
3
0 # Attendre la suppression complète
3
1 aws cloudformation wait stack-delete-complete \
3
2     --stack-name formation-webserver-atelier \
3
3     --region eu-west-3

```



Tip

Résultat attendu :

```

# create-stack :
{
    "StackId": "arn:aws:cloudformation:eu-west-3:123456789012:stack/formation-
webserver-atelier/b2c3d4e5-f6a7-8901-bcde-f12345678901"
}

# (après wait stack-create-complete – ~3 à 5 minutes)
# describe-stacks → WebServerURL :
http://ec2-15-236-78-42.eu-west-3.compute.amazonaws.com

# Le navigateur affiche la page HTML avec "Bienvenue sur votre serveur web
CloudFormation!"

```

Warning

IAM requis pour CloudFormation : Pour déployer ce template, l'utilisateur (ou le rôle IAM) doit avoir les permissions de créer des ressources EC2, VPC, Security Groups. En entreprise, créer un rôle IAM dédié `CloudFormationDeployRole` avec les permissions nécessaires, plutôt que d'utiliser un compte admin.

Danger

`delete-stack` supprime toutes les ressources : La commande `delete-stack` détruit la VPC, le subnet, l'IGW, le Security Group ET l'instance EC2 de manière irréversible. Toutes les données stockées sur l'instance EBS seront perdues. Toujours vérifier `--stack-name` avant d'exécuter.

Points clés de ce template production

Ce qui distingue ce template du premier exemple, ligne par ligne :

Élément	Explication	Bonne pratique
VPC CIDR 10.0.0.0/16	Classe privée standard pour les VPC	Utiliser RFC 1918 (10.x, 172.16.x, 192.168.x)
Subnet 10.0.1.0/24	Plage de 251 adresses disponibles	Laisser de l'espace pour futurs subnets
IGW + Route 0.0.0.0/0	Rend le subnet public et accessible d'Internet	Nécessaire pour un serveur web public
Security Group restrictif	HTTP (80) + SSH (22) explicites	En prod : limiter SSH à des IPs spécifiques
UserData script	Configure Apache automatiquement au lancement	Évite la configuration manuelle post-lancement
Outputs	Affiche l'URL et l'IP après déploiement	Indispensable pour que l'utilisateur sache comment accéder
Tags	Identification et traçabilité	Permet le suivi des ressources pour la facturation

Retenez surtout les Outputs et les Tags : ce sont les deux éléments les plus souvent oubliés dans un premier template, alors qu'ils deviennent indispensables dès que la stack sort du cadre d'un simple test.

3.4 Déployer le template avec AWS CLI

Une fois le template rédigé, on peut le déployer depuis la ligne de commande :

Info

Valider le template avant déploiement : Avant de créer une stack, il est conseillé de valider la syntaxe YAML avec `aws cloudformation validate-template --template-body file://mon-fichier.yaml`. Cette commande vérifie la syntaxe mais pas la validité des valeurs (AMI ID, type d'instance, etc.).

```

1  # 1. Créer la stack (remplacer mon-fichier.yaml par le chemin réel)
2  aws cloudformation create-stack \
3      --stack-name ma-premiere-stack \
4      --template-body file://mon-fichier.yaml \
5      --parameters ParameterKey=InstanceType,ParameterValue=t2.micro \
6      --region eu-west-1
7
8  # Résultat attendu : StackId
9  # Output : arn:aws:cloudformation:eu-west-1:123456789:stack/ma-premiere-
stack/guid
1
0
1
1  # 2. Suivre la progression du déploiement
1
2  aws cloudformation describe-stack-events \
1
3      --stack-name ma-premiere-stack \
1
4      --region eu-west-1
1
5
1
6  # 3. Une fois CREATE_COMPLETE, afficher les outputs
1
7  aws cloudformation describe-stacks \
1
8      --stack-name ma-premiere-stack \
1
9      --query 'Stacks[0].Outputs' \
2
0      --region eu-west-1
2
1
2
2  # 4. Pour mettre à jour la stack (ex. changer le type d'instance)
2
3  aws cloudformation update-stack \

```



```
2
4  --stack-name ma-premiere-stack \
2
5  --template-body file://mon-fichier.yaml \
2
6  --parameters ParameterKey=InstanceType,ParameterValue=t2.small \
2
7  --region eu-west-1
2
8
2
9  # 5. Supprimer la stack (attention : cela supprime TOUTES les ressources)
3
0  aws cloudformation delete-stack \
3
1  --stack-name ma-premiere-stack \
3
2  --region eu-west-1
```



Tip

Résultat attendu :

```
# create-stack :
{
  "StackId": "arn:aws:cloudformation:eu-west-1:123456789012:stack/ma-
premiere-stack/a1b2c3d4-e5f6-7890-abcd-ef1234567890"
}

# describe-stack-events (extrait) :
{
  "StackEvents": [
    {
      "StackId": "arn:aws:cloudformation:eu-west-1:...",
      "EventId": "...",
      "ResourceStatus": "CREATE_COMPLETE",
      "ResourceType": "AWS::EC2::Instance",
      "LogicalResourceId": "MonInstance",
      "Timestamp": "2026-03-24T10:32:15.000Z"
    },
    {
      "ResourceStatus": "CREATE_COMPLETE",
      "ResourceType": "AWS::CloudFormation::Stack",
      "LogicalResourceId": "ma-premiere-stack"
    }
  ]
}

# describe-stacks (Outputs) :
[
  {
    "OutputKey": "PublicIP",
    "OutputValue": "54.12.34.56",
    "Description": "Adresse IP publique de l'instance EC2"
  },
  {
    "OutputKey": "WebServerURL",
    "OutputValue": "http://54.12.34.56",
    "Description": "URL pour accéder au serveur web"
  }
]
```

Danger

Attention — `delete-stack` supprime TOUTES les ressources ! La commande `aws cloudformation delete-stack` détruit définitivement toutes les ressources créées par la stack (instances EC2, VPC, RDS, S3...). Il n'y a pas de corbeille. Assurez-vous d'avoir des sauvegardes et de cibler la bonne stack avant d'exécuter cette commande.

3.5 Avantages de CloudFormation

Après cette pratique, voici pourquoi CloudFormation reste le choix de référence pour l'Infrastructure as Code sur AWS :

Avantage	Bénéfice pédagogique
Automation complète	Créer/détruire une infrastructure complexe en quelques minutes
Gestion des dépendances	CloudFormation sait que l'IGW doit être créée AVANT d'être attachée à la VPC
Versionning	Stocker les templates dans Git, tracer tous les changements
Reproductibilité	Déployer exactement la même infrastructure en 10 environnements différents
Rollback	Si une erreur survient, CloudFormation annule les changements automatiquement
Coût	CloudFormation est gratuit (on paie seulement les ressources créées)

Warning

CloudFormation Drift — Divergence de configuration : Si vous modifiez manuellement des ressources gérées par CloudFormation (via la console ou la CLI), elles entrent en état de **drift** — elles ne correspondent plus au template. CloudFormation ne détecte pas ces écarts automatiquement. Utilisez `aws cloudformation detect-stack-drift --stack-name <nom>` pour identifier les ressources divergentes. Règle d'or : **ne jamais modifier manuellement une ressource gérée par CloudFormation.**

Référence : [CloudFormation Template Reference](#) 

Réflexe d'automatisation — idempotence CloudFormation

Un template CloudFormation déjà déployé est relancé sans aucune modification. Quel comportement recherche-t-on ?

► Afficher la réponse raisonnée

4. AWS Systems Manager — Automatisation opérationnelle

4.1 Qu'est-ce que Systems Manager ?

AWS Systems Manager (SSM) est un service d'**administration centralisée** et d'**automatisation opérationnelle** pour les ressources AWS et hybrides.

Il remplace le service obsolète **OpsWorks** et fournit des outils modernes pour :

- Exécuter des **scripts à distance** sur des instances EC2 (**Run Command**)
- **Patcher** automatiquement les systèmes (**Patch Manager**)
- Stocker des **configurations centralisées** (**Parameter Store**)
- Automatiser des **tâches complexes** (**Automation Documents**)
- Collectionner un **inventaire** de toutes les ressources (**Inventory**)

Analogie : Systems Manager est comme un TABLEAU DE CONTRÔLE À DISTANCE

Avant (sans SSM) :

Admin doit se connecter à chaque serveur manuellement :

```
ssh ubuntu@10.0.1.100
ssh ubuntu@10.0.1.101
ssh ubuntu@10.0.1.102
... exécuter la même commande 100 fois
```

Avec Systems Manager Run Command :

Admin exécute UNE SEULE commande :

```
aws ssm send-command --document-name "AWS-RunShellScript" \
                      --parameters commands=["apt-get update"]
```

→ Appliquée automatiquement à 100 instances simultaneously

Cette capacité de commande centralisée n'est que l'une des cinq fonctionnalités listées plus haut ; voyons maintenant chacune d'elles en détail.

4.2 Fonctionnalités clés de Systems Manager

Run Command — Exécuter des scripts à distance

Run Command permet d'exécuter des scripts sans accès SSH direct.

Prérequis :

- Instance EC2 doit avoir le rôle IAM `AmazonSSMManagedInstanceCore`
- L'agent SSM est pré-installé sur les AMI récentes

```
1  # Exemple : Installer Apache HTTP Server sur 5 instances
2  aws ssm send-command \
3    --instance-ids i-12345 i-67890 i-abcde i-fghij i-klmno \
4    --document-name "AWS-RunShellScript" \
5    --parameters 'commands=[
6      "apt-get update",
7      "apt-get install -y apache2",
8      "systemctl start apache2",
9      "systemctl enable apache2"
10    ]'
1
1
1
1
2  # Résultat : les 5 instances exécutent les commandes EN PARALLÈLE
1
3  # Voir les résultats dans CloudWatch Logs ou via CLI :
1
4  aws ssm get-command-invocation \
1
5    --command-id <command-id> \
1
6    --instance-id i-12345
```



Tip

Résultat attendu :

```
# send-command :
{
  "Command": {
    "CommandId": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
    "DocumentName": "AWS-RunShellScript",
    "Status": "Pending",
    "TargetCount": 5,
    "CompletedCount": 0
  }
}

# get-command-invocation (après exécution) :
{
  "CommandId": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
  "InstanceId": "i-12345",
  "Status": "Success",
  "StatusDetails": "Success",
  "StandardOutputContent": "Reading package lists...\nBuilding dependency
tree...\nThe following NEW packages will be installed: apache2\nSetting up
apache2 (2.4.52-1ubuntu4)...\n",
  "StandardErrorContent": ""
}
```

Résultat attendu : `send-command` retourne un `CommandId`. `get-command-invocation` affiche le `Status` (`InProgress` → `Success`) et les `StandardOutputContent` avec la sortie de chaque commande exécutée sur l'instance.

Parameter Store — Stocker des configurations centralisées

Parameter Store permet de stocker des variables (secrets, chemins d'accès, configurations) de manière centralisée.

L'avantage clé : vos applications lisent leurs secrets via l'API SSM, sans jamais avoir de valeur en clair dans le code ou un fichier `.env`.

```
1 # Stocker un secret (ex. mot de passe database)
2 aws ssm put-parameter \
3   --name /prod/database/password \
4   --value "SecurePassword123!" \
5   --type "SecureString" \
6   --description "Mot de passe RDS pour production"
7
8 # Récupérer la valeur dans une application
9 aws ssm get-parameter \
10  --name /prod/database/password \
11  --with-decryption
12
13 # Stocker une configuration simple
14 aws ssm put-parameter \
15  --name /app/api-url \
16  --value "https://api.example.com" \
17  --type "String"
```



Tip

Résultat attendu :

```
# put-parameter :
{
  "Version": 1,
  "Tier": "Standard"
}

# get-parameter (avec --with-decryption) :
{
  "Parameter": {
    "Name": "/prod/database/password",
    "Type": "SecureString",
    "Value": "SecurePassword123!",
    "Version": 1,
    "LastModifiedDate": "2026-03-24T10:00:00.000Z",
    "ARN": "arn:aws:ssm:eu-west-
3:123456789012:parameter/prod/database/password",
    "DataType": "text"
  }
}
```

Résultat attendu : `put-parameter` retourne un numéro de version (`"Version": 1`). `get-parameter` retourne le JSON avec `"Value"` déchiffré (grâce à `--with-decryption`). Sans ce flag, `SecureString` serait masqué.

Session Manager — Accès shell sécurisé sans SSH

Session Manager permet de se connecter à une instance EC2 **sans port SSH ouvert**, via la console AWS ou CLI. C'est la méthode recommandée pour accéder aux instances en production — zéro clé SSH à gérer, audit complet automatique.

```
1 # Démarrer une session interactive sur une instance
2 aws ssm start-session \
3   --target i-12345
4
5 # AWS configure le tunnel securely et vous connecte au shell
```




Tip

Résultat attendu :

```
Starting session with SessionId: stagiaire-demo-0abc123def456789
sh-4.2$
```

Un shell bash s'ouvre directement sur l'instance sans passer par SSH. Toutes les commandes saisies sont journalisées dans CloudTrail. Si la commande échoue avec `TargetNotConnected`, vérifiez que l'agent SSM est actif (`systemctl status amazon-ssm-agent`) et que le rôle IAM `AmazonSSMManagedInstanceCore` est attaché à l'instance.

Note : Un shell bash s'ouvre sur l'instance (`sh-4.2$`). Toutes les commandes sont journalisées dans CloudTrail. Si la commande échoue avec `TargetNotConnected`, vérifiez que l'agent SSM est actif et que le rôle IAM `AmazonSSMManagedInstanceCore` est attaché à l'instance.

Avantages :

- Pas besoin d'ouvrir le port 22 → Sécurité renforcée
- Audit complet des sessions dans CloudTrail
- Gestion centralisée des accès via IAM

Une fois l'accès aux instances sécurisé, encore faut-il les maintenir à jour : c'est le rôle du dernier outil de cette section.

Patch Manager — Appliquer les mises à jour automatiquement

Patch Manager scanne les instances et applique les patches de sécurité/OS. On définit d'abord une **Patch Baseline** (quels patches approuver et dans quel délai), puis on associe cette baseline aux instances.

```

1  # Scanner les instances pour les mises à jour manquantes
2  aws ssm describe-instance-patches \
3      --instance-id i-12345
4
5  # Créer un plan de patch automatique
6  aws ssm create-patch-baseline \
7      --name "MonLieuxPatchLineMonthly" \
8      --operating-system "UBUNTU" \
9      --approval-rules 'PatchRules=[{PatchFilterGroup={PatchFilters=
[{{Key=CLASSIFICATION,Values=[SECURITY,BUGFIX]}},ApproveAfterDays=7}}]'

```



Tip

Résultat attendu :

```

# describe-instance-patches :
{
  "Patches": [
    {
      "Title": "linux-aws-headers-5.15.0-1056",
      "KBId": "USN-6819-1",
      "Classification": "SECURITY",
      "Severity": "Important",
      "State": "Missing",
      "InstalledTime": null
    },
    {
      "Title": "libssl3",
      "Classification": "SECURITY",
      "Severity": "Critical",
      "State": "Missing"
    }
  ]
}

# create-patch-baseline :
{
  "BaselineId": "pb-0abc123def456789a",
  "Name": "MonLieuxPatchLineMonthly",
  "OperatingSystem": "UBUNTU",
  "CreateDate": "2026-03-24T10:00:00.000Z"
}

```

Résultat attendu : `describe-instance-patches` liste les patches manquants avec leur sévérité (`Critical`, `Important` ...). `create-patch-baseline` retourne un `BaselineId` (ex. `pb-0abc123`). Cette baseline s'applique ensuite via une **Maintenance Window** planifiée.

Référence : [AWS Systems Manager Documentation](#) 

4.3 AWS OpsWorks — Gestion de configuration avec Chef et Puppet (Contexte Historique)

Important : AWS OpsWorks en fin de vie

AWS OpsWorks était un service de **gestion de configuration** basé sur les outils open source **Chef** et **Puppet**. Depuis 2023, AWS recommande **fortement** de migrer vers **AWS Systems Manager** pour les nouvelles infrastructures.

Statut actuel :

- ❌ OpsWorks for Chef Automate : **fin de vie et désactivé depuis le 5 mai 2024**
- ❌ OpsWorks for Puppet Enterprise : **fin de vie et désactivé depuis le 5 mai 2024**
- ❌ OpsWorks Stacks : **fin de vie et désactivé depuis le 26 mai 2024**

Cette section est donc purement informative, pour reconnaître le service si vous le croisez sur une infrastructure existante, et non pour l'utiliser sur un nouveau projet.

Qu'était AWS OpsWorks ?

OpsWorks permettait de **déployer et configurer des applications** sur des instances EC2 en utilisant des **scripts de configuration déclaratifs** :



Migration depuis OpsWorks vers Systems Manager

Si vous héritez d'une infrastructure avec OpsWorks :

```

1 # 1. Exporter les recipes Chef / configurations Puppet
2 #   → Convertir en shell scripts / PowerShell pour Systems Manager
3
4 # 2. Utiliser Systems Manager Run Command pour exécuter les scripts
5 aws ssm send-command \
6     --instance-ids i-12345678 \
7     --document-name "AWS-RunShellScript" \
8     --parameters 'commands=["yum install httpd -y","systemctl start httpd"]'
9
1
0 # 3. Utiliser Patch Manager pour appliquer les correctifs
1
1 aws ssm create-patch-baseline \
1
2     --name "MonLignePatchMigree" \
1
3     --operating-system "UBUNTU" \
1
4     --approval-rules ...
1
5
1
6 # 4. Archiver les ressources OpsWorks
1
7 # Tous les stacks OpsWorks doivent être supprimés avant le 26 janvier 2024

```



Tip

Résultat attendu :

```

# send-command (migration depuis OpsWorks) :
{
  "Command": {
    "CommandId": "c3d4e5f6-a7b8-9012-cdef-a12345678901",
    "DocumentName": "AWS-RunShellScript",
    "Status": "Pending",
    "TargetCount": 1
  }
}

```

Ressources de migration

Pour approfondir cette migration, AWS et sa communauté fournissent les guides suivants :

🔗 [AWS OpsWorks → Systems Manager Migration Guide]
(<https://docs.aws.amazon.com/systems-manager/latest/userguide/opsworks-migration.html>)
🔗 [Pourquoi OpsWorks est obsolète]
(<https://aws.amazon.com/fr/blogs/france/migration-opsworks-systems-manager/>)

Conclusion pour les stagiaires : Vous ne créez JAMAIS un nouvel OpsWorks stack. Si vous le rencontrez en production, c'est un signal pour moderniser vers Systems Manager.

5. AWS Elastic Beanstalk — Déploiement simplifié d'applications

5.1 Qu'est-ce que Elastic Beanstalk ?

AWS Elastic Beanstalk est une plateforme PaaS managée qui permet de **déployer des applications web** sans gérer l'infrastructure sous-jacente.

Contrairement à CloudFormation où vous décrivez **chaque ressource manuellement**, Beanstalk **abstrait** la complexité : vous uploadez simplement votre code, et Beanstalk s'occupe de :

- Créer/gérer les instances EC2
- Configurer l'Auto Scaling
- Mettre en place le Load Balancer
- Activer le monitoring CloudWatch
- Gérer les mises à jour de l'OS et du runtime

Analogie : Elastic Beanstalk vs CloudFormation

CloudFormation = "Je veux décrire exactement mon infrastructure"

- Créer VPC, subnets, security groups, EC2, ALB, RDS, etc.
- Contrôle total mais responsabilité complète

Elastic Beanstalk = "Je veux juste déployer mon app, pas m'embêter avec l'infra"

- Upload le code (Node.js, Python, Java, .NET)
- Beanstalk crée l'infrastructure automatiquement
- Mise en échelle automatique en cas de charge
- Moins de contrôle mais moins de friction

Ce compromis « moins de contrôle contre moins de friction » n'est intéressant que si votre langage et framework sont effectivement pris en charge par la plateforme.

5.2 Runtimes et plateformes supportées

Elastic Beanstalk supporte plusieurs langages et frameworks, avec un mode Docker en filet de sécurité pour tout le reste :

Langage	Framework	Exemple
Node.js	Express, Fastify	Application web Node.js classique
Python	Flask, Django	Application Flask avec routes
Java	Spring Boot	Application Spring Boot JAR
.NET	ASP.NET Core	Application web .NET
PHP	Laravel, Symfony	Brochure web PHP
Go	Gin, Echo	Microservice Go
Docker	N'importe quel container	Flexibilité maximale

Référence : [Elastic Beanstalk Platforms](#)

5.3 Déployer une app Node.js avec Elastic Beanstalk (CLI)

Elastic Beanstalk gère toute l'infrastructure à votre place : vous fournissez juste le code. La commande `eb create` provisionne automatiquement une instance EC2, un load balancer, un Auto Scaling Group et CloudWatch. Vous n'avez qu'à pousser votre code.

```

1  # 1. Créer une application Node.js simple (app.js)
2  cat > app.js << 'EOF'
3  const express = require('express');
4  const app = express();
5
6  app.get('/', (req, res) => {
7      res.send('Bonjour depuis Elastic Beanstalk !');
8  });
9
1
0  const PORT = process.env.PORT || 3000;
1
1  app.listen(PORT, () => {
1
2      console.log(`Server running on port ${PORT}`);
1
3  });
1
4  EOF
1
5
1
6  # 2. Créer un package.json
1
7  cat > package.json << 'EOF'
1
8  {
1
9      "name": "monappbeanstalk",
2
0      "version": "1.0.0",
2
1      "description": "Petite app Express sur Beanstalk",
2
2      "main": "app.js",
2
3      "scripts": {

```

```

2
4     "start": "node app.js"
2
5 },
2
6     "dependencies": {
2
7         "express": "^4.18.2"
2
8     }
2
9 }
3
0 EOF
3
1
3
2 # 3. Initialiser un environnement Elastic Beanstalk
3
3 eb init -p node.js-18 monappbeanstalk --region eu-west-1
3
4
3
5 # 4. Créer et déployer l'environnement
3
6 # AWS crée automatiquement :
3
7 # - Une application Beanstalk
3
8 # - Un environnement (Dev, Staging, Prod, etc.)
3
9 # - 1+ instances EC2 (t2.micro par défaut)
4
0 # - Un Load Balancer
4
1 # - Auto Scaling Group
4
2 # - Monitoring CloudWatch

```



```
4
3  eb create monappbeanstalk-env --instance-type t2.micro --envvars
NODE_ENV=production
4
4
4
5  # 5. Vérifier l'état du déploiement
4
6  eb status
4
7
4
8  # 6. Voir l'URL publique de l'application
4
9  eb open
5
0
5
1  # 7. Voir les logs en temps réel
5
2  eb logs -f
5
3
5
4  # 8. Augmenter le nombre minimum d'instances
5
5  eb scale 3
5
6
5
7  # 9. Deployer une nouvelle version du code
5
8  # (après modifi app.js)
5
9  eb deploy
6
0
6
1  # 10. Supprimer l'application et l'environnement
```

6

2 `eb terminate monappbeanstalk-env`

Tip

Résultat attendu :

```
# eb create (extrait de la progression) :
Creating application version archive "app-v1".
Uploading monappbeanstalk/app-v1.zip to S3. This may take a while.
Upload Complete.
Environment details for: monappbeanstalk-env
  Application name: monappbeanstalk
  Region: eu-west-1
  Deployed Version: app-v1
  Environment ID: e-abc123defg
  Platform: arn:aws:elasticbeanstalk:eu-west-1::platform/Node.js 18 running on
64bit Amazon Linux 2023
  Tier: WebServer-Standard-1.0
  CNAME: monappbeanstalk-env.eu-west-1.elasticbeanstalk.com
  Updated: 2026-03-24 10:45:00.000000+00:00
  Status: Launching
  Health: Grey
...
INFO: Successfully launched environment: monappbeanstalk-env

# eb status :
Environment details for: monappbeanstalk-env
  Status: Ready
  Health: Green
  CNAME: monappbeanstalk-env.eu-west-1.elasticbeanstalk.com

# L'application répond "Bonjour depuis Elastic Beanstalk !" à
http://monappbeanstalk-env.eu-west-1.elasticbeanstalk.com
```

Warning

Cold start Elastic Beanstalk : Le premier déploiement (`eb create`) prend généralement 5 à 10 minutes car AWS provisionne l'infrastructure complète (EC2, ELB, Auto Scaling Group). Les déploiements suivants (`eb deploy`) sont plus rapides (1 à 3 minutes). Si `eb status` reste sur `Launching` trop longtemps, consultez les logs avec `eb logs`.

Résultat attendu : `eb create` affiche la progression en temps réel (création VPC, EC2, Load Balancer...) et se termine avec l'URL publique de l'application. `eb open` ouvre cette URL dans votre navigateur — vous devez voir “Bonjour depuis Elastic Beanstalk !”.

5.4 Avantages et limitations

Après cette prise en main, voici comment situer Elastic Beanstalk par rapport à une approche CloudFormation complète :

Avantage	Limitation
Déploiement simple du code	Contrôle limité sur l'infrastructure
Scalabilité automatique	Moins flexible que CloudFormation
Monitoring intégré	Pas adapté aux architectures très complexes
Mises à jour OS automatiques	Coûts potentiellement plus élevés

Référence : [Elastic Beanstalk Documentation](#) 

5.5 Elastic Beanstalk vs Lambda — Quand choisir quoi ?

Ces deux services répondent à la même question — “comment déployer du code sans gérer de serveurs” — mais avec des philosophies radicalement différentes.

	Beanstalk	Lambda
Exécution	EC2 toujours allumé, Load Balancer, Auto Scaling Group, CloudWatch	Conteneur éphémère, créé à la demande et détruit après exécution
Paradigme	Lift & Shift	Event-Driven

Cette différence de paradigme se décline sur une dizaine de critères concrets, résumés dans le tableau détaillé ci-dessous.

Tableau de comparaison

Voici le détail critère par critère, qui explique pourquoi ces deux services ne sont pas réellement interchangeables :

Critère	Elastic Beanstalk	Lambda
Paradigme	PaaS — application toujours en cours	FaaS — fonction déclenchée à la demande
Infrastructure	EC2 + ELB + ASG (gérés automatiquement)	Aucune instance visible
Démarrage	Toujours chaud	Cold start possible (ms à quelques s)

Warning

Lambda Cold Starts : Lors du premier appel d'une fonction Lambda (ou après une longue période d'inactivité), AWS doit initialiser le conteneur d'exécution — c'est le **cold start**. La latence peut aller de quelques dizaines de ms (Node.js/Python) à plusieurs secondes (Java). Solutions : **Provisioned Concurrency** (maintient N conteneurs chauds en permanence, payant), ou choisir un runtime léger (Node.js/Python) pour les APIs sensibles à la latence.

| **Durée max d'exécution** | Illimitée | **15 minutes** | | **Mémoire max** | Celle de l'instance (jusqu'à 384 Go) | **10 Go** | | **Stockage local** | EBS persistant | **/tmp : 10 Go seulement** | | **Langages supportés** | Java, Node.js, Python, Ruby, PHP, Go, .NET | Java, Node.js, Python, Ruby, Go, .NET, Rust + custom runtime | | **Modèle de coût** | EC2 à la seconde (même si pas de requêtes) | À l'invocation (1M req gratuites/mois) | | **Scaling** | Auto Scaling Group (minutes) | Instantané, jusqu'à 1 000 exécutions parallèles | | **État** | Stateful possible (session, fichiers) | Stateless obligatoire | | **Réseau** | VPC natif, Security Groups | VPC optionnel | | **Déploiement** | ZIP, WAR, Docker, `eb deploy` | ZIP, container, `aws lambda update-function-code` |

Le critère « Modèle de coût » mérite un chiffrage détaillé, car c'est souvent celui qui fait basculer la décision.

Modèle de coût détaillé

Elastic Beanstalk :

Beanstalk en lui-même = GRATUIT

Vous payez les ressources qu'il crée :

Exemple : app Node.js standard

1× EC2 t3.small	→ 0,023 \$/h	→ ~17 \$/mois
1× ELB Application	→ 0,008 \$/h	→ ~6 \$/mois + 0,008 \$/LCU
Stockage EBS 20 Go	→	→ ~2,5 \$/mois

TOTAL (1 instance) → → ~26 \$/mois
(même si 0 utilisateur cette nuit-là)

Lambda :

Free Tier permanent : 1 000 000 requêtes/mois + 400 000 Go-secondes/mois

Au-delà :

Requêtes : 0,20 \$ / million

Durée : 0,0000000167 \$ / Go-seconde

Exemple : API Lambda 128 Mo RAM, 200 ms d'exécution, 1M requêtes/mois

Durée : $1\,000\,000 \times 0,128\text{ Go} \times 0,2\text{ s} = 25\,600\text{ Go-secondes} \rightarrow \text{GRATUIT} (< 400\,000)$

Requêtes : 1 000 000 $\rightarrow$ GRATUIT ($< 1\text{M}$)

TOTAL : 0 \$ (dans le Free Tier)

Exemple : 10M requêtes/mois

Durée : 256 000 Go-s supplémentaires $\rightarrow 256\,000 \times 0,0000000167 = \sim 0,004\text{ \$}$

Requêtes : 9M supplémentaires $\rightarrow 9 \times 0,20 = 1,80\text{ \$}$

TOTAL : $\sim 1,80\text{ \$/mois}$

Ces deux exemples confirment l'intuition : Lambda reste imbattable en dessous du Free Tier et pour un trafic modéré, tandis que Beanstalk facture un socle fixe indépendant du trafic réel.

Quand utiliser lequel ?

En croisant les critères techniques et le modèle de coût vus ci-dessus, voici une grille de décision pratique :

CHOISIR BEANSTALK si :

- ✓ Application web traditionnelle (Django, Express, Spring Boot, WordPress)
- ✓ Traitement long (> 15 minutes)
- ✓ Besoin de sessions persistantes côté serveur
- ✓ Migration d'une app existante ("lift & shift")
- ✓ Besoin d'accès à des fichiers locaux entre requêtes
- ✓ Équipe non familière avec l'architecture event-driven

CHOISIR LAMBDA si :

- ✓ API REST légère (avec API Gateway)
- ✓ Traitement d'événements (upload S3, message SQS, stream DynamoDB)
- ✓ Tâches planifiées (cron CloudWatch Events)
- ✓ Traitement de fichiers (redimensionnement images, parsing CSV)
- ✓ Webhooks, notifications, automatisations
- ✓ Trafic très variable (pics et creux importants)
- ✓ Budget serré avec faible volumétrie

💡 **En pratique** : beaucoup d'architectures modernes combinent les deux. Beanstalk pour le frontend/API principale, Lambda pour les traitements en arrière-plan (envoi d'emails, génération de rapports, nettoyage de données).

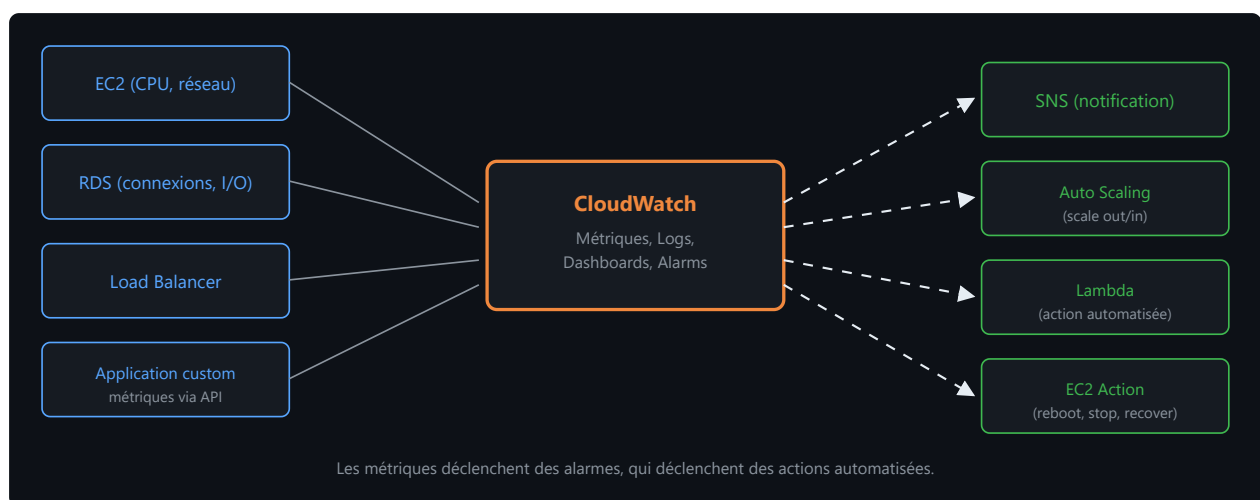
👤 [Elastic Beanstalk vs Lambda — AWS Blog](#)

6. Amazon CloudWatch — Supervision et alarmes

6.1 Qu'est-ce que CloudWatch ?

Amazon CloudWatch est le service de **monitoring centralisé** d'AWS. Il collecte, stocke et affiche des métriques sur :

- **Instances EC2** : CPU, mémoire réseau, I/O disque
- **Bases RDS** : connexions actives, CPU, I/O
- **Load Balancers** : requêtes/seconde, latence
- **Applications custom** : envoi de métriques via API



6.2 Créer une alarme CloudWatch (CLI)

Une alarme surveille une métrique en continu et déclenche une action (ici, une notification par email via SNS) dès qu'un seuil est franchi de façon répétée :

```

1  # 1. Créer une alarme sur la métrique CPU d'une instance EC2
2  # L'alarme se déclenche si CPU > 80% pendant 2 périodes consécutives (10 min)
3  aws cloudwatch put-metric-alarm \
4      --alarm-name "MonInstance-CPU-Élevé" \
5      --alarm-description "Alerte CPU élevé instance EC2" \
6      --metric-name CPUUtilization \
7      --namespace AWS/EC2 \
8      --statistic Average \
9      --period 300 \
10     --threshold 80 \
11
12     --comparison-operator GreaterThanThreshold \
13
14     --evaluation-periods 2 \
15
16     --dimensions Name=InstanceId,Value=i-12345
17
18 # 2. Ajouter une action SNS (envoyer un email)
19
20 # D'abord, créer un sujet SNS
21
22 aws sns create-topic --name MonTopicAlarmes
23
24 # Output : TopicArn: arn:aws:sns:eu-west-1:123456789:MonTopicAlarmes
25
26 # S'abonner au sujet (recevoir les notifications)
27
28 aws sns subscribe \
29
30     --topic-arn arn:aws:sns:eu-west-1:123456789:MonTopicAlarmes \
31
32     --protocol email \

```

```

2
4  --notification-endpoint admin@example.com
2
5
2
6 # 3. Modifier l'alarme pour envoyer une notification
2
7 aws cloudwatch put-metric-alarm \
2
8  --alarm-name "MonInstance-CPU-Élevé" \
2
9  --alarm-description "Alerte CPU élevé instance EC2" \
3
0  --metric-name CPUUtilization \
3
1  --namespace AWS/EC2 \
3
2  --statistic Average \
3
3  --period 300 \
3
4  --threshold 80 \
3
5  --comparison-operator GreaterThanThreshold \
3
6  --evaluation-periods 2 \
3
7  --dimensions Name=InstanceId,Value=i-12345 \
3
8  --alarm-actions arn:aws:sns:eu-west-1:123456789:MonTopicAlarms
3
9
4
0 # 4. Lister toutes les alarmes
4
1 aws cloudwatch describe-alarms
4
2

```



```
4
3  # 5. Supprimer une alarme
4
4  aws cloudwatch delete-alarms --alarm-names "MonInstance-CPU-Élevé"
```

Tip

Résultat attendu :

```
# sns create-topic :
{
  "TopicArn": "arn:aws:sns:eu-west-1:123456789012:MonTopicAlarmes"
}

# sns subscribe :
{
  "SubscriptionArn": "pending confirmation"
}
# → Un email est envoyé à admin@example.com avec un lien de confirmation

# put-metric-alarm : (pas de sortie si succès – code HTTP 200)

# describe-alarms (extrait) :
{
  "MetricAlarms": [
    {
      "AlarmName": "MonInstance-CPU-Élevé",
      "AlarmDescription": "Alerte CPU élevé instance EC2",
      "StateValue": "OK",
      "MetricName": "CPUUtilization",
      "Threshold": 80.0,
      "Period": 300,
      "EvaluationPeriods": 2
    }
  ]
}
```

6.3 Envoyer des métriques custom depuis une application

Applications peuvent envoyer des métriques CloudWatch pour monitorer des KPI métier :

```
1 # Exemple : envoyer une métrique custom "OrdersPerMinute"
2 aws cloudwatch put-metric-data \
3   --namespace "MonApplication" \
4   --metric-name "OrdersPerMinute" \
5   --value 42 \
6   --unit Count \
7   --timestamp 2025-03-24T14:30:00Z
```



Résultat attendu :

```
# put-metric-data : pas de sortie si succès (HTTP 200)
# La métrique est visible dans CloudWatch Console sous "MonApplication >
OrdersPerMinute"
# après environ 1 minute de délai d'ingestion.
```

```

1  # Ou dans un script Python :
2  import boto3
3
4  cloudwatch = boto3.client('cloudwatch')
5
6  # Envoyer une métrique custom
7  cloudwatch.put_metric_data(
8      Namespace='MonApplication',
9      MetricData=[
10
11          {
12
13              'MetricName': 'PedidosProcessados',
14
15              'Value': 42,
16
17              'Unit': 'Count',
18
19              'Timestamp': datetime.utcnow()
20
21          }
22      ]
23  )

```

Une fois plusieurs métriques (standard et custom) collectées, il devient utile de les visualiser toutes ensemble sur un même écran plutôt que de les consulter une par une.

6.4 Tableaux de bord CloudWatch

CloudWatch permet de créer des **dashboards** personnalisés affichant plusieurs métriques :

```

1  # Créer un dashboard JSON
2  cat > dashboard.json << 'EOF'
3  {
4      "widgets": [
5          {
6              "type": "metric",
7              "properties": {
8                  "metrics": [
9                      [ "AWS/EC2", "CPUUtilization", { "stat": "Average" } ],
10                     [ ".", "NetworkIn", { "stat": "Sum" } ]
11                 ],
12                 "period": 300,
13                 "stat": "Average",
14                 "region": "eu-west-1",
15                 "title": "Métriques EC2"
16             }
17         }
18     ]
19 }
20 EOF
21
22 # Créer le dashboard
23 aws cloudwatch put-dashboard \

```

```
2
4  --dashboard-name "MonDashboard" \
2
5  --dashboard-body file://dashboard.json
```



Tip

Résultat attendu :

```
# put-dashboard :
{
  "DashboardValidationMessages": []
}
# Le tableau de bord "MonDashboard" est maintenant visible dans la console
CloudWatch.
# Accès : CloudWatch → Dashboards → MonDashboard
```

Référence : [CloudWatch Documentation](#)

6.5 AWS SDK pour Développeurs — Automatisation Programmatique

Jusque-là, nous avons utilisé la **CLI AWS** pour exécuter des commandes manuellement. Mais les **SDK AWS** permettent d'intégrer AWS directement dans du code applicatif (Python, Node.js, Java, Go, etc.).

Qu'est-ce que le SDK AWS ?

SDK = kit de développement fourni par AWS dans plusieurs langages pour interagir avec les services AWS par programmation.

Analogie : CLI vs SDK

CLI (AWS CLI)



Outil en ligne de commande

Exécute des commandes manuellement ou dans des scripts bash

Exemple : `aws ec2 describe-instances`

SDK (boto3, SDK.js, SDK.java)



Bibliothèque logicielle intégrée dans votre code

Votre application Python/Node/Java appelle AWS directement

Exemple : `ec2_client.describe_instances()`

Ce choix entre CLI et SDK n'est pas exclusif — les deux s'appuient sur les mêmes API et peuvent cohabiter dans un même projet. AWS propose un SDK officiel pour chaque langage majeur.

SDK AWS Disponibles

AWS maintient officiellement un SDK pour chaque langage majeur, avec la même couverture fonctionnelle que la CLI :

Langage	Nom SDK	Cas d'usage
Python	boto3	Data science, Lambda, backend
JavaScript/Node.js	AWS SDK for JavaScript	Applications web, serverless
Java	AWS SDK for Java	Entreprise, Spring Boot
Go	AWS SDK for Go	CLI tools, microservices
C#/.NET	AWS SDK for .NET	Windows, applications d'entreprise
PHP	AWS SDK for PHP**	Applications web, Laravel

Le reste de cette section se concentre sur `boto3`, le SDK Python, très utilisé pour l'automatisation et les fonctions Lambda.

Introduction à boto3 (Python)

boto3 est la **SDK AWS officielle pour Python**. Elle est utilisée dans :

- Scripts d'automatisation
- Applications Lambda
- Tâches cron de maintenance
- Outils de gestion d'infrastructure

Avant de l'utiliser dans du code, il faut bien sûr l'installer dans votre environnement Python.

Installation de boto3

Comme toute bibliothèque Python, boto3 s'installe via pip :

```
1 # Installer boto3
2 pip install boto3
3
4 # Vérifier l'installation
5 python3 -c "import boto3; print(boto3.__version__)"
```



Résultat attendu :

```
Collecting boto3
  Downloading boto3-1.34.69-py3-none-any.whl (139 kB)
Successfully installed boto3-1.34.69 botocore-1.34.69 s3transfer-0.10.1
1.34.69
```

Exemple 1 : Lister les instances EC2

Premier réflexe avec boto3 : créer un client pour le service ciblé, puis appeler la méthode correspondant à l'action CLI équivalente :

```

1  import boto3
2
3  # Créer un client EC2
4  ec2_client = boto3.client('ec2', region_name='eu-west-3')
5
6  # Lister toutes les instances
7  response = ec2_client.describe_instances()
8
9  # Parcourir les instances
1
0  for reservation in response['Reservations']:
1
1     for instance in reservation['Instances']:
1
2         instance_id = instance['InstanceId']
1
3         instance_type = instance['InstanceType']
1
4         state = instance['State']['Name']
1
5
1
6         # Afficher les informations
1
7         print(f"Instance : {instance_id}")
1
8         print(f"  Type : {instance_type}")
1
9         print(f"  État : {state}")
2
0         print()

```

Résultat :


```
Instance : i-0123456789abcdef0
Type : t3.micro
État : running

Instance : i-0987654321abcdef0
Type : t3.small
État : stopped
```

Ce premier exemple ne fait que lire l'existant ; voyons maintenant comment créer une nouvelle ressource avec la même logique client/appel/réponse.

Exemple 2 : Créer une snapshot EBS

Ce même client EC2 sert aussi bien à consulter l'existant qu'à créer de nouvelles ressources :

```

1  import boto3
2
3  # Créer un client EC2
4  ec2_client = boto3.client('ec2', region_name='eu-west-3')
5
6  # Créer un snapshot du volume vol-12345678
7  response = ec2_client.create_snapshot(
8      VolumeId='vol-12345678',
9      Description='Sauvegarde avant migration',
10
11      TagSpecifications=[
12
13          {
14
15              'ResourceType': 'snapshot',
16
17              'Tags': [
18
19                  {'Key': 'Name', 'Value': 'backup-migration-2026-03-24'},
20
21                  {'Key': 'Environment', 'Value': 'production'}
22
23              ]
24
25          }
26
27      ]
28
29  )
30
31
32
33  # Afficher l'ID du snapshot créé
34
35  snapshot_id = response['SnapshotId']
36
37
38  progress = response['Progress']

```

```

2
4
2
5 print(f"Snapshot créé : {snapshot_id}")
2
6 print(f"Progression : {progress}")

```

Ce snapshot protège les données ; il reste à voir comment agir sur le cycle de vie de l'instance elle-même.

Exemple 3 : Arrêter une instance EC2

À l'inverse d'une création, certaines opérations modifient l'état d'une ressource existante — comme arrêter une instance en cours d'exécution :

```

1 import boto3
2
3 # Créer un client EC2
4 ec2_client = boto3.client('ec2', region_name='eu-west-3')
5
6 # Arrêter une instance
7 instance_id = 'i-0123456789abcdef0'
8
9 response = ec2_client.stop_instances(InstanceIds=[instance_id])
1
0
1
1 # Vérifier que l'arrêt est en cours
1
2 for instance in response['StoppingInstances']:
1
3     print(f"Instance {instance['InstanceId']} est en cours d'arrêt")
1
4     print(f"État précédent : {instance['PreviousState']['Name']}")
1
5     print(f"État courant : {instance['CurrentState']['Name']}")

```

Ces trois premiers exemples ont ciblé EC2 ; boto3 s'utilise de la même façon pour n'importe quel autre service, comme CloudWatch.

Exemple 4 : Créer une alarme CloudWatch

Ce dernier exemple ramène au thème de la section 6 : créer via boto3 la même alarme CloudWatch qu'on a créée plus haut en CLI, pour montrer l'équivalence entre les deux approches :

```

1 import boto3
2
3 # Créer un client CloudWatch
4 cloudwatch_client = boto3.client('cloudwatch', region_name='eu-west-3')
5
6 # Créer une alarme si CPU > 80%
7 cloudwatch_client.put_metric_alarm(
8     AlarmName='CPU-Haute-Production',
9     ComparisonOperator='GreaterThanThreshold',
10
11     EvaluationPeriods=2,
12
13     MetricName='CPUUtilization',
14
15     Namespace='AWS/EC2',
16
17     Period=300, # 5 minutes
18
19     Statistic='Average',
20
21     Threshold=80.0,
22
23     ActionsEnabled=True,
24
25     AlarmActions=['arn:aws:sns:eu-west-3:123456789:AlertesProduction'],
26
27     Dimensions=[
28         {
29             'Name': 'InstanceId',
30
31             'Value': 'i-0123456789abcdef0'
32         }
33     ]

```

```
2
4 )
2
5
2
6 print("Alarme CloudWatch créée avec succès")
```

Ces quatre exemples suivent tous le même schéma (client, appel, traitement de la réponse) ; pour les généraliser en code de production, quelques règles s'imposent.

Bonnes pratiques boto3

Avant d'utiliser ces scripts au-delà d'un simple test, voici les réflexes à adopter systématiquement :

- ✓ Utiliser des variables d'environnement ou des profils AWS pour les credentials
- ✓ Gérer les erreurs avec try/except
- ✓ Utiliser des context managers ou des sessions boto3
- ✓ Documenter chaque appel API avec un commentaire
- ✓ Tester en environnement non-production d'abord
- ✓ Utiliser des rôles IAM appropriés (pas de clés d'accès root)

Le premier et le dernier point se rejoignent : ne jamais coder en dur des clés d'accès, qu'elles soient celles d'un utilisateur IAM classique ou, pire, celles du compte root.

7. AWS Well-Architected Framework — Mise en pratique

Le Chapitre 1 a introduit les six piliers du **AWS Well-Architected Framework** (Operational Excellence, Security, Reliability, Performance Efficiency, Cost Optimization, Sustainability) avec leurs bonnes pratiques respectives. Maintenant que vous avez manipulé IAM, S3, EC2, Lambda, RDS, VPC et CloudFormation, vous disposez de tous les services nécessaires pour appliquer concrètement ce framework à un cas réel.

Info

Besoin d'un rappel des 6 piliers ? Retournez au Chapitre 1, section 7 — définitions, questions clés et bonnes pratiques par pilier y sont détaillées.

7.1 Cas d'étude : Application "CloudPizza"

Imaginons une application de commande de pizzas. Appliquons les 6 piliers :

Pilier	Décision	Justification
Operational Excellence	Déployer via CloudFormation + CI/CD	Répétabilité, traçabilité
Security	IAM par service, KMS pour BDD, bucket S3 privé	Moindre privilège, conformité
Reliability	Multi-AZ, ALB, RDS Multi-AZ, snapshots EBS quotidiens	RTO 1h, RPO 1h
Performance	Lambda + DynamoDB au lieu de serveurs	Scalabilité illimitée automatique
Cost Optimization	Reserved Instances pour serveurs stables, Spot pour batch	Réduire 40% des coûts
Sustainability	Déployer en Irlande (énergies renouvelables), Lambda sans serveur	Réduire l'empreinte carbone

Ce tableau ne donne que la synthèse des décisions ; le pilier Cost Optimization s'appuie en réalité sur un outil précis, déjà présenté au Chapitre 4, qu'il vaut la peine de relier explicitement à ce cas d'usage.

7.2 Rappel — AWS Compute Optimizer et le pilier Cost Optimization

Le Chapitre 4 a détaillé le fonctionnement d'**AWS Compute Optimizer** (collecte CloudWatch, analyse ML, recommandations chiffrées) — c'est l'outil concret qui alimente le pilier **Cost Optimization** vu ci-dessus : dans le cas CloudPizza, c'est lui qui permettrait de vérifier a posteriori que les Reserved Instances choisies sont bien dimensionnées à l'usage réel.

 Info

Besoin d'un rappel du fonctionnement de Compute Optimizer ? Retournez au Chapitre 4, section 7 — commande CLI complète, exemple de sortie JSON et cas concret `t3.large` → `t3.small` y sont détaillés.

8. Services complémentaires — Queues et événements

8.1 Amazon SQS — File d'attente de messages

Amazon SQS (Simple Queue Service) est une **file d'attente de messages** complètement gérée.

Cas d'usage : Découpler des composants d'une application.

Analogie : SQS est comme une BOÎTE AUX LETTRES

Sans SQS (couplage fort) :

Producteur (app web) → appelle directement Consommateur (worker)

Si worker est en panne → producteur attend → demande utilisateur bloquée

Avec SQS (découplage) :

Producteur → envoie message dans SQS → retour immédiat

Consommateur → consomme messages quand il est prêt (même en panne, pas de perte)

Voici la séquence complète : créer la queue, envoyer un message, le lire, puis le supprimer. La suppression explicite est obligatoire — SQS ne supprime pas automatiquement un message après lecture (pour éviter la perte en cas d'échec).


```
1 # Créer une queue SQS
2 aws sqs create-queue --queue-name MonQueue
3
4 # Envoyer un message
5 aws sqs send-message \
6     --queue-url https://sqs.eu-west-1.amazonaws.com/123456789/MonQueue \
7     --message-body "Bonjour depuis CloudFormation"
8
9 # Consommer un message (avec delete)
10
11 aws sqs receive-message \
12     --queue-url https://sqs.eu-west-1.amazonaws.com/123456789/MonQueue \
13     --max-number-of-messages 1
14
15 # Supprimer le message de la queue
16
17 aws sqs delete-message \
18     --receipt-handle <receipt-handle>
```



Tip

Résultat attendu :

```
# create-queue :
{
  "QueueUrl": "https://sqs.eu-west-1.amazonaws.com/123456789012/MonQueue"
}

# send-message :
{
  "MD5ofMessageBody": "9c7c6f0f3f748bdfa5a5e6e7c8d9e0f1",
  "MessageId": "msg-0abc123def456789a"
}

# receive-message :
{
  "Messages": [
    {
      "MessageId": "msg-0abc123def456789a",
      "ReceiptHandle": "AQEB...longstring...",
      "MD5ofBody": "9c7c6f0f3f748bdfa5a5e6e7c8d9e0f1",
      "Body": "Bonjour depuis CloudFormation"
    }
  ]
}

# delete-message : pas de sortie si succès (HTTP 200)
```

8.2 Amazon SNS — Notifications pubsub

Amazon SNS (Simple Notification Service) est un service de **notifications pub/sub**.

Différence SQS vs SNS :

SQS : 1 producteur → 1 queue → 1 consommateur (FIFO ou parallèle)

SNS : 1 producteur → N abonnés (email, SMS, SQS, Lambda, HTTP)

Voici comment créer un topic SNS, y abonner une adresse email, et publier un message qui sera envoyé à tous les abonnés simultanément :

```

1  # Créer un sujet SNS
2  aws sns create-topic --name MonTopicAlarmes
3
4  # Ajouter un abonnement email
5  aws sns subscribe \
6    --topic-arn arn:aws:sns:eu-west-1:123456789:MonTopicAlarmes \
7    --protocol email \
8    --notification-endpoint admin@example.com
9
10 # Publier un message
11
12 aws sns publish \
13
14    --topic-arn arn:aws:sns:eu-west-1:123456789:MonTopicAlarmes \
15
16    --message "Alerte : CPU élevé détecté !"

```



Tip

Résultat attendu :

```

# create-topic :
{
  "TopicArn": "arn:aws:sns:eu-west-1:123456789012:MonTopicAlarmes"
}

# subscribe :
{
  "SubscriptionArn": "pending confirmation"
}
# → Un email est envoyé avec un lien de confirmation

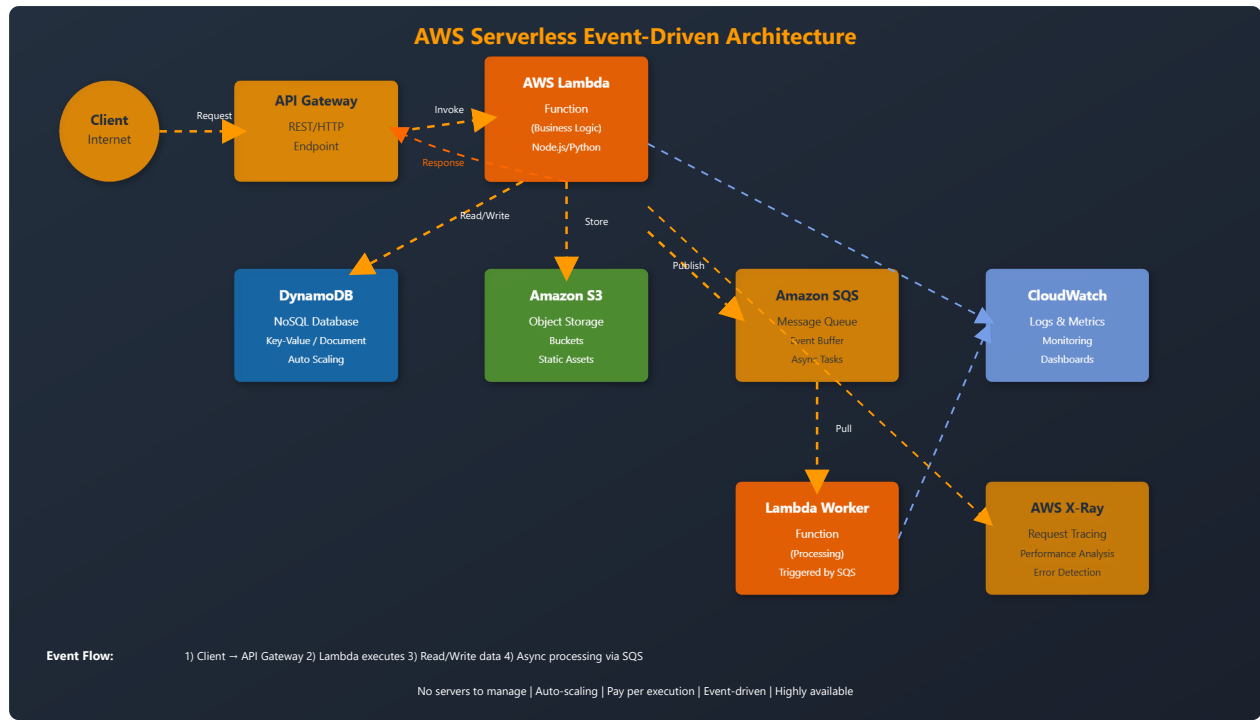
# publish :
{
  "MessageId": "pub-0abc123def456789a"
}
# → Tous les abonnés (email, SQS, Lambda) reçoivent le message instantanément

```

Référence : [Amazon SQS](#) Référence : [Amazon SNS](#)

8.3 Architectures découplées, microservices et sans serveur

Le schéma suivant rassemble les briques d'une architecture sans serveur orientée événements. L'objectif n'est pas de mémoriser une succession d'icônes : suivez le trajet d'une requête, puis identifiez les points où l'application absorbe un pic, conserve un état ou isole une panne.



API Gateway reçoit les appels synchrones, tandis qu'une file ou un bus d'événements permet de différer certains traitements. Lambda exécute le code sans serveur à administrer, mais le client reste responsable des permissions IAM, des dépendances, de l'idempotence, des erreurs partielles et du cycle de vie des données.

SQS et SNS résolvent le découplage entre deux composants pris isolément. Une architecture applicative complète va plus loin : elle assemble plusieurs services managés pour qu'aucun composant ne dépende directement de la disponibilité d'un autre, et que chaque brique puisse évoluer, tomber en panne ou être remplacée sans effet domino sur le reste du système. C'est le principe des **microservices** : découper une application monolithique en plusieurs services indépendants, chacun responsable d'une capacité métier précise (paiement, catalogue, notifications), communiquant entre eux par API ou par messages plutôt que par appels de fonction directs en mémoire.

Couplage fort vs couplage faible — le test décisif

Couplage fort (monolithe classique) :

- Service A appelle directement Service B (HTTP synchrone ou appel de fonction)
- Si B est lent ou en panne, A attend, se bloque, ou échoue en cascade
- Faire évoluer B (changer sa techno, sa capacité) impose de coordonner A

Couplage faible (architecture découplée) :

- Service A publie un événement/message, sans savoir qui le consommera
- B (et C, D...) consomment à leur rythme, indépendamment de la disponibilité de A
- B peut tomber, redémarrer, ou être remplacé sans qu'A ne le sache

Le test décisif pour repérer un couplage fort évitable : si le service producteur doit attendre une réponse synchrone du consommateur pour continuer son propre travail, alors qu'il n'a besoin d'aucune information en retour, c'est un candidat naturel au découplage via SQS ou SNS.

Amazon API Gateway — le point d'entrée unifié

Dans une architecture microservices, chaque service pourrait exposer sa propre adresse réseau — mais cela oblige les clients (applications mobiles, sites web, partenaires) à connaître et gérer N adresses différentes, et complique la sécurisation (authentification à répliquer partout). **Amazon API Gateway** résout ce problème en offrant un point d'entrée HTTP unique, qui route chaque requête vers le bon service backend (Lambda, conteneur, serveur EC2) selon l'URL et la méthode appelées.

Client mobile/web



API Gateway

/users → Lambda A

/orders → Lambda B

/catalog → ECS Fargate

← authentification centralisée, throttling, cache

API Gateway prend en charge, sans code supplémentaire à écrire dans chaque microservice : l'authentification (intégration IAM, Cognito, ou clé API), la limitation de débit (*throttling*) pour protéger les backends d'un pic de trafic, la mise en cache des réponses, et la transformation de requêtes/réponses (mapping de formats).

```

1  # Créer une API REST
2  aws apigateway create-rest-api --name "MonAPI-Catalogue"
3
4  # Créer une ressource sous la racine
5  aws apigateway create-resource \
6    --rest-api-id <api-id> \
7    --parent-id <root-resource-id> \
8    --path-part "produits"
9
10 # Associer une méthode GET à une fonction Lambda (intégration proxy)
11
12 aws apigateway put-integration \
13   --rest-api-id <api-id> \
14   --resource-id <resource-id> \
15   --http-method GET \
16   --type AWS_PROXY \
17   --integration-http-method POST \
18   --uri arn:aws:apigateway:eu-west-1:lambda:path/2015-03-
31/functions/arn:aws:lambda:eu-west-1:123456789:function:lister-
produits/invocations

```



Tip

Résultat attendu (create-rest-api) :

```
{
  "id": "a1b2c3d4e5",
  "name": "MonAPI-Catalogue",
  "createdDate": "2026-07-27T10:00:00Z",
  "apiKeySource": "HEADER",
  "endpointConfiguration": {
    "types": ["EDGE"]
  }
}
```

AWS Step Functions — orchestrer plusieurs services dans un workflow

Certains traitements ne se résument pas à un simple message transmis d'un service à l'autre : ils enchaînent plusieurs étapes avec de la logique conditionnelle (si le paiement échoue, annuler la réservation), des étapes parallèles (vérifier le stock et calculer les frais de port en même temps), et des reprises sur erreur. **AWS Step Functions** modélise ce type de workflow sous forme de machine à états (*state machine*), où chaque état est typiquement une fonction Lambda, un appel à un autre service AWS, ou une branche de décision.

```

1  {
2    "Comment": "Traitement d'une commande e-commerce",
3    "StartAt": "VerifierStock",
4    "States": {
5      "VerifierStock": {
6        "Type": "Task",
7        "Resource": "arn:aws:lambda:eu-west-1:123456789:function:verifier-stock",
8        "Next": "StockDisponible"
9      },
10
11      "StockDisponible": {
12
13        "Type": "Choice",
14
15        "Choices": [
16
17          {
18
19            "Variable": "$.stock_ok",
20
21            "BooleanEquals": true,
22
23            "Next": "TraiterPaieement"
24          }
25        ],
26
27        "Default": "AnnulerCommande"
28      },
29
30      "TraiterPaieement": {
31
32        "Type": "Task",
33
34        "Resource": "arn:aws:lambda:eu-west-1:123456789:function:traiter-
paieement",

```



```

2
4     "Catch": [
2
5         {
2
6             "ErrorEquals": ["PaielementRefuse"],
2
7             "Next": "AnnulerCommande"
2
8         }
2
9     ],
3
0     "Next": "ConfirmerCommande"
3
1 },
3
2     "ConfirmerCommande": {
3
3         "Type": "Task",
3
4         "Resource": "arn:aws:lambda:eu-west-1:123456789:function:confirmer-
commande",
3
5         "End": true
3
6     },
3
7     "AnnulerCommande": {
3
8         "Type": "Task",
3
9         "Resource": "arn:aws:lambda:eu-west-1:123456789:function:annuler-
commande",
4
0         "End": true
4
1     }

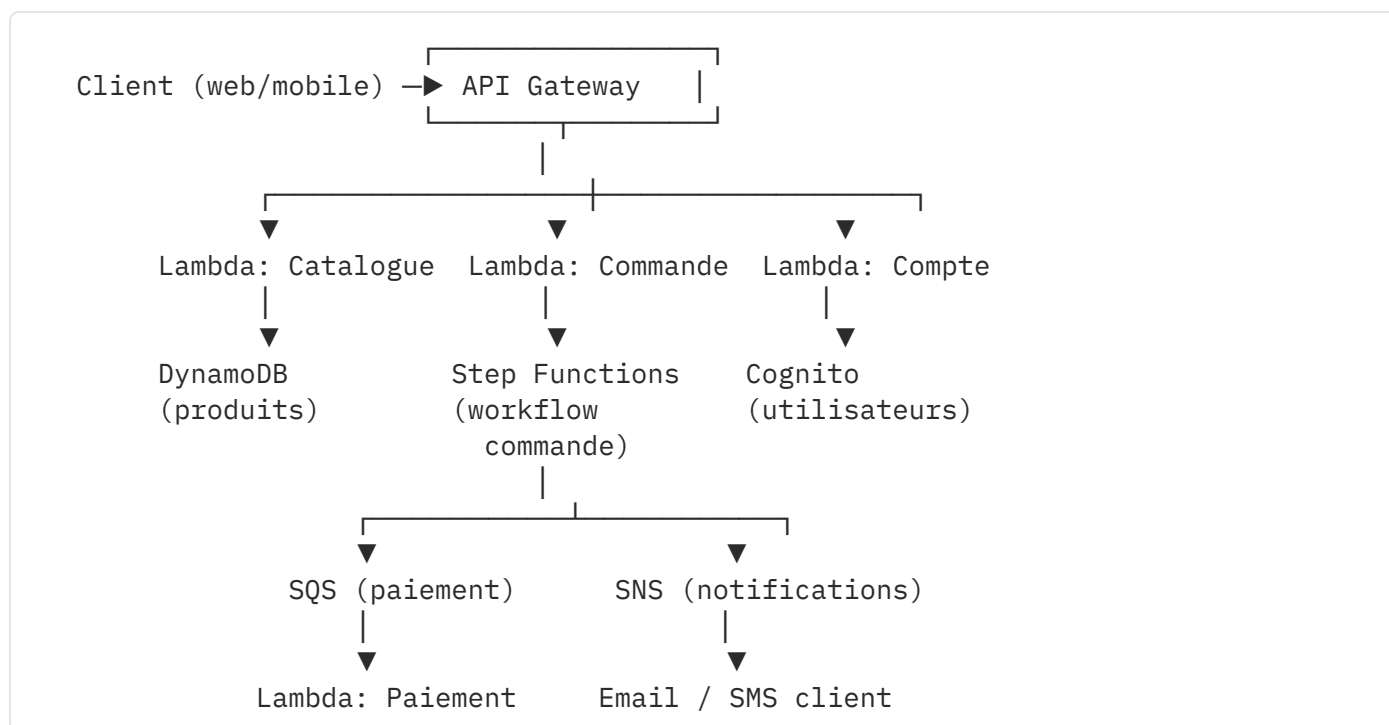
```

```
4
2    }
4
3    }
```

L'intérêt par rapport à enchaîner ces appels directement dans le code d'une seule fonction Lambda : chaque état est visible, monitorable et rejouable indépendamment dans la console AWS (Step Functions affiche un diagramme visuel de l'exécution, avec l'état exact atteint en cas d'échec) — un débogage bien plus direct qu'une pile d'appels imbriqués dans les logs CloudWatch d'une fonction monolithique.

Architecture microservices sans serveur complète — exemple

L'illustration ci-dessous assemble les briques vues dans ce chapitre et les précédents en une architecture microservices sans serveur cohérente pour une application de commande en ligne :



Aucun composant de ce schéma ne connaît directement l'adresse réseau d'un autre composant en amont : le client ne connaît que l'URL d'API Gateway, les Lambdas ne connaissent que les ARN des ressources qu'elles invoquent, et la communication asynchrone (SQS/SNS) élimine toute dépendance temporelle stricte entre le traitement de la commande et l'envoi de la notification. C'est cette absence de dépendance directe qui permet à chaque brique d'être mise à l'échelle, remplacée ou de tomber en panne sans effet domino sur le reste de l'architecture — le principe même du découplage appliqué à l'échelle d'un système complet.

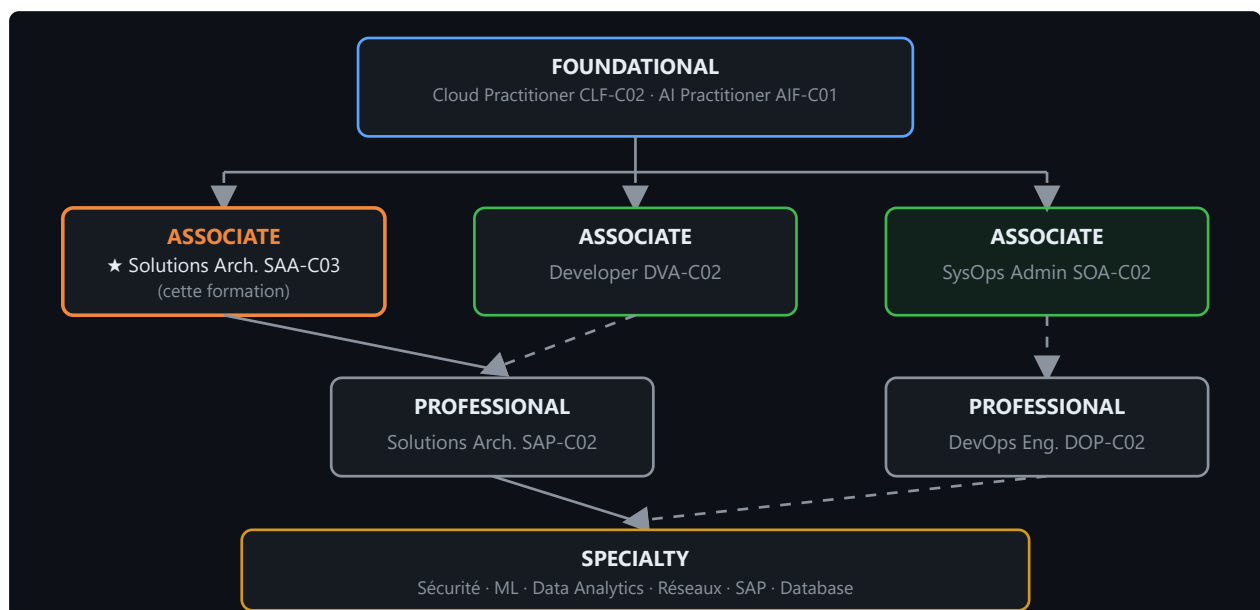
⚠ Warning

Piège fréquent : multiplier les microservices sans réel besoin métier augmente la complexité opérationnelle (plus de composants à surveiller, plus de latence réseau entre services, plus de scénarios d'échec partiel à gérer) sans bénéfice proportionnel. Le découpage en microservices se justifie quand des équipes différentes doivent déployer indépendamment, quand des composants ont des besoins de mise à l'échelle très différents (le service de paiement encaisse un pic le vendredi soir, le catalogue reste stable), ou quand la résilience d'un composant ne doit jamais bloquer les autres. Un monolithe bien structuré reste souvent le bon choix pour une application simple ou une petite équipe.

Référence : [Amazon API Gateway](#) Référence : [AWS Step Functions](#)

9. Certifications AWS — Objectif SAA-C03

Le Chapitre 1 a présenté les quatre niveaux de certification AWS (Fondamental, Associate, Professional, Specialty) et pourquoi cette formation cible la **Solutions Architect Associate (SAA-C03)**. Maintenant que vous avez vu l'ensemble des services du programme, voici ce qui compte vraiment pour préparer concrètement cet examen : la pondération réelle des domaines testés.



i Info

Besoin d'un rappel des 4 niveaux de certification ? Retournez au Chapitre 1, section 1.3.

9.3 Domaines couverts par la SAA-C03

L'examen SAA-C03 répartit ses questions selon quatre domaines pondérés :

Domaine	Pondération
Design d'architectures résilientes	26 %
Design d'architectures haute performance	24 %
Design d'architectures sécurisées	30 %
Design d'architectures optimisées en coût	20 %

Les quatre piliers de cette formation correspondent exactement à ces quatre domaines.

9.4 Certifications spécialisées

Les certifications **Specialty** valident une expertise approfondie sur un domaine précis. Elles nécessitent généralement une expérience pratique significative.

Specialty	Code	Domaine
Security	SCS-C02	IAM, chiffrement, conformité, détection des menaces
Machine Learning	MLS-C01	SageMaker, data pipelines, modèles ML
Advanced Networking	ANS-C01	VPC avancé, Direct Connect, Transit Gateway
Data Analytics	DAS-C01	Redshift, Athena, EMR, Glue, QuickSight
Database	DBS-C01	RDS, DynamoDB, Neptune, ElastiCache
SAP on AWS	PAS-C01	Déploiement SAP sur infrastructure AWS

Ces codes suivent une logique simple : les deux ou trois premières lettres identifient le domaine (SCS = Security, MLS = Machine Learning, ANS = Advanced Networking, DAS = Data Analytics, DBS = Database, PAS = SAP), et le suffixe **-C0x** numérote la version de l'examen — comme pour SAA-C03 vu plus haut, un chiffre plus élevé signale une version plus

récente qui remplace la précédente. Deux services mentionnés dans la colonne Domaine n’ont pas encore été détaillés dans cette formation : **QuickSight** est le service AWS de tableaux de bord et de visualisation de données (BI) qui se branche directement sur Redshift, Athena ou S3 pour construire des graphiques interactifs sans infrastructure à gérer ; **Neptune** est une base de données de graphes managée, pensée pour les données fortement connectées entre elles (réseaux sociaux, moteurs de recommandation, détection de fraude) là où RDS ou DynamoDB modélisent plutôt des tables ou des documents indépendants.

Pour aller plus loin : [Parcours de certifications AWS](#) — programme officiel AWS (Solution Architect, Developer, SysOps, DevOps...)

La boucle est bouclée : IAM sécurise, VPC isole, EC2/S3/ECS exécutent, CloudFormation automatise. Ce que les cinq chapitres ont posé un par un, c’est maintenant à vous de le recomposer sur des cas réels — dans les ateliers Go Deploy et AWS Academy, puis en conditions d’examen si vous visez la certification Solutions Architect Associate.

Travaux pratiques — Automatisation et résilience

AWS Academy — Cloud Foundations

Type	Référence	Intitulé
Module	Module 9	Architecture cloud et Well-Architected Framework
Module	Module 10	Auto Scaling et surveillance
Atelier	Atelier 6	Mise à l’échelle et équilibrage des charges de votre architecture — 100 points
Évaluation	Fin du parcours	Évaluation du cours Cloud Foundations — seuil indiqué : 70/100

Type	Référence	Intitulé
Module	Module 10	Mise en œuvre de la surveillance, de l'élasticité et de la haute disponibilité
Module	Module 11	Automatisation de votre architecture
Module	Module 13	Création d'architectures découplées
Module	Module 14	Création de microservices et d'architectures sans serveur
Module	Module 15	Modèles d'ingénierie des données
Module	Module 16	Planification des sinistres
Module	Module 17	Passerelle vers la certification
Atelier guidé	Module 10	Création d'un environnement hautement disponible
Atelier guidé	Module 11	Automatisation de l'infrastructure avec AWS CloudFormation
Atelier Défi	Module 11	Automatisation du déploiement d'infrastructures
Atelier guidé	Module 13	Création d'applications découplées à l'aide d'Amazon SQS
Atelier guidé	Module 14	Mise en œuvre d'une architecture sans serveur sur AWS
Atelier Défi	Module 14	Mise en œuvre d'une architecture sans serveur pour le café
Atelier guidé	Module 16	Configuration du stockage hybride et migration avec AWS Storage Gateway
Projet	Synthèse	Projet basé sur le cas pratique du café
Évaluation	Fin du parcours	Évaluation du cours Cloud Architecting

Go Deploy

Lab	Intitulé
Lab 22	Surveillance avec CloudWatch : alarmes et tableaux de bord
Lab 24	Utiliser AWS Systems Manager pour installer Apache sur EC2
Lab 25	Gérer des files d'attente avec Amazon SQS
Lab 26	Créer des topics SNS, s'y abonner et ajouter un événement SNS sur S3
Lab 28	Créer un environnement hautement disponible

► Parcours guidé et validation

Ressources

Documentation officielle AWS

Pour approfondir les services d'automatisation présentés dans ce chapitre, voici les références officielles correspondantes :

- [AWS CloudFormation Documentation](#) 
- [AWS Systems Manager Documentation](#) 
- [AWS Elastic Beanstalk Documentation](#) 
- [Amazon CloudWatch Documentation](#) 
- [AWS Well-Architected Framework](#) 
- [AWS Certification](#) 

Le quiz ci-dessous permet de vérifier l'acquisition des notions clés de ce dernier chapitre avant de passer à la pratique.

Quiz interactif du chapitre

Choisissez une ou plusieurs réponses selon la question. La correction expliquée apparaît immédiatement et les propositions changent d'ordre à chaque nouvelle tentative.

Le quiz interactif reste disponible dans la version web du support.

Mini-jeu de synthèse — AWS Quest

Résolvez cinq décisions factuelles pour consolider les fondamentaux avant d'ouvrir le chapitre suivant. Le jeu est une simulation locale : il ne se connecte à aucun compte AWS.

Le quiz interactif reste disponible dans la version web du support.

Référence rapide des commandes CLI et des templates types utilisés dans la formation. Pour les définitions des termes, voir le glossaire.

AWS CLI dans CloudShell

Les commandes sont exécutées depuis **AWS CloudShell** dans la console temporaire du lab. Ne lancez pas `aws configure` et ne copiez aucune clé sur votre poste.

```
1 aws sts get-caller-identity           # vérifier l'identité courante
2 aws configure get region              # afficher la région par défaut du lab
3 aws ec2 describe-regions --output table # lister les régions visibles
```


IAM

```
1  # Utilisateurs et groupes
2  aws iam create-user --user-name alice
3  aws iam list-users
4  aws iam delete-user --user-name alice
5  aws iam create-group --group-name Developers
6  aws iam add-user-to-group --user-name alice --group-name Developers
7
8  # Politiques
9  aws iam list-policies --scope Local
1
0  aws iam create-policy --policy-name MaPolicy --policy-document
file://policy.json
1
1  aws iam attach-user-policy --user-name alice --policy-arn
arn:aws:iam::aws:policy/ReadOnlyAccess
1
2  aws iam attach-group-policy --group-name Developers --policy-arn <arn>
1
3
1
4  # Rôles
1
5  aws iam create-role --role-name MonRole --assume-role-policy-document
file://trust.json
1
6  aws iam attach-role-policy --role-name MonRole --policy-arn <arn>
1
7  aws iam list-roles --query 'Roles[*].[RoleName,Arn]' --output table
1
8
1
9  # Clés d'accès
2
0  aws iam create-access-key --user-name alice
```

```
2
1  aws iam list-access-keys --user-name alice
2
2  aws iam delete-access-key --user-name alice --access-key-id <id>
```

Priorité d'évaluation IAM : Deny explicite > Allow explicite > Deny implicite (par défaut).

Structure d'une policy JSON :

```
1  {
2    "Version": "2012-10-17",
3    "Statement": [
4      {
5        "Effect": "Allow",
6        "Action": ["s3:GetObject", "s3:ListBucket"],
7        "Resource": ["arn:aws:s3:::mon-bucket", "arn:aws:s3:::mon-bucket/*"]
8      }
9    ]
10 }
```

STS

```
1  aws sts get-caller-identity
2
3  aws sts assume-role \
4    --role-arn arn:aws:iam::123456789012:role/mon-role \
5    --role-session-name ma-session \
6    --duration-seconds 3600
7  # Retourne : AccessKeyId, SecretAccessKey, SessionToken, Expiration
```

S3

```
1  # Buckets
2  aws s3 mb s3://mon-bucket --region eu-west-3
3  aws s3 ls
4  aws s3 rb s3://mon-bucket --force
5
6  # Objets
7  aws s3 cp fichier.txt s3://mon-bucket/
8  aws s3 cp s3://mon-bucket/fichier.txt ./
9  aws s3 sync ./dossier s3://mon-bucket/dossier/ --delete
1
0  aws s3 ls s3://mon-bucket/ --recursive
1
1  aws s3 rm s3://mon-bucket/fichier.txt
1
2  aws s3 rm s3://mon-bucket/ --recursive
1
3  aws s3 presign s3://mon-bucket/rapport.pdf --expires-in 3600
1
4
1
5  # Configuration via s3api
1
6  aws s3api put-bucket-versioning --bucket mon-bucket \
1
7      --versioning-configuration Status=Enabled
1
8  aws s3api put-bucket-policy --bucket mon-bucket --policy file://policy.json
1
9  aws s3api get-bucket-policy --bucket mon-bucket
2
0  aws s3api put-public-access-block --bucket mon-bucket \
2
1      --public-access-block-configuration
"BlockPublicAcls=true,IgnorePublicAcls=true,BlockPublicPolicy=true,RestrictPublicBu
ckets=true"
```

Classes de stockage S3 :

Classe	Usage	Disponibilité
Standard	Données fréquentes	99,99 %
Standard-IA	Accès rare, récup rapide	99,9 %
One Zone-IA	Accès rare, 1 AZ	99,5 %
Glacier Instant	Archive, récup ms	99,9 %
Glacier Flexible	Archive, récup mn-h	99,99 %
Glacier Deep Archive	Archive longue durée	99,99 %
Intelligent-Tiering	Accès variable, auto	99,9 %

EC2

```
1  # Instances
2  aws ec2 describe-instances --query 'Reservations[*].Instances[*].
[InstanceId,State.Name,PublicIpAddress]' --output table
3  aws ec2 run-instances \
4    --image-id ami-0abcdef1234567890 \
5    --instance-type t3.micro \
6    --key-name ma-cle \
7    --security-group-ids sg-xxx \
8    --subnet-id subnet-xxx \
9    --count 1
1
0  aws ec2 start-instances --instance-ids i-xxx
1
1  aws ec2 stop-instances --instance-ids i-xxx
1
2  aws ec2 terminate-instances --instance-ids i-xxx
1
3  aws ec2 create-tags --resources i-xxx --tags Key=Name,Value=mon-serveur
1
4
1
5  # Key pairs
1
6  aws ec2 create-key-pair --key-name ma-cle --query 'KeyMaterial' --output text >
ma-cle.pem
1
7  chmod 400 ma-cle.pem
1
8  aws ec2 describe-key-pairs
1
9
2
0  # Security Groups
```

```

2
1  aws ec2 create-security-group --group-name mon-sg --description "Mon SG" --vpc-
id vpc-xxx
2
2  aws ec2 authorize-security-group-ingress --group-id sg-xxx --protocol tcp --
port 22 --cidr 0.0.0.0/0
2
3  aws ec2 describe-security-groups --group-ids sg-xxx
2
4
2
5  # AMIs
2
6  aws ec2 describe-images --owners self
2
7  aws ec2 create-image --instance-id i-xxx --name "mon-ami" --no-reboot

```

Types d'instances courants :

Famille	Usage	Exemple
t3/t4g	Usage général (burstable)	t3.micro, t3.small
m6i	Usage général équilibré	m6i.large
c6i	Calcul intensif	c6i.xlarge
r6i	Mémoire optimisée	r6i.large
p3/g4	GPU	p3.2xlarge

VPC & Réseau

```
1  # VPC
2  aws ec2 create-vpc --cidr-block 10.0.0.0/16
3  aws ec2 describe-vpcs
4  aws ec2 modify-vpc-attribute --vpc-id vpc-xxx --enable-dns-hostnames
5
6  # Subnets
7  aws ec2 create-subnet --vpc-id vpc-xxx --cidr-block 10.0.1.0/24 --availability-
zone eu-west-3a
8  aws ec2 modify-subnet-attribute --subnet-id subnet-xxx --map-public-ip-on-
launch
9
1
0  # Internet Gateway
1
1  aws ec2 create-internet-gateway
1
2  aws ec2 attach-internet-gateway --internet-gateway-id igw-xxx --vpc-id vpc-xxx
1
3
1
4  # NAT Gateway
1
5  aws ec2 allocate-address --domain vpc
1
6  aws ec2 create-nat-gateway --subnet-id subnet-xxx --allocation-id eipalloc-xxx
1
7
1
8  # Route Tables
1
9  aws ec2 create-route-table --vpc-id vpc-xxx
2
0  aws ec2 create-route --route-table-id rtb-xxx --destination-cidr-block
0.0.0.0/0 --gateway-id igw-xxx
```

```
2
1 aws ec2 associate-route-table --route-table-id rtb-xxx --subnet-id subnet-xxx
```

SG vs NACL : Security Group = stateful, niveau instance, allow only. NACL = stateless, niveau subnet, allow + deny.

RDS

```
1 aws rds create-db-subnet-group \  
2   --db-subnet-group-name mon-subnet-group \  
3   --db-subnet-group-description "Subnet group formation" \  
4   --subnet-ids subnet-aaa subnet-bbb  
5  
6 aws rds create-db-instance \  
7   --db-instance-identifier mon-db \  
8   --db-instance-class db.t3.micro \  
9   --engine mysql \  
10  --master-username admin \  
11  --master-user-password MonMotDePasse! \  
12  --allocated-storage 20 \  
13  --db-subnet-group-name mon-subnet-group \  
14  --no-publicly-accessible  
15  
16 aws rds wait db-instance-available --db-instance-identifier mon-db  
17  
18 aws rds describe-db-instances --query 'DBInstances[0].Endpoint.Address'  
19  
20 aws rds stop-db-instance --db-instance-identifier mon-db  
21  
22 aws rds start-db-instance --db-instance-identifier mon-db  
23  
24 # Snapshots
```

2

```
2 aws rds create-db-snapshot --db-instance-identifier mon-db --db-snapshot-  
identifier mon-snapshot
```

2

```
3 aws rds describe-db-snapshots
```

2

```
4 aws rds delete-db-instance --db-instance-identifier mon-db --skip-final-  
snapshot
```

Lambda

```
1 zip function.zip lambda_function.py
2
3 aws lambda create-function \
4   --function-name ma-fonction \
5   --runtime python3.11 \
6   --role arn:aws:iam::123456789012:role/LambdaRole \
7   --handler lambda_function.lambda_handler \
8   --zip-file fileb://function.zip \
9   --timeout 30 \
10
11   --memory-size 128
12
13
14
15
16 aws lambda invoke \
17
18   --function-name ma-fonction \
19
20   --payload '{"key": "value"}' \
21
22   --cli-binary-format raw-in-base64-out \
23
24   response.json
25
26
27
28
29 aws lambda update-function-code --function-name ma-fonction --zip-file
fileb://function-v2.zip
30
31
32 aws lambda update-function-configuration --function-name ma-fonction --timeout
60 --memory-size 256
33
34
35 aws lambda list-functions --query 'Functions[*].
[FunctionName,Runtime,LastModified]' --output table
```

```
2
1 aws logs tail /aws/lambda/ma-fonction --follow
```

CloudFormation

```
1 aws cloudformation validate-template --template-body file://template.yaml
2
3 aws cloudformation create-stack \
4   --stack-name mon-stack \
5   --template-body file://template.yaml \
6   --parameters ParameterKey=Env,ParameterValue=prod \
7   --capabilities CAPABILITY_IAM
8
9 aws cloudformation wait stack-create-complete --stack-name mon-stack
1
0 aws cloudformation describe-stacks --stack-name mon-stack --query
'Stacks[0].Outputs'
1
1 aws cloudformation describe-stack-events --stack-name mon-stack
1
2 aws cloudformation update-stack --stack-name mon-stack --template-body
file://template.yaml
1
3 aws cloudformation delete-stack --stack-name mon-stack
1
4 aws cloudformation detect-stack-drift --stack-name mon-stack
```

Fonctions intrinsèques :

Fonction	Usage
<code>!Ref LogicalId</code>	Référencer une ressource ou un paramètre
<code>!GetAtt Resource.Attr</code>	Attribut d'une ressource
<code>!Sub 'texte \${Var}'</code>	Substitution de variable
<code>!Join [sep, [a,b]]</code>	Concaténer des valeurs
<code>!Select [idx, liste]</code>	Sélectionner dans une liste
<code>!If [cond, vrai, faux]</code>	Condition
<code>!ImportValue nom</code>	Importer un Output d'un autre stack

Template de référence — VPC + EC2 + Security Group :

```

1  AWSTemplateFormatVersion: '2010-09-09'
2  Parameters:
3    InstanceType:
4      Type: String
5      Default: t3.micro
6  Resources:
7    VPC:
8      Type: AWS::EC2::VPC
9      Properties:
10
11      CidrBlock: 10.0.0.0/16
12
13    SubnetPublic:
14
15      Type: AWS::EC2::Subnet
16
17      Properties:
18
19      VpcId: !Ref VPC
20
21      CidrBlock: 10.0.1.0/24
22
23      AvailabilityZone: !Select [0, !GetAZs '']
24
25      MapPublicIpOnLaunch: true
26
27    InternetGateway:
28
29      Type: AWS::EC2::InternetGateway
30
31    IGWAttachment:
32
33      Type: AWS::EC2::VPCGatewayAttachment
34
35      Properties:
36
37      VpcId: !Ref VPC

```

```

2
4     InternetGatewayId: !Ref InternetGateway
2
5 RouteTable:
2
6     Type: AWS::EC2::RouteTable
2
7     Properties:
2
8         VpcId: !Ref VPC
2
9 RouteDefault:
3
0     Type: AWS::EC2::Route
3
1     DependsOn: IGWAttachment
3
2     Properties:
3
3         RouteTableId: !Ref RouteTable
3
4         DestinationCidrBlock: 0.0.0.0/0
3
5         GatewayId: !Ref InternetGateway
3
6 SubnetRouteTableAssociation:
3
7     Type: AWS::EC2::SubnetRouteTableAssociation
3
8     Properties:
3
9         SubnetId: !Ref SubnetPublic
4
0         RouteTableId: !Ref RouteTable
4
1 SecurityGroup:
4
2     Type: AWS::EC2::SecurityGroup

```

```

4
3   Properties:
4
4   GroupDescription: SG formation – HTTP/HTTPS
4
5   VpcId: !Ref VPC
4
6   SecurityGroupIngress:
4
7       - {IpProtocol: tcp, FromPort: 80, ToPort: 80, CidrIp: 0.0.0.0/0}
4
8       - {IpProtocol: tcp, FromPort: 443, ToPort: 443, CidrIp: 0.0.0.0/0}
4
9   EC2Instance:
5
0   Type: AWS::EC2::Instance
5
1   Properties:
5
2   ImageId: ami-0c55b159cbfafa1f0
5
3   InstanceType: !Ref InstanceType
5
4   SubnetId: !Ref SubnetPublic
5
5   SecurityGroupIds: [!Ref SecurityGroup]
5
6   Outputs:
5
7   PublicIP:
5
8   Value: !GetAtt EC2Instance.PublicIp

```

Template de référence — Lambda + API Gateway HTTP :


```

1  AWSTemplateFormatVersion: '2010-09-09'
2  Resources:
3    LambdaRole:
4      Type: AWS::IAM::Role
5      Properties:
6        AssumeRolePolicyDocument:
7          Version: '2012-10-17'
8          Statement:
9            - {Effect: Allow, Principal: {Service: lambda.amazonaws.com}, Action:
sts:AssumeRole}
1
0        ManagedPolicyArns:
1
1          - arn:aws:iam::aws:policy/service-role/AWSLambdaBasicExecutionRole
1
2    HelloFunction:
1
3      Type: AWS::Lambda::Function
1
4      Properties:
1
5        Runtime: python3.11
1
6        Role: !GetAtt LambdaRole.Arn
1
7        Handler: index.handler
1
8        Code:
1
9        ZipFile: |
2
0          def handler(event, context):
2
1            return {'statusCode': 200, 'body': 'Bonjour depuis Lambda !'}
2
2    HttpApi:
2
3      Type: AWS::ApiGatewayV2::Api

```

```

2
4   Properties:
2
5       ProtocolType: HTTP
2
6       Target: !GetAtt HelloFunction.Arn
2
7   LambdaPermission:
2
8       Type: AWS::Lambda::Permission
2
9       Properties:
3
0           FunctionName: !Ref HelloFunction
3
1           Action: lambda:InvokeFunction
3
2           Principal: apigateway.amazonaws.com
3
3           SourceArn: !Sub arn:aws:execute-
api:${AWS::Region}:${AWS::AccountId}:${HttpApi}/*
3
4   Outputs:
3
5       ApiUrl:
3
6           Value: !Sub https://${HttpApi}.execute-api.${AWS::Region}.amazonaws.com/

```

Auto Scaling & Load Balancing

```
1 aws elbv2 create-load-balancer --name mon-alb --subnets subnet-111 subnet-222 -  
-security-groups sg-xxx --type application  
2 aws elbv2 create-target-group --name mon-tg --protocol HTTP --port 80 --vpc-id  
vpc-xxx  
3  
4 aws autoscaling create-auto-scaling-group \  
5   --auto-scaling-group-name mon-asg \  
6   --launch-template "LaunchTemplateName=mon-template" \  
7   --min-size 1 --max-size 5 --desired-capacity 2 \  
8   --vpc-zone-identifier "subnet-111,subnet-222"
```

Elastic Beanstalk

```
1 eb init -p node.js-18 monapp --region eu-west-3  
2 eb create monapp-env --instance-type t3.small --min-instances 1 --max-instances  
3  
3 eb status  
4 eb open  
5 eb logs  
6 eb deploy  
7 eb scale 2  
8 eb terminate monapp-env
```

Systems Manager

```
1  # Run Command
2  aws ssm send-command \
3    --instance-ids i-xxx \
4    --document-name "AWS-RunShellScript" \
5    --parameters 'commands=["apt-get update","apt-get install -y apache2"]'
6  aws ssm get-command-invocation --command-id <id> --instance-id i-xxx
7
8  # Parameter Store
9  aws ssm put-parameter --name /prod/database/password --value "..." --type
"SecureString"
1
0  aws ssm get-parameter --name /prod/database/password --with-decryption
1
1
1
2  # Session Manager (SSH sans port 22 ouvert)
1
3  aws ssm start-session --target i-xxx
```

ElastiCache

```
1 aws elasticache create-cache-cluster \  
2   --cache-cluster-id formation-redis \  
3   --cache-node-type cache.t3.micro \  
4   --engine redis --engine-version 7.0 \  
5   --num-cache-nodes 1 \  
6   --vpc-security-group-ids sg-xxx \  
7   --cache-subnet-group-name my-subnet-group  
8  
9 aws elasticache wait cache-cluster-available --cache-cluster-id formation-redis  
1  
0 aws elasticache describe-cache-clusters --cache-cluster-id formation-redis
```

Route 53

```
1 aws route53 create-hosted-zone --name example.com --caller-reference $(date
+ %s)
2
3 aws route53 change-resource-record-sets --hosted-zone-id ZONE_ID --change-batch
'{
4     "Changes": [{
5         "Action": "CREATE",
6         "ResourceRecordSet": {"Name": "www.example.com", "Type": "A", "TTL": 300,
"ResourceRecords": [{"Value": "93.184.216.34"}]}
7     }]
8 }'
9
1
0 aws route53 create-health-check --health-check-config '{
1
1     "Type": "HTTP", "IPAddress": "93.184.216.34", "Port": 80,
1
2     "ResourcePath": "/health", "RequestInterval": 30, "FailureThreshold": 3
1
3 }'
```

SQS & SNS

```
1 # SQS
2 aws sqs create-queue --queue-name MonQueue
3 aws sqs send-message --queue-url <url> --message-body "Bonjour"
4 aws sqs receive-message --queue-url <url> --max-number-of-messages 1
5 aws sqs delete-message --queue-url <url> --receipt-handle <receipt-handle>
6
7 # SNS
8 aws sns create-topic --name MonTopic
9 aws sns subscribe --topic-arn <arn> --protocol email --notification-endpoint
admin@example.com
1
0 aws sns publish --topic-arn <arn> --message "Alerte"
```

CloudWatch

```
1 aws cloudwatch put-metric-alarm \
2   --alarm-name "CPU-Élevé" \
3   --metric-name CPUUtilization --namespace AWS/EC2 \
4   --statistic Average --period 300 --threshold 80 \
5   --comparison-operator GreaterThanThreshold --evaluation-periods 2 \
6   --dimensions Name=InstanceId,Value=i-xxx \
7   --alarm-actions arn:aws:sns:eu-west-3:123456789012:MonTopic
8
9 aws cloudwatch describe-alarms
1
0 aws cloudwatch delete-alarms --alarm-names "CPU-Élevé"
1
1 aws cloudwatch put-metric-data --namespace "MonApp" --metric-name "Ventes" --
value 42 --unit Count
```

AWS Backup

```
1 aws backup create-backup-vault --backup-vault-name mon-vault
2 aws backup create-backup-plan --backup-plan file://backup-plan.json
3 aws backup create-backup-selection --backup-plan-id <id> --backup-selection
file://selection.json
4 aws backup list-recovery-points-by-resource --resource-arn <arn>
5 aws backup start-restore-job --recovery-point-arn <arn> --iam-role-arn <role-
arn>
```

Well-Architected Framework — 6 piliers

Pilier	Principe clé
Excellence opérationnelle	Automatiser, observer, améliorer
Sécurité	Moindre privilège, chiffrement, traçabilité
Fiabilité	Redondance, reprise, tests de panne
Performance	Choisir le bon service, élasticité
Optimisation des coûts	Pay-as-you-go, rightsizing, Reserved Instances
Durabilité	Réduire l'empreinte carbone

Régions et zones de disponibilité

```
1 aws ec2 describe-regions
2 aws ec2 describe-availability-zones --region eu-west-3
```


Région	Nom	AZ
Paris	eu-west-3	eu-west-3a/b/c
Irlande	eu-west-1	eu-west-1a/b/c
Frankfurt	eu-central-1	eu-central-1a/b/c

Filtres et sortie CLI

```

1  # JMESPath (--query)
2  aws ec2 describe-instances --query 'Reservations[0].Instances[0].InstanceId'
3  aws ec2 describe-instances --query 'Reservations[*].Instances[?
State.Name=='running'].[InstanceId,PublicIpAddress]' --output table
4  aws s3api list-buckets --query 'length(Buckets)'
5
6  # Formats de sortie
7  aws ec2 describe-instances --output json      # défaut, idéal pour scripts
8  aws ec2 describe-instances --output table    # idéal pour lecture humaine
9  aws ec2 describe-instances --output text     # idéal pour scripts bash
1
0  aws ec2 describe-instances --output yaml      # AWS CLI v2
1
1
1
2  # Filtres serveur
1
3  aws ec2 describe-instances --filters Name=tag:Name,Values=mon-serveur
1
4  aws ec2 describe-instances --filters Name=instance-state-name,Values=running

```

Tags et diagnostic

```
1 # Tagger et rechercher par tag
2 aws ec2 create-tags --resources i-xxx --tags Key=Environment,Value=Production
3 aws ec2 describe-instances --filters "Name=tag:Environment,Values=Production"
4
5 # Coûts
6 aws ce get-cost-and-usage \
7     --time-period Start=2026-01-01,End=2026-01-31 \
8     --granularity MONTHLY --metrics "UnblendedCost" \
9     --group-by Type=TAG,Key=Project
1
10
11
12 # Limites de compte
13
14
15 aws service-quotas list-service-quotas --service-code ec2
```

Nommage des ressources — convention :

Ressource	Convention	Exemple
VPC	vpc-[env]-[projet]	vpc-prod-webapp
Subnet	sn-[public/private]-[az]	sn-public-a
EC2	[role]-[env]-[numéro]	web-prod-01
Security Group	sg-[role]-[env]	sg-web-prod
S3 bucket	[org]-[projet]-[env]-[région]	acme-backup-prod-eu
IAM Role	role-[service]-[permissions]	role-ec2-s3read

Services clés — tableau de référence

Service	Description	Console
IAM	Gestion identités et permissions	iam.amazonaws.com
EC2	Machines virtuelles	console.aws.amazon.com/ec2
S3	Stockage objet	console.aws.amazon.com/s3
VPC	Réseau privé virtuel	console.aws.amazon.com/vpc
RDS	Bases de données managées	console.aws.amazon.com/rds
Lambda	Fonctions serverless	console.aws.amazon.com/lambda
CloudFormation	IaC AWS	console.aws.amazon.com/cloudformation
CloudWatch	Monitoring et logs	console.aws.amazon.com/cloudwatch
Route 53	DNS managé	console.aws.amazon.com/route53
ELB	Load balancing	console.aws.amazon.com/ec2/v2#LoadBalancers
Auto Scaling	Scalabilité automatique	console.aws.amazon.com/ec2/autoscaling
ECS	Conteneurs Docker managés	console.aws.amazon.com/ecs
EKS	Kubernetes managé	console.aws.amazon.com/eks
Secrets Manager	Gestion des secrets	console.aws.amazon.com/secretsmanager
KMS	Gestion des clés de chiffrement	console.aws.amazon.com/kms

Les termes sont organisés par grand thème de la formation puis par ordre alphabétique.

Fondamentaux du Cloud

AWS (Amazon Web Services) Plateforme de cloud computing d'Amazon lancée en 2006. Plus de 200 services couvrant le calcul, le stockage, les bases de données, la sécurité, l'IA, le DevOps.

CAPEX (Capital Expenditures) Modèle économique traditionnel : investissement initial important (achat de serveurs), amorti sur plusieurs années.

OPEX (Operational Expenditures) Modèle économique du Cloud : dépenses variables basées sur l'usage réel, sans investissement initial.

Cloud Computing Modèle de mise à disposition de ressources informatiques (calcul, stockage, réseau) à la demande, facturées à l'usage, plutôt que possédées et exploitées en interne.

Cloud hybride Modèle de déploiement combinant Cloud privé et Cloud public, avec circulation de données entre les deux environnements.

Cloud privé Modèle de déploiement où l'infrastructure est dédiée à une seule organisation, non mutualisée avec d'autres clients.

Cloud public Modèle de déploiement où les ressources sont hébergées dans les datacenters du fournisseur et mutualisées entre plusieurs clients, avec isolation logique.

Conteneurisation Méthode d'isolation applicative où plusieurs applications partagent le même noyau OS, dans des conteneurs légers — par opposition à la virtualisation complète.

IaaS (Infrastructure as a Service) Modèle de service où le fournisseur met à disposition des ressources de base (calcul, stockage, réseau) que le client configure lui-même. Exemple : EC2.

NIST — Cinq caractéristiques du Cloud Référentiel définissant le Cloud Computing par cinq critères : libre-service à la demande, accès réseau étendu, mise en commun des ressources, élasticité rapide, mesurabilité du service.

PaaS (Platform as a Service) Modèle de service où le fournisseur gère l'infrastructure et le runtime, le client se concentre sur son code. Exemple : Elastic Beanstalk.

SaaS (Software as a Service) Modèle de service où le fournisseur gère toute la pile applicative, le client utilise le logiciel via un navigateur ou une API. Exemple : AWS WorkMail.

Virtualisation Technologie permettant d'exécuter plusieurs machines virtuelles isolées sur un même serveur physique, via un hyperviseur.

Well-Architected Framework Ensemble de bonnes pratiques AWS structurées en six piliers : Excellence opérationnelle, Sécurité, Fiabilité, Performance, Optimisation des coûts, Durabilité.

Sécurité & IAM

Access Key Paire de credentials (Access Key ID + Secret Access Key) permettant l'accès programmatique à AWS via CLI ou SDK. Ne jamais committer dans un dépôt Git.

AssumeRole Action STS permettant à une identité de prendre temporairement les permissions d'un rôle IAM, via des credentials temporaires.

ARN (Amazon Resource Name) Identifiant unique et universel de toute ressource AWS, au format

`arn:partition:service:région:compte-id:type/nom`.

Cognito Identity Pool Service échangeant un token externe (Cognito User Pool, Google, SAML) contre des credentials AWS temporaires, permettant à une application d'appeler directement S3, DynamoDB, etc.

Cognito User Pool Annuaire d'utilisateurs managé pour applications web et mobiles — inscription, connexion, MFA, réinitialisation de mot de passe.

CloudTrail Service d'audit qui enregistre tous les appels API effectués sur un compte AWS — qui a fait quoi, quand, depuis où.

Explicit Deny Refus explicite dans une politique IAM, prioritaire sur tout Allow sans exception.

Fédération d'identité Mécanisme permettant à des utilisateurs authentifiés par un système externe (Active Directory, Okta) d'accéder à AWS sans compte IAM natif.

IAM (Identity and Access Management) Service centralisé de gestion des identités et des permissions AWS — définit qui peut faire quoi, sur quelle ressource, dans quelles conditions.

IAM Identity Center Service de Single Sign-On centralisé pour accéder à plusieurs comptes AWS et applications avec un seul identifiant.

MFA (Multi-Factor Authentication) Authentification à plusieurs facteurs — mot de passe + application TOTP, clé physique FIDO2, ou biométrie.

Permission Boundary Politique IAM définissant le plafond maximal de permissions qu'une identité peut avoir, même si des politiques plus permissives lui sont attachées.

Policy IAM Document JSON définissant des permissions accordées ou refusées, structuré en `Version`, `Statement` (`Effect`, `Action`, `Resource`, `Condition`).

Principal Entité à qui s'applique une politique IAM — utilisateur, rôle, service AWS, compte, ou tous (*).

Rôle IAM Identité AWS sans credentials permanents, assumée temporairement par un utilisateur, un service ou un compte tiers.

SCP (Service Control Policy) Politique appliquée au niveau d'une OU ou d'un compte dans AWS Organizations, limitant les permissions maximales même pour le root.

Secrets Manager Service de gestion sécurisée de secrets (mots de passe, clés API), avec rotation automatique et intégration native RDS.

STS (Security Token Service) Service émettant des credentials temporaires (AccessKeyId, SecretAccessKey, SessionToken), utilisé pour AssumeRole et la fédération d'identité.

Trust Policy Politique attachée à un rôle IAM définissant qui a le droit de l'assumer — distincte de la policy de permissions qui définit ce que le rôle peut faire.

Utilisateur IAM Identité permanente associée à des identifiants (login/mot de passe ou clés d'accès), représentant une personne ou une application.

Stockage S3

ACL (Access Control List) Mécanisme legacy de permissions sur un bucket ou un objet S3 — préférer les Bucket Policies.

Bucket S3 Conteneur de stockage d'objets, au nom unique mondial, rattaché à une région.

Bucket Policy Document JSON attaché directement à un bucket S3, définissant qui peut accéder à quoi.

Classe de stockage S3 Niveau de tarification et de disponibilité d'un objet S3 : Standard, Standard-IA, One Zone-IA, Glacier (Instant/Flexible/Deep Archive), Intelligent-Tiering.

Durabilité 11×9 Propriété de S3 Standard : 99,999999999 % de durabilité des données stockées.

Lifecycle Policy (S3) Règle automatisant la transition d'objets entre classes de stockage ou leur suppression après un délai défini.

Object (S3) Unité de base de S3 — une clé, des données (jusqu'à 5 To), des métadonnées et un ETag.

Presigned URL URL temporaire signée cryptographiquement, permettant un accès direct à un objet S3 privé sans credentials AWS.

S3 (Simple Storage Service) Service de stockage d'objets à haute durabilité et disponibilité, accessible via HTTPS.

S3 Transfer Acceleration Fonctionnalité accélérant les uploads vers S3 en passant par le réseau Edge de CloudFront.

SSE (Server-Side Encryption) Chiffrement côté serveur pour S3 — SSE-S3 (clé gérée par AWS), SSE-KMS (clé gérée dans KMS, avec audit), SSE-C (clé fournie par le client).

Versioning (S3) Fonctionnalité conservant toutes les versions d'un objet S3, protégeant contre la suppression ou l'écrasement accidentels.

Calcul EC2 & services managés

AMI (Amazon Machine Image) Image disque prête à l'emploi pour lancer une instance EC2 — OS, logiciels préinstallés, configuration.

Auto Scaling Group Groupe gérant automatiquement le nombre d'instances EC2 selon la charge, entre un minimum, un maximum et une capacité désirée.

Availability Zone (AZ) Centre de données isolé physiquement au sein d'une région AWS — déployer sur plusieurs AZ garantit la haute disponibilité.

Elastic Beanstalk Service PaaS gérant automatiquement l'infrastructure (EC2, Load Balancer, Auto Scaling) autour du code applicatif fourni.

EBS (Elastic Block Store) Volume de stockage bloc attaché à une instance EC2, persistant après l'arrêt de l'instance.

EFS (Elastic File System) Système de fichiers NFS managé, partageable entre plusieurs instances EC2 simultanément.

ECS (Elastic Container Service) Service d'orchestration de conteneurs Docker managé par AWS, sur EC2 ou en mode serverless Fargate.

EKS (Elastic Kubernetes Service) Service Kubernetes managé — AWS gère le control plane, le client gère (ou délègue à Fargate) les workers.

Fargate Mode d'exécution serverless pour ECS/EKS — aucune instance à gérer, facturation à la ressource consommée.

Instance Store Stockage temporaire attaché physiquement à l'hôte EC2, perdu à l'arrêt ou au redémarrage de l'instance.

Key Pair Paire de clés SSH (publique/privée) utilisée pour l'authentification sécurisée à une instance EC2.

Lambda Service de calcul sans serveur (FaaS) exécutant du code en réponse à un événement, facturé à l'invocation et à la durée d'exécution.

Launch Template Modèle de configuration (AMI, type, Security Group, user data) utilisé pour lancer des instances EC2, notamment via un Auto Scaling Group.

Reserved Instance Engagement d'utilisation EC2 sur 1 ou 3 ans, avec réduction tarifaire pouvant atteindre 75 % par rapport au tarif à la demande.

Savings Plans Engagement de dépense horaire (\$/h) sur 1 ou 3 ans, plus flexible que les Reserved Instances, couvrant EC2, Lambda et Fargate.

Security Group Pare-feu stateful appliqué au niveau de l'instance EC2 — règles entrantes et sortantes, allow uniquement.

Spot Instance Capacité EC2 inutilisée louée à prix réduit (jusqu'à 90 %), interruptible par AWS avec 2 minutes de préavis.

Type d'instance EC2 Combinaison de vCPU, RAM, réseau et stockage — familles t (burstable), m (équilibrée), c (calcul), r (mémoire), g/p (GPU).

User Data Script exécuté au premier démarrage d'une instance EC2, pour l'installation ou la configuration initiale.

Réseau VPC

CIDR (Classless Inter-Domain Routing) Notation définissant une plage d'adresses IP — par exemple `10.0.0.0/16` pour 65 536 adresses.

ENI (Elastic Network Interface) Carte réseau virtuelle attachée à une instance EC2, portant IP privée/publique, adresse MAC et Security Groups.

Internet Gateway (IGW) Passerelle permettant la communication entre un subnet public et Internet. Gratuite, une seule par VPC.

NACL (Network Access Control List) Pare-feu stateless au niveau du subnet — règles numérotées, entrantes et sortantes évaluées séparément.

NAT Gateway Service permettant à des instances en subnet privé d'initier des connexions vers Internet sans être accessibles depuis l'extérieur.

Peering VPC Connexion réseau privée entre deux VPC, non transitive — $A \leftrightarrow B$ et $B \leftrightarrow C$ ne créent pas $A \leftrightarrow C$.

Route Table Table de routage associée à un subnet, déterminant où envoyer le trafic selon sa destination.

Subnet Subdivision d'un VPC rattachée à une zone de disponibilité — public (route vers IGW) ou privé (route vers NAT Gateway ou aucune route Internet).

Transit Gateway Hub réseau centralisé interconnectant plusieurs VPC, comptes AWS et réseaux on-premise via un seul attachement par ressource.

VPC (Virtual Private Cloud) Réseau privé virtuel isolé dans AWS, avec sa propre plage d'adresses IP, ses subnets, tables de routage et règles de sécurité.

VPC Endpoint Point de connexion privé permettant d'accéder à un service AWS depuis une VPC sans transiter par Internet.

VPN Site-to-Site Connexion chiffrée entre un réseau on-premise et un VPC AWS, via deux tunnels IPsec redondants.

Bases de données

Aurora Moteur de base de données relationnel propriétaire AWS, compatible MySQL et PostgreSQL, avec réplication native sur 6 copies dans 3 zones de disponibilité.

Aurora Serverless v2 Version auto-scalante d'Aurora, facturée en ACU-heures, capable de s'ajuster en quelques secondes selon la charge.

DynamoDB Base de données NoSQL serverless stockant des paires clé-valeur ou documents JSON, avec latence inférieure à la milliseconde.

ElastiCache Service de cache en mémoire managé — moteur Redis (structures avancées, persistance) ou Memcached (cache simple).

Hot Partition Situation où une Partition Key DynamoDB à faible cardinalité concentre toutes les requêtes sur une seule partition, provoquant un throttling.

RCU / WCU Unités de capacité de lecture et d'écriture DynamoDB en mode provisionné — 1 RCU = une lecture fortement cohérente d'un item ≤ 4 Ko, 1 WCU = une écriture d'un item ≤ 1 Ko.

RDS (Relational Database Service) Service de bases de données relationnelles managé — MySQL, PostgreSQL, MariaDB, Oracle, SQL Server, Aurora.

RDS Multi-AZ Déploiement RDS en haute disponibilité — instance principale et standby synchrone dans une autre zone, failover automatique en ~30 secondes.

Read Replica Copie en lecture seule d'une base RDS ou Aurora, en réplication asynchrone, utilisée pour décharger les lectures de l'instance principale.

AWS DMS (Database Migration Service) Service managé de migration de bases de données, combinant une copie complète initiale (Full Load) et une réplication continue des changements (CDC).

DNS & découplage

Alias Record (Route 53) Type d'enregistrement DNS propre à AWS pointant vers une ressource AWS (ALB, CloudFront, S3), sans les limitations d'un CNAME classique.

Dead Letter Queue (DLQ) File SQS ou topic SNS recevant les messages non traités avec succès après plusieurs tentatives.

Health Check (Route 53) Vérification périodique de la disponibilité d'une ressource, utilisée pour déclencher un routage de failover.

Route 53 Service DNS managé d'AWS — enregistrement de domaines, résolution DNS, health checks, politiques de routage avancées.

SNS (Simple Notification Service) Service de messagerie pub/sub — un message publié sur un topic est distribué à tous les abonnés (email, SMS, SQS, Lambda).

SQS (Simple Queue Service) Service de file d'attente de messages managée, découplant un producteur d'un consommateur.

Automatisation & Infrastructure as Code

API Gateway Service exposant un point d'entrée HTTP unique, routant chaque requête vers le service backend adapté (Lambda, conteneur, EC2).

CloudFormation Service d'Infrastructure as Code AWS — décrit une infrastructure en YAML/JSON (template) et la déploie sous forme de stack.

CloudFormation Drift Divergence entre l'état réel d'une ressource et son template CloudFormation, survenant après une modification manuelle.

CloudFormation Stack Ensemble de ressources AWS créées et gérées ensemble par un même template, avec création/mise à jour/suppression atomiques.

CloudWatch Service de supervision AWS — métriques, logs, alarmes, tableaux de bord.

EventBridge Service d'événements serverless routant des événements entre services AWS, applications SaaS et fonctions Lambda selon des règles.

IaC (Infrastructure as Code) Paradigme consistant à décrire et gérer l'infrastructure via du code versionnable plutôt que par des actions manuelles.

Quick Start Template CloudFormation préconfiguré et validé par AWS, déployant une architecture complète éprouvée.

RPO (Recovery Point Objective) Quantité maximale de données qu'une organisation accepte de perdre en cas de sinistre, exprimée en durée.

RTO (Recovery Time Objective) Durée maximale d'indisponibilité acceptable d'un service avant que l'impact métier devienne intolérable.

AWS Systems Manager (SSM) Suite d'outils d'administration à distance sans SSH — Run Command, Session Manager, Patch Manager, Parameter Store.

Step Functions Service d'orchestration de workflows serverless sous forme de machine à états, coordonnant Lambda et d'autres services AWS.

Ce glossaire couvre les cinq chapitres de la formation. Pour la syntaxe des commandes CLI correspondantes, voir le cheat-sheet.
